The Foundation Model Handbook

Architecture, Pretraining, Post-training, Reinforcement Learning, Inference, and Systems

pig7selene

Version 1.0 · September 2026

Contents

Part I — Foundations

Chapter 1: Tokenization and Input Representations

Abstract

Tokenization defines the discrete alphabet through which a language model receives text and emits predictions. This chapter develops that interface from a string-to-token map through subword vocabulary construction and embedding lookup. It emphasizes the consequences of tokenization for sequence length, parameter allocation, reproducibility, and the engineering contracts that connect data preparation to model execution.

1.1 Introduction

Text reaches a Foundation Model through a deliberately narrow interface. The model does not receive characters, words, or bytes as linguistic objects; it receives a finite sequence of integer identifiers. That reduction is easy to overlook because tokenization is usually hidden behind a library call. It nevertheless fixes the model's input alphabet, determines how much of a context window a document occupies, and allocates a substantial parameter table. A tokenizer is therefore part of the model specification, not merely preprocessing.

The immediate goal of this chapter is to make the sequence entering a decoder-only Transformer mathematically explicit. Its longer-term purpose is practical: training systems count compute in tokens, form batches from token sequences, and reuse the same identifiers in the output softmax. The discussion treats tokenization as a stable contract between corpus processing, model training, evaluation, and generation.

1.2 From Text to Discrete Sequences

Let $\mathcal{S}$ denote the set of text strings after a chosen character encoding, and let $\mathcal{V}$ be a finite vocabulary. A tokenizer is a deterministic map from an input string to a finite sequence of vocabulary elements:

$$\tau : \mathcal{S} \to \mathcal{V}^*. \tag{1}$$

The star denotes the set of all finite sequences. In an implementation, each vocabulary element is assigned an integer identifier, so the output of τ is more conveniently written as

$$x = (x_1, \ldots, x_T), \quad x_t \in \{0, \ldots, |\mathcal{V}| - 1\}. \tag{2}$$

Here T is the tokenized length of the particular input, not a property of the vocabulary. Two strings with similar character counts can have very different values of T, and the computational cost of a Transformer is governed primarily by token positions rather than character positions.

> **Definition 1: Tokenizer.** A tokenizer consists of a finite vocabulary $\mathcal{V}$, a convention for converting text into an initial sequence of atomic symbols, a segmentation procedure that returns elements of $\mathcal{V}$, and an inverse decoding convention on the subset of sequences that it regards as valid.

The definition includes more than a merge list. Normalization, whitespace handling, the choice between Unicode characters and bytes, and the treatment of reserved symbols all affect the map in Equation 1. A tokenizer need not be reversible over arbitrary input strings: a normalizer may intentionally collapse distinctions. It should, however, make its contract explicit. In a training system, the same contract must be used for corpus preparation, training examples, evaluation prompts, and generation-time decoding. Otherwise the model is trained and evaluated on different discrete languages.

The vocabulary is usually much smaller than the space of possible strings. A word-level vocabulary makes each frequent word inexpensive in sequence length, but it either assigns rare forms to an unknown symbol or grows without bound. A character-level vocabulary avoids most out-of-vocabulary failures, but represents common words by long sequences. Subword tokenization occupies the useful middle ground: common fragments may become single tokens, while uncommon strings are composed from smaller pieces. Sennrich, Haddow, and Birch adapted BPE to this open-vocabulary setting and showed that subword units can replace a fixed-vocabulary back-off mechanism in neural translation [1].

1.3 Vocabulary Design and Reserved Symbols

A vocabulary contains ordinary text pieces and a small set of symbols with control semantics. Typical examples include a beginning-of-sequence marker, an end-of-sequence marker, padding, and reserved delimiters for message roles or document boundaries. These are token IDs whose embeddings are learned in exactly the same way as ordinary token embeddings, but whose positions carry a convention established by the training data and runtime.

The crucial design rule is that a special token must be handled as an atomic unit before ordinary segmentation. If a chat delimiter is sometimes decomposed into text pieces and sometimes inserted as one reserved ID, the model sees two incompatible representations of the same structural event. Conversely, a token that is reserved but never appears in training has no learned semantics merely because it has a vocabulary entry. Tokenizer configuration and data formatting must therefore be versioned together.

Vocabulary size also creates a direct parameter trade-off. A larger vocabulary can shorten sequences by storing more frequent substrings, but it enlarges both the input embedding table and, in many decoder-only models, the output classifier. It also changes the empirical distribution of targets: a very common token receives many updates, while a rare token receives few. There is no vocabulary size that is independently optimal; the appropriate choice depends on corpus coverage, languages, model width, and the intended context length.

1.4 Byte Pair Encoding

BPE originated as a compression algorithm that repeatedly replaces frequent adjacent symbol pairs with newly introduced symbols [2]. Its appeal for neural text processing is not compression alone. The same procedure constructs a vocabulary that can represent frequent strings compactly while retaining a base alphabet for rare strings. Modern BPE tokenizers differ in details, but the merge construction is sufficiently simple to derive directly.

Consider a corpus represented as sequences over an initial alphabet $\mathcal{V}_0$. The alphabet may be characters, bytes, or a reversible encoding of bytes. At merge step m, let $c_{m(a,b)}$ count occurrences of the adjacent pair (a, b) in the current representation. BPE selects a most frequent pair,

$$(a_m, b_m) = \arg\max_{a,b} c_{m(a,b)}, \tag{3}$$

introduces a new symbol u_m denoting the concatenation of a_m and b_m, and replaces the selected pair according to a fixed overlap convention. After M merges, the learned vocabulary is the base vocabulary together with the introduced symbols:

$$\mathcal{V}_M = \mathcal{V}_0 \cup \{u_1, ..., u_M\}. \tag{4}$$

The recurrence exposes the central inductive bias. BPE does not discover linguistic morphemes by definition; it merges pairs that are frequent under its current representation. A frequent substring can become one token even if it is not a word, while a morphologically meaningful but rare substring may remain split. This is often appropriate for language modeling, whose immediate objective is to assign probabilities to symbol sequences, but it prevents us from treating token boundaries as a ground-truth linguistic analysis.

> **Algorithm 1: BPE vocabulary training.** Start from an initial corpus representation over $\mathcal{V}_0$. At each step, count adjacent pairs, choose a pair with maximum count under a deterministic tie rule, introduce its concatenated symbol, and replace its eligible occurrences throughout the corpus. Record the ordered merge. Stop after the prescribed number of merges or when no pair is eligible.

Training the merge vocabulary and encoding a new string are separate procedures. Training estimates which pairs deserve vocabulary entries from a corpus. Encoding begins from the base symbols of a new string and applies only merges recorded during training, in their learned priority order. Implementations commonly store a rank for each merge and repeatedly select the best available adjacent pair. The rank order, base-symbol construction, and pre-tokenization policy are all part of the tokenizer artifact; changing any of them changes the IDs in Equation 2.

Byte-level BPE makes the base alphabet cover bytes rather than an application-specific set of characters. Provided the byte encoding itself is reversible, every input byte sequence can then be represented without an unknown token; rare text simply falls back to more base symbols. GPT-2 used a byte-level version of BPE, illustrating how this choice supplies a fixed vocabulary while retaining coverage of arbitrary text bytes [3]. By contrast, SentencePiece emphasizes training subword models directly from raw sentences, avoiding a language-specific requirement for pre-tokenized word boundaries [4]. These choices are implementation conventions, not interchangeable defaults: they influence whitespace behavior, multilingual coverage, and the reproducibility of a dataset pipeline.

1.5 Embedding Lookup

The output of a tokenizer becomes useful to a neural network through an embedding table. Let d be the model width and let

$$E \in R^{|\mathcal{V}| \times d} \tag{5}$$

be a trainable matrix. The vector associated with token ID x_t is the corresponding row,

$$e_t = E[x_t] \in R^d. \tag{6}$$

This operation is a row lookup, although automatic-differentiation systems often implement it as multiplication by a one-hot vector. If q_t is the one-hot representation of x_t, then $e_t = q_t^T E$. The lookup view is operationally more useful: only the rows selected by a batch receive direct embedding gradients. A vocabulary item that appears rarely receives few such updates, while a control token inserted into every sequence may receive many.

For a batch of B padded sequences of maximum length T, the token-ID tensor has shape $B \times T$ and the embedding output has shape $B \times T \times d$. The embedding table contributes $|\mathcal{V}|d$ trainable scalar parameters before any positional representation or Transformer layer is considered. This count is elementary, yet it explains why tokenization choices cannot be separated from model budgeting.

1.6 Positional Input Representations

The embedding table associates a vector with a token identity, not with a location in a sequence. The word piece for a name has the same lookup vector whether it occurs first or last. A Transformer consequently needs an additional source of positional information. In the original Transformer formulation, position-dependent vectors are added to token embeddings before the stacked attention and feed-forward layers [5]. We write the initial state at position t as

$$h_t^0 = e_t + p_t, \tag{7}$$

where $p_t \in R^d$ is a positional representation. It may be a learned absolute embedding, a deterministic vector, or part of a later attention computation. The present point is narrower than a comparison of position-encoding schemes: tokenization fixes the positions to which any such scheme is applied. Altering a tokenizer can therefore change both the length and the positional geometry of every training example, even when the underlying text is unchanged.

Some implementations share the input embedding matrix with the output projection used to score the next token. Weight tying reduces parameters and links the geometry of input and output symbols, but it is an architectural option rather than a tokenizer requirement. The original Transformer described shared embedding and pre-softmax weights in some settings [5].

1.7 Token Counts and Engineering Implications

For a corpus $\mathcal{D}$, define its token count under tokenizer τ by

$$N_{\tau(\mathcal{D})} = \sum_{s \in \mathcal{D}} |\tau(s)|. \tag{8}$$

This quantity recurs throughout Foundation Model engineering. Pretraining budgets are normally reported in tokens, batches are assembled in token positions, and the cost of processing a fixed document collection varies with $N_{\tau(\mathcal{D})}$. A tokenizer that splits a corpus into longer sequences consumes more context positions and can increase the attention work associated with a training example. It may nevertheless be preferable if its base representation improves coverage or avoids brittle text

normalization. The correct comparison is therefore not a slogan such as "fewer tokens is better," but a joint assessment of coverage, modeling behavior, parameter cost, and systems cost.

The same caution applies to evaluation. Perplexity and average negative log-likelihood are defined per model token, so their numerical values are meaningful only relative to a specified tokenizer and normalization pipeline. Comparing token-level losses across incompatible tokenizations can confound changes in modeling quality with changes in the units being predicted.

1.8 Implementation Contracts

A minimal tokenizer implementation should expose four testable operations: encode text to IDs, decode valid IDs to text, identify reserved tokens, and serialize the complete vocabulary plus segmentation rules. Round-trip tests should be performed on representative text, including whitespace, non-ASCII characters, delimiters, and malformed inputs where the contract specifies behavior. The test does not prove that a tokenizer is linguistically desirable; it establishes that the discrete interface is stable.

For BPE, a small implementation is especially instructive. Begin with a corpus of short strings, represent each string as a sequence of base symbols, count adjacent pairs, apply the merge rule in Equation 3, and store the resulting ranks. Then encode held-out strings using those ranks. The exercise makes two facts concrete: merge training is corpus dependent, and inference-time segmentation is constrained by a fixed learned artifact. Both facts become important when training data, checkpoints, or evaluation sets are revised.

1.9 Summary

Tokenization is one of the first irreversible modeling decisions in a language-model pipeline. It defines the symbols that a model will embed, predict, cache, shard, and evaluate. A sound implementation makes its discrete mapping, special-token conventions, merge rules, and serialization format explicit. The resulting sequence of IDs is the input to the decoder-only computation graph developed in Chapter 2, **Transformer Architecture**.

References

[1] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2016, pp. 1715–1725. doi: 10.18653/v1/P16-1162.

[2] P. Gage, "A New Algorithm for Data Compression," *C Users Journal*, vol. 12, no. 2, pp. 23–38, 1994, [Online]. Available: https://www.derczynski.com/papers/archive/BPE_Gage.pdf

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[4] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, 2018, pp. 66–71. doi: 10.18653/v1/D18-2012.

[5] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

Part II — Architecture

Chapter 2: Transformer Architecture

Abstract

A decoder-only Transformer maps a batch of token IDs to vocabulary logits through a sequence of shape-preserving residual updates. This chapter gives the complete high-level computation graph before treating attention, positional encoding, normalization, and optimization in dedicated technical chapters. It defines the hidden-state and tensor-shape contracts of the model, contrasts Pre-Norm and Post-Norm block organization, and identifies the parameter, activation, and sequence-length terms that dominate later systems analysis.

2.1 Introduction

A decoder-only Transformer is often introduced as a stack of Self-Attention layers. That description is useful but incomplete. A language model has a stable external contract: it receives a tensor of token IDs and produces one vocabulary-sized vector of logits at every position. Attention is only one transformation in the path between those two objects. The remaining transformations determine where normalization acts, how information is accumulated, which tensor dominates activation memory, and how the output vocabulary is scored.

This chapter gives that path a precise, high-level form. The aim is not to rederive attention, positional encoding, normalization, or feed-forward networks in isolation. Instead, the chapter establishes the computation graph that makes those details meaningful: a sequence enters a residual stream, each block writes updates to that stream, and a final projection turns the resulting states into logits for an autoregressive language-model objective.

2.2 Decoder-Only Transformer Architecture

2.2.1 From Token IDs to Hidden States

Consider a batch of B token sequences, each represented to length T. Let

$$X \in \{0, ..., |\mathcal{V}| - 1\}^{B \times T} \tag{9}$$

be the integer token-ID tensor, let d denote the model dimension, and let $E \in R^{|\mathcal{V}| \times d}$ be the token-embedding matrix. Embedding lookup produces a rank-three tensor

$$H^0 = E[X] \in R^{B \times T \times d}. \tag{10}$$

The superscript identifies depth rather than a power. Thus H^0 is the sequence representation before the first Transformer block, while H^ℓ will denote the representation after block ℓ. If the architecture uses additive absolute positions, the input is instead $H^0 = E[X] + P$, where $P \in R^{T \times d}$ is broadcast across the batch. Other positional schemes modify later computations rather than this initial tensor. In every case, the central object is the same: for batch item b and position t, $H^0_{b,t,:}$ is a length-d hidden-state vector.

The distinction between IDs and states is fundamental. X is discrete and has no useful local geometry: token ID 800 is not intrinsically closer to ID 801 than to ID 7. Each slice of H^0 is continuous and trainable. Subsequent layers can mix information across positions and features without changing the sequence length.

2.2.2 Stacked Blocks and Causal Processing

Let L be the number of Transformer blocks. A decoder-only model applies a sequence of functions $F_1, ..., F_L$ to the input representation:

$$H^\ell = F_{\ell(H^{\ell-1})}, \quad \ell \in \{1, ..., L\}. \tag{11}$$

Each F_ℓ preserves the shape $B \times T \times d$. The expression **decoder-only** refers to the causal dependency pattern within these functions: the state at position t may draw on positions at most t, but not on later tokens. It does not mean that the block processes positions one at a time during training. The full tensor is usually evaluated in parallel under a causal Attention Mask. The sequential aspect of generation emerges only when the model is used to append a new token to an existing prefix.

The original Transformer paired an encoder stack with a decoder stack for sequence-to-sequence tasks [6]. A Foundation Model used for Causal Language Modeling retains the masked, autoregressive

branch and omits encoder-decoder cross-attention. This reduction creates a uniform contract: a single stream of token positions is transformed repeatedly, then scored against the vocabulary. GPT-2 is an influential example of this decoder-only pattern [7].

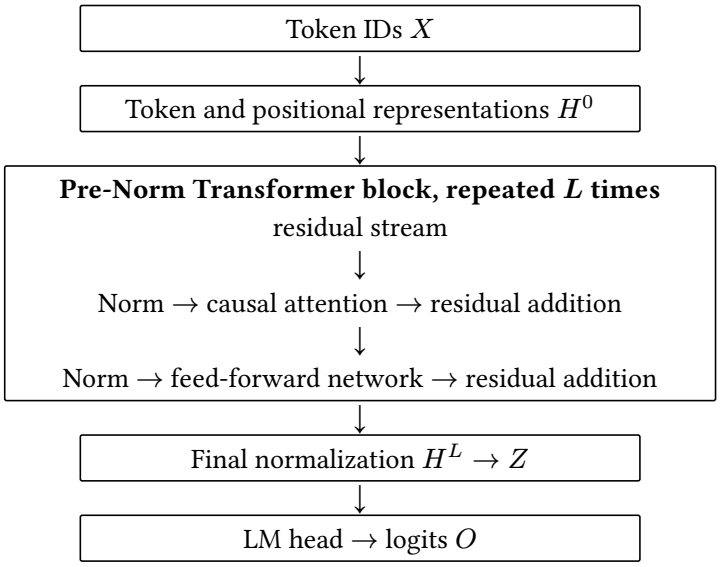

Figure 1: A high-level decoder-only Transformer computation path. Each block reads the current residual stream and writes an update to it.

The diagram in Figure 1 omits internal parameter matrices on purpose. It exposes the architectural invariant that matters at this stage: every block receives a hidden-state tensor and returns a tensor of the same shape. Depth is therefore a controlled composition of representation updates rather than a sequence of incompatible interfaces.

2.3 Transformer Block

A standard decoder-only block has two learned sublayers: causal Self-Attention and a position-wise feed-forward network (FFN). Attention allows the representation at one position to use an allowed prefix of the sequence; the FFN applies the same nonlinear transformation independently at every position. Residual connections preserve a direct route around both sublayers, while normalization controls the scale and distribution of the vectors passed into them.

For a Pre-Norm block, write A_ℓ for the attention transformation, M_ℓ for the FFN transformation, and N_ℓ^a and N_ℓ^m for their respective normalizations. The two updates are

$$U^\ell = H^{\ell-1} + A_{\ell(N_\ell^a(H^{\ell-1}))}, \tag{12}$$

followed by

$$H^\ell = U^\ell + M_{\ell(N_\ell^m(U^\ell))}. \tag{13}$$

These equations describe control flow, not an attempt to hide the details in notation. A_ℓ contains the query, key, value, mask, softmax, and output operations. M_ℓ contains the expansion, activation or gate, and contraction. Their outputs have the same final feature dimension d as their inputs, so they can be added to the residual stream.

The order of the two sublayers is conventional rather than mathematically necessary. The important point is that attention and the FFN have complementary roles. Attention mixes **positions**: a token representation can incorporate information from its prefix. The FFN mixes **features** within one position: it turns the resulting feature vector into a richer nonlinear update. A model needs both forms of computation to make context-sensitive predictions without treating every feature interaction as an attention operation.

2.4 Residual Stream

The phrase **residual stream** is a useful abstraction for H^ℓ. At any depth, it is the common sequence representation carried through the network. Attention does not replace that representation; it reads a

normalized view of it and adds a context-dependent update. The FFN then reads the updated stream and adds a position-wise update. The paired updates in Equation 12 and Equation 13 therefore describe an additive computation in which information can persist across many layers while new features are written into the same width-d state.

This view has two benefits. First, it makes tensor shapes easy to audit: the residual stream remains $B \times T \times d$ from input to final normalization. Second, it discourages a misleading picture in which a Transformer repeatedly discards and reconstructs a sequence. Each block instead makes bounded modifications to a shared representation. The identity terms in the residual connections provide direct forward and backward routes, while the learned sublayers specialize in producing useful corrections.

2.5 Pre-Norm and Post-Norm

The original Transformer uses a Post-Norm organization, in which normalization follows each residual addition. For a generic sublayer S, that pattern is

$$y = N(x + S(x)). \tag{14}$$

By contrast, Pre-Norm applies normalization to the sublayer input and leaves the residual path itself unnormalized:

$$y = x + S(N(x)). \tag{15}$$

Neither expression is merely a rearrangement of the other. In Post-Norm, the next sublayer receives a normalized mixture of the previous stream and the current update. In Pre-Norm, the identity path from x to y remains explicit, while the learned update is driven by a normalized argument. Consequently, normalization placement changes both forward activation statistics and the paths followed by gradients.

This distinction became practically consequential as Transformer stacks grew deeper. Xiong et al. analyze the initialization behavior of Pre-Norm and Post-Norm Transformers and relate Post-Norm's output-layer gradient scale to the usefulness of warmup [8]. Modern decoder-only architectures commonly adopt a Pre-Norm family of designs; GPT-2, for example, places LayerNorm at the input of each sub-block [7]. These observations do not make Pre-Norm universally superior. They identify a stability trade-off whose interaction with optimizer choice, residual scaling, precision, and depth must be evaluated as a system.

2.6 From Hidden States to Logits

After the final block, many decoder-only models apply a final normalization N_f:

$$Z = N_{f(H^L)}, \quad Z \in R^{B \times T \times d}. \tag{16}$$

The language-model head is an affine map from each final hidden vector to one score per vocabulary item. With output matrix $W_{\text{out}} \in R^{d \times |\mathcal{V}|}$ and bias $b_{\text{out}} \in R^{|\mathcal{V}|}$,

$$O = ZW_{\text{out}} + b_{\text{out}}, \quad O \in R^{B \times T \times |\mathcal{V}|}. \tag{17}$$

The element $O_{b,t,v}$ is a logit, not yet a probability: it is the unnormalized score assigned to vocabulary item v at position t. An autoregressive objective applies a masked cross-entropy loss to these scores. At inference time, a decoding rule converts the final position's logits into a distribution or selected next token.

When input and output embeddings are tied, the model reuses the token-embedding table and takes $W_{\text{out}} = E^T$, up to the orientation convention of the implementation. Weight tying reduces parameter count and connects the geometry used to read a token with the geometry used to score it. It is an architectural choice, not an identity required by decoder-only modeling. The original Transformer discusses shared embedding and pre-softmax weights in some configurations [6].

2.7 Tensor Shapes Through the Model

The following table records the shape contracts most useful when implementing or debugging the high-level forward pass. Let h be the number of query heads, d_h the per-head dimension, and d_{ff} the FFN intermediate width; ordinarily $d = hd_h$.

Quantity	Shape	Role
Token IDs X	$B \times T$	Discrete input indices.
Embeddings / residual stream H^ℓ	$B \times T \times d$	Sequence representation at depth ℓ.
Queries, keys, values	$B \times T \times h \times d_h$	Head-partitioned attention inputs.
Attention output	$B \times T \times d$	Contextual update projected back to model width.
FFN intermediate	$B \times T \times d_{\text{ff}}$	Expanded position-wise representation.
Logits O	$B \times T \times \lvert\mathcal{V}\rvert$	Vocabulary scores for the language-model objective.

Table 1: Symbolic tensor shapes in a decoder-only Transformer. The exact internal layout of heads may vary by implementation, but the residual-stream and logit contracts are stable.

The table distinguishes model-width tensors from vocabulary-width tensors. Most block activations have width d, whereas logits have width $|\mathcal{V}|$. For a large vocabulary, materializing logits for every position can itself be substantial. Conversely, the residual stream persists at every block and is a major source of activation memory during training. Shape reasoning is therefore not bookkeeping after the fact; it is the first step toward a reliable memory and compute model.

2.8 Computational Structure

At a high level, each block contains four dense projection families associated with attention: query, key, value, and output. In a conventional full-width implementation, they contribute on the order of $4d^2$ parameters per layer. The FFN contributes roughly $2dd_{\text{ff}}$ parameters for a two-projection design, or a different constant factor for gated variants such as SwiGLU. These counts are only first-order approximations, but they already explain why changing d, L, or d_{ff} changes the parameter budget materially.

The principal sequence-length dependence arises in attention. For a length-T sequence, each position can compare against a prefix whose total pairwise work grows on the order of T^2d for dense Self-Attention. The FFN instead performs work on each position independently, scaling on the order of Tdd_{ff}. The relative bottleneck therefore changes with sequence length, width, batch size, hardware, and kernel implementation. Cache-aware decoding and other inference methods alter the execution regime without changing the basic architectural contract.

Activation memory has a different emphasis from parameter count. During training, backward computation requires access to selected forward intermediates or to recomputed equivalents. The residual stream alone stores BTd scalars per layer before accounting for attention and FFN internals. During autoregressive decoding, the model instead retains key and value representations for prior positions in a KV Cache. The forward equations are the same in spirit, but the dominant persistent state changes with the execution regime.

2.9 Summary

A decoder-only Transformer maps token IDs to embeddings, repeatedly applies residual attention and FFN updates, normalizes the final stream, and projects each hidden state to vocabulary logits. The stack in Equation 11 supplies the architectural spine; the Pre-Norm updates in Equation 12 and Equation 13 make the residual stream explicit; and the LM head in Equation 17 closes the path back to the discrete vocabulary.

The high-level view deliberately treats causal attention, positional encoding, normalization, and optimization as modules. A subsequent independent chapter can derive attention masks, head variants, and positional encodings; another can study FFNs, normalization, residual scaling, and gradient flow. The computation graph developed here supplies the common tensor contracts that those analyses require.

References

[6] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[8] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.

Chapter 3: Attention and Position Encoding

Abstract

Attention is the mechanism by which a decoder-only Transformer turns a sequence of hidden states into context-dependent representations. This chapter derives Scaled Dot-Product Attention with explicit tensor shapes, explains why causal masking must precede Softmax, and relates Multi-Head Attention, Multi-Query Attention, and Grouped-Query Attention to KV Cache cost. It then develops Rotary Position Embedding as a position-dependent rotation of queries and keys whose attention score depends on relative displacement.

3.1 Introduction

The previous chapter treated causal Self-Attention as one sublayer in a decoder-only Transformer block. That abstraction is sufficient for the high-level computation graph, but it conceals the operation that gives the model access to its prefix. At every position, attention constructs a query-dependent weighted average of value vectors drawn from eligible positions. The weights are not fixed convolution coefficients or recurrent gates. They are recomputed from the current hidden states, so the same layer can retrieve a nearby syntactic cue in one context and a distant factual cue in another.

This chapter uses a batch-major hidden-state tensor $H \in R^{B\times T\times d}$, where B is batch size, T is sequence length, and d is model width. Let h denote the number of query heads, let d_h be the head dimension, and assume $d = hd_h$ for the conventional full-width case. In Grouped-Query Attention, g will denote the number of key-value heads, with g dividing h. The symbols i and j index query and key positions respectively, and each ranges from 1 to T unless stated otherwise.

3.2 Scaled Dot-Product Attention

3.2.1 Query, Key, and Value Projections

Attention begins by giving each hidden state three learned roles. A query represents the information sought by a position; a key represents the kind of information a position offers; and a value is the representation that will be mixed into the output. With projection matrices W_Q, W_K, W_V, the usual batched projections are reshaped into head-major tensors,

$$Q = \mathrm{reshape}\big(HW_Q\big) \in R^{B\times h\times T\times d_h}, \tag{18}$$

$$K = \mathrm{reshape}(HW_K) \in R^{B\times g\times T\times d_h}, \quad V = \mathrm{reshape}(HW_V) \in R^{B\times g\times T\times d_h}. \tag{19}$$

For ordinary Multi-Head Attention (MHA), $g = h$, and all three projection matrices have d output features. The more general notation in Equation 19 anticipates key-value sharing: W_K and W_V have gd_h output features when $g < h$. The reshaping operation changes only the view of the projected features. It does not mix token positions; all position mixing occurs in the score and weighted-sum operations below.

3.2.2 Scores, Scaling, and Weighted Values

Let $\rho(r)$ identify the key-value head used by query head r. The unmasked score from query position i to key position j is

$$S_{b,r,i,j} = \frac{q_{b,r,i}^{T} k_{b,\rho(r),j}}{\sqrt{d_h}}. \tag{20}$$

Thus S has shape $B \times h \times T \times T$. The division by $\sqrt{d_h}$ is the scale used in the original Transformer formulation [9]. If the components of queries and keys have comparable variance, their inner product typically grows in scale with d_h; the normalization keeps the logits entering Softmax in a range that does not become systematically sharper solely because the head width changed.

After a mask has been added, Softmax is applied along the key-position axis:

$$A_{b,r,i,j} = \mathrm{softmax}_{j(S_{b,r,i,j}+M_{i,j})}, \tag{21}$$

where M is discussed in Section 3.3. Each row $A_{b,r,i,:}$ sums to one over its legal keys. The head output is then

$$O_{b,r,i} = \sum_{j=1}^{T} A_{b,r,i,j} v_{b,\rho(r),j} \in R^{d_h}. \tag{22}$$

The operation in Equation 22 is a content-addressed aggregation: the query selects a distribution over positions through query-key compatibility, and the selected values supply the output features. It is important that values are not themselves normalized across positions. The weights carry the positional competition; values carry the information to aggregate.

3.3 Causal Masking

A decoder-only language model must not use a future token to predict an earlier one. During training, however, it is desirable to compute all positions in parallel. A causal mask reconciles these requirements by prohibiting the score entries above the diagonal:

$$M_{i,j} = \begin{cases} 0 & j \leq i \\ -\infty & j > i. \end{cases} \tag{23}$$

Combined with Equation 21, this mask gives zero probability to every future position. The diagonal is permitted, so every query has at least its own position as a legal key. Padding or packed-sequence boundaries require additional mask terms, but they follow the same rule: inadmissible entries receive zero probability by being excluded before normalization.

The ordering is operationally significant. A correct implementation adds the mask to logits before the Softmax, often through a fused attention kernel. Setting future probabilities to zero after Softmax is not equivalent unless the remaining probabilities are renormalized. In reduced precision, implementations commonly use a representable large negative mask value or a kernel-level causal flag rather than materializing an arithmetic negative infinity. The numerical contract is nevertheless the same: masked logits contribute zero probability, and rows with no legal keys must never be normalized.

3.4 Multi-Head Attention

One attention head computes one compatibility function and one weighted mixture. MHA repeats this computation over h learned subspaces. For the conventional case with $g = h$, the head outputs are concatenated and mixed back into model width:

$$Y = \text{concat}(O_1, ..., O_h) W_O, \quad Y \in R^{B \times T \times d}, \tag{24}$$

where $W_O \in R^{d \times d}$ is the output projection. Different heads can specialize in different compatibility patterns because they use different query, key, and value projections. The heads are not independent layers: their outputs are concatenated, linearly mixed by W_O, and then added to the residual stream by the enclosing Transformer block. MHA was introduced as part of the Transformer architecture to let the model attend jointly to information from different representation subspaces [9].

The phrase “multiple heads” should not be interpreted as a guarantee of interpretable or disjoint functions. It is an architectural factorization of the attention width. Its value is that several query-key geometries can coexist at the same depth while keeping each score computation at width d_h.

3.5 Key-Value Head Sharing: MQA and GQA

During autoregressive decoding, every layer retains the keys and values of prior tokens in a KV Cache. MHA stores a distinct key and value vector for every query head. This design is expressive, but repeatedly reading that cache can become memory-bandwidth intensive as the prefix grows. Multi-Query Attention (MQA) keeps h query heads but uses a single shared key head and a single shared value head, reducing the cached key-value tensors substantially [10].

Grouped-Query Attention (GQA) interpolates between the two choices. Partition the h query heads into g contiguous groups of equal size and set

$$\rho(r) = 1 + \text{floor}\left(\frac{r-1}{\frac{h}{g}} \right). \tag{25}$$

Each group uses one key-value head. MHA is recovered by $g = h$, MQA by $g = 1$, and intermediate $1 < g < h$ values define GQA. Ainslie et al. introduce this organization and report that it can approach MHA quality while retaining much of MQA's decoding advantage [11]. The appropriate group count is therefore a model-and-serving trade-off, not a change to the causal attention objective.

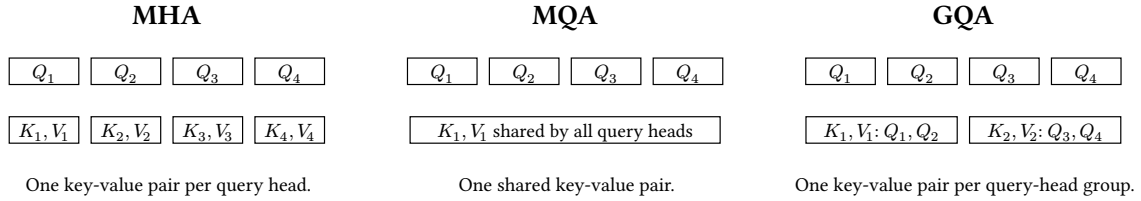

Figure 2: Key-value organization for four query heads. MHA uses four key-value heads, MQA one, and this GQA example two.

3.6 Positional Information and Rotary Position Embedding

The score in Equation 20 is permutation equivariant when the hidden states carry no positional signal: permuting the sequence permutes the projected queries, keys, values, and outputs in the same way. Token identity alone therefore cannot tell attention whether one occurrence precedes another. A Transformer needs a positional convention either in its input states or directly in its attention computation.

Rotary Position Embedding (RoPE) takes the second route. It rotates paired coordinates of each query and key by an angle determined by position before their inner product is formed. Let d_h be even and index coordinate pairs by $m \in \left\{0, \ldots, \frac{d_h}{2} - 1\right\}$. For a frequency θ_m, define the two-dimensional rotation

$$R_{t,m} = \begin{pmatrix} \cos(t\theta_m) & -\sin(t\theta_m) \\ \sin(t\theta_m) & \cos(t\theta_m) \end{pmatrix}. \tag{26}$$

The block-diagonal matrix R_t contains one such rotation for each coordinate pair. Standard RoPE choices use a geometric schedule such as $\theta_m = \theta_0^{-2\frac{m}{d_h}}$. If query q_t and key k_s occupy positions t and s, the score uses $\hat{q}_t = R_t q_t$ and $\hat{k}_s = R_s k_s$. The values are not rotated for the purpose of the attention score.

3.6.1 Relative Displacement in the Attention Score

The useful algebraic property follows from orthogonality and composition of planar rotations:

$$\hat{q}_t^T \hat{k}_s = q_t^T R_t^T R_s k_s = q_t^T R_{s-t} k_s. \tag{27}$$

Although the rotation assigned to each vector uses an absolute position, the resulting query-key interaction depends on the relative displacement $s - t$. RoPE therefore gives the attention score an explicit relative-position structure without adding a separate position vector to the residual stream. Su et al. introduce this construction and analyze its relative-position form [12]. In practice, implementations apply the pairwise rotation after the Q and K projections and before attention-score computation; the rotation is parameter-free once the frequency schedule is fixed.

RoPE does not by itself determine a model's usable context length or extrapolation behavior. Those properties also depend on training positions, frequency choices, scaling variants, and the rest of the architecture. Its immediate role is narrower and fundamental: it makes otherwise order-agnostic dot products sensitive to position and displacement.

3.7 Tensor Shapes, Compute, and KV Cache

3.7.1 Shape and Cache Accounting

The general g-head notation makes the principal storage effect of MQA and GQA explicit. The table records the per-layer cache in scalars for a prefix of length T. A complete model multiplies this value by its number of layers and by the byte width of its cache dtype.

Configuration	KV heads	Per-layer K or V shape	Per-layer cache scalars
MHA	$g = h$	$B \times h \times T \times d_h$	$2BThd_h = 2BTd$
GQA	$1 < g < h$	$B \times g \times T \times d_h$	$2BTgd_h$
MQA	$g = 1$	$B \times T \times d_h$	$2BTd_h$

Table 2: Key-value tensor and cache size per Transformer layer. The scalar count includes both keys and values but excludes temporary attention workspaces.

The score and probability tensors retain the query-head dimension in all three configurations: $S, A \in R^{B\times h\times T\times T}$. Key-value sharing changes the source tensors that those scores read, not the number of query rows. It therefore targets cache capacity and memory traffic more directly than the quadratic score shape of dense attention.

3.7.2 Compute in Training and Decoding

For a full training sequence, forming score products and weighted value sums costs on the order of $BhT^2d_h = BT^2d$ arithmetic operations, apart from constant factors and the Q, K, V, and output projections. A naive implementation also materializes a score or probability tensor with BhT^2 elements. Memory-efficient attention kernels can avoid retaining that full matrix, but they do not alter the underlying all-pairs dependency pattern.

Autoregressive decoding has a different shape. To generate one new token after a prefix of length T, each query head compares one new query with T cached keys and then combines T cached values. The attention portion is linear in prefix length for that decoding step, while the persistent cache follows Table 2. MQA and GQA reduce the number of distinct cached key-value vectors that must be read, which is why their benefit is especially relevant in bandwidth-constrained incremental serving. Projection cost, batching, kernel design, and model width still affect end-to-end latency; cache size alone is not a complete performance model.

3.8 Summary

Scaled Dot-Product Attention projects a residual stream into queries, keys, and values, turns scaled query-key inner products into a distribution over legal positions, and uses that distribution to aggregate values. Causal masking makes the parallel training computation compatible with autoregressive prediction. MHA supplies multiple learned query-key geometries; MQA and GQA preserve multiple query heads while reducing key-value cache storage and traffic. RoPE inserts position through rotations of queries and keys, giving the resulting inner product a relative-displacement form.

These mechanisms define the main sequence-mixing operator in a modern decoder-only Transformer. The following architectural topics can now treat attention as a concrete tensor program rather than a black box: normalization and residual design control the scale of its inputs, inference systems manage its cache, and distributed systems partition its projections and communication. Chapter 30 revisits MHA, MQA, and GQA as KV-state designs and separates those architectural choices from FlashAttention-style kernel execution.

References

[9] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[10] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: https://arxiv.org/abs/1911.02150

[11] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.

[12] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv preprint arXiv:2104.09864*, 2021, [Online]. Available: https://arxiv.org/abs/2104.09864

Chapter 4: Feed-Forward Networks, Normalization, and Residual Connections

Abstract

Attention moves information across token positions, but it is not the only source of computation in a Transformer block. This chapter develops the position-wise Feed-Forward Network as a learned expansion, nonlinear transformation, and contraction; derives gated variants and SwiGLU; and compares LayerNorm with RMSNorm at the level of their statistics and axes. It then treats residual connections, normalization placement, initialization, and gradient flow as one coupled design problem, concluding with parameter, compute, activation, and numerical implementation contracts relevant to modern decoder-only models.

4.1 Introduction

A decoder-only Transformer block alternates two qualitatively different operations. Self-Attention mixes information across positions, whereas the Feed-Forward Network (FFN) transforms the features of each position independently. Normalization determines the scale at which each sublayer reads the residual stream, and the residual connection determines how the resulting update is accumulated. These components are sometimes presented as architectural details surrounding attention. In a deep language model they are instead part of the mechanism that makes repeated sequence mixing expressive and trainable.

Let $H \in R^{B\times T\times d}$ denote a residual-stream tensor with batch size B, sequence length T, and model width d. The FFN intermediate width is denoted by m. Every operation considered here preserves the outer shape $B \times T$; normalization and the FFN act independently on the final feature axis, while residual addition requires the sublayer output to return to width d.

4.2 Position-Wise Feed-Forward Networks

4.2.1 Expansion, Nonlinearity, and Contraction

The original Transformer applies the same two-layer network to every sequence position [13]. For a single feature vector $x \in R^d$, a conventional FFN is

$$\mathrm{FFN}(x) = \varphi\big(xW_{\mathrm{up}} + b_{\mathrm{up}}\big)W_{\mathrm{down}} + b_{\mathrm{down}}, \tag{28}$$

with $W_{\mathrm{up}} \in R^{d\times m}$, $b_{\mathrm{up}} \in R^m$, $W_{\mathrm{down}} \in R^{m\times d}$, and $b_{\mathrm{down}} \in R^d$. Applied to H, the first affine map produces a tensor in $R^{B\times T\times m}$; the second returns it to $R^{B\times T\times d}$. The weights are shared across all positions and batch items. Consequently, Equation 28 mixes features but never token positions.

The width m is usually larger than d. Expansion gives the nonlinearity a higher-dimensional feature space in which to form an update; contraction makes that update compatible with the residual stream. Without the nonlinear function φ, the two affine maps would collapse into a single affine transformation, apart from their combined bias, and the expanded representation would add no expressive depth. Ignoring biases, Equation 28 contains $2dm$ parameters. For each token it also requires approximately $2dm$ scalar multiply-accumulates: dm for each projection. If one multiplication and one addition are counted as two floating-point operations, this is approximately $4dm$ FLOPs. Stating the counting convention matters because systems reports use both conventions.

4.2.2 ReLU, GELU, and SiLU

The original Transformer used the Rectified Linear Unit,

$$\mathrm{ReLU}(z) = \max(0, z), \tag{29}$$

applied elementwise. Later language models commonly use smoother functions. The Gaussian Error Linear Unit is

$$\mathrm{GELU}(z) = z\Phi(z), \tag{30}$$

where Φ is the standard normal cumulative distribution function [14]. The Sigmoid Linear Unit, usually implemented as SiLU or Swish, is

$$\mathrm{SiLU}(z) = z\sigma(z), \tag{31}$$

where $\sigma(z) = \frac{1}{1+\exp(-z)}$. ReLU removes every negative input exactly. GELU and SiLU instead attenuate negative inputs smoothly and retain a small nonzero response over part of the negative half-line. The choice changes optimization and representation behavior, but its effect cannot be reduced to smoothness alone; model scale, initialization, precision, and the surrounding gated architecture also matter.

4.3 Gated Feed-Forward Networks and SwiGLU

4.3.1 Multiplicative Gating

A gated FFN replaces the single transformed branch in Equation 28 with the elementwise product of two learned projections. In a bias-free form,

$$\mathrm{GFFN}(x) = \big(\varphi(xW_g) \odot (xW_u)\big)W_d, \tag{32}$$

where $W_g, W_u \in R^{d\times m}$, $W_d \in R^{m\times d}$, and $\odot$ denotes the Hadamard product. One branch determines a data-dependent gate and the other supplies candidate features. Because both branches depend on x, the product introduces a multiplicative interaction that is absent from a single activation applied after one projection.

SwiGLU chooses SiLU for the gated branch:

$$\mathrm{SwiGLU}(x) = \big(\mathrm{SiLU}(xW_g) \odot (xW_u)\big)W_d. \tag{33}$$

Shazeer compared several GLU variants in Transformer FFNs and introduced the SwiGLU naming and formulation [15]. Modern decoder-only models often combine SwiGLU with Pre-Norm and RMSNorm; LLaMA is a prominent documented example [16]. This convention is empirical rather than a mathematical requirement: the gate, normalization, residual organization, and intermediate width remain separable architectural choices.

4.3.2 Parameter-Matched Intermediate Width

The extra projection changes the appropriate comparison of widths. A conventional FFN of width m_v has approximately $2dm_v$ weights, whereas a gated FFN of width m_g has approximately $3dm_g$. Matching their leading parameter counts gives

$$2dm_v \approx 3dm_g \quad \Rightarrow \quad m_g \approx \frac{2}{3}m_v. \tag{34}$$

Thus a vanilla expansion $m_v = 4d$ corresponds to a gated width near $8\frac{d}{3}$ at the same leading parameter budget. Implementations usually round m_g to a hardware-friendly multiple. Comparing a $4d$ vanilla FFN directly with a $4d$ SwiGLU block silently gives the gated model roughly fifty percent more projection parameters and multiply-accumulates.

4.4 Normalization over the Feature Axis

Normalization in a Transformer is applied independently to each token vector. It does not aggregate statistics across the batch or across sequence positions. Let $x \in R^d$, let $\varepsilon > 0$ be a numerical stabilizer, and let $\gamma, \beta \in R^d$ be learned elementwise affine parameters where present.

4.4.1 LayerNorm

LayerNorm computes the feature mean and population variance

$$\mu(x) = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2(x) = \frac{1}{d}\sum_{i=1}^{d} (x_i - \mu(x))^2, \tag{35}$$

then returns

$$\mathrm{LayerNorm}\,(x)_i = \gamma_i \frac{x_i - \mu(x)}{\sqrt{\sigma^2(x) + \varepsilon}} + \beta_i. \tag{36}$$

The mean and variance are recomputed for each token at both training and inference time; they do not depend on batch-level running statistics [17]. LayerNorm is invariant to a uniform shift of all features before the learned affine transformation and is also invariant to positive rescaling, up to the effect of ε.

4.4.2 RMSNorm

RMSNorm removes mean subtraction and normalizes by the root mean square,

$$\mathrm{rms}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}, \tag{37}$$

$$\mathrm{RMSNorm}\,(x)_i = \gamma_i \frac{x_i}{\mathrm{rms}(x)}. \tag{38}$$

In its usual form there is no learned bias β. RMSNorm therefore uses the second raw moment rather than the variance: it provides rescaling invariance but not recentering invariance. Zhang and Sennrich proposed RMSNorm as a computationally simpler alternative to LayerNorm and found recentering dispensable in their studied settings [18]. That empirical result does not make the two transformations identical; a feature vector with a large nonzero mean is treated differently by Equation 36 and Equation 38.

Method	Statistic	Core transform	Typical affine parameters
LayerNorm	Mean and centered variance	Center, then divide by standard deviation	γ, β
RMSNorm	Root second moment	Divide by RMS without centering	γ

Table 3: LayerNorm and RMSNorm operate over the feature axis of each token vector. The table describes their standard forms; implementation variants may alter the affine parameters.

4.4.3 Numerical Implementation

Both normalizers reduce d values into one scale, so their implementation must balance bandwidth, precision, and kernel fusion. The stabilizer ε belongs inside the square root in Equation 36 and Equation 37. In mixed-precision training, it is common to accumulate the reduction in higher precision than the stored activations and then cast the normalized result as required by the surrounding kernel. A mathematically equivalent rearrangement is not automatically numerically equivalent: computing the variance as $E[x^2] - E[x]^2$, for example, can suffer cancellation when the mean is large.

The affine parameters are broadcast over the B and T axes. A frequent implementation error is to normalize over more than the last feature axis, which couples tokens or batch items and changes the model. Another is to insert a second, unintended ε or to use different values across training and inference. These choices are part of the checkpoint's executable specification.

4.5 Residual Connections and Block Ordering

4.5.1 The Residual Stream

Let F_l denote the learned transformation of sublayer l, including either attention or an FFN. A residual update has the shape contract

$$H^l = H^{l-1} + \Delta H^l, \quad H^{l-1}, \Delta H^l \in R^{B\times T\times d}. \tag{39}$$

The tensor H is called the residual stream because every sublayer reads from and writes an update to a common width-d representation. The additive path can preserve existing information while the learned branch contributes a correction. It also imposes a strict interface: an FFN may expand internally to m, but its contraction must return to d before residual addition.

4.5.2 Post-Norm and Pre-Norm

The original Transformer used Post-Norm sublayers [13],

$$y = N(x + F(x)), \tag{40}$$

where normalization follows residual addition. A Pre-Norm sublayer instead computes

$$y = x + F(N(x)). \tag{41}$$

The two forms are not interchangeable. Pre-Norm presents a normalized input to F while leaving an exact identity path from x to y. Post-Norm normalizes the combined stream, so the identity branch

also passes through the Jacobian of N. GPT-2 documented moving normalization to the input of each sub-block and adding a final normalization after the stack [19]; Pre-Norm families have since become common in decoder-only models.

For a Pre-Norm stack, repeated substitution exposes the additive structure

$$H^L = H^0 + \sum_{l=1}^{L} F_{l\left(N_{l(H^{l-1})}\right)}. \tag{42}$$

This equation does not imply that layers are independent: each update depends on the accumulated stream. It does show that a forward identity route persists through arbitrary depth, and the backward Jacobian of one sublayer has the form

$$\frac{\partial y}{\partial x} = I + J_F J_N. \tag{43}$$

For Post-Norm, the corresponding local Jacobian is $J_{N(I+J_F)}$. The placement of J_N changes how gradients compose across depth, which is one reason normalization location affects optimization rather than merely activation scale.

4.6 Initialization, Gradient Flow, and Stability

4.6.1 Variance-Preserving Scale

Initialization should keep activations and gradients in a useful numerical range before learning has organized the network. Consider a bias-free projection $y = xW$ with $W \in R^{d_{\text{in}} \times d_{\text{out}}}$. If the components of x and W are independent, zero mean, and have variances q and s^2, then

$$\operatorname{Var}\left(y_j\right) \approx d_{\text{in}} s^2 q. \tag{44}$$

Choosing s^2 on the order of $\frac{1}{d_{\text{in}}}$ therefore preserves variance to first order. Activations and gates change the constant, and correlations accumulated during training invalidate the independence assumptions, so Equation 44 is an initialization heuristic rather than a guarantee. It is nevertheless a useful audit: a projection whose standard deviation ignores fan-in can make scale grow or shrink immediately with width.

4.6.2 Residual Accumulation with Depth

Residual addition creates a second scale problem. Under the simplifying assumption that the residual branch updates are zero mean and uncorrelated with the incoming stream,

$$\operatorname{Var}(H^L) \approx \operatorname{Var}(H^0) + \sum_{l=1}^{L} \operatorname{Var}\left(\Delta H^l\right). \tag{45}$$

If every branch contributes the same fixed variance, the residual-stream variance grows on the order of L. Practical architectures counter this tendency through normalization, fan-aware initialization, and sometimes depth-dependent scaling of residual projections. GPT-2, for example, reports scaling residual-layer weights at initialization by $\frac{1}{\sqrt{N}}$ for N residual layers to account for accumulation with depth [19]. The exact factor depends on what is counted as a residual branch and where it is applied; copying a constant without matching the architecture defeats the purpose of the derivation.

4.6.3 Interpreting Gradient-Flow Claims

The identity term in Equation 43 supplies a direct gradient contribution, but it does not make optimization automatically stable. The learned Jacobian can still produce large updates, normalization can amplify small-variance inputs, and finite-precision arithmetic, optimizer state, learning rate, and data distribution all interact with the architecture. Conversely, Post-Norm can train successfully when initialization and learning-rate warmup are chosen appropriately.

Xiong et al. analyze gradients at initialization and connect Post-Norm's placement to large output-layer gradients and the usefulness of warmup, while finding better-behaved initial gradients for their Pre-Norm setting [20]. The correct conclusion is conditional: normalization placement changes the gradient paths and therefore the optimization regime. It is not a theorem that every Pre-Norm model is stable or that every Post-Norm model is unstable.

4.7 Parameter, Compute, and Memory Accounting

The leading costs of the FFN are summarized below. Biases, normalization parameters, elementwise activations, and residual additions are lower order in parameter count relative to the dense projections. The activation-memory column describes logical intermediates; fused kernels, recomputation, and checkpointing can reduce what must be retained.

FFN form	Leading parameters	MACs per token	Width-m intermediates
Vanilla	$2dm$	$2dm$	One projected tensor before contraction
Gated / SwiGLU	$3dm$	$3dm$	Two projections before elementwise gating

Table 4: Leading projection costs for bias-free FFNs. One multiply-accumulate is counted as one MAC; a FLOP convention that counts multiplication and addition separately doubles these values.

For a batch of sequences, multiply the per-token arithmetic by BT. A vanilla FFN therefore performs on the order of $2BTdm$ MACs, while a gated FFN performs $3BTdm$. Normalization and residual addition require only $O(BTd)$ arithmetic, but they read and write full residual-stream tensors and can be limited by memory bandwidth. Operator fusion is valuable because it avoids materializing every normalized vector, bias result, activation, gate, and residual update as a separate memory round trip.

The FFN often owns more parameters than attention within a dense Transformer layer. With $m = 4d$, a vanilla FFN has approximately $8d^2$ weights, compared with roughly $4d^2$ for full-width query, key, value, and output projections in conventional MHA. This comparison concerns projection weights, not total runtime: at long sequence lengths, attention's pairwise position cost can dominate even when its parameter count is smaller.

4.8 Implementation Contracts

A reliable implementation should make shapes and axes explicit at module boundaries. Given input $B \times T \times d$, an FFN must return exactly $B \times T \times d$; normalization must reduce only the feature axis; and residual addition must not rely on accidental broadcasting. For gated FFNs, checkpoint conversion must preserve the identity and ordering of the gate and value projections. Swapping them changes Equation 33 because SiLU is applied to only one branch.

Numerical checks should include finite outputs for nearly constant vectors, agreement between training and inference normalization, and a reference comparison in higher precision. Initialization tests can measure empirical output variance for random inputs and verify that every intended residual projection received its depth scaling. These checks do not replace end-to-end training, but they catch silent specification errors before optimizer behavior is blamed for an incorrect tensor program.

4.9 Summary

The position-wise FFN expands each token representation, applies nonlinear or gated feature interactions, and contracts the result back to the residual width. SwiGLU adds a learned multiplicative gate and therefore requires a narrower intermediate dimension when compared at fixed parameter count. LayerNorm centers and rescales each token vector, whereas RMSNorm rescales by its second raw moment without centering. Residual connections accumulate sublayer updates into a shared stream, and the placement of normalization determines whether the identity path itself passes through a normalizer.

These choices jointly determine more than a block diagram. They set the leading FFN parameter and compute budget, the shape of saved activations, the numerical behavior of reductions, and the routes by which gradients traverse depth. Initialization and residual scaling should therefore be derived from the actual block organization rather than selected as isolated defaults.

References

[13] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[14] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016, [Online]. Available: https://arxiv.org/abs/1606.08415

[15] N. Shazeer, "GLU Variants Improve Transformer," *arXiv preprint arXiv:2002.05202*, 2020, [Online]. Available: https://arxiv.org/abs/2002.05202

[16] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023, [Online]. Available: https://arxiv.org/abs/2302.13971

[17] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016, [Online]. Available: https://arxiv.org/abs/1607.06450

[18] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019.

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[20] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.

Part III — Pretraining

Chapter 5: Pretraining Objective and Language Modeling

Abstract

This chapter formalizes the Causal Language Modeling pretraining objective from the vocabulary logits produced by a decoder-only Transformer. It derives autoregressive sequence factorization, next-token prediction, and the relationship between token-level negative log-likelihood and cross-entropy. It distinguishes vocabulary size, sequence length, logits, probability distributions, targets, and loss aggregation, then explains why teacher forcing and causal masking allow eligible token losses to be evaluated in parallel even though generation proceeds one token at a time.

5.1 Introduction

The first four chapters established how a token sequence enters a decoder-only Transformer, how causal Self-Attention and Feed-Forward Networks update its residual stream, and how final hidden states produce vocabulary logits. Pretraining assigns a statistical task to that computation graph: to learn a probability distribution over token sequences. For a tokenized sequence $x = (x_1, ..., x_T)$, the model does not choose an entire text from an unstructured space at once. Instead, at each position it assigns probabilities to possible next tokens conditioned on the prefix that has already appeared.

This objective is central to modern autoregressive language models. It turns unlabeled text into a supervised signal: the same sequence supplies both context and target. Neural probabilistic language modeling seeks to model the joint probability of token sequences [21]; GPT-style models scale this principle into the pretraining objective of large decoder-only Transformers [22].

This chapter retains the token notation of Chapter 1. Let $\mathcal{V}$ denote the vocabulary and let $V = |\mathcal{V}|$ be its size. Let T denote the token length of a particular sequence; it is not a property of the vocabulary. For a batch, let B be the number of sequences. The language-model head introduced in Chapter 2 produces a logit vector of length V for every batch item and position. The remaining question is how these vectors become conditional probabilities and a training loss.

5.2 Autoregressive Language Modeling

Autoregressive Language Modeling represents a sequence from left to right. At prediction position t, the model may use only the token prefix strictly to its left, denoted by $x_{\{<t\}} = \left(x_1, ..., x_{\{t-1\}}\right)$. With parameters θ, its conditional distribution over any vocabulary token v is written as

$$p_{\theta\left(v \mid x_{\{<t\}}\right)}, \quad v \in \mathcal{V}. \tag{46}$$

This quantity is not a single score for a known target; it is a complete distribution over the vocabulary. It is nonnegative and normalized: $p_{\theta\left(v \mid x_{\{<t\}}\right)} \geq 0$ and $\sum_{v \in \mathcal{V}} p_{\theta\left(v \mid x_{\{<t\}}\right)} = 1$. During generation, this distribution provides candidates for the next token. During pretraining, the corpus supplies the token that actually follows the prefix, so the model can evaluate the probability it assigned to that target.

For variable-length text, sequence termination must also be represented by the probability model. Implementations usually include an end-of-sequence token and train the model to predict it at an appropriate position. To avoid adding boundary notation to each derivation, the sequence $x_1, ..., x_T$ below is understood to include the required boundary convention.

5.3 Sequence Probability Factorization

The probability chain rule factorizes any joint distribution. An autoregressive model uses parameterized conditional distributions to instantiate that factorization:

$$p_{\theta(x_1,...,x_T)} = \prod_{t=1}^{T} p_{\theta\left(x_t \mid x_{\{<t\}}\right)}. \tag{47}$$

The factorization does not declare tokens independent. On the contrary, the term at position t may depend on the full prefix. It specifies the direction of dependence: the model may summarize prior tokens in its hidden state, but it may not read x_t itself or any future token when predicting x_t.

The causal Attention Mask described in Chapter 3 is the Transformer mechanism that enforces this information constraint.

The product form expresses a probability, but it is inconvenient for numerical optimization. Taking a logarithm turns the sequence log-probability into a sum of conditional log-probabilities:

$$\log p_{\theta(x_1,\ldots,x_T)} = \sum_{t=1}^{T} \log p_{\theta\left(x_t \mid x_{\{<t\}}\right)}. \tag{48}$$

Consequently, maximizing the likelihood of whole sequences is equivalent to maximizing the conditional log-probability of every observed next token. This equivalence is why a local next-token training signal defines a sequence-level probability model.

5.4 Next-Token Prediction

In training code, next-token prediction is usually implemented by a shift. If a beginning-of-sequence token x_0 is introduced, the model input is $\left(x_0, x_1, \ldots, x_{\{T-1\}}\right)$, and the output at position $t-1$ predicts the target x_t. Implementations without an explicit x_0 commonly shift the inputs and labels of a packed sequence by one position: the logits at position t align with the token at position $t+1$. Both conventions have the same semantics: the prediction of target x_t is conditioned only on $x_{\{<t\}}$.

The target token is an integer ID, not a one-hot vector and not a token generated by the model. For batch item b and supervised position t, write the target as $y_{b,t} \in \{0, \ldots, V-1\}$. Under the common right-shift convention, $y_{b,t}$ is the next token in the input sequence. An off-by-one error in this alignment changes the training objective itself; it is not a small discrepancy that an optimizer can correct.

5.5 Logits and Token Probabilities

Chapter 2 denotes the output of the language-model head by $O \in R^{B\times T\times V}$. The vector $O_{b,t,:}$ has length V and is called the logits at position (b,t). Its entries are unnormalized scores, not probabilities. For fixed (b,t), softmax defines

$$p_{\theta(v \mid x_{b,<t})} = \frac{\exp\left(O_{b,t,v}\right)}{\sum_{j=0}^{V-1} \exp\left(O_{b,t,j}\right)}. \tag{49}$$

Softmax maps an arbitrary real-valued vector to the vocabulary simplex. Adding a common constant to all logits leaves Equation 49 unchanged, so the absolute zero point of a logit has no probabilistic meaning; only relative scores among vocabulary items determine the distribution. The vocabulary size V determines the width of each logit vector, whereas T determines the number of prediction positions. They are distinct quantities.

Quantity	Symbol or shape	Meaning
Vocabulary size	$V = \lvert\mathcal{V}\rvert$	Number of possible token IDs and width of every logit vector.
Sequence length	T	Number of token positions in one sequence; it varies with the tokenized text.
Logits	$O_{b,t,:} \in R^V$	Unnormalized scores at one prediction position.
Probability distribution	$p_{\theta(\cdot \mid x_{b,<t})}$	Softmax-normalized categorical distribution over the vocabulary.
Target token	$y_{b,t}$	Observed integer token ID used to select one probability.
Token-level loss	$\ell_{b,t}$	Negative log-probability assigned to one observed target.
Sequence / batch loss	$\mathcal{L}_b$, $\mathcal{L}_{\text{batch}}$	Aggregations of token losses with an explicit reduction rule.

Table 5: Objects that are often conflated in language-model training. A target is one index; logits and probabilities are length-V vectors; losses reduce these objects to scalars.

5.6 Cross-Entropy and Negative Log-Likelihood

For one supervised position, define the empirical target distribution $q_{b,t}$ to place all of its probability mass on $y_{b,t}$. The cross-entropy from this one-hot target distribution to the model distribution is

$$\ell_{b,t} = -\sum_{v=0}^{V-1} q_{b,t}(v) \log p_{\theta(v \mid x_{b,<t})} = -\log p_{\theta(y_{b,t} \mid x_{b,<t})}. \tag{50}$$

The final expression is the token-level negative log-likelihood (NLL). When the target is one-hot, cross-entropy and NLL refer to the same numerical training term. Their conceptual emphases differ: cross-entropy compares two distributions, whereas NLL emphasizes the probability assigned by the model to an observed sample. Standard information theory relates cross-entropy to both optimal code length and likelihood [23].

Expanding the softmax gives an expression directly in terms of logits:

$$\ell_{b,t} = -O_{b,t,y_{b,t}} + \log \sum_{v=0}^{V-1} \exp\bigl(O_{b,t,v}\bigr). \tag{51}$$

The first term encourages a high logit for the correct token. The second, commonly called the log-sum-exp term, accounts for every competing vocabulary item. The derivative with respect to any logit is

$$\frac{\partial \ell_{b,t}}{\partial O_{b,t,v}} = p_{\theta(v \mid x_{b,<t})} - \delta_{v,y_{b,t}}, \tag{52}$$

where $\delta_{v,y}$ is one when $v = y$ and zero otherwise. Gradient descent therefore raises the target logit and lowers competing logits in proportion to their predicted probabilities. This local expression also shows that exact full-softmax loss requires the language-model head to produce scores for all V vocabulary items.

Numerical implementations should not first compute $\exp\bigl(O_{b,t,v}\bigr)$ directly. Let $a = \max_v O_{b,t,v}$. The algebraically equivalent identity

$$\log \sum_v \exp\bigl(O_{b,t,v}\bigr) = a + \log \sum_v \exp\bigl(O_{b,t,v} - a\bigr) \tag{53}$$

ensures that the arguments to the exponentials are no greater than zero. In practice, frameworks commonly fuse this stable log-sum-exp calculation with the target gather operation rather than materializing a separate probability tensor solely for the loss.

5.7 Teacher Forcing and Causal Masking

Teacher forcing means that the prefix supplied during training is drawn from the observed corpus rather than from the model's earlier samples. The term predates the Transformer [24], but the principle applies directly: when scoring x_t, the model receives $x_{\{<t\}}$ from the data and evaluates a token-level loss against the known target. Every eligible token position in a sequence can therefore contribute a learning signal.

Causal masking makes teacher forcing statistically valid for autoregressive modeling. In a decoder-only attention layer, position t may attend only to permitted earlier positions; future tokens are masked before the attention softmax. The mask prevents a representation from encoding the very answer it will be asked to predict. The mask does not itself define the loss: the loss is defined by target alignment and token-level cross-entropy. The original Transformer applies such a mask in decoder self-attention so that a prediction cannot depend on subsequent positions [25].

This causal dependency does not require a sequential training loop. Given a complete input tensor, projections, masked attention scores, FFNs, the language-model head, and token losses at all $B \times T$ positions can be computed with batched tensor operations. The mask imposes the same allowed-prefix rule on every row of this parallel computation. Generation is different because the next input token is not yet known: the model produces a distribution for the current prefix, selects or samples a token, appends it, and only then can compute the next distribution. Parallel training and sequential generation are thus two execution regimes of the same conditional model, not competing definitions of autoregression.

5.8 Loss Averaging Across Tokens and Batches

The NLL of a complete sequence is the sum of its token-level losses:

$$\mathcal{L}_b = -\log p_{\theta(x_{b,1},\ldots,x_{b,T})} = \sum_{t=1}^{T} \ell_{b,t}. \tag{54}$$

Training usually reports and differentiates a mean loss rather than a raw sum. Let $m_{b,t} \in \{0,1\}$ indicate whether position (b,t) has a valid supervised target, and let $N = \sum_{b=1}^{B}\sum_{t=1}^{T} m_{b,t}$. The token-averaged batch loss is then

$$\mathcal{L}_{\text{batch}} = \frac{1}{N}\sum_{b=1}^{B}\sum_{t=1}^{T} m_{b,t}\ell_{b,t}. \tag{55}$$

The loss mask m differs from the attention mask. A causal attention mask constrains information flow; a loss mask specifies which output positions contribute to the scalar objective. Padding positions, positions without a shifted target, and intentionally excluded labels should have $m_{b,t} = 0$. Dividing by N gives each valid token the same weight even when sequences have different effective lengths. By contrast, averaging within each sequence and then averaging sequences gives short and long sequences equal weight. Neither reduction is universally incorrect, but the choice changes the objective and should be recorded with the reported loss.

5.9 Perplexity

With natural logarithms and the token-averaged NLL $\mathcal{L}_{\text{batch}}$, perplexity is defined as

$$\text{PPL} = \exp(\mathcal{L}_{\text{batch}}). \tag{56}$$

For a single sequence without masked positions, it is the reciprocal geometric mean of the probabilities assigned to its observed tokens. A value near one means that the model assigns high probability to the evaluated token stream; a larger value denotes greater average uncertainty. If cross-entropy is measured in bits with base-two logarithms, the equivalent expression is 2^H.

Perplexity converts average log-loss into an interpretable effective branching factor, but it is not a tokenizer-independent measure of language understanding. As Chapter 1 explains, tokenization changes both the event space and the number of prediction positions. Token-level PPL values obtained under different vocabularies, normalization policies, or loss masks are not directly comparable. Nor does a lower PPL determine which decoding rule should be used during generation.

5.10 Training Objective versus Generation

During pretraining, likelihood evaluation asks a non-counterfactual question: given the true prefix from the corpus, how much probability does the model assign to the true next token? Although the complete sequence is available in memory, causal masking ensures that each prediction uses only its permitted prefix. The result is a scalar objective that aggregates many such conditional probabilities. During generation, only the prefix is available. The model produces $p_{\theta}\left(\cdot \mid x_{\{<t\}}\right)$, and a decoding rule selects or samples a token to extend the prefix. That generated token becomes part of the context for the next step. Maximum-likelihood training specifies how to fit the conditional distribution; it does not prescribe greedy selection, temperature scaling, or any other decoding policy. These inference choices belong to later chapters, but the distinction is essential: token-level training loss evaluates probabilities under data prefixes, whereas generation constructs trajectories under model-produced prefixes.

5.11 Implementation Contracts

A minimal Causal Language Modeling training step should make four aligned tensors explicit: input token IDs of shape $B \times T$, target IDs of shape $B \times T$, logits of shape $B \times T \times V$, and a loss mask of shape $B \times T$. At every supervised position, the target must be the token immediately following the input prefix represented by the corresponding logit. Wherever the loss mask is one, target IDs must be integer indices in $[0, V-1]$, not floating-point probabilities.

The loss implementation should consume raw logits, use a stable fused cross-entropy kernel, and reduce only over valid targets. Tests should check the shift explicitly on short hand-written sequences, verify that masked targets contribute zero, and compare a small batch against a direct probability calculation in high precision. A causality test should alter a future input token and confirm that logits at earlier positions do not change. These contracts turn the mathematical objective into a reproducible tensor program.

5.12 Summary

Causal Language Modeling assigns a probability to a token sequence by factorizing its joint probability into next-token conditionals. A decoder-only Transformer produces a vocabulary-sized logit vector at each position; softmax converts it into a conditional distribution, and the negative log-probability of the observed target gives the token-level cross-entropy. Summing these terms yields the sequence NLL, while a loss-masked token average yields the usual batch loss and its corresponding perplexity. Teacher forcing provides observed prefixes during training, and causal masking prevents those prefixes from containing future answers. Together, they allow position-wise losses across an entire batch to be evaluated in parallel. Autoregressive generation remains sequential because each newly selected token changes the prefix for the next step. The objective is compact, but tensor alignment, masking, reduction conventions, and numerical implementation jointly determine the pretraining problem actually being solved.

References

[21] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A Neural Probabilistic Language Model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003, [Online]. Available: https://jmlr.org/papers/v3/bengio03a.html

[22] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language-models.pdf

[23] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006. doi: 10.1002/047174882X.

[24] R. J. Williams and D. Zipser, "A Learning Algorithm for Continually Running Fully Recurrent Neural Networks," *Neural Computation*, vol. 1, no. 2, pp. 270–280, 1989, doi: 10.1162/neco.1989.1.2.270.

[25] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

Chapter 6: Pretraining Data

Abstract

Large-scale pretraining data is not a passive collection of text. It is the result of a sequence of collection, extraction, filtering, deduplication, mixture, tokenization, and sequence-construction decisions that together define the distribution seen by a language model. This chapter develops those decisions at document, sample, sequence, and token levels. It explains how exact and near deduplication differ, why benchmark leakage compromises evaluation, how mixture weights change the learned distribution, and why valid-token budgets are a more useful scale measure than raw document counts.

6.1 Introduction

Chapter 5 defined pretraining as maximum-likelihood learning over token sequences. That formulation is deliberately compact: the loss averages the negative log-probability of observed target tokens. In practice, however, the observed tokens are produced by a data pipeline. Decisions about which documents enter the corpus, how they are cleaned, which duplicates are removed, how sources are sampled, and how text is packed into fixed-length sequences determine the empirical distribution on which the loss is estimated.

Pretraining data is therefore part of the model specification. A decoder-only Transformer trained on the same architecture and objective can learn materially different capabilities, biases, and failure modes when its input distribution changes. Modern open-corpus efforts make this point concrete by documenting mixtures that include filtered web text, scientific material, code, public-domain books, social media, and encyclopedic sources [26]. The objective of this chapter is not to prescribe one universal corpus. It is to establish the distinctions and contracts required to reason about a real language-model data pipeline.

Four granularities must remain separate. A **document** is a source artifact such as a web page, book chapter, repository file, or paper. A **sample** is a retained record after source-specific extraction and filtering; it may be one document or a selected span. A **sequence** is a fixed-length model input constructed from one or more samples. A **token** is the discrete unit on which the loss in Chapter 5 is evaluated. A decision at one level often changes another, but conflating the levels obscures both data provenance and the actual training distribution.

Unit	Typical representation	Typical decisions	Primary invariant
Document	Captured source artifact and metadata	Extraction, language assignment, provenance	Source identity remains recoverable.
Sample	Retained text record or span	Quality filters, deduplication, source weight	Filtering decision and reason are recorded.
Sequence	Fixed-length token-ID tensor	Packing, truncation, boundary policy	Inputs, labels, and masks are aligned.
Token	Vocabulary index	Loss eligibility and accounting	Counted under a declared tokenizer and mask.

Table 6: The four levels of a pretraining pipeline. A document is not automatically a training sequence, and a document count is not a token budget.

6.2 Data Sources

Large pretraining corpora commonly combine broad web collections with sources that have more explicit editorial, technical, or domain structure. Web data contributes scale and topical breadth, but it contains boilerplate, duplication, spam, malformed pages, and uneven language coverage. Books, reference works, scientific papers, code, forums, and licensed or public-domain collections can provide valuable distributions that are sparse in a generic crawl. The Pile illustrates a mixture-oriented design:

it assembled 22 components, including academic and professional sources, rather than treating a single web corpus as sufficient [27].

A source category is not a quality label. A book may be scanned incorrectly; a code repository may contain generated files, secrets, or copied dependencies; a web page may contain unusually careful technical writing. Source selection should instead state a hypothesis about useful coverage, permitted use, and the processing that will be applied. A corpus inventory should retain source name, acquisition method, snapshot or release identifier, jurisdictional and license information when available, language coverage, and the intended sampling policy. These fields make later audits possible even when the raw source cannot be redistributed.

6.3 Data Collection and Crawling

Web-scale collection begins with a capture policy. A crawler or web archive records URLs, retrieval times, response metadata, and page payloads; a non-web source may instead provide a release manifest, file hashes, and version identifiers. The collection stage should preserve a stable document key independent of later cleaning. Otherwise, a pipeline cannot explain whether two retained samples came from the same raw artifact or whether a filter changed between reruns.

Collection is necessarily incomplete and selective. Crawl seeds, link-following rules, host restrictions, robots and access policies, retry behavior, time window, and archive availability all influence the captured distribution before any quality filter is applied. The Colossal Clean Crawled Corpus (C4), for example, begins from Common Crawl snapshots and applies a documented cleaning pipeline rather than claiming that a crawl is already a language-model corpus [28]. For every source, collection metadata and the raw-to-extracted mapping should be versioned alongside the model-facing data.

6.4 Text Extraction and Normalization

An HTML response is not a text document. Extraction removes markup, scripts, navigation menus, cookie notices, repeated templates, and other material that is useful for rendering a page but not necessarily for language modeling. The extraction rule must be evaluated on representative samples because aggressive boilerplate removal can erase tables, mathematical notation, code blocks, lists, or the main content of unusual page layouts. For structured sources, a parser should preserve distinctions that carry semantic meaning, such as paragraph boundaries, code indentation, document title, and section order, whenever the downstream representation can support them.

Normalization makes equivalent surface forms more consistent, but it is not a license to rewrite content. Common operations include Unicode canonicalization, removal of invalid control characters, repair of ill-formed byte sequences, and a declared treatment of whitespace. Replacing every newline with a space can destroy code and table structure; collapsing visually similar Unicode characters can alter identifiers; stripping all markup before extraction can concatenate unrelated navigation text. The appropriate output is a clean text record with a documented normalization version, not a vaguely defined string called “clean text.”

6.5 Language Identification

Language identification assigns a language label, a probability, or both to an extracted unit. It can operate at document, paragraph, or sentence level, and the choice matters. Document-level identification is efficient and supports monolingual corpus construction, but it may discard legitimate multilingual documents or misclassify short, code-heavy, transliterated, and mixed-language pages. Finer-grained identification improves control but introduces boundary and aggregation decisions.

At web scale, language identification is a filter with a nonzero error rate, not an oracle. CCNet combines language identification, document deduplication, and quality selection to produce monolingual data from Common Crawl [29]. A production pipeline should retain classifier version, confidence score, threshold, and treatment of uncertain or mixed-language records. This permits a later analysis of what a label such as “English corpus” actually means and prevents an accidental threshold change from silently redefining the dataset.

6.6 Quality Filtering

Quality filtering aims to remove text that is unlikely to contribute useful learning signal under the intended objective. Typical candidates include empty extraction results, pages dominated by navigation or advertising, repeated character runs, machine-generated keyword lists, malformed encodings, extremely short fragments, and documents with implausible ratios of punctuation, digits, or stop words. Filters are often applied after extraction and language assignment, because their features are meaningful only on the retained textual representation.

No scalar quality score captures every useful property. A terse API reference, a mathematical proof, conversational dialogue, and literary prose can all look atypical under the same heuristic. Filters should be evaluated by inspecting both retained and rejected samples across source domains, languages, and length ranges. A conservative pipeline often separates clearly invalid material from uncertain material, records rejection reasons, and measures how each rule changes source composition. This makes it possible to distinguish a deliberate quality decision from an unintended collapse in diversity.

6.7 Heuristic and Model-Based Filtering

Heuristic filters use observable properties such as length, character composition, line structure, URL patterns, repetition, or a blocklist. They are fast, interpretable, and useful for removing obvious artifacts, but they encode a limited definition of quality. Model-based filters instead score a sample using a classifier, a language model, or similarity to a trusted reference distribution. CCNet uses perplexity-based filtering to select documents closer to high-quality corpora, showing one way in which a learned score can supplement hand-built rules [29].

Model-based filtering does not remove judgment; it relocates it into the training data, objective, calibration, and threshold of the scoring model. A classifier trained to recognize encyclopedic prose may remove useful informal dialogue or underrepresented dialects. Conversely, accepting every high-scoring sample can concentrate the corpus around a narrow style. RefinedWeb demonstrates that careful filtering and deduplication can yield a large high-quality web corpus [30], but the general lesson is not that one score is universally correct. It is that filter behavior must be measured against the target mixture and reviewed as a distributional intervention.

6.8 Deduplication

Duplication is introduced by crawl revisits, mirrors, syndicated pages, quoted material, templates, and repeated fragments within or across documents. It consumes token budget without adding independent evidence, can amplify narrow sources, and makes memorization more likely. Deduplication is performed on a declared representation: hashing raw bytes, normalized text, extracted paragraphs, or token sequences answers different questions. The representation, scope, and threshold must therefore be recorded with the data release.

Exact deduplication removes units that are identical under the selected canonicalization. A cryptographic hash of normalized document text is an efficient document-level test; exact substring methods can additionally find long repeated spans even when the surrounding documents differ. Exact matching has high precision but cannot recognize lightly edited copies, translation variants, or pages that differ only in a date, header, or advertising block.

Near deduplication identifies units that are sufficiently similar rather than identical. A common document-level approach converts a document into a set of token or character n-grams, uses MinHash or a related sketch to approximate Jaccard similarity, and applies locality-sensitive hashing to retrieve likely matches before an exact threshold check. The threshold controls a real tradeoff: a low threshold may remove independent documents that share a genre or template, whereas a high threshold leaves nearly identical copies. Lee et al. distinguish exact substring matching from approximate document matching and show that thorough deduplication can reduce memorized text and train-test overlap [31].

6.9 Contamination and Benchmark Leakage

Evaluation contamination occurs when material that reveals an evaluation item, its answer, or a close paraphrase appears in training data. The concern is not limited to an exact copy of a benchmark test question. Solutions, explanations, answer keys, mirrors, and derivative pages can permit recognition rather than the intended generalization. A reported benchmark score then mixes capability with exposure, and the magnitude of the error depends on both the overlap rule and the benchmark design.

Contamination control should begin before training. A pipeline can maintain protected benchmark identifiers and canonicalized text, search candidate documents for exact or approximate overlap, and remove or quarantine matched records. The evaluation set itself should also be deduplicated internally and against the retained training set. The deduplication study of Lee et al. found train-validation overlap in standard datasets and emphasizes that it can distort evaluation [31]. Because no overlap detector is complete, a rigorous report states the protected benchmarks, matching representation, thresholds, scope, date of the data snapshot, and residual limitations rather than claiming contamination-free evaluation without evidence.

6.10 Data Mixtures and Sampling

After filtering and deduplication, a corpus is usually a family of source distributions rather than one homogeneous dataset. Let P_k denote the distribution induced by retained source component k, and let π_k be the probability that a training sequence is drawn from that component. The effective mixture is

$$P_{\mathrm{mix}(x)} = \sum_{k=1}^{K} \pi_k P_{k(x)}, \quad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1. \tag{57}$$

Under the Causal Language Modeling objective of Chapter 5, expected training loss is taken with respect to P_{mix}, not an abstract distribution of all available text. Increasing π_k upweights the conditional patterns, vocabulary, styles, and domains represented by component k; decreasing it does the opposite. Source size alone does not determine this effect. Sampling with replacement can repeat a small component many times, while a large component can be capped or downsampled. The Pile and Dolma both make source composition an explicit corpus-design choice rather than a by-product of raw storage volume [26], [27].

Mixture weights should be defined at the point where sampling occurs. If a pipeline first selects a document source, then packs documents into sequences, the intended document-level weights may differ from the realized token-level weights because documents have different lengths and rejection rates. Monitoring should therefore report both planned sampling probabilities and realized valid-token fractions after tokenization, truncation, packing, and loss masking.

6.11 Domain Balancing

Domain balancing is the deliberate choice to avoid allowing one abundant source to determine the entire mixture. Technical text, code, reference material, books, conversational text, and web pages each contain different conditional distributions. Upweighting a small component can preserve expertise or linguistic variety that would otherwise vanish in a web-dominated corpus. Downweighting can prevent repetitive, low-value, or legally sensitive sources from consuming a disproportionate share of the budget.

There is no source-independent optimal balance. A mixture should be evaluated against its intended use, language coverage, safety and governance requirements, and held-out distributions that are not themselves protected benchmark answers. Overweighting a polished subset can improve short-run loss while reducing stylistic and topical diversity. Conversely, preserving every noisy source at its raw frequency can waste training tokens. A useful practice is to make mixture changes as explicit experimental variables, with component-level ablations and token-accounting reports, instead of treating them as invisible preprocessing.

6.12 Tokenization and Sequence Construction

After document- and sample-level processing, retained text is converted to token IDs using the tokenizer described in Chapter 1. The tokenizer version, vocabulary, normalization configuration, and special-token policy must be pinned: changing any of them changes token counts, sequence boundaries, and the model's input alphabet. Tokenization is therefore a model-facing transformation, not a storage detail that can be swapped after data mixture weights have been chosen.

Long samples must be divided, truncated, or streamed into model-length sequences. A deterministic prefix truncation rule can systematically underrepresent conclusions, appendices, and late-file code; random windows can improve coverage but must be seeded for reproducibility. Empty tokenizations, malformed special tokens, and documents that exceed a maximum length require explicit handling. At this stage, a sample can yield zero, one, or many sequences, so source proportions should be recomputed after the transformation rather than inferred from document counts.

6.13 Packing and Document Boundaries

Packing fills a fixed sequence length by concatenating tokenized material from multiple samples, often inserting an end-of-sequence marker between documents. It improves token utilization relative to padding every short sample to the context length. It also creates a boundary policy. If ordinary causal attention is used across the whole packed tensor, a token in a later document may attend to tokens from an earlier document. The end-of-sequence token signals a transition, but it does not mathematically prevent cross-document attention.

Some pipelines accept this behavior as an efficient approximation to a stream of documents. Others construct a block-diagonal attention mask or reset positional state at boundaries so that attention cannot cross documents. These choices change the conditional distribution being trained: the first trains on continuations conditioned on preceding packed material, while the second trains each document segment independently within the same tensor. The causal Attention Mask and loss mask have different roles, as Chapter 5 explains. Attention masking controls which prefix tokens are visible; a loss mask controls which target positions contribute to the scalar loss.

Padding, packing, and truncation must preserve target alignment. In a right-shift implementation, the final token of one packed unit can become the context for the first token of the next unless the boundary policy prevents it. Padding tokens should neither provide useful context nor contribute token-level loss. The data loader should make these facts observable through input IDs, target IDs, attention masks, loss masks, segment identifiers where used, and tests containing short hand-constructed examples.

6.14 Data Quality versus Data Quantity

Filtering aggressively can improve the average usefulness of a token, but it can also remove rare domains, minority language varieties, informal registers, or unconventional but valid technical text. Data quality is not simply the fraction of documents that resemble a preferred reference corpus. It is the suitability of a retained distribution for the intended learning problem, together with its provenance, diversity, duplication rate, and governance constraints.

Quantity still matters because a language model must observe broad lexical, factual, stylistic, and compositional variation. The relevant tradeoff is not a choice between "clean" and "large" data. It is an iterative allocation problem: estimate which filters remove artifacts, measure the diversity lost by each decision, inspect downstream behavior, and reserve enough unique material to avoid excessive repetition. RefinedWeb's result that extensively filtered web data can remain large at scale is a useful counterexample to the claim that data quality and quantity are always opposed [30].

6.15 Dataset Scale and Token Budgets

Raw document count is a poor scale measure. Documents vary from a title fragment to a book, and their character counts do not map consistently to model work. Let $\tau(d)$ be the tokenization of retained document d. A corpus's unique-token inventory can be summarized as

$$N_{\text{unique}} = \sum_{d \in \mathcal{D}} |\tau(d)|, \tag{58}$$

provided that the tokenizer and document-level deduplication policy are declared. Training scale is more directly described by the number of valid target tokens actually consumed. For sequences $s = 1, \ldots, S$, with length L_s and loss mask $m_{s,t}$, define

$$N_{\text{drawn}} = \sum_{s=1}^{S} \sum_{t=1}^{L_s} m_{s,t}. \tag{59}$$

This quantity accounts for padding, excluded targets, sequence construction, and repeated sampling. It can exceed N_{unique} when the corpus is traversed for multiple epochs or when components are upsampled. Conversely, a large stored corpus may contribute fewer tokens than expected if filtering, truncation, or a source cap removes much of it. Compute-optimal scaling studies explicitly treat the number of training tokens as a variable coupled to model size and compute budget [32]. Any token-budget report should therefore name the tokenizer, whether counts include inputs or supervised targets, the mixture schedule, and the number of effective passes through each component.

6.16 Data Governance and Reproducibility

Data governance begins with provenance. Each retained sample should be traceable, as permitted by source policy, to a source component, capture or release version, extraction rule, language decision, filter decisions, deduplication cluster, and tokenizer version. A manifest can record content hashes rather than redistributing raw text when redistribution is restricted. This lineage supports removal requests, incident response, contamination investigation, and analysis of a model behavior that may be tied to a source component.

Reproducibility requires more than releasing a list of URLs. It requires deterministic or explicitly randomized pipeline stages, software and model versions, configuration files, thresholds, sampling seeds, corpus statistics before and after each filter, and immutable dataset manifests. Dolma's release pairs a large corpus with curation details and tooling, illustrating the value of making construction decisions inspectable [26]. Some sources cannot be preserved indefinitely, so an honest data card should distinguish a reproducible procedure from an exactly reproducible byte-level snapshot.

6.17 Implementation Contracts

A data pipeline should expose a typed schema at every boundary. A raw document record needs a stable identifier, source metadata, capture metadata, and payload reference. An extracted sample needs normalized text, language information, quality scores or rejection reasons, deduplication state, and provenance links. A model sequence needs input IDs, target IDs, attention mask, loss mask, segment or document-boundary information where applicable, and a source-component identifier for accounting.

Validation should run at every granularity. Document-level checks detect missing provenance and corrupt payloads. Sample-level checks measure acceptance rate, language distribution, quality-score distribution, and duplicate clusters by source. Sequence-level checks verify maximum length, special-token placement, packing policy, and the absence of invalid IDs. Token-level checks verify that valid loss positions contain targets in the vocabulary range and that the aggregate count agrees with Equation 59. A compact audit set of known duplicates, benchmark strings, multilingual pages, empty extractions, and boundary cases is more valuable than a pipeline that merely completes without throwing an exception.

6.18 Summary

Pretraining data is a sequence of distributional decisions rather than a static text archive. Collection and extraction define the document population; language identification, quality filtering, and deduplication decide which samples remain; mixture weights and domain balancing determine how often their patterns are observed. Exact and near deduplication answer different similarity questions, while contamination control protects evaluation from exposure to benchmark material and derivatives.

Tokenization and sequence construction translate retained samples into the tensors used by the Causal Language Modeling objective. Packing, truncation, document boundaries, attention masks, and loss masks jointly determine which context is visible and which targets are counted. For this reason, a trustworthy pretraining corpus is described not by a document count alone but by source provenance, processing versions, mixture policy, unique-token inventory, and the valid-token budget actually drawn during training.

References

[26] L. Soldaini *et al.*, "Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research," *arXiv preprint arXiv:2402.00159*, 2024, [Online]. Available: https://arxiv.org/abs/2402.00159

[27] L. Gao *et al.*, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," *arXiv preprint arXiv:2101.00027*, 2021, [Online]. Available: https://arxiv.org/abs/2101.00027

[28] C. Raffel *et al.*, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020, [Online]. Available: https://jmlr.org/papers/v21/20-074.html

[29] G. Wenzek *et al.*, "CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data," *arXiv preprint arXiv:1911.00359*, 2020, [Online]. Available: https://arxiv.org/abs/1911.00359

[30] G. Penedo *et al.*, "The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only," *arXiv preprint arXiv:2306.01116*, 2023, [Online]. Available: https://arxiv.org/abs/2306.01116

[31] K. Lee *et al.*, "Deduplicating Training Data Makes Language Models Better," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2022, pp. 8424–8445. doi: 10.18653/v1/2022.acl-long.577.

[32] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022, doi: 10.48550/arXiv.2203.15556.

Chapter 7: Optimization for Pretraining

Abstract

Pretraining minimizes a token-level likelihood objective, but the objective alone does not specify a viable training procedure. This chapter develops the optimization machinery that turns the loss of Chapter 5 into parameter updates: mini-batch gradients, momentum, Adam, AdamW, learning-rate control, batch sizing, gradient accumulation, clipping, and optimizer-state accounting. The central theme is that an optimizer configuration is a coupled system. Its update count, valid-token count, learning-rate schedule, regularization, and numerical contracts must refer to the same training event.

7.1 Introduction

Chapter 5 expressed Causal Language Modeling as the minimization of token-level negative log-likelihood, and Chapter 6 described how a data pipeline supplies the valid target tokens on which that loss is measured. Optimization closes the loop: it maps an estimate of the loss gradient to a change in the parameters. The mapping is consequential. At pretraining scale, a small mistake in loss normalization, update counting, learning-rate scheduling, or optimizer-state restoration can alter every subsequent update.

Let z denote a valid target-token training event, including the context that predicts it, and let $\ell(\theta; z)$ be its negative log-likelihood under parameters θ. The population objective is

$$\mathcal{J}(\theta) = \mathrm{E}_{z\sim P}[\ell(\theta; z)], \tag{60}$$

where P is the effective data distribution induced by the mixture and sequence-construction policy of Chapter 6. The full expectation is unavailable, so each optimizer update uses a finite collection of valid token events. This chapter assumes that gradients of the decoder-only Transformer have already been obtained by backpropagation. The questions are how to combine them, scale them, regularize the resulting update, and account for the state required to repeat the procedure.

7.2 Mini-Batch Optimization

At update t, let $\mathcal{B}_t$ contain n_t valid target-token events. The mini-batch gradient is

$$g_t = \frac{1}{n_t} \sum_{z\in\mathcal{B}_t} \nabla_\theta \ell(\theta_t; z). \tag{61}$$

Here n_t is the number of loss-eligible tokens, not necessarily the number of sequences in device memory. Padding, document-boundary policies, and loss masks can make those quantities differ substantially. The implementation should therefore use the same valid-token convention for the loss reported in Chapter 5 and for the gradient that drives the update.

If $\mathcal{B}_t$ is sampled from P under the stated mixture policy, g_t is an estimate of $\nabla\mathcal{J}(\theta_t)$. Its randomness is useful: it makes optimization feasible without processing the whole corpus before every update. It also means that a training run is governed by a stochastic process rather than by a deterministic descent curve. Changing the number of valid tokens per update changes both the computational work and the distribution of this estimate.

Plain Stochastic Gradient Descent (SGD) applies

$$\theta_{t+1} = \theta_t - \eta_t g_t, \tag{62}$$

where $\eta_t > 0$ is the learning rate. This equation remains the reference point for more elaborate optimizers. Every additional mechanism can be understood as changing the direction g_t, the scale η_t, or both.

7.3 SGD and Momentum

The gradient from one mini-batch can fluctuate because token events are a limited and heterogeneous sample. Momentum reduces the influence of fluctuations that reverse from update to update while preserving directions that recur. With a velocity u_t and momentum coefficient μ, a common form is

$$u_t = \mu u_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta_t u_t, \quad 0 \le \mu < 1. \tag{63}$$

Expanding the recurrence shows that u_t is an exponentially weighted history of gradients. A coherent direction is reinforced across updates; a rapidly alternating component tends to cancel. The exact momentum convention varies across software libraries, including whether the learning rate is placed inside the velocity. What matters is to identify the convention before transferring hyperparameters. Momentum has long been central to practical deep-network optimization, although its success depends on the interaction among initialization, curvature, and the momentum schedule [33].

SGD with momentum uses one scalar learning rate for every parameter coordinate. That can be effective, but Transformer pretraining contains parameters with different gradient scales and nonstationary statistics. An adaptive optimizer addresses this by maintaining coordinate-wise estimates of gradient scale.

7.4 Adam and AdamW

7.4.1 First and Second Moment Estimates

Adam maintains two state tensors for each optimized parameter tensor. The first moment m_t is an exponential moving average of the gradient, while the second moment v_t is an exponential moving average of its elementwise square:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2. \tag{64}$$

The square in Equation 64 is elementwise. Thus m_t resembles momentum, whereas v_t records a recent uncentered second moment for each coordinate. Dividing by the square root of v_t makes a coordinate with persistently large gradients receive a smaller normalized step than one with small gradients. Adam was introduced as a first-order method based on adaptive estimates of these moments [34].

The state tensors are usually initialized at zero. Consequently, their early values are biased toward zero, particularly when β_1 or β_2 is close to one. Adam corrects this initialization bias by defining

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}. \tag{65}$$

The adaptive update is then

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{66}$$

where division and the square root are elementwise, and $\varepsilon > 0$ protects the denominator. The pair (β_1, β_2) sets the memory of the two statistics; ε is part of the update rule rather than an incidental implementation detail.

Adam is common in language-model pretraining not because it removes optimization choices, but because it combines gradient smoothing with coordinate-wise scaling. This is attractive when a deep model has heterogeneous parameter groups, when gradient scales change during early training, and when a global SGD learning rate would be difficult to tune. Its state is also simple to serialize and its cost per parameter is predictable. The price of this adaptivity is additional memory and additional hyperparameters, discussed in Section 7.8 and Section 7.9.

7.4.2 Decoupled Weight Decay

L2 regularization adds a penalty $\lambda \frac{\|\theta\|_2^2}{2}$ to the objective. Its gradient adds $\lambda\theta_t$ to the data gradient. With plain SGD, the resulting update is

$$\theta_{t+1} = \theta_t - \eta_t(g_t + \lambda\theta_t) = (1-\eta_t\lambda)\theta_t - \eta_t g_t. \tag{67}$$

This algebra explains why L2 regularization and weight decay are often treated as equivalent in SGD: both shrink parameters by the scalar factor $1-\eta_t\lambda$. The equivalence does not survive Adam's coordinate-wise preconditioner. Let D_t be the diagonal operator whose jth diagonal entry is

$$(D_t)_{j,j} = \frac{1}{\sqrt{\hat{v}_{t,j}} + \varepsilon}. \tag{68}$$

Coupling the L2 penalty to Adam then gives $\theta_{t+1} = \theta_t - \eta_t D_t(g_t + \lambda\theta_t)$. The shrinkage is scaled by D_t and therefore differs across coordinates.

AdamW instead applies the decay independently of the adaptive data-gradient update:

$$\theta_{t+1} = (1 - \eta_t \lambda)\theta_t - \eta_t D_t g_t. \tag{69}$$

This is decoupled Weight Decay. It restores a direct, coordinate-independent shrinkage step while retaining Adam's adaptive update for the data loss. Loshchilov and Hutter formalized the distinction and showed that treating L2 regularization as Weight Decay in adaptive methods is misleading [35]. In practice, a parameter-group policy must state which tensors decay. It is common to exempt bias and normalization parameters, but that is a design choice to record and validate, not a consequence of the AdamW equation.

7.5 Learning-Rate Control

The learning rate determines the size of an optimizer step after all other transformations. In AdamW, it appears in both the adaptive update and the decoupled shrinkage factor. A schedule is therefore a time-dependent definition of the optimizer, not a cosmetic post-processing operation.

7.5.1 Warmup

Warmup starts with a small learning rate and increases it over an initial interval of W optimizer updates. Linear warmup to a peak value $\eta_{\max}$ is

$$\eta_t = \eta_{\max} \min\left(1, \frac{t}{W}\right). \tag{70}$$

At the start of training, random initialization, incomplete moment estimates, and rapidly changing activation statistics make a peak learning rate unnecessarily risky. A large early update can move parameters into a regime from which the loss recovers slowly or not at all. Warmup limits this transient while the model and optimizer establish their initial scales. Large-mini-batch experiments demonstrated that an early-training warmup can resolve optimization difficulties that arise when scaling the learning rate with batch size [36]; studies of Transformer normalization also connect warmup sensitivity to the scale of early gradients [37].

Warmup is not a substitute for a viable peak learning rate. It changes the beginning of the trajectory, whereas the peak value, optimizer coefficients, batch size, and model parameterization continue to determine its subsequent behavior. Its duration should be stated in optimizer updates and, for cross-run comparison, in valid tokens seen.

7.5.2 Schedules

After warmup, a schedule usually holds or reduces the learning rate as training progresses. One common decay is a cosine transition from $\eta_{\max}$ to $\eta_{\min}$ over updates $W \le t \le U$:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \times \left(1 + \cos\left(\pi \frac{t - W}{U - W}\right)\right). \tag{71}$$

Linear decay, constant-with-decay, and other schedules are also used. No formula is inherently canonical; the schedule must be specified together with its clock. A schedule indexed by updates changes meaning when gradient accumulation changes the number of optimizer steps per token. A schedule indexed by valid tokens avoids that ambiguity but still requires an explicit mapping from tokens to update boundaries. Compute-optimal language-model studies accordingly treat the learning-rate schedule as an experimental variable coupled to model and token budgets [38].

7.6 Batch Size, Gradient Noise, and Accumulation

7.6.1 Batch Size and Gradient Noise

Under an idealized independent-sampling model, the conditional variance of the mini-batch estimate scales approximately as

$$\mathrm{Var}[g_t \texttt{ mid } \theta_t] \approx \frac{\Sigma(\theta_t)}{n_t}, \tag{72}$$

where $\Sigma(\theta_t)$ is the covariance associated with one valid target-token event. Language-model tokens within a sequence are correlated, and mixture sampling introduces further structure, so this equation is a guide rather than an exact accounting identity. Its central implication remains useful: a larger effective batch usually reduces the stochastic variation of the update estimate.

This variation is called gradient noise. It is not simply an error term. Moderate noise can help exploration of a nonconvex objective, while excessive noise makes the loss trajectory erratic and can trigger instability. As batch size grows, the marginal reduction in noise eventually becomes small compared with the extra work required to form an even larger batch. Empirical work on large-batch training uses the gradient noise scale to characterize this trade-off [39].

Learning rate and batch size must be tuned together. Increasing the effective batch often permits a larger learning rate because the gradient estimate is less variable, but a linear scaling rule is a useful hypothesis, not a universal law. It depends on the optimizer, the data distribution, the schedule, and the regime of the model. When batch size changes, warmup duration, total updates, schedule clock, and clipping rate are quantities to revisit rather than constants to inherit blindly.

7.6.2 Gradient Accumulation

The physical batch is the amount of data whose forward and backward pass fits at one time. When it is too small to reach a desired effective batch, gradient accumulation processes K micro-batches without updating the parameters, sums their gradients, and takes one optimizer step. For micro-batch k with n_k valid target tokens, the desired aggregate is

$$g_t = \frac{1}{N} \sum_{k=1}^{K} \sum_{z \in \mathcal{B}_{t,k}} \nabla_\theta \ell(\theta_t; z), \quad N = \sum_{k=1}^{K} n_k. \tag{73}$$

Thus an implementation must weight each micro-batch by its valid-token count. Averaging K already-averaged losses is wrong when the numbers of valid tokens differ. Accumulation changes the effective batch without storing all activations simultaneously, but it is not identical to increasing the physical batch in every respect. It requires K separate forward and backward passes, has a different reduction order and runtime profile, and can differ when a model includes batch-dependent operations or stochastic behavior that is not controlled consistently. It also changes the number of optimizer steps per token unless the surrounding schedule is adjusted.

The decisive contract is that AdamW updates m_t, v_t, the learning-rate clock, and Weight Decay once after the aggregate gradient has been formed—not once per micro-batch. Applying any of these per micro-batch creates a different optimizer.

7.7 Gradient Clipping

Gradient clipping limits the influence of an unusually large update direction. Global-norm clipping replaces g_t by

$$\tilde{g}_t = g_t \min\left(1, \frac{c}{\|g_t\|_2}\right), \tag{74}$$

where c is a configured threshold. Gradients below the threshold are unchanged; a gradient above it is rescaled without changing direction. The technique was introduced as a practical response to exploding gradients [40], and it remains a useful guardrail for rare loss spikes or abnormal batches during pretraining.

Clipping does not repair a chronically unsuitable learning rate, corrupted data, or an incorrect loss reduction. A high clipping frequency is diagnostic information. Moreover, clipping each micro-batch before accumulation differs from clipping the aggregate gradient in Equation 73. The latter corresponds to the effective update whose norm is being limited and should be the default meaning unless the implementation explicitly chooses another policy.

7.8 Optimizer State and Memory Cost

For P optimized scalar parameters, AdamW stores a first-moment tensor and a second-moment tensor with the same logical shape as the parameters. If each state element occupies b_s bytes, the moment states alone require approximately

$$M_{\text{optimizer}} \approx 2Pb_s. \tag{75}$$

This excludes the parameters themselves, gradients, and any optional master-parameter copy. If the parameter, gradient, and moment-state representations occupy b_w, b_g, and b_s bytes per scalar respectively, a useful model-side lower-bound accounting is

$$M_{\text{parameters-plus-states}} \approx P\big(b_w + b_g + 2b_s\big). \tag{76}$$

For example, half-precision parameters and gradients with full-precision moment states consume $2P + 2P + 4P + 4P = 12P$ bytes before any master copy, activations, temporary buffers, or distributed-training considerations. The exact layout is framework- and precision-policy-dependent, but the qualitative conclusion is not: optimizer state is a first-class memory cost. A checkpoint that omits m_t, v_t, parameter-group settings, or the update index cannot resume the same AdamW trajectory.

7.9 Hyperparameter Interactions

The most important pretraining hyperparameters form coupled groups rather than an independent checklist. Table Table 7 summarizes the interactions that should be reasoned about together.

Configuration change	Immediate effect	Required accompanying check
Larger effective batch	Less variable gradient estimate and fewer updates per token at fixed token budget	Retune learning rate and warmup; state whether schedule is indexed by updates or valid tokens.
More accumulation steps	Larger effective batch without larger activation residency	Normalize by total valid tokens; step AdamW, decay, and scheduler only after accumulation.
Higher peak learning rate	Larger adaptive parameter updates	Revisit warmup, clipping frequency, decay strength, and loss-spike monitoring.
Different β_1 or β_2	Different time scale for gradient statistics	Reset and checkpoint moment states consistently; do not compare schedules by learning rate alone.
Different Weight Decay	Different parameter shrinkage over updates	Check the learning-rate schedule, decay exclusions, and total number of optimizer steps.

Table 7: Optimization settings that must be interpreted jointly. Each row changes the meaning of at least one other control.

The accumulation example is especially easy to miss. If total training is specified in tokens and the effective batch doubles, the number of optimizer updates is approximately halved. A warmup defined as a fixed number of updates then covers roughly twice as many tokens; a cosine decay defined over a fixed update count no longer ends at the intended token budget. The experiment should choose one primary clock and derive the other explicitly.

7.10 Implementation Contracts

The loss-reduction contract must preserve the objective in Chapter 5. For variable-length micro-batches, accumulate the **sum** of loss over valid target tokens and divide by the total valid-token count N of the accumulation cycle. Padding tokens, ignored labels, and masked positions must contribute neither to this numerator nor denominator. The logged loss, gradient normalization, and token counter should agree on the same mask.

The optimizer-step contract is equally strict. Compute all micro-batch gradients at fixed θ_t, form the aggregate gradient, apply global-norm clipping if configured, update AdamW's moments and bias-correction index once, apply its decoupled decay once, and advance the learning-rate schedule once. Clearing gradients or stepping the scheduler inside the accumulation loop violates this contract.

The parameter-group contract should enumerate the learning rate, (β_1, β_2), ε, Weight Decay, and decay-exclusion policy for every group. The checkpoint contract should preserve parameters, both moment tensors, update index, scheduler state, parameter-group configuration, and counters for valid tokens and samples. Finally, monitoring should record at least loss, learning rate, global gradient norm before clipping, clipping events, update count, and valid tokens seen. These traces make an optimization failure diagnosable rather than anecdotal.

7.11 Summary

Pretraining optimization estimates the gradient of a token-level likelihood objective with mini-batches and maps that estimate to parameter updates. SGD supplies the basic update, momentum smooths a history of gradients, and Adam combines first-moment smoothing with coordinate-wise scaling from second moments. AdamW separates adaptive data-gradient updates from Weight Decay, avoiding the false equivalence between L2 regularization and decay under an adaptive preconditioner.

Learning rate, warmup, schedule, effective batch size, and gradient accumulation define the timescale and stochasticity of training together. Gradient clipping bounds exceptional update norms, while AdamW's moment tensors make optimizer state a substantial memory and checkpointing concern. A reliable pretraining run is therefore not defined by an optimizer name alone: it is defined by a coherent set of equations, clocks, normalization rules, state tensors, and auditable implementation contracts.

References

[33] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the Importance of Initialization and Momentum in Deep Learning," in *Proceedings of the 30th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 1139–1147. [Online]. Available: https://proceedings.mlr.press/v28/sutskever13.html

[34] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, 2015. doi: 10.48550/arXiv.1412.6980.

[35] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[36] P. Goyal *et al.*, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," *arXiv preprint arXiv:1706.02677*, 2017, doi: 10.48550/arXiv.1706.02677.

[37] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.

[38] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022, doi: 10.48550/arXiv.2203.15556.

[39] S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team, "An Empirical Model of Large-Batch Training," *arXiv preprint arXiv:1812.06162*, 2018, doi: 10.48550/arXiv.1812.06162.

[40] R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proceedings of the 30th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 1310–1318. [Online]. Available: https://proceedings.mlr.press/v28/pascanu13.html

Chapter 8: Numerical Precision and Training Stability

Abstract

Large-language-model pretraining is executed with finite representations, finite accumulators, and a finite tolerance for abnormal updates. This chapter distinguishes numerical range from precision; compares FP32, FP16, and BF16; and develops the mixed-precision contracts that keep low-precision computation compatible with reliable optimization. It derives stable forms of Softmax, Log-Sum-Exp, and cross-entropy, explains loss scaling and higher-precision accumulation, and separates representational failures from genuine optimization instability. The chapter concludes with diagnostic signals and implementation invariants for a stable pretraining run.

8.1 Introduction

The pretraining objective in Chapter 5 is written over real-valued logits, probabilities, and losses. The optimizer in Chapter 7 is written as an update over real-valued gradients and moment tensors. A training system, however, evaluates neither expression over the real numbers. It stores tensors in finite floating-point formats, rounds intermediate results, and chooses a precision for every reduction and parameter update. These choices affect memory traffic and arithmetic throughput, but they also decide whether a finite mathematical quantity is represented faithfully, rounded coarsely, flushed to zero, or converted to an infinity.

Numerical stability is therefore not synonymous with successful optimization. A run can be numerically invalid because a finite intermediate overflows, even when its learning rate would otherwise be reasonable. It can also remain perfectly finite while diverging because the learning rate, data, initialization, or model dynamics produce unsuitable updates. The purpose of this chapter is to make that distinction operational. It builds on the stable cross-entropy path introduced in Chapter 5, the normalization and initialization analysis in Chapter 4, and the clipping and optimizer contracts in Chapter 7. It does not address distributed execution or communication.

8.2 Floating-Point Representation

8.2.1 Range and Precision

A normalized binary floating-point number has a sign, an exponent, and a significand. Abstractly, a nonzero normalized value can be written as

$$x = (-1)^s 2^{e-b}(1+f), \tag{77}$$

where s is the sign bit, e is the encoded exponent, b is an exponent bias, and f is the fractional significand. The exponent determines the broad range of magnitudes that can be represented. The significand determines the spacing between adjacent representable values at a given magnitude. These are different resources: more exponent bits extend range, whereas more fraction bits improve relative precision.

For a normal floating-point operation rounded to nearest, the standard error model is often summarized as

$$\mathrm{fl}(x) = x(1+\delta), \quad |\delta| \le u, \tag{78}$$

where fl denotes the stored floating-point result and u is the unit roundoff of the destination format. The model excludes overflow, underflow, and exceptional values; it is a local approximation, not a promise that an entire neural-network computation has small error. It nevertheless explains why many individually small rounding errors can accumulate in long reductions [41].

Range answers whether a magnitude can be represented at all. **Precision** answers how closely a representable number near that magnitude can approximate it. A format may have broad range and coarse increments, or narrow range and fine increments. Conflating the two leads directly to poor mixed-precision decisions.

8.2.2 FP32, FP16, and BF16

The three formats most relevant to modern training are FP32, IEEE binary16 (usually called FP16), and BF16. Table Table 8 gives their layout-level distinction. Fraction-bit counts exclude the implicit

leading bit of normalized binary values. IEEE 754 specifies the binary floating-point formats, rounding behavior, and exceptional values that underlie FP32 and FP16 [42].

Format	Sign / exponent / fraction bits	Largest finite magnitude	Smallest normal magnitude	Primary consequence
FP32	$\frac{1}{8}$ $\frac{}{23}$	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	Wide range and fine relative precision.
FP16	$\frac{1}{5}$ $\frac{}{10}$	65504	$\approx 6.1 \times 10^{-5}$	More fraction bits than BF16, but a much narrower exponent range.
BF16	$\frac{1}{8}$ $\frac{}{7}$	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	FP32-like range with substantially coarser relative precision.

Table 8: Format properties for normalized finite values. FP16 and BF16 use the same storage width, but allocate their bits differently; subnormals and hardware handling of them are not shown.

FP16 preserves more fraction bits than BF16, so it distinguishes more nearby values at the same scale. Its five exponent bits, however, make its largest finite value only 65504. BF16 instead keeps FP32's eight exponent bits and sacrifices fraction bits. It can therefore represent roughly the same order of magnitudes as FP32 but rounds more coarsely at each order of magnitude.

This exponent-range property is why BF16 is widely preferred for large-model training when efficient hardware support is available. Activations, gradients, and logits can vary across many orders of magnitude, and BF16 greatly reduces the risk that a finite FP32-scale value overflows or underflows solely because it was stored in a 16-bit tensor. BF16 does not make computation exact: its short significand still loses small relative changes, so important reductions and optimizer state commonly use FP32. The empirical BF16 study of Kalamkar et al. identifies its FP32-like range as the central advantage over FP16-style half precision for training [43].

8.3 Finite-Precision Failure Modes

8.3.1 Rounding, Underflow, and Overflow

Rounding replaces a real result by a nearby representable value. The effect is relative to scale: a small increment can disappear when added to a much larger value because the format has no representable number between the larger value and its next neighbor. This is one reason that updating a large FP16 parameter directly with a small FP16 increment can silently produce no change.

Underflow occurs when a nonzero mathematical quantity is too small for the destination representation. A format may retain a subnormal value near zero, round the result to zero, or use a kernel policy that flushes subnormals to zero for performance. In all of these cases, a stored zero need not mean that the real-valued quantity was zero. Small gradients are especially vulnerable in FP16 because its normal range begins near 6.1×10^{-5}. Underflow is a representational event, not evidence that the corresponding derivative is mathematically absent.

Overflow occurs when a finite mathematical magnitude exceeds the largest finite value available in the destination format. Under ordinary round-to-nearest behavior it typically produces a signed infinity. Subsequent invalid expressions, such as infinity minus infinity or zero times infinity, can produce a Not a Number (NaN). Overflow is also distinct from an exploding gradient. An exploding gradient is a model- or optimization-level statement that a derivative norm has become very large; overflow is a format-level statement that some finite quantity no longer fits. A large gradient may overflow in FP16 while remaining finite in FP32 or BF16, and a gradient can be genuinely explosive without yet crossing any format limit.

8.3.2 Forward Precision and Accumulation Precision

The precision used to store or multiply inputs is not necessarily the precision used to accumulate a result. For a dot product,

$$r = \sum_{i=1}^{n} a_i b_i, \tag{79}$$

an implementation may read a_i and b_i from BF16 or FP16 tensors, form low-precision products, and accumulate the partial sum in FP32 before rounding the output for storage. This is materially different from accumulating the entire sum in the input format. The former retains the bandwidth and matrix-multiply advantages of a 16-bit data path while reducing error in the reduction; the latter repeatedly rounds a running total that may contain thousands of products.

The same distinction applies to loss sums, normalization statistics, gradient norms, and optimizer moments. In this chapter, **forward precision** means the format in which a tensor participates in the main forward and backward arithmetic, whereas **accumulation precision** means the format of partial sums, reductions, or update state. A mixed-precision policy must name both. Saying only that a run uses "BF16" leaves the behavior of its reductions unspecified.

8.4 Mixed-Precision Training

Mixed-precision training assigns different numerical roles to different tensors and operations. A common policy stores activations, model-facing weights, and many gradients in a 16-bit format; performs selected reductions and matrix accumulations in FP32; and preserves an FP32 parameter representation or optimizer state for updates. This separation reduces memory and often accelerates compute without asking every operation to tolerate the same error budget. The original mixed-precision recipe explicitly combines low-precision model tensors with FP32 master weights, higher-precision accumulation, and loss scaling for FP16 gradients [44].

8.4.1 Master Weights and Optimizer State

Let θ^{master} denote a higher-precision parameter copy and let θ^{model} be the rounded copy used by forward and backward kernels. A simplified update path is

$$\theta_{t+1}^{\text{master}} = \text{Update}(\theta_t^{\text{master}}, g_t), \quad \theta_{t+1}^{\text{model}} = \text{cast}(\theta_{t+1}^{\text{master}}). \tag{80}$$

The master copy protects small updates that would be lost if a low-precision weight were updated in place. The cast model copy may still be coarse, but the optimizer does not repeatedly discard sub-unit updates from the parameter trajectory. Chapter 7 explains the AdamW update itself; the numerical point here is that its first and second moment tensors, its denominator, and its parameter update should be maintained in a precision compatible with their dynamic range and required resolution.

An FP32 master copy is particularly important in the classical FP16 recipe. BF16's wider range reduces overflow and gradient-underflow pressure, but it does not provide FP32-level parameter resolution. Frameworks therefore often retain FP32 optimizer states and may retain FP32 master weights even when BF16 is the model-facing format. This is a configuration decision, not an invariant of the name BF16: the checkpoint must record which copy is authoritative and which tensors are merely compute casts.

8.4.2 Gradient Scaling and Loss Scaling

In FP16, many mathematically valid gradients can be too small to survive the backward pass. Loss scaling multiplies the scalar loss by a positive scale S before differentiation:

$$\mathcal{L}_S = S\mathcal{L}, \quad g_S = \nabla_\theta \mathcal{L}_S = Sg. \tag{81}$$

Before the optimizer operates, the implementation divides the gradient by the same scale,

$$g = \frac{g_S}{S}. \tag{82}$$

After this unscaling, the intended objective and optimizer update are unchanged, apart from finite-precision rounding. Loss scaling does not redefine maximum likelihood, increase the learning rate, or create additional signal; it shifts intermediate gradient magnitudes into a format's representable range and then reverses the shift. The loss-scaling method is useful only when the unscaling occurs before operations whose thresholds have semantic meaning, such as gradient clipping, optimizer updates, and gradient-norm logging. Clipping g_S with the ordinary threshold c, for example, is not equivalent to clipping g with c.

Dynamic loss scaling adjusts S during training. A typical policy detects nonfinite scaled gradients, skips the affected optimizer update, lowers S, and retries future batches at the lower scale. After a

sustained interval of finite updates it may increase S cautiously. The finite-value check is part of numerical control, not an optimization step. BF16 often avoids the need for loss scaling because its exponent range matches FP32's, but this does not eliminate the need to check for NaNs, infinities, or inaccurate low-precision reductions.

8.5 Stable Exponentials and Cross-Entropy

8.5.1 Log-Sum-Exp and Softmax

Chapter 5 defines a vocabulary logit vector $O_{b,t,:}$ and its Softmax distribution. For this section, write the logits at one supervised position as $o_1, \ldots, o_V$ and let

$$c = \max_{1 \le j \le V} o_j. \tag{83}$$

The Log-Sum-Exp function has the algebraically equivalent shifted form

$$\mathrm{LSE}(o) = \log \sum_{j=1}^{V} \exp\bigl(o_j\bigr) = c + \log \sum_{j=1}^{V} \exp\bigl(o_j - c\bigr). \tag{84}$$

Because $o_j - c \le 0$, every exponential in the right-hand sum lies in $(0, 1]$ over the real numbers. The largest term is exactly one, so a very large positive logit cannot overflow merely because it is exponentiated. Softmax uses the same shift,

$$\mathrm{softmax}\,(o)_v = \frac{\exp(o_v - c)}{\sum_{j=1}^{V} \exp\bigl(o_j - c\bigr)}. \tag{85}$$

The shift changes neither the exact probability distribution nor the exact Log-Sum-Exp value; it subtracts the same constant from every logit. It changes the floating-point computation decisively. Unshifted $\exp(o_v)$ can overflow for a large positive logit, while shifted exponentials do not. Conversely, a logit far below c can produce an exponential that underflows to zero. That output is a numerical zero, not a claim that the real exponential is zero. Its contribution to the denominator is often below the working precision and therefore harmless, but it should not be confused with a mathematical deletion of the token. Numerical analyses of shifted Log-Sum-Exp and Softmax formalize why these algebraically equivalent expressions behave differently in finite precision [45].

8.5.2 A Stable Cross-Entropy Path

For target token index y, the token-level negative log-likelihood can be written directly from logits as

$$\ell = \mathrm{LSE}(o) - o_y = c + \log \sum_{j=1}^{V} \exp\bigl(o_j - c\bigr) - o_y. \tag{86}$$

This is the numerical form that a cross-entropy kernel should implement. Computing p_y from a separately materialized Softmax vector and then evaluating $-\log p_y$ is less robust. If o_y is much smaller than the maximum logit, p_y can underflow to stored zero even though Equation 86 remains a finite loss. The stable logit-space expression also avoids constructing a length-V probability tensor solely to gather one target probability.

The max subtraction in Equation 84 does not solve every numerical problem. The sum remains a reduction, logits may already contain infinities or NaNs, and a low-precision output may still round. Its role is narrower and essential: it prevents avoidable overflow from the exponential while preserving the mathematical objective introduced in Chapter 5.

8.6 Reductions, Norms, and Statistics

Reductions are numerically sensitive because they combine many values into one. Under the standard model in Equation 78, a sequential summation of n terms has a representative error bound

$$|\hat{s} - s| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \quad \gamma_k = \frac{ku}{1 - ku}, \tag{87}$$

when $ku < 1$, where $s = \sum_i x_i$ and $\hat{s}$ is the computed sum. The bound is not a prediction for one particular kernel, but it captures the two relevant facts: rounding error grows with the number of additions, and cancellation makes a small final sum difficult to compute relative to the magnitude of

its inputs [41]. Tree reductions change the order of additions and therefore can change low-order bits even when they are mathematically equivalent.

This applies directly to the valid-token loss sum in Chapter 5 and the accumulated gradient in Chapter 7. Accumulate loss numerators, gradient statistics, and large dot products in higher precision where the kernel permits. The denominator for a masked loss must be counted under the same mask, but it is an integer accounting quantity rather than a low-precision activation reduction.

LayerNorm and RMSNorm add a further case: they estimate a scale from many feature values. Chapter 4 notes that the variance identity $E[x^2] - E[x]^2$ can suffer cancellation when the mean is large. Higher-precision accumulation and a stable variance algorithm reduce this risk. The normalizer's ε prevents division by a zero or extremely small estimated scale, but it cannot repair an activation distribution that has already become pathological.

Global gradient norms require the same care. The expression $\|g\|_2 = \sqrt{\sum_i g_i^2}$ squares each component before summing, so a finite component can overflow during squaring in a narrow format. Compute norms and clipping decisions from unscaled gradients in an appropriately wide accumulation precision. This preserves the meaning of the clipping threshold from Chapter 7 rather than making it a hidden function of the compute dtype.

8.7 Stability as a Training-System Property

8.7.1 Initialization and Scale

Numerical failures often expose an earlier scale problem. Chapter 4 derived how fan-aware initialization and residual scaling aim to keep activation and gradient variance in a useful range. Those arguments become more—not less—important in low precision. A poorly scaled projection can create activation outliers; outliers widen logit and gradient distributions; wide distributions increase the chance of overflow, loss-scale backoff, or inaccurate reductions.

Monitoring should therefore observe distributions, not only scalar loss. Useful per-block measurements include activation RMS, maximum absolute activation, high quantiles, logit range, gradient RMS, and global gradient norm. A single extreme layer can be invisible in a batch-averaged loss until it contaminates a reduction or parameter update. In contrast, a broadly rising activation scale across many layers may point to residual accumulation, normalization, or learning-rate behavior. These measurements do not identify a cause by themselves, but they localize the first boundary at which the run stopped resembling its finite baseline.

8.7.2 Numerical and Optimization Instability

Numerical instability is a failure of the implemented arithmetic to represent or evaluate the intended computation reliably. Its immediate signals include NaN or Inf tensors, a sudden loss-scale reduction, nonfinite optimizer moments, or a loss that changes drastically when only the compute dtype is widened. Optimization instability is a failure of the update trajectory: the loss may spike, gradients may grow, or the model may fail to improve while every tensor remains finite. It can be caused by an unsuitable learning-rate transition, a data anomaly, a batch-size change, or an architectural scale problem.

The two failures interact but should not be diagnosed as one. An optimization-induced gradient spike can trigger FP16 overflow. Conversely, repeated FP16 underflow can suppress useful gradients and create an apparent optimization plateau. Widening selected operations to FP32 is a useful isolation experiment: if a failure disappears while the data, parameters, and update schedule are held fixed, a numerical pathway is implicated; if it persists with finite FP32 tensors, the cause is more likely in the optimization or data path. Neither outcome is a proof, so logs and reproducible failing batches remain necessary.

8.8 Practical Stability Diagnostics

Table Table 9 summarizes common failure signals. The right response is to record the earliest abnormal tensor and isolate one variable at a time, not to apply a larger clipping threshold or a smaller learning rate indiscriminately.

Observed signal	Possible interpretation	First isolation step
NaN or Inf after one operation	Overflow, invalid arithmetic, corrupt input, or a nonfinite value entering the operation	Locate the first nonfinite tensor; rerun the same batch with selected operations widened.
Sudden finite loss spike	Data anomaly, learning-rate transition, abnormal logits, or emerging optimization instability	Compare the batch, learning-rate state, logit range, and per-layer norms with the last finite update.
Abnormal gradient norm	True gradient growth, incorrect loss reduction, missed unscaling, or low-precision norm accumulation	Verify unscaling and masks, then inspect layerwise gradient statistics in wider precision.
Activation or logit outlier	Initialization, residual-scale, normalization, data, or kernel-path issue	Identify the first affected block and compare its inputs, normalizer statistics, and residual update.
Nonfinite optimizer state	A prior nonfinite gradient or invalid update has entered the state tensors	Skip the update, preserve the failing state, and resume only from a known finite checkpoint after diagnosis.

Table 9: Diagnostic signals are evidence, not diagnoses. The goal is to identify the earliest abnormal boundary while preserving the exact batch and training state that produced it.

Sudden loss spikes deserve particular restraint. A spike can be a transient consequence of stochastic mini-batch sampling, a corrupted or unusual sequence, a learning-rate boundary, a numerical overflow that has not yet surfaced as NaN, or a genuine model instability. The decisive question is not whether the scalar loss is large, but whether the surrounding tensors, masks, learning-rate state, and optimizer state remain finite and consistent with the declared contracts. A stable system logs enough information to answer that question after the fact.

8.9 Implementation Contracts

A precision policy must be explicit and checkpointed. It should declare the model-facing dtype, the accumulation dtype for matrix products and reductions, the authoritative parameter dtype, the dtype of optimizer moments, and the cast boundaries between them. A configuration described only as "mixed precision" is underspecified. The policy should also state whether subnormal flushing, fused kernels, or dynamic loss scaling are active when those choices can affect reproducibility.

The loss contract follows Chapter 5: consume raw logits, evaluate cross-entropy through a stable fused Log-Sum-Exp path, and reduce only valid target positions. In an FP16 loss-scaling path, test scaled gradients for finiteness, then unscale successful gradients before computing gradient norms, applying clipping, logging optimizer-facing statistics, or updating parameters. The valid-token loss numerator, denominator, and accumulation convention must remain identical to those used by Chapter 7.

The finite-state contract checks parameters, gradients, optimizer moments, loss, and selected activation statistics at declared boundaries. A nonfinite gradient must not be allowed to update a master parameter or moment tensor. Preserve the failing batch identifiers, random state, optimizer state, precision policy, and recent scalar traces before skipping an update or restoring a checkpoint. Finally, a reference test should compare a small deterministic batch against a higher-precision implementation for logits, loss, selected gradients, and one optimizer update. This is not a requirement for bitwise equality; it is a contract that low-precision deviations are bounded, expected, and not silently changing the computation.

8.10 Summary

Floating-point formats trade range against precision. FP16 has a narrower exponent range despite more fraction bits than BF16; BF16 preserves FP32-like range but has coarser relative resolution. This makes BF16 a strong default for modern large-model compute when hardware supports it, while FP32 remains valuable for reductions, master parameters, and optimizer state.

Mixed-precision training works because not every operation needs the same representation. Higher-precision accumulation protects large reductions, master weights retain small updates, and loss scaling shifts FP16 gradients into range before being reversed prior to the optimizer. Stable Log-Sum-Exp and max-shifted Softmax preserve the cross-entropy objective while avoiding preventable exponential

overflow. Reliable training then requires more than finite loss: it requires explicit dtype boundaries, correct unscaling and clipping order, monitored activation and gradient statistics, nonfinite-value containment, and a disciplined separation between numerical failure and optimization instability.

References

[41] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Society for Industrial, Applied Mathematics, 2002. doi: 10.1137/1.9780898718027.

[42] IEEE, "IEEE Standard for Floating-Point Arithmetic," technical report IEEE Std 754–2019, 2019. doi: 10.1109/IEEESTD.2019.8766229.

[43] D. Kalamkar *et al.*, "A Study of BFLOAT16 for Deep Learning Training," *arXiv preprint arXiv:1905.12322*, 2019, doi: 10.48550/arXiv.1905.12322.

[44] P. Micikevicius *et al.*, "Mixed Precision Training," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=r1gs9JgRZ

[45] P. Blanchard, D. J. Higham, and N. J. Higham, "Accurately Computing the Log-Sum-Exp and Softmax Functions," *IMA Journal of Numerical Analysis*, vol. 41, no. 4, pp. 2311–2330, 2020, doi: 10.1093/imanum/draa038.

Chapter 9: Scaling Laws and Compute

Abstract

Scaling laws summarize measured relationships among model size, training tokens, and compute; they are useful for planning only when their accounting conventions and empirical domain are explicit. This chapter distinguishes parameter count, corpus inventory, drawn-token budget, training FLOPs, and wall-clock time. It develops power-law fits and a simple compute-constrained allocation model, then contrasts the model-heavy Kaplan-style frontier with the more balanced Chinchilla-style result. The final sections describe data-limited and compute-limited regimes, the limits of extrapolation, and the accounting contracts required for an auditable pretraining plan.

9.1 Introduction

Chapters 5 through 8 describe what a decoder-only language model optimizes, which tokens reach that objective, how its parameters are updated, and how that computation remains numerically reliable. Scaling asks a different question: given a finite pretraining budget, how should a team allocate resources among model capacity, training tokens, and computation? The answer is not obtained by maximizing a single number such as parameter count. A larger model sees fewer tokens under a fixed compute budget; a longer run limits the model size that can be afforded. The relevant choice is a balance.

Empirical scaling laws make this balance measurable. They fit held-out loss across controlled runs at different scales and use the fitted relationship to estimate a useful frontier. Their value is practical: a small family of well-instrumented runs can rule out expensive configurations that are predictably data-starved or capacity-limited. Their status is equally important: they are regressions over a specified model family, dataset, objective, and optimization recipe. They are not fundamental laws of language or guarantees about downstream behavior.

This chapter uses the token-accounting distinctions of Chapter 6 and the valid-token loss of Chapter 5. It does not address how a fixed FLOP budget is distributed across devices; parallelism, communication, and cluster topology belong to Chapter 10.

9.2 Scale Accounting

9.2.1 Parameters, Documents, and Tokens

Let P denote a declared parameter count for the model. It is an architectural quantity: it counts stored trainable degrees of freedom under a stated convention. A report may count all trainable parameters, non-embedding parameters, or active parameters in a sparse model; these are not interchangeable. For the dense Transformer calculations below, P is the parameter count used by the FLOP convention.

Let R denote a raw corpus size, such as a document count, byte count, or source-record count. It is a property of an archive, not necessarily of training. Documents vary greatly in length; filtering, deduplication, and tokenization change their contribution; packing and loss masking can further change the number of supervised positions. Chapter 6 therefore distinguishes the unique token inventory N_{unique} from the drawn valid-token budget N_{drawn}. Here write

$$U = N_{\text{unique}}, \quad D = N_{\text{drawn}}, \tag{88}$$

where U is the inventory after a declared processing pipeline and D is the total valid target-token count actually consumed by the objective. The latter can exceed U when data are revisited or components are upsampled. For a scaling study, D is usually the relevant data variable because it tracks the token-level training work. Raw document count R is not a substitute for D.

Quantity	Symbol and unit	What it measures
Parameter count	P parameters	Declared model capacity. The counting convention must specify, for example, whether embeddings or inactive sparse parameters are included.
Raw corpus size	R documents, bytes, or records	An archive property. It does not determine sequence length, valid targets, or model work.
Unique token inventory	U tokens	Retained tokenized content before repeated sampling, under a specified tokenizer and deduplication policy.
Effective token budget	D valid target tokens	The token positions actually drawn by the training objective, including mixture sampling and any reuse.
Training compute	C_{train} FLOPs	Arithmetic work implied by the model, token budget, and FLOP-accounting convention.
Wall-clock time	t_{wall} seconds	Elapsed execution time, which also depends on sustained throughput and operational interruptions.

Table 10: Scale variables have different units and different roles. A credible plan reports each one rather than treating raw corpus size, token budget, FLOPs, and elapsed time as synonyms.

The ratio $\frac{D}{P}$, often called tokens per parameter, is a useful diagnostic only after both numerator and denominator have been defined. It compares training exposure to model capacity under a particular accounting convention; it does not measure data quality, coverage, or the number of optimization updates. It is also not fixed by the mathematics of language modeling. It emerges from a fitted frontier and may change when the data mixture or model family changes.

9.2.2 Training FLOPs and Wall-Clock Time

For a dense decoder-only Transformer, a common planning approximation expresses pretraining arithmetic as

$$C_{\text{train}} \approx kPD, \quad k \approx 6. \tag{89}$$

The factor near six summarizes an approximate forward-and-backward cost per parameter-token pair under a conventional dense-model counting rule. It is useful because it exposes the dominant first-order trade-off: at fixed C_{train}, increasing P requires reducing D, and conversely. It is not a precise bill of materials. Attention work, embeddings, sequence length, activation recomputation, sparsity, kernel choices, and the definition of a FLOP change the constant and sometimes the form of the estimate. Kaplan et al. use a related parameter-and-token accounting to study compute-efficient language-model training [46].

FLOPs are a work estimate, whereas wall-clock time is an execution outcome. If r_{eff} is the measured sustained training rate in FLOPs per second for a fixed implementation, then a planning estimate is

$$t_{\text{wall}} \approx \frac{C_{\text{train}}}{r_{\text{eff}}}. \tag{90}$$

Peak accelerator throughput is not r_{eff}. Input stalls, non-matmul work, memory traffic, numerical precision, and recovery time lower sustained performance. This chapter needs only the distinction: two runs with the same declared FLOPs can take different elapsed time, and two runs with the same elapsed time can execute different FLOPs. The mechanisms behind sustained throughput are a systems topic, not a replacement for FLOP accounting.

9.3 Empirical Power-Law Behavior

A common one-variable scaling fit is

$$\mathcal{L}(x) = Ax^{-\alpha} + B, \quad A > 0, \quad \alpha > 0. \tag{91}$$

Here $\mathcal{L}$ is a measured validation loss and x can stand for a scale variable such as P, D, or an optimally allocated compute budget. The coefficient A sets the scale of the reducible loss, α controls the rate of improvement, and B is a fitted asymptote or floor. Within this equation, B is unrelated to the batch-size notation used in Chapter 7. To avoid that collision below, write the asymptote as $\mathcal{L}_{\text{inf}}$.

The practical content of Equation 91 is diminishing return rather than a promise of unlimited improvement. If x is multiplied by r, then the excess loss $\mathcal{L}(x) - B$ changes by $r^{-\alpha}$. A positive exponent therefore predicts consistent relative improvement on a log-log scale, but progressively smaller absolute gains as the reducible term shrinks. After subtracting a justified fitted floor, the relationship is linear in logarithms:

$$\log(\mathcal{L}(x) - B) = \log A - \alpha \log x. \tag{92}$$

Kaplan et al. reported smooth power-law relationships between autoregressive validation loss and model size, dataset size, and compute in their experimental setting [46]. Such a fit should be judged by held-out residuals, by its stability across nearby scales, and by whether its accounting conventions match the proposed run. A straight segment on one log-log plot does not justify extrapolation across a new tokenizer, data distribution, architecture, or training procedure.

9.4 Compute-Constrained Allocation

9.4.1 A Representative Joint Loss Surface

A one-variable fit hides the fact that a model can be limited by either capacity or training exposure. A useful illustrative surface is

$$\hat{\mathcal{L}}(P, D) = \mathcal{L}_{\text{inf}} + a_P P^{-\alpha} + a_D D^{-\beta}, \quad a_P, a_D, \alpha, \beta > 0. \tag{93}$$

The P term represents the capacity-limited contribution and the D term the token-limited contribution. This separable form is a planning model, not the only valid scaling-law parameterization. Real fits can include interaction terms, finite-data corrections, learning-rate-schedule effects, or a different functional form. Its advantage is that it makes the allocation logic explicit.

Suppose the planned runs obey the approximate compute relation Equation 89, so that $D = \frac{C_{\text{train}}}{kP}$. Substituting this constraint into Equation 93 gives a loss along one compute frontier:

$$\hat{\mathcal{L}}(P; C_{\text{train}}) = \mathcal{L}_{\text{inf}} + a_P P^{-\alpha} + a_D \left(k \frac{P}{C_{\text{train}}} \right)^{\beta}. \tag{94}$$

The first term decreases with model size; the second increases because a larger model consumes the same FLOPs over fewer tokens. Setting the derivative with respect to P to zero equates their marginal contributions:

$$\alpha a_P P^{-\alpha} = \beta a_D D^{-\beta}. \tag{95}$$

This yields the qualitative compute-optimal exponents

$$P^* = c_P C_{\text{train}}^{\frac{\beta}{\alpha+\beta}}, \quad D^* = c_D C_{\text{train}}^{\frac{\alpha}{\alpha+\beta}}, \quad c_P, c_D > 0. \tag{96}$$

The fitted exponents determine where additional compute goes. If α and β are similar, model size and token budget grow at similar rates. If they differ, the balance shifts. The derivation does not establish the values of α and β; it shows why different empirical fits can recommend different allocations under the same high-level FLOP constraint.

9.4.2 Kaplan-Style and Chinchilla-Style Frontiers

Table 11 contrasts two influential results. The difference is historical and empirical, not a contradiction in algebra. The two studies fit different data and modeling assumptions, then minimized their respective fitted loss models subject to a compute budget.

Study	Compute-optimal implication	Planning consequence
Kaplan et al. (2020)	The reported compute-efficient frontier grew parameters approximately as $C^{0.73}$ and data more slowly, approximately as $C^{0.27}$.	A relatively model-heavy allocation, with models stopped well before convergence on a much larger token inventory.
Hoffmann et al. (2022)	The controlled fit found that model size and training tokens should grow in roughly equal proportion as compute increases.	Allocate materially more tokens per parameter; the published Chinchilla run used 70B parameters and 1.4T tokens.

Table 11: Two influential compute frontiers. The exponents and ratios summarize specific empirical fits; neither row supplies a tokenizer-, dataset-, or architecture-independent prescription.

Kaplan et al. found a model-heavy allocation in which the fitted parameter scale grew approximately as $C_{\text{train}}^{0.73}$ and the required data scale approximately as $C_{\text{train}}^{0.27}$ [46]. In this regime, a very large model could be compute-efficient even though it was trained far short of convergence on the available corpus. This recommendation was consequential because it separated compute-efficient training from the intuition that every model should be trained until its loss no longer improves.

Hoffmann et al. revisited the allocation with more than four hundred Transformer runs spanning model sizes and token budgets, and found a substantially more balanced frontier: doubling model size should be accompanied by a doubling of training tokens for compute-optimal training [47]. Their compute-matched Chinchilla model used 70 billion parameters and 1.4 trillion training tokens, about 20 tokens per parameter, while using the same pretraining compute budget as the 280-billion-parameter Gopher model. It outperformed Gopher on their reported evaluations [47]. This result changed the default planning question from "how large can the model be?" to "which model-and-token pair uses the budget most effectively?"

9.5 Regimes, Reuse, and Bottlenecks

9.5.1 Undertraining and Overtraining

A model is **undertrained** relative to a chosen scaling frontier when it has too little token exposure for its parameter count and compute budget. Its loss can still be decreasing rapidly when training stops, and a smaller model trained on more data may achieve better validation loss for the same C_{train}. Undertraining is therefore not a claim that the model has seen few documents; it is a relationship among P, D, the data distribution, and the training recipe.

The opposite label, **overtraining**, must be used carefully. It can describe a run that spends many additional tokens for small marginal validation improvement, or one that repeatedly reuses a limited corpus until generalization deteriorates. These are different mechanisms. A large supply of new, high-quality, diverse tokens can continue to improve a fixed model beyond a compute-optimal point chosen for pretraining efficiency. By contrast, repeated exposure to duplicates or narrow domains can inflate D without comparable new information. As Chapter 6 emphasizes, mixture weights, deduplication, and provenance determine what a drawn-token count means.

No universal tokens-per-parameter threshold separates these regimes. The Chinchilla ratio is a result of a particular controlled study, not a license to treat every token as interchangeable or every parameter count as comparable. Evaluation goals also matter: a model selected for low pretraining loss under a one-time compute budget can differ from a model selected for low inference cost across a long deployment horizon.

9.5.2 Data-Limited and Compute-Limited Planning

In a compute-limited regime, the team has enough validated data to consider several (P, D) pairs satisfying a chosen compute budget. The principal task is to estimate a frontier from compatible pilot runs and select a pair near its minimum. The learning-rate schedule, batch convention, and sequence construction must stay comparable across those pilots; otherwise the fitted difference mixes scale with recipe changes.

In a data-limited regime, the supply of high-quality, policy-compliant, sufficiently diverse tokens constrains D before the compute budget is exhausted. Replaying the same inventory changes the data-reuse rate rather than creating a larger corpus. The planning problem then includes a data decision: accept reuse with an explicit risk assessment, improve curation and collection, reduce model capacity, or spend compute elsewhere. Calling the run "Chinchilla-optimal" without stating the available unique inventory conceals this constraint.

9.6 Scaling Predictions and Their Limits

Scaling laws are most defensible as interpolation tools. A team can train a family of smaller models using the intended tokenizer, data mixture, sequence length, loss reduction, optimizer schedule, and evaluation set; fit a loss surface; and compare its predictions with additional held-out scales. The validation loss being predicted must be measured on a fixed, protected evaluation distribution. A changing benchmark or contaminated validation set can create a persuasive but meaningless curve.

Extrapolation is harder. Changes in architecture, active-parameter sparsity, context length, data quality, curriculum, precision policy, optimization stability, or evaluation distribution can alter both the fitted constants and exponents. A loss floor may be unidentifiable from a limited range, so subtracting a guessed B in Equation 92 can make a log-log fit appear straighter than its evidence warrants. More fundamentally, cross-entropy loss is only one outcome: downstream reliability, calibration, safety, and task-specific capability need not follow the same scaling relationship.

The correct conclusion is neither that scaling laws are exact nor that they are useless. They are empirical summaries that can convert a large, uncertain budget into testable predictions. Their assumptions, residuals, data identity, and out-of-sample checks are part of the result.

9.7 Practical Planning Under a Fixed Budget

Begin with four independently measured constraints: a maximum training FLOP budget, a wall-clock deadline, a verified unique-token inventory, and an intended evaluation target. Convert each candidate architecture into a declared P, estimate feasible token budgets D with Equation 89, and reject pairs that exceed the data, compute, or time constraints. The result is a short set of feasible configurations, not yet an optimal choice.

Next, run scaled pilots with the same data-processing policy and optimization semantics that the full run will use. Record validation loss against both D and C_{train}, not merely against update count. A pilot that changes model size while also changing tokenizer, sequence length, mixture, warmup clock, or loss-mask reduction cannot identify a model-size effect cleanly. Chapter 7's schedule and batch conventions remain part of the experimental definition even though this chapter does not rederive them.

Finally, choose a conservative point near the fitted frontier rather than a single extrapolated optimum. Preserve a reserve for failed runs, checkpoint recovery, ablations, and validation. If inference cost is material, include it as a separate deployment constraint: a smaller model trained longer may be preferable even when a pretraining-only frontier would select a larger one. The plan should state which objective it optimizes, because "best model" has no meaning without a budget and a use case.

9.8 Implementation Contracts

A scaling report must make its counters auditable. It should record the parameter-count convention, including whether embeddings, tied heads, inactive experts, or frozen tensors are included. It should record raw corpus size R, unique inventory U, and drawn valid-token budget D separately, together with tokenizer version, data manifest, mixture schedule, sequence-construction policy, and loss-mask convention. A token counter that includes padding or ignored labels is not a counter for the objective in Chapter 5.

The compute contract must name the FLOP formula, its constant k, and any excluded work. It should distinguish estimated compute from measured elapsed time and record the measured sustained rate used in Equation 90. Checkpoints and experiment logs should preserve the update count, valid-token count, data-reuse rate, learning-rate state, model configuration, and evaluation-dataset version. These

fields make it possible to reproduce a scaling point and to detect when two nominally equal-compute runs are not actually comparable.

Before relying on an extrapolation, retain pilot configurations that test both model-limited and token-limited sides of the proposed optimum, hold out at least one scale for validation, and compare prediction error in the loss domain rather than only in a visually appealing log-log plot. A scaling fit is an implementation artifact as well as a mathematical fit: its conclusions are only as reliable as the counters and controlled variables that produced it.

9.9 Summary

Pretraining scale has several independent units. Parameter count measures declared model capacity; raw document count describes an archive; unique and drawn tokens describe different forms of data exposure; FLOPs estimate arithmetic work; and wall-clock time measures execution. Token count is usually the meaningful scale variable for the pretraining objective because it records valid target positions actually consumed, whereas document count does not.

Empirical power laws describe diminishing improvements over a measured regime, and a joint loss model explains why a fixed compute budget creates a trade-off between larger models and more training tokens. Kaplan-style fits favored a more model-heavy allocation, while Chinchilla-style results favored substantially more data relative to parameters. Neither recommendation is universal. Data quality, reuse, model family, objective, and deployment requirements all change the decision. A sound pretraining plan therefore uses scaling laws as validated, fully accounted predictions rather than as a single immutable ratio.

References

[46] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020, doi: 10.48550/arXiv.2001.08361.

[47] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022, doi: 10.48550/arXiv.2203.15556.

Chapter 10: Distributed Training

Abstract

Large language models exceed the memory and throughput limits of a single accelerator, so pretraining must divide data, model state, and computation across coordinated ranks. This chapter derives gradient synchronization for Distributed Data Parallel, contrasts replication with sharding, and explains Tensor, Pipeline, and Sequence Parallelism. It then develops the memory logic of ZeRO and Fully Sharded Data Parallel, relates communication to realized scaling efficiency, and gives the contracts required to make a multidimensional parallel plan correct and recoverable.

10.1 Introduction

The pretraining budget in Chapter 9 is expressed in tokens and FLOPs, but that budget must be executed by an actual training system. A dense model may not fit in one accelerator's memory once its parameters, gradients, optimizer state, and activations are present. Even when it does fit, one accelerator can make the planned token budget take an impractical amount of wall-clock time. Distributed training addresses both constraints by placing different pieces of a training step on multiple ranks.

The central difficulty is not merely placing tensors on more devices. All ranks must evaluate the same mathematical objective, keep the intended parameter state coherent, and communicate the information that a local computation no longer possesses. Parallelism therefore has two distinct purposes. **Data parallelism** replicates a model and divides examples across replicas. **Model parallelism** divides the model computation or state itself. The right system usually combines them because a training job needs both more aggregate throughput and less per-rank state.

This chapter builds on the token-level objective in Chapter 5, optimizer state in Chapter 7, numerical state in Chapter 8, and the compute-versus-wall-time distinction in Chapter 9. It does not prescribe a vendor-specific runtime, network topology, or inference-serving stack.

10.2 Parallelism, Ranks, and Batches

Let W be the total number of ranks. A rank is one independently scheduled participant in a distributed process group; it need not be identified with a particular physical device for the mathematical discussion. A **replicated** tensor has a full copy on every rank in a group. A **sharded** tensor is partitioned so that each rank holds only a disjoint piece, with an explicit rule for reconstructing or communicating the whole.

The local batch and global batch must also be separated. Let n_D be the Data Parallel degree, let b_{mu} be the microbatch size on each Data Parallel replica, and let M be the number of such microbatches accumulated into one optimizer step. In the simple equal-size case,

$$B_{\mathrm{local}} = Mb_{\mathrm{mu}}, \quad B_{\mathrm{global}} = n_D M b_{\mathrm{mu}}. \tag{97}$$

Tensor, Pipeline, and Sequence Parallel ranks collaborate on the **same** local microbatch; they do not multiply the number of examples in Equation 97. Variable sequence lengths and loss masks require token-weighted accounting rather than this compact example, as Chapter 5 explains.

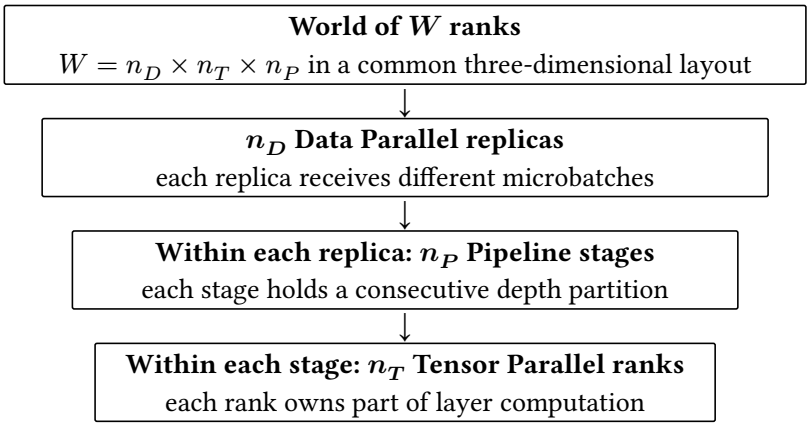

Figure 3: A common three-dimensional decomposition. Sequence Parallelism commonly reuses the Tensor Parallel group to partition selected activations, so it need not introduce another independent factor in world size.

Figure 3 is a logical hierarchy, not a claim that every process group is nested in exactly this order. It records a useful accounting rule: in a conventional three-dimensional layout, $W = n_D n_T n_P$, where n_T and n_P are the Tensor and Pipeline Parallel degrees. Sequence Parallelism often operates over the same n_T ranks rather than adding a fourth independent factor. Some systems add other dimensions, but each must specify which ranks share data, parameters, activations, and collectives.

10.3 Data Parallelism and Gradient Synchronization

In ordinary Data Parallelism, every rank begins a step with the same parameters θ_t and receives a different local batch. Let g_r be rank r's local gradient for an equal-weight local objective. Distributed Data Parallel (DDP) computes the global gradient by an all-reduce average:

$$g = \frac{1}{n_D}\,\mathrm{AllReduce}_{\mathrm{sum}(g_1,\dots,g_{n_D})} = \frac{1}{n_D}\sum_{r=1}^{n_D} g_r. \tag{98}$$

Every participant receives the same g, so applying the deterministic optimizer update from Chapter 7 produces the same θ_{t+1} on every replica. This is the mathematical reason replicated DDP is coherent: independent forward and backward passes are permitted because their gradients are reconciled before the shared update. The DDP design and its overlap of gradient communication with backward computation are described by Li et al. [48].

For Causal Language Modeling, the local loss may contain a different number of valid target tokens due to packing or masking. Let $S_r = \sum_{b,t} m_{b,t}\ell_{b,t}$ be a local loss numerator and let $N_r = \sum_{b,t} m_{b,t}$ be its valid-token count. The globally token-averaged objective is

$$\mathcal{L}_{\mathrm{global}} = \frac{\sum_{r=1}^{n_D} S_r}{\sum_{r=1}^{n_D} N_r}. \tag{99}$$

An unweighted average of local **mean** losses equals Equation 99 only when the N_r are equal. A distributed implementation must therefore equalize valid-token counts, scale each local loss against the global denominator, or explicitly all-reduce correctly weighted gradient numerators. More ranks do not excuse a changed objective.

10.3.1 All-Reduce and the Replication Limit

All-reduce is a collective operation: it combines a tensor across a process group and makes the result available to every member. In DDP, the payload is usually the gradient buffer. A ring implementation of an all-reduce with payload q bytes moves approximately

$$q_{\mathrm{ring}} \approx 2\frac{n_D - 1}{n_D} q \tag{100}$$

bytes per rank, ignoring protocol overhead and using a common reduce-scatter plus all-gather decomposition. Other algorithms trade latency, bandwidth, and topology differently, but no all-reduce is free.

The group must finish the necessary collective before its replicas can safely take the next synchronized update.

DDP reduces the data and activation work per rank as n_D grows, but it does not reduce the replicated training state. Let P be the declared parameter count, let b_θ, b_g, and b_{opt} be bytes per parameter for parameters, gradients, and total optimizer state, and let M_{act} be local activation memory. A schematic DDP memory model is

$$M_{\text{DDP}} \approx P\big(b_\theta + b_g + b_{\text{opt}}\big) + M_{\text{act}}. \tag{101}$$

Adding DDP ranks can shorten the wall-clock time of a feasible model, but every rank must still hold this full state. This is why ordinary DDP eventually fails to make a larger model trainable even when more accelerators are available: the model state is replicated rather than sharded.

10.4 Tensor Parallelism

Tensor Parallelism partitions a computation **inside** a Transformer layer across a small group of ranks. It is model parallelism, not a data-replication scheme. Consider a linear map $Y = XW$ with $X \in R^{b\times d_{\text{in}}}$ and $W \in R^{d_{\text{in}}\times d_{\text{out}}}$. If W is split by output columns across n_T ranks, then

$$W = \Big[W_1, ..., W_{n_T}\Big], q \quad Y = \Big[XW_1, ..., XW_{n_T}\Big]. \tag{102}$$

Each rank computes a different feature slice of Y locally. This is a column-parallel linear layer. If instead the input features and corresponding rows of W are split, each rank forms a partial result and the full output requires a sum:

$$Y = \sum_{j=1}^{n_T} X_j W_j. \tag{103}$$

The right side of Equation 103 is normally materialized by an all-reduce. Transformer MLPs can arrange the expansion projection as column-parallel, apply its activation locally, then arrange the contraction as row-parallel. In Self-Attention, heads can likewise be divided across ranks before an output projection requires reconciliation. This intra-layer approach is the central Megatron-LM idea for training models that do not fit on one accelerator [49].

Tensor Parallelism reduces parameter and some activation storage per rank, but it introduces frequent collectives within the critical path of each layer. Enlarging n_T also reduces local matrix dimensions, which can make matrix multiplications less efficient. For this reason, Tensor Parallel groups are commonly kept modest and mapped to ranks with the highest available communication bandwidth. The exact group size is a performance decision, not a property of the Transformer equations.

10.5 Pipeline Parallelism

Pipeline Parallelism partitions model depth. With n_P stages, stage j owns a consecutive subset of the L Transformer blocks, receives activations from stage $j-1$, and sends its output activations to stage $j+1$. Backpropagation sends the corresponding activation gradients in the reverse direction. Unlike Tensor Parallelism, which cooperates within one layer, Pipeline Parallelism assigns different layers to different ranks. GPipe formalized the idea of executing layer partitions on separate accelerators with a split batch schedule [50].

If the full local batch is sent through all stages as one unit, all but one stage wait much of the time. Splitting it into M microbatches permits a stage to process one microbatch while another stage processes a different microbatch. Under an idealized pipeline with balanced stages, the utilization is approximately

$$u_{\text{pipe}} \approx \frac{M}{M + n_P - 1}, \quad 1 - u_{\text{pipe}} \approx \frac{n_P - 1}{M + n_P - 1}. \tag{104}$$

The second expression is the pipeline-bubble fraction. More microbatches improve utilization by amortizing the fill and drain periods, but they can increase activation memory, scheduling complexity, or the delay before one optimizer step completes. Pipeline schedules such as one-forward-one-backward change the activation-memory and timing trade-off, but they do not remove the dependency

boundaries between stages. Megatron-LM studies these schedules and the composition of pipeline, tensor, and data parallelism at large scale [51].

10.6 Sequence Parallelism

Sequence Parallelism partitions selected activation tensors along the sequence dimension across ranks that already participate in Tensor Parallelism. It is neither Data Parallelism nor a replacement for the attention computation described in Chapter 3. Its purpose is to avoid replicating activations for operations that act independently across token positions, such as dropout, residual operations, and normalization. When an operation needs the full representation in a different layout, the system uses the appropriate gather or reduce-scatter boundary.

This distinction matters because activation memory can dominate after parameter state has been divided. If a replicated activation has shape $B_{\text{local}} \times T \times d$, a sequence partition over n_T ranks can reduce the stored local slice toward $B_{\text{local}} \times \left(\frac{T}{n_T}\right) \times d$ for compatible operations. The saving is conditional: attention, tensor layouts, and communication boundaries determine which tensors can remain partitioned. Korthikanti et al. combine Sequence Parallelism with Tensor Parallelism to reduce activation-recomputation pressure in large Transformers [52].

10.7 Sharding with ZeRO and FSDP

ZeRO changes the replication policy inside a Data Parallel group. Rather than treating parameters, gradients, and optimizer state as three fully replicated objects, it shards them progressively. For a group of n_D ranks, the steady-state state terms in Table 12 are a useful first approximation. Activation memory and temporary communication buffers are additional terms.

Scheme	State placement in one Data Parallel group	Approximate persistent state per rank
DDP	Parameters, gradients, and optimizer state replicated.	$P\left(b_\theta + b_g + b_{\text{opt}}\right)$
ZeRO Stage 1	Optimizer state sharded; parameters and gradients replicated.	$P\left(b_\theta + b_g + \frac{b_{\text{opt}}}{n_D}\right)$
ZeRO Stage 2	Optimizer state and gradients sharded; parameters replicated.	$P\left(b_\theta + \frac{b_g + b_{\text{opt}}}{n_D}\right)$
ZeRO Stage 3 / full shard	Parameters, gradients, and optimizer state sharded.	$\frac{P\left(b_\theta + b_g + b_{\text{opt}}\right)}{n_D}$

Table 12: Persistent model-state accounting under ZeRO-style sharding. Full-shard methods temporarily materialize needed parameters for computation, so peak memory also includes gathered parameters, activations, and communication buffers.

ZeRO Stage 1 removes replicated optimizer state; Stage 2 also removes replicated gradients; Stage 3 shards parameters as well. The progression preserves Data Parallel training semantics while changing which rank owns each state element between collectives. Rajbhandari et al. introduced this staged state-partitioning approach to reduce model-state memory without requiring conventional model parallelism for every case [53].

Fully Sharded Data Parallel (FSDP) is a Data Parallel implementation of the full-shard idea. It does not mean that individual layer matrix multiplications are Tensor Parallel. Instead, a rank holds parameter shards at rest, all-gathers the full parameters of a wrapped module when that module must compute, and reduce-scatters or otherwise shards gradients after the corresponding backward work. A well-chosen wrapping boundary limits how much full parameter state is materialized at once. PyTorch FSDP is an implementation family designed around this behavior and its communication-memory trade-offs [54].

FSDP therefore differs from ordinary replicated DDP in its state ownership and communication schedule. DDP retains full parameter, gradient, and optimizer copies on every rank and mainly all-reduces gradients. FSDP retains shards and introduces parameter materialization around module execution. Both are Data Parallel in the sense that different ranks work on different data, but their memory and collective contracts differ substantially.

10.8 Communication, Overlap, and Scaling Efficiency

The arithmetic work in Chapter 9 is only one component of a training step. A compact communication model for n_{coll} collectives with total payload q is

$$t_{\text{comm}} \approx n_{\text{coll}}\alpha + q\beta, \tag{105}$$

where α is a latency-like cost per collective and β is an inverse-bandwidth-like cost per byte. The model is deliberately simple: real performance depends on the collective algorithm, placement, contention, and message shape. Its main lesson is durable. Many small collective operations can be latency-bound, whereas large gradient or parameter exchanges can be bandwidth-bound.

Backward propagation exposes gradients in reverse layer order. A DDP runtime can place ready gradients into buckets and start their all-reduces while backward computation continues through earlier layers. Tensor and FSDP implementations can similarly prefetch or overlap some communication. The useful step time is therefore not always $t_{\text{compute}} + t_{\text{comm}}$; it is closer to compute plus the portion of communication that remains uncovered. Overlap improves utilization only when enough independent computation exists and when communication is initiated early enough.

For a fixed workload, let t_1 be a comparable one-rank or lower-scale reference time and let t_W be the distributed time on W ranks. Strong-scaling efficiency is

$$\eta(W) = \frac{t_1}{W t_W}. \tag{106}$$

An ideal value is one. Actual efficiency falls because of collective communication, pipeline bubbles, load imbalance, data stalls, synchronization waits, and smaller local matrix multiplications. Consequently, theoretical division of FLOPs by W is not an execution prediction. Measured throughput and memory headroom should decide whether an additional parallel dimension is beneficial.

10.9 Failure, Synchronization, and Practical Strategy

Distributed training is synchronized state evolution. A collective mismatch—one rank calls a collective that another rank does not call, or ranks use incompatible tensor shapes—can deadlock or corrupt progress. A slow or failed rank can stall every replica at a synchronization point. A skipped optimizer step due to numerical control in Chapter 8 must be agreed upon by the relevant process group; allowing only some replicas to advance destroys parameter consistency.

Checkpointing must preserve more than model tensors. A sharded checkpoint needs the parallel configuration, shard layout, optimizer shards, scheduler state, random-state conventions, valid-token counters, and data-sampler state needed to resume the same global objective. Restoring under a changed world size can be supported only when the checkpoint format and resharding procedure explicitly permit it. A collection of rank-local files without this metadata is not a reliable recovery artifact.

A practical parallelism strategy begins with the bottleneck. If a complete training state fits per rank, replicated DDP is the simplest baseline. If model state does not fit, ZeRO or FSDP reduces replication. If one Transformer layer is too large or requires faster intra-layer computation, introduce a small Tensor Parallel group. If depth partitions remain necessary, add Pipeline Parallelism and choose enough microbatches to control Equation 104 without exhausting activation memory. Sequence Parallelism becomes attractive when replicated activations remain the limiting term. The Megatron-LM results illustrate that Tensor, Pipeline, and Data Parallelism can be composed, but the group sizes must be chosen for the measured communication and memory regime rather than copied as a universal configuration [51].

10.10 Implementation Contracts

The parallelism contract must declare the world size, every process group, and the mapping from rank to Data, Tensor, Pipeline, and Sequence Parallel roles. It must state which tensors are replicated, which are sharded, their shard axes, and the collective that reconstructs or reduces each tensor. All members of a process group must execute collectives in the same order with compatible shapes. A test that

uses a small deterministic batch should compare distributed logits, loss, selected gradients, and one optimizer update with a single-process reference within the expected numerical tolerance.

The objective contract preserves Chapter 5 exactly. It must define the microbatch-to-global-batch mapping in Equation 97, shard input examples without unintended duplication, and aggregate valid-token loss numerators and denominators consistently with Equation 99. Gradient accumulation, pipeline microbatches, and Data Parallel replicas are different clocks; the optimizer and learning-rate schedule from Chapter 7 advance once per declared global update, not once per local microbatch or rank.

The reliability contract coordinates finite-value detection, skipped updates, clipping, checkpoint barriers, and restart behavior across all relevant ranks. Monitoring should record per-rank and aggregate memory, collective wait time, communication volume, pipeline utilization, token throughput, update count, and valid-token count. These observables convert a slow or stalled job from an anecdote into a diagnosable execution trace.

10.11 Summary

Distributed training makes large-model pretraining feasible by separating two questions: how data, state, and computation are placed, and how ranks communicate the information needed for one coherent update. DDP replicates the model and averages gradients, which scales throughput but leaves replicated state as a memory limit. Tensor Parallelism splits layer computation; Pipeline Parallelism splits depth and pays bubble costs; Sequence Parallelism partitions compatible activation work. Their combination creates a multidimensional layout whose batch and world-size accounting must remain explicit.

ZeRO and FSDP reduce Data Parallel memory replication by sharding optimizer state, gradients, and eventually parameters, at the cost of additional communication and runtime complexity. The resulting system is not characterized by device count alone. Real scaling efficiency depends on collective cost, overlap, microbatching, load balance, memory peaks, and failure coordination. A correct training run is therefore one in which the distributed placement and synchronization rules preserve the same objective and optimizer trajectory intended by the single-process equations.

References

[48] S. Li *et al.*, "PyTorch Distributed: Experiences on Accelerating Data Parallel Training," *arXiv preprint arXiv:2006.15704*, 2020, doi: 10.48550/arXiv.2006.15704.

[49] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019, doi: 10.48550/arXiv.1909.08053.

[50] Y. Huang *et al.*, "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019. doi: 10.48550/arXiv.1811.06965.

[51] D. Narayanan *et al.*, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021. doi: 10.1145/3458817.3476209.

[52] V. A. Korthikanti *et al.*, "Reducing Activation Recomputation in Large Transformer Models," in *Proceedings of Machine Learning and Systems*, MLSys, 2023. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html

[53] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. doi: 10.1109/SC41405.2020.00024.

[54] Y. Zhao *et al.*, "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023, doi: 10.14778/3611540.3611569.

Chapter 11: Evaluation, Checkpointing, and Training Diagnostics

Abstract

Pretraining is an extended stochastic computation rather than a single optimization step. This chapter separates the losses that measure its progress from the signals that diagnose its execution, then develops a checkpoint as a resumable training state rather than a collection of model weights. It explains held-out validation, perplexity, online health metrics, loss spikes, failure localization, checkpoint cadence, sharded recovery, checkpoint selection, and experiment tracking. The central requirement is continuity: a monitor or restart is useful only when its counters, data identity, numerical state, and optimizer semantics remain explicit.

11.1 Introduction

Chapters 5 through 10 described the pretraining objective, data pipeline, optimizer, numerical controls, scale accounting, and distributed execution needed to update a language model. A long training run still needs an answer to a different question: how can its operators tell whether those components are producing the intended trajectory? The scalar loss is necessary evidence, but it is not a complete diagnosis. A training run can have a plausible loss while silently consuming an incorrect token stream, advancing its scheduler on the wrong clock, accumulating corrupted optimizer state, or failing to write a recoverable checkpoint.

Monitoring, evaluation, and checkpointing form one control loop. Monitoring records observations from the current step. Evaluation estimates performance on a protected distribution. A checkpoint preserves enough state to repeat the next update after interruption. The loop should detect deviations early, support isolation of their cause, and make recovery semantically meaningful. This chapter concerns pretraining health and reproducibility; it does not define downstream benchmark suites or post-training evaluation.

11.2 Training Loss, Validation Loss, and Perplexity

The training loss is the token-weighted Causal Language Modeling loss used to form gradients. Let $\mathcal{D}_{\text{train}}$ be the effective training distribution induced by the data mixture, packing policy, and loss mask, and let $\mathcal{D}_{\text{val}}$ be a fixed held-out evaluation distribution. For parameters θ_t at update t, write their population cross-entropies as

$$\mathcal{L}_{\text{train}(\theta_t)} = \mathrm{E}_{z\sim\mathcal{D}_{\text{train}}}[\ell(\theta_t; z)], \quad \mathcal{L}_{\text{val}(\theta_t)} = \mathrm{E}_{z\sim\mathcal{D}_{\text{val}}}[\ell(\theta_t; z)]. \tag{107}$$

The two quantities use the same token-level negative log-likelihood defined in Chapter 5, but they answer different questions. The training loss estimates the objective whose gradient changes θ_t. The validation loss estimates how that same probabilistic model scores examples withheld from the training stream. A validation set is not protected merely because it is stored in another file: it must be excluded from data mixture construction, deduplication decisions that would carry labels across the split, and any tuning procedure that would turn repeated evaluation into indirect training.

Neither loss is usually observed as the exact expectation in Equation 107. A training dashboard reports a finite, often short-window estimate whose variation includes sampling noise and changes in valid-token count. A validation event evaluates a finite, versioned slice under an explicit tokenizer, sequence policy, and loss mask. The evaluated token count and reduction convention must accompany both values; otherwise a lower number may merely reflect different padding, truncation, or masking.

11.2.1 Perplexity and Evaluation Distributions

For a token-averaged validation loss measured with natural logarithms, validation perplexity is

$$\mathrm{PPL}_{\text{val}} = \exp(\mathcal{L}_{\text{val}}). \tag{108}$$

This is the same transform introduced in Chapter 5, now applied to a protected distribution. Perplexity makes a log-loss easier to compare within one fixed evaluation protocol, but it is not a tokenizer-independent measure and is not a task score. Changing the vocabulary, normalization, sequence boundary convention, or valid-token mask changes the event space being measured.

Validation perplexity also differs from downstream benchmark performance. It measures the likelihood assigned to observed continuation tokens under data prefixes; a downstream benchmark can require a different prompt distribution, an answer-extraction rule, a decoding policy, or a task-specific scoring function. Work on language-model evaluation illustrates that likelihood alone does not exhaust the behavioral properties one may want to measure [55]. A pretraining team should therefore use validation loss to monitor the declared objective while keeping downstream claims separate from it.

11.3 Evaluation Cadence and Online Metrics

Evaluation is not free. It consumes accelerator time, data-loader capacity, and attention from the operators who must interpret the result. Evaluating too rarely can let a bad trajectory consume a large token budget before it is recognized; evaluating too frequently can materially reduce training throughput or lead operators to overreact to small fluctuations. A practical cadence is defined in valid tokens or optimizer updates, records its own clock, and includes occasional larger evaluations in addition to lightweight frequent probes.

The online metrics surrounding a loss value provide its context. The learning rate and optimizer-step count explain the intended update scale. The consumed valid-token count connects a curve to Chapter 9's token budget rather than to an implementation-dependent batch count. Throughput exposes stalls even when loss is healthy. Gradient norms, activation summaries, logit ranges, finite-value counts, and optimizer-state summaries expose failures before they necessarily appear as a divergent scalar loss. Chapter 8 explains why these reductions require a declared precision policy.

The point is not to log every tensor indiscriminately. A durable log separates fast scalar metrics from sampled distributional summaries. For example, record the training loss numerator and valid-token denominator, global gradient norm, learning rate, tokens per second, and finite-value flag at every update or a short interval. Record per-layer activation RMS, maxima or high quantiles, parameter norms, and moment statistics at a less frequent schedule or around an anomaly. Each series needs a stable name, unit, reduction rule, and association with the exact training state.

11.4 Diagnosing Spikes and Divergence

A loss spike is an observation, not a diagnosis. A **transient** spike is an unusually high loss that is followed by a return to the prior range while parameters, gradients, and optimizer state remain finite. **Sustained divergence** is a persistent deterioration, often accompanied by growing norms, repeated nonfinite values, stalled throughput, or a failure to recover after the next scheduled updates. The distinction matters because an automatic rollback for every outlying mini-batch can discard useful stochastic progress, while continuing through a corrupted update can make later evidence uninterpretable. Loss spikes have been documented as a practical problem in large-language-model pretraining; Takase et al. connect one important class to sudden gradient-norm growth [56].

The first task is to localize the earliest abnormal boundary. A data failure may be restricted to particular source records, malformed token sequences, unexpected loss masks, or a shifted mixture component. An optimization failure can follow a learning-rate transition, a batch-size change, a warmup-clock error, or a growing gradient norm while arithmetic remains finite. A numerical failure is indicated by NaN or Inf values, loss-scale backoff, or strong sensitivity to widened arithmetic. A distributed failure can show as inconsistent rank-local counters, collective stalls, or an update taken by only part of a process group. A checkpoint failure can show only after restore, when missing state produces a discontinuity that the saved weights alone cannot explain.

Observed signal	First scope-preserving check	Failure class to investigate
One high but finite loss	Compare the exact batch, valid-token denominator, logit range, and learning-rate state with the preceding update.	Data anomaly, stochastic variation, or an optimization transition.
Repeated rising loss or norms	Hold the recipe fixed and inspect the first layer, parameter group, or schedule boundary that departs from baseline.	Optimization or architectural-scale instability.
NaN or Inf	Locate the first nonfinite tensor; rerun the preserved batch with selected reductions widened.	Numerical failure, invalid input, or a nonfinite update entering state.
Rank disagreement or stall	Compare rank-local counters, tensor shapes, and collective order before any restart.	Distributed execution or data-sharding fault.
Loss discontinuity after restore	Compare model, optimizer, scheduler, scaler, random, and sampler states against checkpoint metadata.	Incomplete or inconsistent checkpoint.

Table 13: A diagnostic table should preserve scope before changing a hyperparameter. The earliest abnormal boundary narrows a failure class; it does not establish a cause by itself.

Table 13 is deliberately ordered by tests that preserve the existing experiment. Lowering the learning rate, changing precision, or discarding a data source can be useful isolation experiments, but each also changes the trajectory. Preserve the failing batch identifiers, parameter revision, optimizer state, random state, data manifest, parallel configuration, and recent traces before applying a mitigation.

11.5 Checkpoints as Resumable Training States

A weight snapshot answers a narrow question: which parameters produced this model output? A resumable checkpoint answers a stronger question: which state is required for the next update to have the intended semantics? Let Ξ_t denote the optimizer state, σ_t the learning-rate scheduler state, s_t the gradient-scaler state when applicable, ρ_t the random-number-generator states, d_t the data-loader or sampler state, and π_t the distributed layout and sharding metadata. A conceptual checkpoint at update t is

$$\mathcal{C}_t = (\theta_t, \Xi_t, \sigma_t, s_t, \rho_t, d_t, \pi_t, t, N_{\text{consumed}}, \mu), \tag{109}$$

where N_{consumed} is the consumed valid-token count and μ records immutable run identity such as the model configuration, tokenizer version, data manifest, code revision, and precision policy. Some systems also record an in-progress gradient-accumulation state, pending asynchronous writes, or an evaluation cursor. The exact container varies, but the distinction in Equation 109 does not: model parameters are one component of the future computation.

The optimizer state matters because AdamW's first and second moments determine the next update, as Chapter 7 derives. Scheduler state matters because an update-indexed or token-indexed learning rate cannot be reconstructed safely from a vague training duration. The scaler state matters when dynamic loss scaling is active. Random states and sampler state matter because dropout, data order, augmentation, and worker initialization can change the subsequent gradient sequence. Restoring only θ_t can create a useful inference model, but it does not normally recreate the same training trajectory.

11.5.1 Full and Sharded Checkpoints

A **full** checkpoint materializes a logically complete state in a layout that can be read by one process or by a standard loading path. It is convenient for archival, interchange, or offline inspection, but gathering a large sharded model merely to write it can exceed memory or produce an expensive write bottleneck. A **sharded** checkpoint stores partitions across ranks or files, preserving the distributed placement needed for scalable I/O. It reduces gathering pressure, but its manifest must identify every shard, the model and optimizer mappings, the parallel layout, and the procedure for reconstructing or resharding the state.

Large-language-model checkpointing therefore trades storage, write interruption, recovery time, and implementation complexity. Let t_{save} be the training-visible cost of one checkpoint, let τ be a time interval between checkpoints, and let λ be an approximate failure rate per unit time. A simple steady-state accounting for the fractional overhead is

$$h(\tau) \approx \frac{t_{\text{save}}}{\tau} + \frac{\lambda \tau}{2}. \tag{110}$$

The first term rewards less frequent saves; the second represents, in expectation, the work lost since the most recent save after a failure. This is a planning model, not a universal formula: failures are not always memoryless, restores have a cost, and an asynchronous save can overlap some of t_{save}. Its purpose is to make the trade-off explicit. LLM checkpointing systems must also account for the large, distributed model and optimizer state that turns a naive frequent write into a substantial I/O burden [57].

11.5.2 Resume Semantics and Checkpoint Validation

An **exact** resume is a demanding target: after reload, the next batch, masks, random draws, optimizer update, and counters agree with an uninterrupted run up to the stated numerical tolerance. This requires more than setting one global seed. It requires that data workers, rank-local random streams, sampler positions, accumulation boundaries, loss normalization, and distributed collectives be restored with compatible conventions. Bitwise identity can remain impossible across changed kernel versions, hardware, or collective-reduction order; this should be stated as a limitation rather than hidden as an unexplained drift.

A **semantic** resume is weaker but still meaningful. It preserves the model, optimizer, scheduler, data policy, and declared global counters even if a supported resharding operation changes the number of ranks or shard boundaries. The new run is not claimed to reproduce every random bit, but it remains an explicit continuation of the same objective. A change in world size is not automatically safe: the checkpoint format and loader must specify how parameter and optimizer shards map to the new layout. Modern distributed checkpoint interfaces provide such resharding mechanisms only when the state representation and metadata support them [58].

Checkpoint validation should occur before a failure makes it urgent. A writer should complete a manifest only after every required shard is durable and verifiable. A reader should check schema and version compatibility, file existence or checksums, tensor keys and shapes, finite values, optimizer parameter-group identity, and counter monotonicity. Periodically restore a recent checkpoint into an isolated process, run a small deterministic evaluation, and confirm that the measured loss and selected state summaries match the saved record. A checkpoint that has never been loaded successfully is a storage artifact, not a recovery plan.

11.6 Checkpoint Selection and Stopping Decisions

The checkpoint selected for an application need not be the most recent checkpoint. If the declared target is held-out Causal Language Modeling loss on a fixed validation distribution, select using a versioned validation protocol and retain the evaluation record with the artifact. If the target is a downstream capability, the selection criterion must be a separate task-level evaluation; it cannot be inferred from perplexity alone. Ranking checkpoints on a repeatedly inspected validation set also creates selection pressure, so the final report should distinguish the development validation stream from any independently protected test or benchmark evaluation.

Classical early stopping halts training when validation performance no longer improves, using held-out data to avoid continuing into overfitting. Its criterion and patience window are themselves choices [59]. In large-scale pretraining, early stopping is less straightforward. The run may be planned around a fixed token or compute budget, validation loss may continue to improve slowly long after a narrow patience rule would trigger, evaluations can be expensive, and the available corpus may be far from exhausted. A stopped run may also be inferior under a later deployment objective even when it is locally best on one validation slice.

Early stopping is therefore a decision rule, not a default proof of generalization. It is most defensible when a protected metric has sustained deterioration, data reuse or distribution shift makes continued optimization harmful, or a controlled budget objective explicitly favors the best validation checkpoint. For a long planned pretraining run, validation should more often guide investigation and checkpoint retention than act as an unexamined stop signal.

11.7 Experiment Tracking and Reproducibility

Reproducibility begins with identity rather than with a random seed. Every measured curve should be associated with an immutable run configuration: architecture and parameter-count convention, tokenizer, source and data-manifest versions, mixture and packing policy, loss mask, optimizer and schedule, precision policy, distributed layout, code revision, hardware or runtime version where material, and evaluation datasets. The valid-token counter provides the common time axis connecting training loss, validation loss, learning rate, throughput, and checkpoints.

An experiment record should distinguish configuration from observation. Configuration specifies what the run intended to do. Observation records what it did: losses with numerators and denominators, gradient and activation summaries, learning rate, optimizer updates, consumed tokens, throughput, checkpoint identifiers, failures, restarts, and any manual intervention. This separation makes it possible to compare two runs that share a model name but not a data stream or schedule, and to diagnose why their outcomes differ.

Reproducibility is not an assertion that every implementation reproduces every low-order bit. It is the ability to reconstruct the declared computation closely enough to audit its trajectory, identify controlled differences, and resume it under stated semantics. That standard is stronger than archiving weights and more realistic than promising hardware-independent determinism.

11.8 Implementation Contracts

The metric contract must define the training and validation data identities, tokenizer version, sequence and masking policy, token-level loss numerator, valid-token denominator, evaluation cadence, and the counter that indexes each value. It must prevent evaluation examples from entering the training mixture and record any validation-set revision. Logged perplexity should be derived from the same natural-log token average as Equation 108, not from an incompatible sequence average.

The health contract should log learning rate, optimizer update, consumed valid-token count, throughput, loss, gradient norm, finite-value status, and checkpoint identifier under stable reduction rules. It should sample activation, logit, and optimizer-state statistics at a declared cadence, preserve enough recent history to analyze a spike, and make rank-local versus globally reduced metrics explicit. A skipped update or rollback must be recorded with its reason and agreed across the relevant distributed group.

The checkpoint contract must specify the complete state in Equation 109, whether the artifact is full or sharded, its manifest and integrity checks, the save-completion boundary, and its exact or semantic resume target. Test both a direct reload and a failure-style restore in which the process is recreated, data is reinitialized, and the next optimizer step is compared against a controlled reference. Finally, link every selected checkpoint to the validation protocol and experiment record that justified it.

11.9 Summary

Training loss and validation loss measure related but different distributions: one drives updates, while the other estimates how the pretraining objective transfers to protected examples. Perplexity is a monotonic transform of token-averaged validation loss under a fixed protocol; it is not a substitute for task-specific evaluation. A reliable run therefore interprets loss together with token counts, learning rate, throughput, gradient and activation statistics, finite-value checks, and distributed health.

Checkpointing turns monitoring into recoverability only when the artifact contains the state that determines the next update. Parameters alone are not enough: optimizer and scheduler state, scaler state where applicable, random and sampler state, consumed-token count, configuration identity, and sharding metadata define whether a restart is exact or merely a new run from old weights. With

validated checkpoints, protected evaluation, and explicit experiment records, training diagnostics become a disciplined way to preserve and investigate a pretraining trajectory rather than a collection of dashboard curves.

References

[55] C. Meister and R. Cotterell, "Language Model Evaluation Beyond Perplexity," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, 2021, pp. 5328–5339. doi: 10.18653/v1/2021.acl-long.414.

[56] S. Takase, S. Kiyono, S. Kobayashi, and J. Suzuki, "Spike No More: Stabilizing the Pre-training of Large Language Models," *arXiv preprint arXiv:2312.16903*, 2023, doi: 10.48550/arXiv.2312.16903.

[57] A. Maurya, R. Underwood, M. M. Rafique, F. Cappello, and B. Nicolae, "DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024. doi: 10.1145/3625549.3658685.

[58] PyTorch Contributors, "Distributed Checkpoint." 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/distributed.checkpoint.html

[59] L. Prechelt, "Early Stopping—but When?," in *Neural Networks: Tricks of the Trade*, vol. 1524, in Lecture Notes in Computer Science, vol. 1524., Springer, 1998, pp. 55–69. doi: 10.1007/3-540-49430-8_3.

Chapter 12: Practical FSDP and Distributed Training State

Abstract

Fully Sharded Data Parallel (FSDP) makes a model whose training state would not fit on one accelerator executable by keeping parameter, gradient, and optimizer-state shards at rest and materializing a layer only when computation requires it. This chapter develops the resulting execution lifecycle, the transient memory peaks that determine whether a job fits, and the state-dictionary and checkpoint contracts needed for recovery at scale. It treats FSDP as a systems pattern rather than a framework recipe, while using established PyTorch terminology where that terminology makes the operational model precise.

12.1 Introduction

Chapter 10 introduced Fully Sharded Data Parallel (FSDP) as a form of Data Parallelism that replaces persistent replication with sharding. That conceptual distinction is necessary but not sufficient for operating a large training job. An FSDP configuration can have a small **steady-state** footprint and still fail because an All-Gather, a prefetched unit, a full checkpoint export, or an optimizer-state conversion creates a transient peak. Conversely, a configuration that fits can be slow because its sharding boundaries issue too many small collectives or expose communication that the backward pass cannot hide.

This chapter concerns the operational state machine behind those outcomes. Its main setting is a dense model trained with a sharded Data Parallel group of size n_F. Tensor, Pipeline, and Sequence Parallelism may coexist with this group, as Chapter 10 explains, but they are not re-derived here. Likewise, the optimizer semantics and numerical-format requirements remain those of Chapters 7 and 8. The question here is narrower: when model state is distributed, which state lives where, when is a full unit reconstructed, and what must be preserved for the next update or a later restart to retain the intended semantics?

12.2 FSDP as an Execution Lifecycle

Let a model have P parameters, partitioned into FSDP units $U_1, \ldots, U_J$. A unit is a module or explicitly chosen parameter group that will be materialized and released together. At rest, every rank holds a shard of each unit. Ignoring padding and metadata, the parameter component of one rank's persistent model state is approximately $\frac{P}{n_F}$ parameters rather than P. The same sharding principle can apply to gradients and optimizer state. This is the memory idea shared by ZeRO-style stage-3 sharding and FSDP [60], [61].

The reduction is not obtained by making a layer's computation incomplete. Before a rank can execute a local forward operation for a unit, it needs the unit's logical parameters. In a common full-shard schedule, the rank All-Gathers those parameter shards, performs the local forward computation, and releases the reconstructed parameters. Before the corresponding backward computation, it All-Gathers the unit again, forms gradients locally, then Reduce-Scatters the gradient so that each rank retains only its gradient shard. The optimizer updates local parameter and optimizer-state shards. Schematically,

sharded at rest → All-Gather → local forward → reshard
→ All-Gather → local backward → Reduce-Scatter → local update

The exact release points, flattening strategy, and collective implementation vary by runtime. The invariant is more important than any one interface: every operation that requires a complete unit has an explicit materialization boundary, and every state that need not remain replicated is returned to a shard after that boundary. The FSDP design documents this pattern as a practical implementation of fully sharded training [61].

12.2.1 All-Gather and Reduce-Scatter

An **All-Gather** concatenates or otherwise reconstructs the shards of a tensor so that every participant can access the full logical tensor. In this chapter, its usual payload is a unit's parameters. A **Reduce-Scatter** first combines corresponding gradient contributions across ranks and then gives each rank

only its assigned output shard. It therefore supplies the same averaged gradient information that replicated Data Parallelism needs while avoiding a full gradient replica after the collective.

For a unit with P_U parameters stored at b_θ bytes each, the full parameter materialization is on the order of $P_U b_\theta$ bytes per participating rank, while a persistent parameter shard is on the order of $P_U \frac{b_\theta}{n_F}$. Those expressions are accounting aids, not universal allocations: a runtime may pad flattened buffers, retain a communication workspace, use a different dtype for a temporary buffer, or overlap several units. The practical consequence is nevertheless robust. Sharding reduces state held **between** computations; it does not remove the temporary memory and network traffic required to reconstruct a unit.

Chapter 10 derives why a collective must use a consistent process group and tensor shape on all of its participants. FSDP adds a lifecycle requirement: ranks must also issue the collectives in a compatible unit order. One rank skipping a wrapped unit, taking a conditional branch that changes the sequence, or using a different wrapping plan can turn a local program error into a collective hang.

12.3 Units, Wrapping, and Memory Peaks

The choice of FSDP units, often called a **wrapping policy**, is a central systems decision. A unit should be large enough that its communication is not dominated by per-collective latency, but small enough that reconstructing it does not dominate the peak memory. Transformer blocks are frequently useful boundaries because they contain substantial computation and a coherent local parameter set. This is a design heuristic, not a rule: embeddings, unusually large layers, tied weights, and nested model structure can motivate different boundaries.

If one root wrapper contains the entire model, the forward pass may materialize a very large parameter set at once. If every tiny leaf is wrapped independently, the run can issue a large number of small All-Gathers and Reduce-Scatters, increasing launch overhead and making overlap difficult. Nested or block-level units create a middle ground in which the runtime can free one reconstructed unit before materializing the next. A wrapping plan is therefore a memory-and-communication schedule, not merely a way to annotate modules.

Let M_{state} denote the resident sharded parameters, gradient shards, and optimizer-state shards on one rank. A useful peak-memory decomposition is

$$M_{\text{peak}} \approx M_{\text{state}} + M_{\text{materialize}(U_{\text{current}})} + M_{\text{prefetch}} + M_{\text{act}} + M_{\text{temporary}}. \tag{111}$$

Here $M_{\text{materialize}(U_{\text{current}})}$ includes the full parameter and any transient gradient buffers needed for the current unit; M_{prefetch} is the state deliberately reconstructed ahead of use; M_{act} is the activation footprint; and $M_{\text{temporary}}$ covers kernels, communication workspaces, allocator fragmentation, and runtime bookkeeping. The first term scales roughly as

$$M_{\text{state}} \approx \frac{P}{n_F}\left(b_\theta + b_g + b_{\text{opt}}\right), \tag{112}$$

where b_{opt} is the total optimizer-state bytes per parameter. This deliberately compact model excludes activations and temporary full units; treating it as an exact device-memory prediction is a common source of failed capacity plans.

12.3.1 Mixed Precision, Activation Checkpointing, and Offload

FSDP composes with, rather than replaces, the precision policy from Chapter 8. A common design uses a reduced-precision parameter representation for forward and backward compute while retaining the optimizer's update-critical state in a wider format. The bytes in Equation 112 must be counted by actual tensor role: a BF16 model copy, FP32 optimizer moments, reduced-precision gradients, and communication buffers are not interchangeable merely because they refer to the same logical parameter. A configuration should specify which dtype is used for parameters, gradient reduction, optimizer state, and accumulation.

Activation checkpointing addresses a different term in Equation 111. Instead of saving selected forward activations for backward, it saves a smaller boundary state and recomputes the omitted forward portion during backward. This exchanges extra computation for lower M_{act}; it neither shards

parameters nor makes an insufficient FSDP unit safe by itself. The general recomputation principle was formalized by Chen et al. [62]. A capacity investigation should distinguish an activation peak from a parameter-materialization peak before enabling both mechanisms indiscriminately.

CPU offload is another capacity tool, not a free memory optimization. Moving inactive parameter, gradient, or optimizer state to host memory can reduce accelerator residency, but the state must cross a host–device interconnect before use. The transfer can expose latency, contend with data loading, move optimizer computation to the host in some designs, and make an otherwise hidden communication bottleneck dominant. It is most defensible when accelerator capacity is the binding constraint and the resulting transfer schedule has been measured; official FSDP interfaces expose offload controls for this reason [63].

12.4 Communication Scheduling and Overlap

The execution lifecycle above contains communication that is useful only if it arrives before the corresponding computation. The step time can be summarized as

$$t_{\text{step}} \approx t_{\text{compute}} + t_{\text{uncovered-communication}} + t_{\text{recompute}} + t_{\text{serialization}}, \tag{113}$$

where $t_{\text{uncovered-communication}}$ is the portion of collective time that cannot overlap with useful computation. FSDP does not eliminate communication relative to a replicated model; it changes which tensors move and when. A job can therefore be memory-efficient but network-bound.

Forward prefetching starts an All-Gather for a future unit before its forward computation is needed. **Backward prefetching** does the analogous work for an upcoming backward unit. Both can reduce $t_{\text{uncovered-communication}}$ when the execution order is predictable and there is enough computation to hide the transfer. Both also increase M_{prefetch}, and overly aggressive prefetching can convert a communication optimization into an OOM. Current framework documentation explicitly describes this overlap–peak-memory trade-off and its dependence on a stable execution order [63].

The right order of diagnosis is usually: establish a correct non-prefetched baseline; measure collective time and rank imbalance; then introduce a bounded amount of prefetching. Do not infer success from a lower profiler value alone. The candidate must still preserve full-batch semantics, stay within peak memory under the longest sequences, and improve end-to-end valid tokens per second rather than merely shifting idle time from one stream to another.

12.4.1 Memory and Communication Trade-offs

Several control choices move a job along a three-way frontier rather than producing an unconditional improvement. Larger FSDP units reduce collective count but enlarge materialization peaks. More aggressive prefetching can hide network time but retains more full units. Activation checkpointing lowers activation memory but adds recomputation. CPU offload releases accelerator memory but creates host-device traffic. Higher sharding degrees reduce resident state per rank but can increase the number of communication participants and alter the effective network topology.

Control	Primary benefit	Primary cost	First measurement
Smaller FSDP units	Lower full-parameter peak and earlier release.	More collectives and less payload efficiency.	Peak memory and collective count.
Forward or backward prefetch	More communication overlap.	Additional materialized state and order sensitivity.	Uncovered communication and peak memory.
Activation checkpointing	Lower saved-activation memory.	Forward recomputation during backward.	Activation peak and step time.
CPU offload	Lower accelerator-resident state.	Host-device transfer and possible CPU-side work.	Transfer stalls and throughput.

Table 14: FSDP controls alter distinct terms in the memory and time model. A measurement should test the claimed benefit and the compensating cost together.

Table 14 also explains why copying a configuration from a model with a different block size, sequence length, network, or optimizer can fail. The useful configuration is a measured point for a declared architecture and workload, not a collection of enabled flags.

12.5 State Dictionaries and Distributed Checkpoints

Chapter 11 defines a resumable checkpoint as more than model weights. FSDP makes the representation of that state an additional operational choice. A **state dictionary** is a named logical view of model or optimizer state. It may be materialized in a full form, retained as shards, or produced in a rank-local form for a particular runtime. The logical model must be the same, but the memory, I/O, and portability properties of these views differ substantially.

A **full model state dictionary** reconstructs each parameter in an unsharded form. It is convenient for export, evaluation in a non-sharded environment, and interoperability with tools that expect ordinary parameter tensors. It may, however, trigger a large gathering operation and should not be silently requested on every rank. A **sharded model state dictionary** preserves distributed pieces plus enough metadata to identify the logical tensor. It is the natural scalable representation for a large training restart, provided the reader understands the shard mapping.

Optimizer state requires the same care. AdamW moments and parameter-group metadata are attached to logical parameters, while a running job may store them as local shards or flattened buffers. A checkpoint path must provide an unambiguous mapping from the saved optimizer tensors to the logical model parameters. Saving a full model state together with rank-local optimizer shards without declaring that mismatch may create an artifact that looks complete but cannot resume. The FSDP experience report emphasizes the importance of handling optimizer state alongside fully sharded model state [61].

State view	Natural use	Operational advantage	Primary risk
Full state dictionary	Export, inspection, or non-sharded loading.	Simple logical tensors for generic consumers.	Gather peak, duplicate host copies, or single-rank I/O bottleneck.
Sharded state dictionary	Large-scale save and distributed restart.	No mandatory global materialization; parallel I/O.	Requires manifest, tensor layout, and reader support.
Rank-local runtime state	Fast restart under the same controlled topology.	Can match the live storage layout closely.	Weak portability unless its mapping is made explicit.

Table 15: State-dictionary views expose the same logical training state with different materialization and recovery properties. The table is a systems taxonomy, not a promise that every framework supports every view identically.

12.5.1 Save, Restore, and Resharding

A distributed checkpoint writer should serialize shards in parallel, record a manifest only after all required pieces are durable, and preserve the model-state, optimizer-state, scheduler, scaler, random-state, data-state, and counter requirements from Chapter 11. The manifest must additionally identify the logical tensor names, global shapes, dtypes, shard offsets or placement, parameter-group mapping, and the parallel configuration that produced the artifact. These are not optional administrative details: without them, a receiver cannot know whether a local shard belongs to a particular global parameter or whether an optimizer moment has the correct correspondence.

Restore should begin by constructing the intended target state and validating the checkpoint schema before training resumes. A small post-load evaluation, finite-value check, and comparison of global counters can catch a bad mapping before the first costly update. If the new run uses a different FSDP world size, a supported loader may redistribute a logical global tensor into new shards. This **resharding** is distinct from merely copying files to a different number of processes: it requires global metadata and a coordinated data movement plan. Distributed checkpoint interfaces explicitly support load-time resharding only when the saved representation and destination state make that mapping well defined [64].

Changing world size normally weakens an exact-resume claim. Random streams, data-worker assignments, collective reduction order, and local batch decomposition can differ even when model and optimizer tensors are resharded correctly. The appropriate target is usually the semantic-resume contract of Chapter 11: preserve the declared objective and update state, log the topology change, and state the reproducibility tolerance instead of implying bitwise continuity.

12.6 Diagnosing FSDP Failures

An FSDP OOM is meaningful only when its timing and allocation class are known. An OOM at initialization may indicate that an initial full model, a checkpoint load, or an optimizer is being materialized before sharding. An OOM just before a wrapped unit's forward often implicates All-Gather payload, wrapping granularity, or prefetched parameters. An OOM in backward can arise from saved activations, full gradient buffers before Reduce-Scatter, optimizer workspace, or several units being retained concurrently. A late OOM during export may be a full state-dictionary gather rather than a training-step capacity failure.

Observed boundary	First scope-preserving check	Likely class of cause
Before first training step	Record which model and optimizer tensors exist before and after sharding or checkpoint load.	Initialization materialization, optimizer construction, or full-state restore.
At unit forward	Trace current and prefetched unit sizes and allocator peak.	Coarse wrapping, too much prefetch, or communication workspace.
During backward	Separate saved activations, reconstructed parameters, gradients, and recomputation buffers.	Activation peak, gradient materialization, or retained units.
Low throughput without OOM	Measure collective wait, payload sizes, and rank skew around unit boundaries.	Exposed communication, small units, topology mismatch, or load imbalance.
Hang or rank error	Compare unit order, collective shape, process groups, and conditional execution across ranks.	Collective-order divergence or incompatible sharding plan.
Resume discontinuity	Validate model and optimizer mappings, counters, and sharding metadata before the next update.	Incomplete state dictionary, corrupt shard, or unsupported resharding.

Table 16: FSDP failures should be localized to a lifecycle boundary before altering a sharding policy or learning rate. The first check preserves evidence needed to identify the responsible state transition.

Communication diagnosis should report more than a single aggregate bandwidth number. Measure time spent waiting for All-Gather and Reduce-Scatter, payload sizes by unit, overlap with computation, rank-to-rank variance, sequence-length distribution, and any interaction with other parallel dimensions. A long tail on one rank can leave every peer idle at the next collective. Conversely, a high communication percentage may be acceptable if the collective is mostly hidden and the end-to-end valid-token rate improves. The correct objective is a stable training step at the intended global batch and token accounting, not the isolated minimization of one kernel or one collective.

12.7 Implementation Contracts

The sharding contract must name the FSDP process group, unit boundaries, logical parameter ownership, parameter and gradient reduction dtypes, optimizer-state layout, and the points at which state is materialized and released. It must state how the plan composes with Data, Tensor, Pipeline, or Sequence Parallel groups, and it must make tied parameters and conditional module execution explicit. Every rank in a group must agree on the collective order, tensor shapes, and loss-normalization semantics.

The memory contract must be evaluated at the workload's real high-water mark: maximum permitted sequence length, target microbatch, gradient-accumulation setting, prefetch configuration, activation-checkpoint policy, optimizer initialization, and checkpoint path. It should separately record resident sharded state, activation peak, full-unit peak, communication workspace, and allocator reserve. A capacity claim based only on Equation 112 is incomplete.

The recovery contract must identify whether each artifact is full, sharded, or rank-local; provide a durable manifest for every sharded tensor; map optimizer state to logical parameters; and record the model configuration, scheduler, scaler, random and sampler states, data position, consumed-token count, and parallel topology. It must test both an ordinary restart and any supported world-size change. A successful save is not evidence of a usable checkpoint until a fresh process can load it, validate the state, and execute a controlled next step.

12.8 Summary

FSDP reduces persistent model-state replication by storing parameter, gradient, and optimizer shards across a Data Parallel group. Its practical behavior is governed by a lifecycle: reconstruct a unit with All-Gather when local computation needs it, release it when possible, and Reduce-Scatter gradients so that optimizer updates remain local to shards. The resulting steady-state memory reduction does not remove transient full-unit, activation, workspace, and prefetch peaks.

Wrapping determines the granularity of that lifecycle; mixed precision, activation checkpointing, offload, and prefetch each change a different term in the memory–communication–compute balance. Checkpointing must expose the logical model and optimizer state with enough distributed metadata to restore or, where supported, reshard it safely. By measuring lifecycle boundaries and treating state representation as part of the training algorithm, a team can use FSDP as a controlled distributed-training system rather than as a collection of opaque memory-saving options.

References

[60] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. doi: 10.1109/SC41405.2020.00024.

[61] Y. Zhao *et al.*, "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 3848–3860, 2023, doi: 10.14778/3611540.3611569.

[62] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training Deep Nets with Sublinear Memory Cost," *arXiv preprint arXiv:1604.06174*, 2016, doi: 10.48550/arXiv.1604.06174.

[63] PyTorch Contributors, "FullyShardedDataParallel." 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/fsdp.html

[64] PyTorch Contributors, "Distributed Checkpoint." 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/distributed.checkpoint.html

Part IV — Post-training, Alignment, and Adaptation

Chapter 13: Supervised Fine-Tuning

Abstract

Supervised Fine-Tuning adapts a pretrained decoder-only language model to instruction following by training on demonstrations of desired responses. This chapter shows that SFT retains the autoregressive next-token machinery of pretraining while changing the data distribution, conversation serialization, and set of target tokens. It develops assistant-only loss masking for multi-turn conversations, treats Chat Templates as versioned model-data contracts, examines packing and data-mixture choices, introduces full-parameter adaptation and LoRA, and concludes with failure modes, evaluation criteria, and implementation contracts.

13.1 Introduction

Pretraining produces a model that assigns probabilities to text continuations. That capability is broad, but the pretraining distribution does not uniquely specify how the model should respond when a user asks a question, requests a transformation, or supplies a system instruction. A web corpus contains answers, questions, dialogue fragments, quoted conversations, unfinished pages, and many other continuation patterns. Even a strong base model can therefore continue an instruction instead of answering it, imitate both sides of a dialogue, or choose a response style that is plausible under the corpus but unsuitable for an assistant.

Post-training narrows this behavioral ambiguity. Its first major stage is commonly **Supervised Fine-Tuning** (SFT): continue optimizing a pretrained model on examples that pair an instruction or conversational context with a desired assistant response. Early instruction-tuning work demonstrated that training across many tasks expressed through natural-language instructions can improve generalization to unseen tasks [65]. InstructGPT likewise began its post-training pipeline with supervised demonstrations before introducing preference-based stages [66].

This chapter isolates SFT. It does not introduce Reward Models, RLHF, PPO, DPO, GRPO, or Reasoning RL. The central question is more basic: what statistical problem is solved when a pretrained Causal Language Model is trained to imitate high-quality instruction-response demonstrations?

13.2 From Pretraining to Instruction Following

Chapter 5 defined pretraining as maximum likelihood over token sequences drawn from a broad corpus distribution. SFT normally retains the same decoder, language-model head, causal Attention Mask, teacher-forced inputs, and token-level cross-entropy. What changes is the distribution of sequences presented to the model and, often, the subset of their tokens that contributes to the loss.

Let $\mathcal{D}_{\text{pre}}$ denote the effective pretraining distribution and $\mathcal{D}_{\text{SFT}}$ the distribution induced by curated demonstrations. Pretraining estimates conditionals that are useful throughout $\mathcal{D}_{\text{pre}}$. SFT places additional probability mass on sequences in which a structured prompt is followed by a desired response. The optimization objective remains next-token prediction, but the observed conditional patterns now repeatedly associate instructions and role markers with assistant behavior.

This distinction separates an **objective** from the behavior induced by its data. Next-token prediction is the local likelihood objective in both stages. Instruction following is not a different output head or a symbolic execution rule added to the model; it is behavior learned from the conditional distribution represented by SFT examples. The model is shown that after a serialized system-and-user context, certain response tokens should receive high probability. Data coverage, formatting, target selection, and weighting therefore determine which notion of "following" the model learns.

SFT should not be described as supplying all knowledge to an otherwise empty model. The pretrained parameters already encode linguistic and factual regularities; SFT teaches the model how to expose and organize those capabilities under a new interaction protocol. LIMA provides a deliberately narrow empirical illustration: a strong base model acquired substantial response behavior from only 1,000 carefully curated demonstrations [67]. That result does not imply that every model or domain requires

little data, but it makes clear that example quality and the pretrained starting point are inseparable from SFT scale.

13.3 Supervised Fine-Tuning Objective

An SFT record begins as structured data rather than as a token tensor. For record n, let

$$c^n = \left(q_1^n, q_2^n, ..., q_{J_n}^n\right) \tag{114}$$

be an ordered sequence of J_n messages. Each message $q_j = \left(r_j, u_j\right)$ has a role r_j, such as `system`, `user`, or `assistant`, and content u_j. A deterministic serialization function φ applies a Chat Template, and the tokenizer τ then produces

$$x^n = \tau(\varphi(c^n)) = \left(x_1^n, ..., x_{T_n}^n\right). \tag{115}$$

The message structure has disappeared by the time the Transformer receives x^n; the model operates on one token sequence. The preprocessing pipeline must therefore preserve a parallel record of which token positions came from which message spans.

For M serialized records, let $m_t^n \in \{0, 1\}$ indicate whether the target token x_{t+1}^n is supervised. With $N = \sum_n \sum_t m_t^n$ valid targets, the token-averaged SFT loss is

$$\mathcal{L}_{\mathrm{SFT}(\theta)} = -\frac{1}{N} \sum_{n=1}^{M} \sum_{t=1}^{T_n - 1} m_t^n \log p_{\theta(x_{t+1}^n \mid x_{1:t}^n)}. \tag{116}$$

Equation 116 is the same masked autoregressive negative log-likelihood derived in Chapter 5. Its meaning changes through $\mathcal{D}_{\mathrm{SFT}}$, φ, and m. The Causal Attention Mask still determines what each position may read; the loss mask determines which next-token predictions influence the parameter update. Confusing these masks can either leak future information or optimize unintended targets.

13.3.1 Full-Sequence and Assistant-Only Loss

Under **full-sequence loss**, every eligible non-padding token in the serialized conversation contributes to Equation 116. The model is trained to predict system text, user text, role markers, assistant text, and boundaries. This can be appropriate when every span is regarded as modeled data, but it spends gradient weight reproducing prompts that the runtime normally supplies.

Under **assistant-only loss**, prompt targets are masked out and assistant response targets are retained. The system and user tokens remain in the input context, so they still affect assistant hidden states and gradients through the computation graph; they simply do not create direct token-level errors of their own. Llama 2 reports zeroing the loss on user-prompt tokens and backpropagating only on answer tokens during its SFT stage [68].

Consider the multi-turn role sequence

$$\text{System} \rightarrow \text{User} \rightarrow \text{Assistant} \rightarrow \text{User} \rightarrow \text{Assistant}. \tag{117}$$

With assistant-only supervision, targets inside the system message and both user messages receive $m_t = 0$. Targets inside both assistant responses receive $m_t = 1$. The end-of-message token following an assistant response is usually supervised because the model must learn when to stop that turn. Whether the opening assistant-role marker is supervised depends on the inference protocol: if the runtime appends that marker as a generation prompt, it is context and may be masked; if the model must generate it, it is a target. The policy must be defined in token coordinates after serialization, not inferred later from decoded text.

Serialized span	Context	Typical target mask	Reason
System content	Yes	0	Conditions the response but is normally supplied by the application.
User content	Yes	0	Defines the request; reproducing it is not the assistant task.
Assistant content	Yes	1	Provides the demonstrated response behavior.
Assistant turn ending	Yes	1	Teaches the model to terminate or hand back control.
Padding	No	0	Carries neither semantic context nor a valid target.

Table 17: A common assistant-only policy for a serialized conversation. Role and boundary-token treatment remains template-specific and must be declared explicitly.

Table 17 is a policy, not a universal convention. Some instruction-tuning datasets use full-sequence loss; some supervise selected role or boundary tokens; some supervise only the final assistant turn. These objectives assign different weights to the same stored conversation. A reported "SFT loss" is therefore incomplete without its serialization and masking rules.

13.4 Conversations as a Model-Data Interface

13.4.1 Chat Templates, Roles, and Boundaries

A Chat Template is the executable definition of φ in Equation 115. It specifies how messages are ordered, how roles are marked, which separators and end-of-turn tokens are inserted, how an absent system message is handled, and what prefix begins an assistant generation. Official Transformers documentation emphasizes that chat models still consume token sequences and that different models may require different control tokens even when they share a base architecture [69].

The template is therefore a model-data interface contract, not a cosmetic string formatter. If training serializes a user message as `user` followed by one boundary convention but inference supplies a different marker or duplicates beginning- and end-of-sequence tokens, the runtime presents prefixes outside the learned protocol. The model may answer in the wrong role, expose separators, fail to stop, or continue the user turn. Tokenizer version, special-token IDs, template source, whitespace behavior, and the generation-prompt convention must be versioned with the checkpoint.

Special tokens do not possess role semantics merely because they have suggestive names. Their embeddings and conditional effects are learned from their positions in training sequences. A reserved `assistant` marker becomes meaningful because the corpus repeatedly places it at a particular transition. Message boundaries similarly teach turn termination only if their target treatment is consistent. Chapter 1's tokenizer contract therefore extends directly into SFT: a template can refer only to tokens whose identities and segmentation behavior are stable.

13.4.2 Single-Turn and Multi-Turn Supervision

A single-turn example contains one prompt and one response. Its supervision teaches a direct conditional mapping and makes example boundaries simple. A multi-turn example contains alternating messages whose later responses depend on earlier context. It can teach reference resolution, correction, constraint persistence, and the convention that the assistant should respond only after the user has yielded the turn.

Multi-turn training also introduces weighting choices. If every assistant span is supervised, one long conversation contributes several response targets and may dominate a token-averaged mixture. If only the final response is supervised, earlier assistant messages remain teacher-forced context but do not receive direct loss. Neither policy is automatically correct. The chosen unit of sampling, the number of supervised turns, and the valid-token denominator determine how much weight a conversation receives.

Teacher forcing creates a further distinction between training and deployment. During training, the second assistant response is conditioned on the recorded first assistant response. At inference, it is conditioned on the model's own earlier output. SFT does not remove this exposure difference.

Diverse multi-turn data and evaluations with model-generated history can reveal it, but changing that limitation requires methods beyond ordinary supervised likelihood.

13.5 Sequence Construction and Data Mixtures

13.5.1 Packing and Conversation Boundaries

SFT examples are often much shorter than the model context, so packing concatenates several serialized conversations into one fixed-length tensor. As in Chapter 6, packing improves token utilization but creates boundary semantics. A later conversation must not accidentally inherit an earlier conversation as its prompt unless that stream interpretation is intentional. Pipelines can isolate samples with a block-diagonal Attention Mask or accept cross-sample attention while inserting explicit boundaries; the decision changes the conditioning distribution.

Loss alignment must survive packing independently of attention. The first token of a new packed conversation should not become an unintended target of the preceding conversation. Assistant-only masks must be concatenated with exactly the same offsets as token IDs, and padding, truncation, and end-of-turn insertion must preserve N in Equation 116. When truncation cuts an assistant response, the retained fragment may teach a response without its proper termination; when it removes the prompt but retains targets, it creates an invalid example. Segment-aware tests are therefore essential.

13.5.2 Mixtures, Sampling, and Curriculum

An SFT corpus commonly combines task demonstrations, open-ended dialogue, reasoning traces, code, safety behavior, domain-specific records, and synthetic responses. If component k induces distribution P_k and is sampled with weight π_k, the effective demonstration distribution is

$$P_{\mathrm{SFT}(c)} = \sum_{k=1}^{K} \pi_k P_{k(c)}, \quad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1. \tag{118}$$

The mixture weights determine which response patterns receive repeated likelihood pressure. Raw example counts are not realized weights: conversation lengths, assistant-only masks, rejection rates, packing, and resampling change the share of supervised tokens. The FLAN Collection found task balancing and enrichment choices to be consequential in instruction tuning [70]. A reliable pipeline therefore reports both sampling probabilities and realized assistant-target-token fractions.

A curriculum makes π_k or example difficulty depend on training progress. It may begin with clean, direct demonstrations and later increase complex or specialized examples, or it may interleave all components from the start. Curriculum is not intrinsically superior to static mixing: changing the order changes optimizer history, and a late narrow phase can overwrite broad behavior. The schedule should be treated as an experimental variable, indexed by updates or valid target tokens, rather than as undocumented loader behavior.

13.6 Data Quality, Diversity, and Failure Modes

SFT datasets are small relative to pretraining corpora, so each repeated pattern can exert substantial influence. Exact or near-duplicate instructions increase the effective weight of their targets and can make validation optimistic if duplicates cross the split. Low-quality synthetic responses can transfer factual errors, verbosity, refusal habits, and formatting artifacts from the generator. Self-Instruct illustrates both the value of generated instruction data and the need to filter invalid or similar generations before fine-tuning [71].

Conflicting supervision occurs when materially equivalent prompts receive incompatible answers, policies, or styles without contextual features that explain the difference. Maximum likelihood must distribute probability across those targets; it cannot infer a hidden annotation policy. Excessive boilerplate can likewise teach the model to emit preambles or markdown scaffolding regardless of the request. A narrow domain mixture can improve in-domain imitation while reducing breadth, and repeated responses with one rhetorical shape can produce **response-style collapse**: outputs remain grammatical but converge toward a small family of lengths, openings, and structures.

Quality and diversity are therefore joint requirements. Quality asks whether a target is correct, relevant, self-contained, and compatible with the declared policy. Diversity asks whether the corpus

covers distinct tasks, domains, languages, response lengths, interaction patterns, and acceptable styles without relying on superficial paraphrase counts. More examples help only when their marginal supervision is useful. LIMA's curated-data result and FLAN's mixture ablations should be read as complementary evidence: careful examples matter, but coverage and weighting still determine generalization [67], [70].

Two familiar optimization failures acquire a post-training form. **Overfitting** appears when training loss continues to improve while held-out instruction performance, calibration, or response diversity deteriorates. **Catastrophic forgetting** denotes degradation of capabilities retained in the base model as updates specialize the parameters to the SFT distribution. Excessive epochs, a high learning rate, duplicated data, and an overly narrow mixture can intensify both risks. Mitigations include early checkpoint comparison, lower update magnitude, broader rehearsal data, explicit capability-regression suites, and parameter-efficient adaptation; none substitutes for a protected evaluation set.

13.7 Full-Parameter and Parameter-Efficient Adaptation

Full-parameter fine-tuning updates every trainable parameter θ under Equation 116. It gives the optimizer maximal freedom to reshape the model and is a direct continuation of pretraining optimization. Its cost is also direct: gradients and optimizer states are maintained for the full model, each adapted checkpoint stores a full parameter set, and updates can alter broadly useful representations.

Parameter-Efficient Fine-Tuning (PEFT) freezes most pretrained parameters and learns a smaller adaptation state. PEFT reduces trainable-state memory and makes it practical to store multiple task or domain adaptations. It also constrains the update subspace, which can be beneficial regularization but can limit adaptation when the chosen modules or capacity are insufficient. PEFT changes which parameters are optimized, not the definition of the token-level SFT objective.

13.7.1 LoRA as a Low-Rank Update

Low-Rank Adaptation (LoRA) represents the update to a selected weight matrix with two thin trainable matrices [72]. For a frozen pretrained projection $W_0 \in R^{d_{\text{out}} \times d_{\text{in}}}$, write

$$W' = W_0 + \Delta W, \quad \Delta W = BA, \tag{119}$$

where $A \in R^{r \times d_{\text{in}}}$, $B \in R^{d_{\text{out}} \times r}$, and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Because $\text{rank}(BA) \leq r$, the learned change is restricted to a low-rank subspace. The trainable parameter count for this projection is

$$r(d_{\text{in}} + d_{\text{out}}) \quad \text{rather than} \quad d_{\text{in}} d_{\text{out}}. \tag{120}$$

Implementations commonly scale the update by $\frac{\alpha}{r}$ and initialize one factor so that $\Delta W = 0$ at the start, preserving the base function before training. During the forward pass, the layer computes $W_0 x + B(Ax)$; after training, the update can be stored separately or merged into W_0 for deployment. Rank, scaling, target modules, bias treatment, and merge convention are part of the adapter specification. Chapter 14 gives the broader PEFT treatment.

13.8 Evaluating an SFT Model

Held-out SFT loss measures likelihood under a fixed serialization and target mask. It is useful for detecting overfitting and comparing checkpoints within one protocol, but it does not by itself establish instruction following. A model can assign high likelihood to reference wording while failing semantically equivalent prompts, or produce a valid answer different from the single reference.

Evaluation should therefore separate several questions. Task suites measure correctness under explicit answer extraction or execution rules. Format tests measure whether requested schemas, role boundaries, and stopping behavior are obeyed. Human evaluation can assess open-ended helpfulness and clarity when deterministic reference metrics are inadequate. Capability-regression suites compare the tuned model with its base checkpoint on retained knowledge and skills. Safety evaluation belongs in the release process even when safety alignment is not the objective of this chapter.

The evaluation distribution must be protected from duplicate or paraphrased training examples, and decoding settings must be fixed when comparing checkpoints. Single-turn and multi-turn behavior should be evaluated separately; the latter should include model-generated history rather than only gold prior responses. Chapter 11's distinction between validation loss and downstream behavior

remains intact: one monitors the declared likelihood objective, while the other tests whether the learned conditional distribution produces the intended interaction.

13.9 Implementation Contracts

The data contract should preserve the original message sequence, source identity, quality decisions, and split assignment. It should version the Chat Template, tokenizer, special-token IDs, generation-prompt convention, and exact mapping from message spans to token spans. Preprocessing must reject or explicitly handle unsupported roles, empty assistant targets, duplicated boundary tokens, truncated prompts, and conversations whose supervised-token count is zero.

The tensor contract should expose input IDs, shifted targets, causal or segment-aware Attention Masks, loss masks, segment IDs where used, and the valid-target denominator. Tests should serialize hand-written single- and multi-turn conversations and verify every m_t against the intended role span, including assistant opening and closing markers. Packing tests should prove that token IDs, masks, and segment offsets remain aligned and that one sample cannot create an unintended target in another.

The optimization contract should record the base checkpoint, trainable parameter set, full-parameter or PEFT mode, optimizer and schedule, effective batch definition, gradient accumulation, precision policy, and number of valid assistant tokens consumed. A LoRA artifact additionally requires rank, scaling, target-module names, factor orientation, initialization, dtype, bias policy, and merge status. Checkpoint evaluation should retain the template and masking metadata needed to reproduce both validation loss and generation.

Finally, the inference contract must apply the same serialization semantics used in training. It should distinguish formatting a completed training conversation from appending a generation prompt, prevent duplicate special tokens, and specify which stop tokens or message boundaries terminate the assistant. A model checkpoint without this interface metadata is not a complete chat model artifact.

13.10 Summary

Supervised Fine-Tuning is a distributional adaptation of a pretrained Causal Language Model. The architecture and next-token likelihood machinery remain largely unchanged, while instruction-response demonstrations, Chat Templates, and loss masks define a new conditional learning problem. Full-sequence loss models every eligible serialized token; assistant-only loss uses system and user messages as context while assigning direct supervision to selected assistant targets. Multi-turn conversations, packing, and truncation make these token-level boundaries operational rather than merely conceptual.

SFT quality depends on more than example count. Mixture weights determine which behaviors are emphasized; duplicated, synthetic, conflicting, or stylistically narrow targets can distort the learned response distribution. Full-parameter fine-tuning offers unconstrained adaptation at full training-state cost, whereas LoRA learns a low-rank update to selected frozen projections. In either case, held-out loss must be complemented by instruction, format, multi-turn, and capability-regression evaluation. The resulting model is reproducible only when its tokenizer, Chat Template, special tokens, loss policy, packing rules, adaptation state, and inference protocol are treated as one versioned contract.

References

[65] J. Wei *et al.*, "Finetuned Language Models Are Zero-Shot Learners," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=gEZrGCozdqR

[66] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744. doi: 10.52202/068431-2011.

[67] C. Zhou *et al.*, "LIMA: Less Is More for Alignment," in *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-2400.

[68] H. Touvron, L. Martin, K. Stone, and others, "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv preprint arXiv:2307.09288*, 2023, doi: 10.48550/arXiv.2307.09288.

[69] Hugging Face, "Chat Templates." 2026. [Online]. Available: https://huggingface.co/docs/transformers/chat_templating

[70] S. Longpre *et al.*, "The FLAN Collection: Designing Data and Methods for Effective Instruction Tuning," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 22631–22648. [Online]. Available: https://proceedings.mlr.press/v202/longpre23a.html

[71] Y. Wang *et al.*, "Self-Instruct: Aligning Language Models with Self-Generated Instructions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 13484–13508. doi: 10.18653/v1/2023.acl-long.754.

[72] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9

Chapter 14: Parameter-Efficient Fine-Tuning

Abstract

Parameter-Efficient Fine-Tuning (PEFT) adapts a pretrained model while retaining a shared frozen base and training a comparatively small task-specific state. This chapter develops the central low-rank construction behind LoRA, then compares it with quantized-base adaptation in QLoRA, inserted Adapter modules, Prefix Tuning, and Prompt Tuning. The central distinction is not simply the number of trainable parameters: PEFT changes the update parameterization, optimizer state, artifact topology, and sometimes the inference path, while leaving important costs such as activations and the base-model footprint in place.

14.1 Introduction

Chapter 13 introduced Supervised Fine-Tuning (SFT) as a change in the conditional data distribution and loss mask used to adapt a Causal Language Model. Its compact treatment of LoRA established the basic low-rank update, but did not compare the larger family of Parameter-Efficient Fine-Tuning (PEFT) methods. This chapter supplies that comparison. It assumes the SFT objective, Chat Template, and assistant-target masking from Chapter 13, and treats PEFT as a choice about **which state is permitted to change** under that same objective.

Full fine-tuning makes a task-specific copy of the entire pretrained parameter vector θ and optimizes it to obtain θ'. This is expressive, but a separate task or customer can require a separate full checkpoint, its optimizer state during training, and a deployment decision about which complete model to load. PEFT instead keeps a base parameter set θ_0 frozen and introduces a much smaller trainable state φ:

$$\theta = \theta_0 \quad \rightarrow \quad (\theta_0, \varphi), \quad \nabla_{\theta_0} \mathcal{L} = 0, \quad \nabla_{\varphi} \mathcal{L} \neq 0. \tag{121}$$

The expression in Equation 121 is a training contract, not a claim that the base model disappears from memory. The frozen weights must still be stored and read for every forward pass. What PEFT primarily avoids is a gradient and optimizer-state allocation for those weights, while representing adaptation in a portable additional artifact. Whether this is sufficient depends on the task, the base model, the selected modules, and the available compute; PEFT is not a guarantee that a small update matches unrestricted fine-tuning.

14.2 Full Fine-Tuning, Trainable State, and Memory

It is useful to separate four quantities that are often compressed into the phrase "memory efficient." Let P denote the number of base-model parameters, P_φ the number of trainable adaptation parameters, and let the b terms denote bytes per stored element under a declared precision policy. A schematic training-memory decomposition is

$$M_{\text{train}} \approx M_{\text{base}} + P_\varphi \left(b_\varphi + b_{\text{grad}} + b_{\text{opt}} \right) + M_{\text{act}} + M_{\text{temp}}. \tag{122}$$

For ordinary full fine-tuning, $P_\varphi = P$ and M_{base} may share storage with the trainable parameter copy. Under PEFT, M_{base} remains a substantial model-weight term, but $P_\varphi \ll P$ can greatly reduce gradient and optimizer-state memory. Chapter 7 explains why Adam-style moment states make this distinction material. Activation memory M_{act} is governed by batch shape, sequence length, layer implementation, checkpointing, and the backward graph; it need not fall in proportion to the number of trainable parameters. Likewise, inference must normally retain the whole base model. A small adapter artifact reduces per-task storage, not the memory required to load the shared model.

The distinction also separates three operational claims. **Trainable parameter count** determines how many adaptation values receive updates. **Training-state memory** adds their gradients and optimizer state, alongside activations and temporary buffers. **Inference memory** contains the loaded base model, any unmerged adaptation state, and request-dependent state such as the KV Cache described in Chapter 24. PEFT can improve all three in particular designs, but none follows automatically from the first.

14.3 Low-Rank Adaptation

Low-Rank Adaptation (LoRA) is the central PEFT construction because it changes an existing linear transformation without inserting a new hidden layer into the Transformer. For a frozen pretrained matrix $W \in R^{d_{\text{out}} \times d_{\text{in}}}$, LoRA learns a low-rank update

$$\Delta W = BA, \quad A \in R^{r \times d_{\text{in}}}, \quad B \in R^{d_{\text{out}} \times r}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}), \tag{123}$$

and uses the adapted transformation

$$W' = W + \frac{\alpha}{r} BA. \tag{124}$$

Here r is the adaptation rank and α is a scaling hyperparameter. The convention $\frac{\alpha}{r}$ is common, but an implementation must state its actual scale and whether it is folded into a factor. Since $\text{rank}(BA) \leq r$, Equation 123 restricts the update to a low-rank subspace. It does not assert that the ideal task-specific update has low rank in every setting; it is an empirically useful parameterization whose capacity increases with r [73].

The original matrix contains $d_{\text{out}} d_{\text{in}}$ parameters. The two LoRA factors contain

$$P_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}}), \tag{125}$$

which can be much smaller when r is modest relative to both dimensions. For an input $x \in R^{d_{\text{in}}}$, the forward calculation is better understood as two paths:

$$W'x = Wx + \frac{\alpha}{r} B(Ax). \tag{126}$$

The frozen path Wx remains present during training. LoRA adds the narrow projection Ax and its expansion $B(Ax)$; it does not replace the pretrained matrix with a small matrix. This distinction explains both its modularity and why base-weight bandwidth still matters.

14.3.1 Rank, Initialization, and Target Modules

Rank governs a direct trade-off. A larger r enlarges the attainable update space and the trainable state in Equation 125; it also increases optimizer memory and low-rank-path arithmetic. A small r is cheaper and can regularize adaptation, but may underfit a task or prove inadequate for the chosen target modules. The useful rank is consequently an empirical decision tied to data size, task shift, selected projections, and evaluation, rather than a universal property of a model family.

A common initialization samples one factor, for example A, and initializes the other factor B to zero. Then $BA = 0$ at the first update and Equation 124 initially reproduces the base transformation exactly. This avoids an arbitrary perturbation of the pretrained function at initialization while still allowing nonzero gradients for the zero-initialized factor. Factor orientation, initialization distribution, scaling, dropout, and precision are all part of an adapter artifact's specification.

In a Transformer, LoRA can target the Query, Key, Value, and output projections in Self-Attention, or one or more expansion and contraction projections in the Feed-Forward Network. Chapter 3 defines these attention projections and Chapter 4 defines the feed-forward path. Selecting only Query and Value projections yields a different update budget and inductive bias from adapting every attention and MLP projection. There is no architecture-independent target set: attention-only adaptation can be sufficient for one distributional shift, while another task may require feed-forward capacity as well.

14.3.2 Merging and Multiple Adapters

For a compatible ordinary linear layer, the trained update can be folded into a copy of the base weight:

$$W_{\text{merged}} = W + \frac{\alpha}{r} BA. \tag{127}$$

Merging removes the extra low-rank path for that fixed adaptation at inference, but produces a task-specific full weight copy. Keeping the factors separate instead permits one frozen base model to coexist with many small adapters. These alternatives answer different operational needs:

Serving choice	Advantage	Consequence
Merged update	One fixed linear weight and no separate LoRA branch in the execution graph.	Consumes a task-specific full-weight copy; switching requires another merged state.
Separate adapter	One base can load or select many small adaptation artifacts.	Runtime must identify the base, adapter, scale, target modules, and compatible kernels.
Composed adapters	Can express a declared combination of adaptations.	Updates may interfere; order, weights, and evaluation must be explicit.

Table 18: Merging optimizes a fixed deployment path; separate adapters optimize sharing and selection. Neither representation is intrinsically safer without a compatibility contract.

Storing an adapter, loading it into an execution process, switching a request to it, and composing several adapters are distinct operations. In particular, arithmetic addition of two low-rank updates is not evidence that their behaviors compose usefully. A server must associate every adapter with a specific base checkpoint, tokenizer and model configuration, factor orientation, scaling convention, target module map, and merge status.

14.4 QLoRA: Quantized Frozen Bases

QLoRA combines a quantized frozen base model with higher-precision trainable LoRA parameters. Its conceptual path is

$$\text{quantized base} + \text{LoRA factors} \rightarrow \text{adapted model}. \tag{128}$$

The quantization in Equation 128 is not ordinary inference quantization applied after training. The base weights remain quantized and frozen during adaptation; gradients pass through the computation needed to update the LoRA factors, not into an independently updated full-precision base. This reduces the weight-storage term in Equation 122 while preserving a trainable adaptation path. Dettmers et al. use this design to make substantially larger frozen bases practical under constrained accelerator memory [74].

Storage precision, computation precision, and trainable-parameter precision must remain separate. A four-bit representation can store a block of frozen weights compactly, while a kernel dequantizes or computes using a higher-precision representation, and the LoRA factors, gradients, and optimizer states may use another dtype. Thus "4-bit fine-tuning" does not mean every operation or every trainable quantity is four-bit.

14.4.1 NF4, Double Quantization, and Paged Optimizers

QLoRA introduced NormalFloat4 (NF4), a four-bit data type designed for weights with an approximately normal distribution, together with block-wise quantization. At the chapter's level of abstraction, NF4 chooses a compact set of representable values better matched to that assumed weight distribution than a uniform integer grid; it is not a claim that every tensor is normally distributed or that NF4 is always the correct format. Chapter 25 develops the general distinction between quantization representations, scales, reconstruction error, and kernel support.

Each quantized block also needs quantization constants, such as scales. **Double quantization** compresses these constants themselves, reducing a secondary metadata cost. It should not be confused with applying two independent quantizers to the base weight: its purpose is to shrink the representation overhead surrounding block quantization.

Paged optimizers address a different term in the memory picture. Temporary optimizer or activation-related allocation spikes can exceed a device's capacity even when steady-state accounting appears feasible. The QLoRA implementation uses a paging mechanism backed by unified memory to manage such spikes. This can enlarge the set of feasible experiments, but page migration is not free: it can introduce transfer overhead and does not eliminate activation, sequence-length, or kernel constraints [74].

The resulting qualitative decomposition is

$$M_{\text{QLoRA}} \approx M_{\text{quantized base}} + M_{\text{LoRA factors, gradients, and optimizer}} + M_{\text{activations}} + M_{\text{temporary buffers}} \tag{129}$$

The first term can fall sharply relative to a full-precision frozen base, but long sequences and large microbatches can still make the remaining terms decisive. QLoRA is therefore a memory-budgeting technique, not a substitute for checking the effective batch, activation policy, and numerical behavior described in Chapters 7 and 8.

14.5 Inserted and Prompt-Based Adaptation

LoRA parameterizes a modification of an existing linear map. Other PEFT methods place the small trainable state at different points in the computation. This changes what is learned, what must execute at inference, and how readily the state can be attached to a compatible base.

14.5.1 Adapters

An Adapter layer inserts a small trainable bottleneck into a Transformer block while leaving most pretrained parameters frozen. A representative residual Adapter maps a hidden vector $h \in R^d$ to

$$h' = h + W_{\text{up}}\varphi(W_{\text{down}}h), \quad W_{\text{down}} \in R^{m\times d}, \quad W_{\text{up}} \in R^{d\times m}, \tag{130}$$

where $m \ll d$ is the bottleneck width and φ is a nonlinearity. Like LoRA rank, m controls an efficiency–capacity trade-off. Adapters introduce new modules into the execution graph rather than changing an existing projection by a low-rank update. They are consequently compact and modular, but can add inference work and architectural integration requirements at every insertion point. Houlsby et al. established this frozen-backbone, bottleneck-module pattern for parameter-efficient transfer [75].

14.5.2 Prefix Tuning and Prompt Tuning

Prefix Tuning learns continuous task-specific states that condition attention while the language-model parameters remain frozen. In a decoder-only layer, the learned prefix can be viewed conceptually as additional Key/Value-like states available to later tokens. These vectors are not ordinary textual tokens with vocabulary IDs; they are trainable continuous states whose placement and layer-wise parameterization must be specified. Prefix Tuning therefore changes conditioning at attention interfaces rather than directly changing W in Equation 124 [76].

Prompt Tuning learns a smaller sequence of **soft prompt** embeddings prepended to the ordinary token embeddings. If $E_{\text{soft}} \in R^{p\times d}$ contains p learned prompt vectors and $E(x)$ is the embedding sequence for tokenized input x, the Transformer receives

$$[E_{\text{soft}}; E(x)]. \tag{131}$$

Textual prompts are discrete strings passed through the tokenizer; soft prompts are optimized vectors and need not correspond to readable vocabulary items. Prompt Tuning changes the input embedding sequence, whereas Prefix Tuning supplies learned conditioning states to attention layers. Both preserve the frozen base weights, but they are not synonyms and have different capacity, context-budget, and implementation behavior [77].

14.6 Comparing PEFT Methods

The methods differ along more than one axis. The following table is deliberately qualitative: trainable share, quality, and throughput depend on model scale, target modules, task shift, sequence length, and implementation.

Method	Trainable state	Base or graph change	Typical inference implication	Primary trade-off
Full fine-tuning	All or nearly all model parameters	Updates base weights	One full adapted checkpoint	Maximum update freedom; highest trainable-state cost.
LoRA	Low-rank factors on selected projections	Frozen base; additive update	Can merge or select separate adapters	Target and rank determine capacity.
QLoRA	LoRA factors with quantized frozen base	Quantized base during adaptation	Quantization-compatible serving path	Reduces base storage; adds quantization constraints.
Adapters	Bottleneck modules	Adds modules to Transformer blocks	Extra module execution unless fused	Modular capacity with architectural overhead.
Prefix Tuning	Layer-level continuous prefixes	Conditions attention states	Prefix state participates in execution	Small state; per-layer conditioning design.
Prompt Tuning	Input-level soft prompt	Extends embedding sequence	Consumes context positions	Very small state; capacity may be limited.

Table 19: PEFT methods choose different locations for task-specific state. The table compares mechanisms, not guaranteed quality rankings.

For a deployment that serves many narrowly differentiated tasks, separate small adapters can reduce artifact storage and enable explicit task selection. For one fixed production task, merging a LoRA update can simplify the execution path. For memory-constrained training, a quantized frozen base can be more important than reducing the adapter size further. Conversely, a method with few trainable parameters can underperform unrestricted fine-tuning when the required behavioral shift is broad, the prompt or prefix capacity is too limited, or the selected LoRA modules omit the computation that must change.

PEFT does not relax the data and evaluation obligations of SFT. A low-rank update can still overfit duplicated instruction data, memorize stylistic artifacts, or cause capability regression. The protected evaluation and compatibility checks of Chapter 13 remain necessary; the reduced trainable state changes the adaptation interface, not the definition of a trustworthy result.

14.7 Failure Modes

Several failure modes arise from treating a PEFT artifact as merely a small tensor file. A rank that is too small can leave an adaptation under-capacitated; a needlessly large rank can erase its resource advantage and overfit limited data. A poor target-module choice can create the same symptom even with a large rank, because the required transformation is inaccessible. Unstable learning rates, inappropriate scaling, or incorrectly frozen base weights can make a supposedly small update destructive.

QLoRA adds representation-specific risks. Poorly calibrated or unsupported low-precision kernels can remove the expected memory or speed benefit, and quantization error can interact with a task-sensitive layer or distribution. A reported "quantized" configuration is incomplete unless it identifies the base checkpoint, quantization scheme and block rules, compute dtype, adapter dtype, and kernel implementation. Activation pressure can still cause out-of-memory failures even when the weight footprint fits.

At serving time, the most serious errors are often compatibility errors: attaching an adapter to the wrong base revision, mixing factor orientations or scaling conventions, merging the wrong update twice, or composing adapters whose joint behavior was never evaluated. A correct deployment therefore treats the adapter identifier and its declared base-model contract as inseparable.

14.8 Implementation Contracts

The **base-model contract** must identify the exact checkpoint, architecture configuration, tokenizer and Chat Template where the adaptation is conversational, weight dtype, and target-module name map. It must state which base parameters are frozen and include a test that their values and gradients remain unchanged after an optimization step.

The **adaptation contract** must identify the PEFT family, trainable parameter count, factor orientation, rank or bottleneck width, scaling, initialization, dropout, target modules, and any merge state.

For QLoRA it must additionally record the quantization format, block policy, quantization-constant representation, computation dtype, and optimizer paging policy. A shape test should verify every A, B, Adapter, prefix, or soft-prompt tensor against its declared model interface before loading weights.
The **optimization contract** retains the SFT data, masking, optimizer, precision, effective batch, sequence-length, gradient-accumulation, and checkpoint conventions from Chapters 7, 8, and 13. It should report both the number of trainable parameters and the observed peak memory by category where possible; reporting only adapter size can hide an activation or temporary-buffer bottleneck.
Finally, the **serving contract** must specify whether updates are merged, separately selected, or composed; the adapter revision; all compatibility metadata; and the evaluation used to validate the exact assembled model. Tests should reject a mismatched base identifier, an unknown target module, inconsistent factor shapes, duplicate merging, and an adapter that changes a frozen base artifact on disk.

14.9 Summary

PEFT adapts a pretrained language model by preserving a shared frozen base and training a smaller task-specific state. Its principal savings arise in trainable parameters, gradients, optimizer state, and per-task artifact storage; base-weight, activation, and request-state memory remain independent accounting terms. Full fine-tuning and PEFT therefore differ in parameterization and operational topology, not merely in the number printed beside a model.
LoRA expresses an update as scaled low-rank factors and can target selected attention or feed-forward projections. QLoRA combines this update path with a quantized frozen base, using NF4, compressed quantization metadata, and paging techniques to improve constrained-memory training. Adapters insert bottleneck modules, while Prefix Tuning and Prompt Tuning learn continuous conditioning states at different locations in the Transformer computation.
The appropriate method follows from a declared task, memory budget, serving model, and evaluation plan. Rank, target modules, quantization policy, merge status, and base-model identity are all executable compatibility conditions. A PEFT artifact is useful only when those conditions preserve both the intended base model and the intended adaptation behavior.

References

[73] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9

[74] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023, doi: 10.48550/arXiv.2305.14314.

[75] N. Houlsby *et al.*, "Parameter-Efficient Transfer Learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 2790–2799. [Online]. Available: https://proceedings.mlr.press/v97/houlsby19a.html

[76] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, 2021, pp. 4582–4597. doi: 10.18653/v1/2021.acl-long.353.

[77] B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2021, pp. 3045–3059. doi: 10.18653/2021.emnlp-main.243.

Chapter 15: Preference Data and Reward Modeling

Abstract

Supervised Fine-Tuning teaches a language model to imitate demonstrations, but many desired properties of an answer are comparative: one response can be more helpful, faithful, safe, or appropriately concise than another. This chapter introduces preference data as a collection of such judgments and develops the Reward Model used by classical RLHF to convert pairwise comparisons into a scalar ranking signal. It derives the Bradley-Terry likelihood, distinguishes a reward score from a calibrated probability or universal utility, examines collection and evaluation protocols, and explains the biases, distribution shifts, and reward-hacking risks that constrain later policy optimization.

15.1 Introduction

Chapter 13 described Supervised Fine-Tuning (SFT) as maximum-likelihood training on demonstrations of desired responses. A demonstration answers the question, "what response should follow this prompt?" It does not necessarily express the finer judgment that arises when several acceptable responses differ in factuality, relevance, safety, brevity, tone, or reasoning quality. For many open-ended prompts, there is no single target whose token sequence exhausts that judgment.

Preference learning supplies a different supervision primitive. Given a prompt and two candidate responses, an annotator identifies the one that better satisfies a stated criterion. The comparison is cheaper to express than a complete ideal response in many settings and preserves a useful fact: the label is relative to the particular prompt, candidates, rubric, and annotator population. Early work on learning from human preferences used comparisons to infer goals that were difficult to encode as a hand-written reward function [78]. In language modeling, the familiar classical pipeline is

$$\text{Pretraining} \rightarrow \text{SFT} \rightarrow \text{Preference Data} \rightarrow \text{Reward Modeling} \rightarrow \text{RLHF}. \tag{132}$$

Pretraining provides a broad language prior; SFT supplies an instruction-following policy and often the source of initial candidates; preference data records comparative judgments; a Reward Model (RM) learns to rank candidate responses; and a later policy-optimization stage uses that learned signal. This chapter ends at the RM. It does not derive PPO, nor does it develop DPO or GRPO beyond noting that they address later stages of the post-training sequence.

15.2 From Demonstrations to Preferences

An SFT record normally contains a context x and a demonstrated continuation y^*, then optimizes the likelihood $p_{\theta(y^* \mid x)}$. The supervision says that the tokens of y^* are desirable targets under the training serialization and loss mask. A preference record instead contains a prompt x, a chosen response y_{w}, and a rejected response y_{l}. Its label says only that the annotator preferred y_{w} to y_{l} under the annotation policy. It does not claim that y_{w} is globally perfect or that y_{l} is unusable in every context.

This difference matters for data design. Demonstrations place probability mass directly on a response distribution. Comparisons expose a direction of improvement among responses sampled from a particular proposal distribution. A response pair can be informative even when neither member is an ideal answer: one may be factually correct but needlessly verbose, while the other is concise but omits a required constraint. The annotation rubric determines which trade-off the label represents.

The candidates should therefore be treated as part of the data-generating process, not as incidental strings. They may be sampled from an SFT checkpoint, a mixture of checkpoints, human-written alternatives, or carefully filtered synthetic generators. If the pool contains only obviously poor and obviously good completions, a reward model can achieve high held-out pair accuracy without learning the subtle distinctions encountered during later optimization. If it contains candidates from only one response style or length range, the learned ranking will encode that narrow comparison distribution.

15.3 Preference Data and Its Collection

A practical preference record needs more provenance than the final binary label. At minimum it should identify the prompt and conversation context, candidate-generation policy and decoding settings, annotation rubric and task version, annotator or judging source, label outcome, collection time, and

split assignment. For multi-turn dialogue, the complete preceding history and its Chat Template are part of x. As Chapter 13 emphasized, changing the serialized context changes the conditional task presented to a language model and to a judge.

Collection source	What the comparison can provide	Principal limitation to record
Human annotation	Judgment under an explicit human rubric, including domain expertise when the task requires it.	Cost, disagreement, rubric interpretation, and limited coverage.
Expert or audited annotation	Higher-fidelity comparisons for specialized or high-stakes criteria.	Expense and possible narrowing to the experts' operational policy.
Synthetic or AI feedback	Rapidly generated comparisons and coverage of rare or adversarial cases.	The judge's blind spots, prompt sensitivity, and shared model-family biases.
Hybrid collection	Human anchors or audits combined with scalable generated labels.	The route and quality controls for every label must remain recoverable.

Table 20: Preference sources differ in what they measure, not only in cost. A response pair must retain its provenance so that a later RM evaluation can distinguish human and synthetic supervision.

Human annotation is not a single protocol. The task may ask an annotator to choose the more helpful response, identify the less harmful response, compare factual accuracy using supplied evidence, or break ties according to a detailed style policy. Randomizing response order avoids turning a fixed presentation position into an accidental label. Blinded candidate identities avoid source-model reputation effects. Multiple labels for selected examples make disagreement visible rather than silently collapsing it into a single asserted preference. InstructGPT collected rankings of model outputs after SFT as the source for its Reward Model stage [79].

Synthetic preference data replaces or augments human choice with a model-based judge, a rule-based verifier, or a constitution-like critique-and-revision process. It can make coverage and iteration substantially cheaper, but it is not automatically human preference data. Constitutional AI and RLAIF are examples of pipelines that use AI-generated feedback to scale parts of the supervision process [80], [81]. A synthetic label inherits the criterion, failure modes, and distributional limits of its generating judge. It should therefore record the judge revision, prompt, decoding policy, and any filtering or audit rule, just as human data records its rubric.

15.3.1 Noise, Ties, and Inconsistent Preferences

Preference labels are observations, not ground truth utilities. Annotators can misread a response, apply the rubric differently, or reasonably disagree because the comparison presents a real trade-off. Inconsistency also arises when a label depends on information absent from the stored prompt, when candidates differ in subtle ways that the rubric does not settle, or when the same comparison is presented with a different order or framing.

A binary dataset often resolves a tie by asking for a forced choice. That can be operationally convenient, but it turns ambiguity into label noise. Systems that collect ties, strength-of-preference labels, or repeated annotations can represent more of the uncertainty, though their modeling and aggregation choices must then be explicit. The basic pairwise objective below deliberately models a binary observed choice; it does not prove that every human has one stable, transitive ranking over all possible responses.

15.4 Pairwise Preference Modeling

Let $r_{\psi(x,y)}$ be a scalar score assigned by an RM with parameters ψ to response y in context x. The Bradley-Terry model represents the probability that the chosen response wins a pairwise comparison as

$$P(y_{\mathrm{w}} \succ y_{\mathrm{l}} \mid x) = \sigma\big(r_{\psi(x,y_{\mathrm{w}})} - r_{\psi(x,y_{\mathrm{l}})}\big), \quad \sigma(z) = \frac{1}{1+\exp(-z)}. \tag{133}$$

The score difference, not either score in isolation, determines the probability in Equation 133. When both scores are equal, the model assigns probability $\frac{1}{2}$ to the observed ordering. Increasing the chosen-minus-rejected margin raises the probability. Reversing the candidates negates the margin and swaps

the probability. This is a useful statistical model of noisy comparative choices, not evidence that human judgments literally arise from a logistic latent utility.

For a dataset $\mathcal{P} = \{(x_i, y_{\mathrm{w}}^i, y_{\mathrm{l}}^i)\}_{i=1}^N$, maximum likelihood yields the negative log-likelihood

$$\mathcal{L}_{\mathrm{RM}(\psi)} = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma\big(r_{\psi(x_i, y_{\mathrm{w}}^i)} - r_{\psi(x_i, y_{\mathrm{l}}^i)}\big). \tag{134}$$

Writing $\Delta_i = r_{\psi(x_i, y_{\mathrm{w}}^i)} - r_{\psi(x_i, y_{\mathrm{l}}^i)}$ makes one example's contribution

$$-\log \sigma(\Delta_i) = \log(1 + \exp(-\Delta_i)). \tag{135}$$

Thus a correctly ordered pair with a large positive margin contributes little loss; an incorrectly ordered pair with a negative margin contributes heavily. The derivative with respect to Δ_i is $\sigma(\Delta_i) - 1$, so uncertain or wrong pairs exert stronger pressure than already confidently correct pairs. In practice, this loss is evaluated with a numerically stable log-sigmoid or log-sum-exp form, following the stability principles of Chapter 8.

The formulation also reveals an identifiability limit. For any function $a(x)$,

$$\big(r_{\psi(x, y_{\mathrm{w}})} + a(x)\big) - \big(r_{\psi(x, y_{\mathrm{l}})} + a(x)\big) = r_{\psi(x, y_{\mathrm{w}})} - r_{\psi(x, y_{\mathrm{l}})}. \tag{136}$$

The pairwise likelihood cannot determine a prompt-dependent common offset. It mainly learns relative ordering inside the comparison task. This is why an RM score should not be read as a portable absolute measurement of helpfulness, safety, or human welfare.

15.4.1 Scores, Probabilities, and Rankings

Three distinctions prevent common misreadings. First, **classification accuracy** asks how often an RM ranks the chosen candidate above the rejected candidate in a held-out labeled pair. It is a useful diagnostic, but it is not reward quality in full: it may ignore ties, calibration, distribution shift, adversarial candidates, and the behavior induced when a policy searches for high scores.

Second, a **reward score** $r_{\psi(x,y)}$ is a model output. The preference **probability** is obtained only after comparing two scores through Equation 133. A score of 3 is not a probability of 3 percent, nor does a score twice as large mean that a response is twice as good. Its scale can change with architecture, training loss, normalization, regularization, and the candidate distribution.

Third, pairwise data naturally identifies a ranking relation rather than a calibrated global score. **Absolute scoring** would require labels with a declared reference scale, such as a rubric-defined rating, and even then demands separate calibration analysis. The Bradley-Terry objective makes a local comparative claim: for this context and pair, one response is predicted to be more likely to win. It does not create a universal ruler across unrelated prompts.

15.5 Reward Models as Sequence-Level Scorers

For a decoder-only RM, the context and candidate response are serialized into one sequence, often using the same tokenizer and Chat Template family as the policy. The Transformer processes the entire prefix causally, and a scalar head maps a chosen hidden state to a sequence-level score. If $h_{T(x,y)}$ denotes the hidden state at the declared terminal response position, a common form is

$$r_{\psi(x,y)} = w^T h_{T(x,y)} + b, \tag{137}$$

where ψ includes the backbone parameters and scalar-head parameters (w, b). Other readout conventions are possible, but the terminal position, end-of-response token handling, padding policy, and truncation rule change the actual function being learned. A reward is called **sequence-level** here because it evaluates a completed candidate response as one object; it is not a token-level next-token likelihood and it need not decompose into rewards for individual tokens.

The RM commonly starts from a pretrained or SFT-capable backbone because it must understand the prompt, response, and their relationship. This initialization is not a guarantee of correct judgments. The scalar head and fine-tuning objective can exploit superficial correlates in the comparison data. Stiennon et al. trained a scalar reward predictor from human summary comparisons and then used it as the reward signal for a later policy stage, making the separation between reward-model fitting and policy optimization explicit [82].

15.5.1 Calibration and Score Normalization

Calibration concerns whether predicted pairwise probabilities correspond to observed frequencies on an appropriate held-out distribution. If the RM assigns roughly 0.8 to many comparisons of a defined type, then roughly 80 percent of those chosen responses should win under the same labeling policy. A temperature $\tau > 0$ can rescale the margin for a held-out calibration procedure,

$$P_{\tau(y_w \succ y_l \mid x)} = \sigma\left(\frac{r_{\psi(x,y_w)} - r_{\psi(x,y_l)}}{\tau}\right). \tag{138}$$

Calibration does not repair a biased preference dataset, an out-of-distribution response, or a misspecified criterion. Nor does it give the raw reward a universal meaning. Mean-variance normalization can make reward magnitudes more convenient for a later optimizer, but it is a training-interface convention, not a discovery of the true scale of human values. The denominator and reference distribution used for any normalization must be stored with the model.

15.6 Bias, Distribution Shift, and Reward Hacking

Reward Models learn patterns predictive of labels, which need not coincide with the intended property. A dataset can reward longer answers because annotators interpret detail as effort, even when extra length obscures the answer. Stiennon et al. explicitly controlled summary length because length can confound preference judgments [82]. A model can similarly learn a preference for polished formatting, a particular refusal style, hedging, citations, or a judge-favored dialect. Such style correlates are not always wrong; they become harmful when they displace task correctness or user intent.

Annotator disagreement is another source of bias rather than merely an inconvenience. A single aggregate label can erase legitimate variation in how people weigh harmlessness, directness, technical detail, or tone. Dataset policy may require aggregation, but evaluation should retain enough metadata to ask where agreement is low, which rubric version applies, and whether the RM amplifies a majority preference beyond its evidential support.

Distribution shift occurs when the prompts, candidates, languages, response lengths, or behaviors evaluated by the RM differ from those on which it was trained. Later policy optimization creates a particularly important shift: it deliberately searches for responses with high predicted reward and can therefore produce unusual candidates. A held-out comparison sampled from the original candidate generator may not expose errors in that searched region.

Reward hacking or **reward overoptimization** is the resulting gap between a rising proxy score and the intended external judgment. The policy is not required to be deceptive for this gap to occur; it need only find a feature that the RM rewards more than humans do. Reward-model ensembles can reduce some idiosyncratic errors, but they do not remove shared blind spots. Eisenstein et al. show that RMs with similar in-distribution behavior can diverge under alignment because of underspecification and distribution shift [83]. This is a reason to preserve human and task-grounded evaluation after RM training, not a reason to treat an ensemble score as an oracle.

15.7 Evaluating Reward Models

The first evaluation is pairwise held-out accuracy: does $r_{\psi(x,y_w)} > r_{\psi(x,y_l)}$ for a protected set of comparisons? Report the split construction, label aggregation, tie policy, candidate sources, and confidence intervals or uncertainty estimates. Accuracy should be broken down by task, domain, response length, safety criterion, and source model when those attributes are available. A single aggregate number can conceal a model that ranks ordinary chat well while failing factual, adversarial, or long-context comparisons.

Pairwise accuracy alone cannot test score calibration, robustness, or the downstream consequences of optimization. A complementary evaluation includes calibration curves for pairwise probabilities, response-order swaps, length-controlled comparisons, repeated labels, and targeted contrast sets where one response has a verifiable flaw. RewardBench was introduced as a benchmark suite containing prompt-chosen-rejected trios across chat, reasoning, and safety, including structured and

out-of-distribution cases [84]. Its role in a workflow is illustrative: evaluate the ranking function on disjoint, diagnostic tasks rather than trusting an in-distribution training loss.

The final evaluation question is behavioral. If an RM is later used for reranking or policy optimization, compare high-RM-score outputs with independent human or task-grounded judgments. Track the gap between proxy reward, held-out RM accuracy, and the external criterion as optimization strength changes. This does not enter PPO mechanics; it establishes the validation boundary that the next RLHF chapter must respect.

15.8 Implementation Contracts

The data contract should version the prompt and conversation serialization, tokenizer, special tokens, Chat Template, candidate generator, decoding settings, annotation rubric, annotator or judge source, candidate order randomization, label aggregation rule, and split assignment. It should preserve both candidates even when a pair is discarded, together with the reason for discard. Duplicate prompts, near-duplicate responses, malformed boundaries, unsupported roles, missing terminal tokens, and labels without a declared policy must be rejected or handled explicitly.

The tensor contract should identify the token span used by the scalar readout in Equation 137, attention and padding masks, pair packing convention, response-order convention, and the loss reduction in Equation 134. Tests should swap y_{w} and y_{l} and verify that the signed margin reverses. They should verify that padding and truncation cannot change a response's readout position unintentionally, and that each pair contributes exactly once to the declared denominator.

The evaluation contract should separate training, validation, calibration, and external-test comparisons by prompt and by sufficiently strong near-duplicate controls. It should record pairwise accuracy, margin distributions, calibration diagnostics, length and source-model slices, disagreement statistics, and any human audit. If the RM is consumed by a later optimizer, checkpoints must retain the tokenizer, template, scalar-head convention, normalization metadata, and provenance needed to reproduce scores. A scalar head without this interface and data-policy information is not a fully specified reward signal.

15.9 Summary

Preference data complements SFT by expressing a relative judgment between candidate responses rather than a single demonstrated continuation. In the classical pipeline, SFT supplies an initial instruction-following policy, comparison data records which candidates better satisfy a rubric, and a Reward Model turns those comparisons into a reusable ranking signal before RLHF optimizes a policy against it.

The Bradley-Terry model converts a reward-score difference into a pairwise preference probability and yields a logistic negative log-likelihood for RM training. Its score is not an absolute utility or a probability by itself: pairwise labels mainly identify local rankings, and common score shifts remain unidentifiable. Calibration and normalization can make the output better behaved for a declared distribution and interface, but they cannot repair a misaligned criterion.

Reward quality is constrained by the data and search distribution. Human and synthetic preferences each carry provenance and bias; length, style, disagreement, and distribution shift can make high pairwise accuracy misleading. The most important discipline before policy optimization is to keep external, human or task-grounded evaluation separate from the learned proxy. The next chapter can then introduce RLHF as an optimization problem with an explicitly limited reward signal rather than as a mechanism that makes the RM authoritative.

References

[78] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep Reinforcement Learning from Human Preferences," in *Advances in Neural Information Processing Systems 30*, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html

[79] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744. doi: 10.52202/068431-2011.

[80] Y. Bai, S. Kadavath, S. Kundu, and others, "Constitutional AI: Harmlessness from AI Feedback," *arXiv preprint arXiv:2212.08073*, 2022, doi: 10.48550/arXiv.2212.08073.

[81] H. Lee *et al.*, "RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback," *arXiv preprint arXiv:2309.00267*, 2023, doi: 10.48550/arXiv.2309.00267.

[82] N. Stiennon *et al.*, "Learning to Summarize from Human Feedback," in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020. doi: 10.48550/arXiv.2009.01325.

[83] J. Eisenstein *et al.*, "Helping or Herding? Reward Model Ensembles Mitigate but Do Not Eliminate Reward Hacking," *arXiv preprint arXiv:2312.09244*, 2023, doi: 10.48550/arXiv.2312.09244.

[84] N. Lambert *et al.*, "RewardBench: Evaluating Reward Models for Language Modeling," *arXiv preprint arXiv:2403.13787*, 2024, doi: 10.48550/arXiv.2403.13787.

Chapter 16: RLHF and PPO

Abstract

Reinforcement Learning from Human Feedback (RLHF) uses a learned preference signal to improve the behavior of an instruction-following language model beyond direct imitation. This chapter formulates an autoregressive language model as a token-level policy, derives the policy-gradient estimator used to optimize sequence-level rewards, and introduces the actor-critic and Generalized Advantage Estimation machinery that reduces its variance. It then develops Proximal Policy Optimization, distinguishes its clipped surrogate from the separate reference-model KL constraint used in language-model RLHF, and closes with the practical PPO loop, common failure modes, and implementation contracts.

16.1 Introduction

Chapter 13 established an SFT policy that imitates instruction-response demonstrations. Chapter 15 then showed how pairwise comparisons can train a Reward Model (RM) to rank complete responses. The next step is not another static supervised dataset update. It is an optimization problem in which a policy generates new responses, receives a score from the RM, and changes its next-token probabilities to increase expected reward while remaining close to useful SFT behavior.

The classical sequence is

$$\text{Pretraining} \rightarrow \text{SFT} \rightarrow \text{Reward Model} \rightarrow \text{RLHF} \rightarrow \text{PPO}. \tag{139}$$

In this chapter, **RLHF** denotes the policy-optimization stage that consumes the SFT initialization, a fixed or slowly refreshed RM, a prompt distribution, and a reference policy. **PPO** is the policy-gradient method used to make that optimization practical. The discussion is deliberately narrow: it supplies the reinforcement-learning notation needed for language-model post-training, but does not attempt a general reinforcement-learning textbook or introduce DPO and GRPO in depth.

16.2 Autoregressive Language Models as Policies

Let x be a prompt sampled from a declared prompt distribution $\mathcal{D}_{\text{prompt}}$. A generated response is $y = (a_0, ..., a_{T-1})$, where each $a_t \in \mathcal{V}$ is a token. At generation step t, define the state as the prompt together with the generated prefix,

$$s_t = (x, a_0, ..., a_{t-1}), \quad a_t \sim \pi_{\theta(\cdot \mid s_t)}. \tag{140}$$

The policy π_θ is the language model's next-token distribution. The environment transition is almost deterministic: appending a_t produces s_{t+1}, except for stopping rules, truncation, and any external tool or simulator that a later application may introduce. A complete response is a trajectory, and its probability factorizes as

$$\pi_{\theta(y \mid x)} = \prod_{t=0}^{T-1} \pi_{\theta(a_t \mid s_t)}. \tag{141}$$

In the common outcome-reward setting, the RM produces one scalar after a response is complete: $r_{\psi(x,y)}$. The model still takes one token action at a time, but the central quality signal is assigned at the end of the trajectory. This delayed, sampled reward is the source of both the flexibility and the difficulty of RLHF.

RL object	LLM realization	Role in the PPO stage
State s_t	Prompt plus all tokens generated before position t.	Conditions the next-token distribution and value estimate.
Action a_t	One sampled vocabulary token.	Receives a policy-gradient contribution through its log probability.
Trajectory y	A terminated or truncated assistant response.	Is scored by the RM and supplies token-level returns.
Policy π_θ	The trainable autoregressive language model.	Is improved from fresh rollouts using the clipped surrogate.
Reference π_{ref}	A fixed SFT checkpoint, commonly token-compatible with the policy.	Defines the KL anchor, not the rollout behavior policy.

Table 21: The RL abstraction preserves the autoregressive structure of a language model. The same completed response is a token trajectory for the policy and a sequence-level object for the Reward Model.

Table 21 exposes a crucial distinction. The RM is a function that scores a completed (x, y) pair. The policy is a conditional distribution that must choose every token along y. Backpropagating through the RM does not directly differentiate through discrete samples a_t or through the combinatorial set of possible continuations. Policy-gradient estimation supplies a way to improve the distribution from sampled trajectories instead.

16.3 Objective, Reward Shaping, and Policy Gradients

Let the unregularized objective be the expected RM score,

$$J_{\text{RM}(\theta)} = \mathrm{E}_{x\sim\mathcal{D}_{\text{prompt}},y\sim\pi_{\theta(\,\cdot\,\mid\,x)}}\Big[r_{\psi(x,y)}\Big]. \tag{142}$$

Optimizing Equation 142 directly is difficult for three linked reasons. The response distribution changes when θ changes, a response contains many sampled discrete decisions, and RM scores are only approximations to the judgment one ultimately wants. A model that changes its distribution aggressively may visit responses on which the RM was never reliable. Classical language-model RLHF consequently regularizes the policy toward a fixed reference model, often the SFT checkpoint.

For each state, define the forward token-distribution divergence

$$\mathrm{KL}_{t(\theta)} = \sum_{a\in\mathcal{V}} \pi_{\theta(a\,\mid\,s_t)} \log \frac{\pi_{\theta(a\,\mid\,s_t)}}{\pi_{\text{ref}(a\,\mid\,s_t)}}. \tag{143}$$

One conceptual regularized objective is

$$J(\theta) = \mathrm{E}_{x\sim\mathcal{D}_{\text{prompt}},y\sim\pi_{\theta(\,\cdot\,\mid\,x)}}\left[r_{\psi(x,y)} - \beta\sum_{t=0}^{T-1}\mathrm{KL}_{t(\theta)}\right], \quad \beta \geq 0. \tag{144}$$

The coefficient β controls the trade-off. A small value permits greater reward-driven deviation; a large value favors staying near the reference and can make learning nearly inert. The expected KL term is nonnegative, though a sampled per-token log-ratio estimate can be positive or negative. For a trajectory sampled by the behavior-policy snapshot π_{old}, many implementations use a sampled estimate to construct shaped rewards,

$$\tilde{r}_t^{\text{old}} = 1_{t=T-1} r_{\psi(x,y)} - \beta \log \frac{\pi_{\text{old}(a_t\,\mid\,s_t)}}{\pi_{\text{ref}(a_t\,\mid\,s_t)}}, \tag{145}$$

where the RM score is placed at the final response position and the KL cost is distributed across generated tokens. At collection time, π_{old} is the current policy snapshot, so Equation 145 estimates the objective in Equation 144. Once stored, these rewards, returns, and advantages are held fixed during PPO epochs; the changing π_θ then appears through the importance ratio. This changes credit assignment, not the intended trade-off: every token is charged for deviation from the reference, while the final response obtains the outcome score. Fine-Tuning Language Models from Human Preferences and InstructGPT are foundational examples of language-model reward optimization with a reference-model constraint [85], [86].

The log-derivative identity gives a policy-gradient estimator. Let $G_t^{\text{old}} = \sum_{k=t}^{T-1} \gamma^{k-t} \tilde{r}_k^{\text{old}}$ be the return from position t, with discount $\gamma \in [0, 1]$. At rollout collection, the estimator is

$$\nabla_\theta J(\theta) = \mathrm{E}_{\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_{\theta(a_t \mid s_t)} G_t^{\text{old}}\right]} . \tag{146}$$

Equation 146 says that actions preceding a high return should become more likely, while actions preceding a low return should become less likely. It does not say that the last token alone caused the RM score. Every earlier token receives a shared, high-variance learning signal through its effect on the sampled continuation. The policy-gradient theorem formalizes this score-function route for parameterized policies [87].

16.3.1 Return, Value, and Advantage

The four quantities below serve different purposes. A **reward** $\tilde{r}_t^{\text{old}}$ is the immediate scalar assigned at one transition. A **return** G_t^{old} is the discounted sum of future rewards. The state value is the expected future return under a policy,

$$V^{\pi(s_t)} = \mathrm{E}_{\left[G_t^{\text{old}} \mid s_t\right]} . \tag{147}$$

The action value $Q^{\pi(s_t, a_t)}$ is the expected return after taking a_t and following π thereafter. The **advantage** is their difference,

$$A^{\pi(s_t, a_t)} = Q^{\pi(s_t, a_t)} - V^{\pi(s_t)}. \tag{148}$$

An advantage asks whether this action was better or worse than the policy's expected continuation from the same state. Subtracting a state-only baseline such as $V^{\pi(s_t)}$ leaves the expected policy gradient unchanged while greatly reducing variance. Thus reward is the supplied signal, return is accumulated reward, value is a prediction of expected return, and advantage is a centered action-quality estimate. Treating them as interchangeable hides the function of the critic.

16.4 Actor-Critic Estimation and GAE

An actor-critic implementation learns both a policy and a value predictor. The **actor** is π_θ, which samples and assigns log probabilities to tokens. The **critic** is $V_{\varphi(s_t)}$, which predicts the shaped future return; it may use a separate model or a value head on a shared backbone. The critic is not the Reward Model. The RM maps a completed response to a preference-derived score and is normally frozen during a PPO update. The critic predicts the future reward that the current rollout process will produce and is trained from rollout returns.

For a sampled trajectory, define the temporal-difference residual

$$\delta_t = \tilde{r}_t^{\text{old}} + \gamma V_{\varphi(s_{t+1})} - V_{\varphi(s_t)}, \tag{149}$$

with $V_{\varphi(s_T)} = 0$ after termination. Generalized Advantage Estimation (GAE) forms

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \lambda \in [0, 1]. \tag{150}$$

When λ is near one, Equation 150 retains longer-horizon information and generally has less bootstrap bias but more variance. When it is near zero, it relies more heavily on the critic and has lower variance but more bias if the value estimate is inaccurate. Many language-model formulations use $\gamma = 1$ because a finite response is not naturally time-discounted; λ then still controls the bias-variance trade-off in the advantage estimator. GAE was introduced as a practical variance-reduction estimator for policy gradients [88].

The value function is commonly trained by regression to a declared target $\hat{G}_t$, for example a bootstrapped return compatible with the chosen GAE convention:

$$\mathcal{L}_{\text{value}(\varphi)} = \mathrm{E}_{t\left[\left(V_{\varphi(s_t)} - \hat{G}_t\right)^2\right]} . \tag{151}$$

Policy loss and value loss therefore have different targets. The policy loss changes action probabilities according to estimated advantage. The value loss teaches a predictor of shaped future return. A low value loss does not prove that the policy is good; a high RM score does not prove that the critic is accurate.

16.5 From Importance Sampling to PPO

Rollouts are collected from a behavior policy π_{old}, a snapshot of the policy before the current optimization epoch. Once those samples have been stored, the probability ratio for their token actions is

$$r_{t(\theta)} = \frac{\pi_{\theta(a_t \mid s_t)}}{\pi_{\text{old}(a_t \mid s_t)}}. \tag{152}$$

Importance sampling uses Equation 152 to estimate how the new policy would weight actions sampled from the old policy. A simple surrogate is $\mathrm{E}\left[r_{t(\theta)}\hat{A}_t\right]$. It enables more than one minibatch pass over a rollout batch, but it becomes unreliable when the new policy moves far from the behavior policy: rare actions can receive large ratios, and a noisy advantage can drive a destructive update.

Proximal Policy Optimization replaces this surrogate with the clipped objective

$$\mathcal{L}^{\text{CLIP}(\theta)} = \mathrm{E}_{\left[\min\left(r_{t(\theta)}\hat{A}_t,\, \text{clip}\left(r_{t(\theta)}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t\right)\right]}. \tag{153}$$

For a positive advantage, increasing the probability ratio is useful only until it exceeds $1+\varepsilon$; the clipped branch then prevents additional objective gain. For a negative advantage, decreasing the ratio is useful only down to $1-\varepsilon$. The minimum in Equation 153 makes the surrogate pessimistic whenever the ratio leaves this interval in the direction that would otherwise improve the objective. PPO therefore constrains the incentive for one batch of noisy rollout data to induce a very large policy step, while still permitting several minibatch updates. PPO was introduced as a practical clipped surrogate for policy-gradient optimization [89].

16.5.1 PPO Clipping and Reference KL Are Not the Same

PPO clipping compares π_θ with π_{old}, the policy that generated the current on-policy rollouts. Its role is local: stabilize reuse of one rollout batch while the policy is updated. Reference KL compares π_θ with π_{ref}, the fixed SFT policy. Its role is global: preserve a behavioral anchor while the RLHF process moves across many rollout-and-update cycles.

Both mechanisms can limit drift, but neither replaces the other. A clipped update can still make steady, harmful movement away from SFT over many batches. A KL penalty alone can still permit an unstable optimization step relative to π_{old}. In language-model RLHF, the reference constraint also addresses a special problem: the RM is a learned proxy whose errors become easier to exploit as the policy searches beyond its training distribution. Chapter 15's reward-overoptimization warning remains active throughout PPO.

16.6 The PPO Loop for Language Models

One RLHF iteration begins by drawing a batch of prompts, not by reusing a static preference pair as though it were an on-policy trajectory. The current policy generates Rollouts under fixed decoding and stopping rules. The RM scores each completed response, the reference model supplies token log probabilities, and the critic supplies values. The system constructs shaped rewards, returns, and advantages, then performs a limited number of PPO minibatch updates before discarding the rollout batch and sampling again.

Stage	Operation	State that must remain attached
1	Sample prompts	Prompt distribution revision, Chat Template, tokenizer, and prompt identifiers.
2	Generate Rollouts	Behavior-policy revision, token IDs, stop reason, sampled log probabilities, and decoding settings.
3	Score and shape reward	RM revision, reference log probabilities, KL coefficient, terminal-score convention, and reward normalization.
4	Estimate critic targets	Value predictions, masks, returns, advantages, and valid-token reductions.
5	Run PPO epochs	Old-policy log probabilities, clip range, policy and value losses, optimizer state, and minibatch schedule.
6	Evaluate and repeat	External evaluation, RM-score diagnostics, KL statistics, response length, and checkpoint identity.

Table 22: A PPO iteration is an on-policy dataflow, not supervised training on static preference pairs. Each stored rollout needs the model revisions and token-level quantities that defined its objective.

The policy, RM, value model, and reference model have different update schedules. The policy and critic usually change during PPO. The reference normally remains fixed for a run or a defined phase. The RM may remain frozen so that the objective is stable, or be refreshed only through a separately versioned preference-data procedure. Mixing these revisions without recording them makes a reward curve uninterpretable.

16.6.1 Stability, Sensitivity, and Reward Overoptimization

PPO reduces but does not eliminate sensitivity. A large policy learning rate, many epochs over one rollout batch, a broad clip range, or a poorly normalized advantage can cause unstable updates. An inaccurate critic makes $\hat{A}_t^{\mathrm{GAE}}$ noisy or biased; a small rollout batch makes its variance high. Batch size changes both the statistical quality of the advantage estimate and the number of tokens through which the RM score is attributed. Chapters 7 and 8 remain relevant: optimizer settings, gradient clipping, precision, and finite-value checks still control the update path.

KL behavior supplies a useful but incomplete diagnostic. Persistently near-zero KL can indicate an overly strong β, a weak reward signal, or an update regime that is not moving the policy. Rapidly growing KL can indicate excessive drift, but a moderate aggregate KL does not prove that every behavior remains safe or useful. Adaptive KL control can target a declared range, yet it cannot compensate for an RM that ranks the wrong property.

Reward overoptimization is visible when RM reward rises while independent human or task-grounded evaluation stagnates or falls. Common mechanisms include response-length exploitation, polished but unresponsive verbosity, mode collapse toward a narrow high-scoring style, and exploitation of RM artifacts outside the preference-data distribution. The remedy is not simply more PPO clipping. It requires protected evaluation, diverse prompts and candidate distributions, RM audits, conservative policy updates, and explicit decisions about the desired trade-off. The language-model preference-learning literature documents that optimizing a learned reward can exploit labeler heuristics and proxy errors [85], [90].

16.7 Implementation Contracts

The rollout contract must version the prompt distribution, Chat Template, tokenizer, current policy checkpoint, sampling parameters, maximum response length, stop-token policy, and response parsing rule. It must retain token IDs, attention masks, action masks, sampled old-policy log probabilities, and the terminal condition. A prompt token is context, not an RL action; padding and post-termination positions must contribute to neither policy loss nor value loss.

The reward contract must name the RM revision, its sequence serialization and scalar-readout convention, the reference revision, the exact KL estimator, the coefficient β, reward normalization policy, and the terminal reward placement in Equation 145. Tests should confirm that reference log probabilities and RM scores are evaluated on exactly the response tokens generated by the behavior policy. A changed Chat Template, stop convention, or response boundary changes the reward function operationally.

The optimization contract must distinguish π_{old} from π_{ref}, preserve the old log probabilities used in Equation 152, and record ε, γ, λ, PPO epoch count, minibatch construction, policy learning rate, value-loss coefficient, gradient controls, and valid-token denominators. It must declare whether actor and critic share parameters and how their losses are weighted. Diagnostic logs should include RM reward, shaped return, policy loss, value loss, advantage statistics, ratio and clip fractions, KL statistics, response length, entropy, finite-value flags, and independent evaluation results.

Finally, a resumable checkpoint requires more than policy weights. It must preserve the critic and any shared-head state, optimizer and scheduler states, RM and reference identities, prompt-data cursor, rollout-buffer semantics when resuming mid-iteration, random states, counters, and the full objective configuration. Chapter 11′s distinction between a model snapshot and a truly resumable training state applies unchanged to RLHF.

16.8 Summary

RLHF turns a preference-trained RM into an optimization signal for an autoregressive policy. A prompt plus generated prefix is the state, the next token is the action, and the completed response is a trajectory scored by the RM. Because the policy samples discrete tokens and reward is commonly delayed, policy gradients use sampled returns to change token probabilities. The critic supplies a value baseline, and GAE converts shaped rewards and value predictions into lower-variance advantage estimates.

PPO uses the old-policy probability ratio and a clipped surrogate to restrain each update on fresh Rollouts. The reference-model KL term serves a different purpose: it anchors the evolving policy to the SFT distribution and makes the reward-versus-drift trade-off explicit through β. Neither mechanism makes the RM correct. Policy loss, value loss, RM score, and external quality are distinct signals that must be monitored separately.

The practical loop is therefore a versioned on-policy system: sample prompts, generate responses, score them, estimate returns and advantages, update the policy and critic cautiously, evaluate externally, and repeat. Its central risk is reward overoptimization, in which a policy improves the learned proxy while degrading the intended behavior. The next post-training methods can be understood only after this distinction between preference-derived reward, policy optimization, and independent evaluation is clear.

References

[85] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019, doi: 10.48550/arXiv.1909.08593.

[86] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744. doi: 10.52202/068431-2011.

[87] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy Gradient Methods for Reinforcement Learning with Function Approximation," in *Advances in Neural Information Processing Systems 12*, MIT Press, 2000. [Online]. Available: https://proceedings.neurips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html

[88] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015, doi: 10.48550/arXiv.1506.02438.

[89] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017, doi: 10.48550/arXiv.1707.06347.

[90] J. Eisenstein *et al.*, "Helping or Herding? Reward Model Ensembles Mitigate but Do Not Eliminate Reward Hacking," *arXiv preprint arXiv:2312.09244*, 2023, doi: 10.48550/arXiv.2312.09244.

Chapter 17: Direct Preference Optimization

Abstract

Direct Preference Optimization (DPO) trains an instruction-following policy directly from chosen–rejected response pairs. This chapter derives the DPO loss from a KL-regularized reward-maximization view of RLHF, showing how a policy and a fixed reference model define an implicit reward difference whose prompt-dependent normalization cancels in pairwise comparisons. It explains sequence-level log probabilities, the role of the coefficient β, the offline training loop, and the practical distinction between DPO and Reward Model plus PPO pipelines. It also identifies the data, reference, and distributional limitations that a simpler objective does not remove.

17.1 Introduction

Chapter 13 introduced Supervised Fine-Tuning (SFT), which imitates demonstrated instruction–response pairs. Chapter 15 then changed the supervision primitive from a demonstration to a comparison, and Chapter 16 showed how classical RLHF can train a Reward Model (RM) on those comparisons and optimize a policy with PPO. Direct Preference Optimization (DPO) takes a different route: it uses the preference pairs to update the policy directly, without first fitting a separately parameterized RM or running an on-policy actor–critic loop.

The two pipelines share their SFT starting point and their dependence on preference data:

$$\text{SFT} \rightarrow \text{Preference Data} \rightarrow \text{Reward Model} \rightarrow \text{RLHF} + \text{PPO}, \quad \text{SFT} \rightarrow \text{Preference Data} \rightarrow \text{DPO}. \tag{154}$$

DPO is not ordinary SFT on the chosen response. Its loss compares how the trainable policy changes the relative likelihood of the chosen and rejected completions **against a fixed reference policy**. That comparison follows from a particular KL-regularized RLHF model under a Bradley–Terry preference assumption [91]. The connection is useful, but it is not a claim that DPO and PPO expose the same operational interface: one consumes a fixed offline dataset, while the other repeatedly generates and scores new Rollouts.

17.2 Preference Pairs and Sequence-Level Policy Scores

Let $\mathcal{P} = \{(x_i, y_{\mathrm{w}}^i, y_{\mathrm{l}}^i)\}_{i=1}^N$ be a dataset of prompts x, chosen responses y_{w}, and rejected responses y_{l}, using the meaning established in Chapter 15. Let π_θ be the trainable policy and π_{ref} be a frozen reference policy, commonly the SFT checkpoint that initializes π_θ. Both policies must assign probabilities under exactly the same tokenizer, Chat Template, role markers, stopping convention, and response boundaries.

If $y = (y_0, \ldots, y_{T-1})$ is a response conditioned on x, its sequence-level log probability is the sum of teacher-forced token log probabilities,

$$\log \pi_{\theta(y \mid x)} = \sum_{t=0}^{T-1} \log \pi_{\theta(y_t \mid x, y_0, \ldots, y_{t-1})}. \tag{155}$$

Thus **token-level log probability** is one conditional term in Equation 155, while **sequence-level log probability** is the log probability of the entire response. In a batched implementation, only assistant-response positions contribute to this sum. Prompt, padding, and positions after termination are context or invalid positions, not response targets. Whether the terminal token is included is a declared modeling choice, but the policy and reference must use the same choice.

The basic quantity in DPO is the log-probability change relative to the reference,

$$q_{\theta(x,y)} = \log \pi_{\theta(y \mid x)} - \log \pi_{\mathrm{ref}(y \mid x)}. \tag{156}$$

$q_{\theta(x,y)}$ is not a reward observed from an annotator. It is a model-dependent likelihood ratio in log space: it is positive when the policy has made this response more likely than the reference and negative when it has made it less likely. Comparing the two responses gives the reference-relative policy margin

$$\Delta_{\theta(x, y_{\mathrm{w}}, y_{\mathrm{l}})} = q_{\theta(x, y_{\mathrm{w}})} - q_{\theta(x, y_{\mathrm{l}})}. \tag{157}$$

Increasing Equation 157 makes the chosen completion relatively more probable and the rejected completion relatively less probable than they were under the reference. The subtraction is essential.

A standalone high likelihood for y_{w} is not enough: a policy can increase both responses, or favor the rejected response even more strongly.

17.3 From KL-Regularized RLHF to an Implicit Reward

The RLHF objective in Chapter 16 conceptualizes a policy as maximizing a response-level reward while paying a KL cost for moving away from a reference. For one prompt x, write that objective as

$$\max_{\pi}; \mathrm{E}_{y\sim\pi(.\mid x)}\left[r(x,y)-\beta\log\frac{\pi(y\mid x)}{\pi_{\mathrm{ref}(y\mid x)}}\right],\quad \beta>0. \tag{158}$$

Here $r(x,y)$ is a latent preference reward and β is the coefficient on reference-relative deviation. Under this objective, the optimal policy has the exponential-tilting form

$$\pi_{r(y\mid x)}=\frac{\pi_{\mathrm{ref}(y\mid x)}\exp\left(\frac{r(x,y)}{\beta}\right)}{Z(x)},\quad Z(x)=\sum_{y'}\pi_{\mathrm{ref}(y'\mid x)}\exp\left(\frac{r(x,y')}{\beta}\right). \tag{159}$$

The normalizer $Z(x)$ sums over complete responses and is not a quantity that ordinary language-model training can enumerate. Taking logarithms and rearranging Equation 159 nevertheless yields

$$r(x,y)=\beta\left[\log\pi_{r(y\mid x)}-\log\pi_{\mathrm{ref}(y\mid x)}\right]+\beta\log Z(x). \tag{160}$$

The final term depends on the prompt but not on which candidate response is compared. The Bradley–Terry model from Chapter 15 uses only a reward difference. For two candidates, the prompt-only term in Equation 160 cancels:

$$r(x,y_{\mathrm{w}})-r(x,y_{\mathrm{l}})=\beta\left[q_{r(x,y_{\mathrm{w}})}-q_{r(x,y_{\mathrm{l}})}\right]. \tag{161}$$

This is the **implicit reward view**. If π_θ is used in place of the unknown optimal policy π_r, then

$$\hat{r}_{\theta(x,y)}=\beta\left[\log\pi_{\theta(y\mid x)}-\log\pi_{\mathrm{ref}(y\mid x)}\right] \tag{162}$$

behaves as a reward parameterization inside the pairwise likelihood. It is not a separately trained scalar Reward Model that can independently score arbitrary responses. It is defined only through the policy and reference probabilities, and its scale depends on β and the chosen reference. The derivation establishes an equivalence under its modeling assumptions; it does not prove that the implicit quantity is a calibrated measure of human value [91].

17.3.1 The Reference Model and β

The reference model has two roles. In the derivation, it defines the KL anchor in Equation 158. In the implemented loss, it makes DPO optimize a **relative** margin rather than merely rewarding frequent chosen strings. A response that the reference already strongly favors receives a different training signal from a response that becomes likely only after adaptation. The reference must remain frozen while a DPO run is interpreted; otherwise the target margin changes as the optimizer moves.

The coefficient β appears both as the KL coefficient in the derivation and as the scale on the preference margin in the resulting logistic loss. It therefore controls the sensitivity of the loss to a given policy-versus-reference change. Larger β makes a fixed relative margin more decisive in the sigmoid, and changing it changes the balance implied by the KL-regularized model. It should not be described as a universal measure of how strongly humans prefer one answer. Its useful range depends on model initialization, pair quality, response lengths, optimizer settings, and the desired amount of deviation from the reference.

17.4 The DPO Objective

Substituting Equation 161 into the Bradley–Terry likelihood gives the DPO loss

$$\mathcal{L}_{\mathrm{DPO}(\theta)}=-\mathrm{E}_{(x,y_{\mathrm{w}},y_{\mathrm{l}})\sim\mathcal{P}}\left[\log\sigma\Big(\beta\left[\log\pi_{\theta(y_{\mathrm{w}}\mid x)}-\log\pi_{\mathrm{ref}(y_{\mathrm{w}}\mid x)}-\log\pi_{\theta(y_{\mathrm{l}}\mid x)}+\log\pi_{\mathrm{ref}(y_{\mathrm{l}}\mid x)}\right]\Big)\right] \tag{163}$$

Equivalently, the sigmoid argument is $\beta\Delta_\theta$ from Equation 157. It is large and positive when the policy has increased the chosen response's log probability relative to the reference more than it has increased the rejected response's. The negative log-sigmoid then becomes small. If the policy favors the rejected response in relative terms, the margin is negative and the loss applies a larger correction. This is a

binary classification loss over pair orderings, but its features are sequence likelihood ratios rather than an independently computed label embedding.

The gradient clarifies why DPO is more than maximizing the chosen likelihood. Let $z = \beta\Delta_\theta$. Since

$$\frac{\partial[-\log\sigma(z)]}{\partial z} = \sigma(z) - 1, \tag{164}$$

the update is strongest for pairs whose current reference-relative ordering is uncertain or wrong. Differentiating z changes $\log \pi_{\theta(y_w \mid x)}$ upward and $\log \pi_{\theta(y_l \mid x)}$ downward, with a coupling determined by the sigmoid. The reference log probabilities are constants in this differentiation. A naive unlikelihood-style objective that ignores the reference-relative weighting does not preserve this particular KL-regularized interpretation.

17.5 Offline Training Loop and Its Trade-Offs

The dataflow is substantially simpler than PPO, but it is still a careful likelihood computation rather than a format-agnostic label update.

Stage	Operation	Invariant to preserve
1	Load a preference pair	Prompt, chosen and rejected response, label orientation, and data split remain recoverable.
2	Serialize two continuations	Chat Template, tokenizer, response boundaries, terminal-token rule, and target masks match exactly.
3	Compute policy and reference log probabilities	Both models score the same valid response-token positions; reference outputs are detached.
4	Form the DPO loss	β, sequence-level reductions, pair weighting, and valid-pair denominator are declared.
5	Update only the policy	Optimizer state, gradient controls, checkpoints, and evaluation revisions are recorded.

Table 23: DPO is an offline pairwise likelihood loop. Each pair needs policy and reference likelihoods for both responses; it needs neither a separately trained RM, an actor–critic target, nor on-policy Rollouts.

The policy begins from SFT or a compatible instruction-following checkpoint. For each pair, the trainable policy and frozen reference score both completions by teacher forcing. The loss in Equation 163 is averaged over valid pairs, backpropagated through the policy only, and optimized with the ordinary training machinery discussed in Chapters 7 and 8. The reference can often be evaluated without gradient storage, but it remains a material memory and throughput cost unless an equivalent implementation strategy is used.

Property	DPO	RM plus PPO RLHF
Primary supervision	Fixed chosen–rejected preference pairs.	Preference pairs train an RM; fresh Rollouts are then scored.
Optimization signal	Reference-relative sequence likelihood margin.	RM reward, shaped returns, and estimated advantages.
Learned auxiliary models	No explicit RM or Value Model is required by the basic objective.	An RM and normally a critic or value head are required.
Policy data during update	Offline dataset; no rollout refresh is inherent.	On-policy responses from the current behavior policy.
Principal strength	Simple, stable supervised-style optimization interface.	Can optimize newly sampled behavior and more flexible reward signals.
Principal limitation	Cannot correct missing coverage by searching for new feedback during a fixed run.	More coupled models, sampling, variance, and stability controls.

Table 24: DPO and PPO-based RLHF use preference information differently. Neither table entry is an unconditional quality ranking: the preferred method depends on the feedback, reward interface, and need for online interaction.

Table 24 identifies the practical transition. DPO avoids explicit RM fitting, rollout-based PPO optimization, Value Model training, and advantage estimation. It does not make preference learning free: it still needs correctly serialized pair data, policy and reference likelihoods for both responses, optimization controls, protected evaluation, and a defensible reference policy. PPO remains useful when a system can collect new on-policy behavior, needs a reward from a verifier, simulator, or other non-pairwise signal, or needs to optimize an objective whose feedback is not fully represented in a static comparison corpus. DPO is therefore a simpler offline preference-optimization pipeline, not a universally superior replacement for online policy optimization.

17.6 Data Limits, Failure Modes, and Extensions

The DPO objective can faithfully optimize the information in a pair while still moving toward an undesirable policy. Incorrect label orientation, duplicated prompts, ambiguous or inconsistent annotations, low-quality synthetic judges, and candidates with systematic formatting artifacts all change the learned margin. The data-quality and annotation-noise analysis of Chapter 15 therefore remains a prerequisite rather than a stage DPO bypasses.

Distribution shift takes a different form from the RM-search problem in PPO. DPO does not deliberately sample new responses during a fixed offline update, so it does not use an RM to rank its own increasingly unusual Rollouts. It also cannot obtain preference supervision for behaviors absent from its dataset. If deployment prompts, response lengths, languages, or failure cases differ from the recorded pairs, the likelihood-ratio objective has no automatic corrective signal. A high held-out pair accuracy or a low DPO loss on an in-distribution split is not evidence of broad behavioral alignment.

Length deserves special attention. Standard DPO uses the sum in Equation 155, so longer responses contain more token log-probability terms. Whether that creates a harmful preference depends on the pair distribution, response-length correlation, stop handling, and evaluation protocol; it cannot be diagnosed from the loss alone. Dividing by length changes the objective and should not be introduced as an innocent numerical stabilization. Independent length-controlled evaluation remains necessary, as it does for RMs and PPO policies.

The reference is another possible failure boundary. A weak or mismatched reference can make relative scores poorly aligned with the intended starting behavior. An overly aggressive optimization configuration can drift from the reference despite the derived interpretation, while a conservative configuration can make little meaningful change. The theoretical assumptions behind the derivation also include a specified preference model and adequate support of the reference policy. Real datasets contain finite, noisy, and often deterministically labeled comparisons; these conditions constrain what can be concluded from the formal equivalence. Analyses of direct preference objectives emphasize that DPO's offline formulation has its own regularization and overfitting behavior [92].

Several direct-preference methods alter the link function, regularization, target margin, or feedback setting. Identity Preference Optimization (IPO), for example, arises from a different choice within a broader preference-optimization formulation and is motivated in part by DPO's potential overfitting behavior under deterministic labels [92]. Such variants are useful reminders that "direct preference optimization" names a family of design choices, not one interchangeable loss. This chapter does not develop GRPO, online preference optimization, or reasoning-specific objectives; those require their own data-collection and credit-assignment analysis.

17.7 Implementation Contracts

The data contract must version the prompt and conversation serialization, tokenizer, Chat Template, role and special tokens, chosen/rejected orientation, annotation or judge source, preference rubric, candidate provenance, deduplication policy, split assignment, and response stop convention. It must reject or explicitly handle a pair with missing boundaries, an empty target span, unsupported roles, identical candidates, malformed terminal tokens, or an undecided label. A role reversal test should change the sign of Equation 157 and convert the pair into its complementary training example.

The scoring contract must define the response-token mask used in Equation 155, whether the end-of-sequence token belongs to the target, truncation and packing behavior, per-sequence reduction, and pair-level averaging. It must ensure that π_θ and π_{ref} score the identical token IDs at identical positions. Tests should verify that padding, prompt tokens, and post-termination positions contribute neither to the policy nor the reference response log probability; they should also verify that the reference path is frozen and detached from gradient updates.

The optimization contract must record the reference checkpoint, β, policy initialization, precision configuration, optimizer and scheduler state, learning rate, effective batch size, gradient accumulation and clipping rules, pair weighting, checkpoint policy, and evaluation datasets. Logs should include chosen and rejected policy log probabilities, their reference-relative margin, DPO loss, response lengths, gradient statistics, finite-value checks, and protected task or human evaluation. Chapter 11's resumability requirements apply: recovering only the policy weights does not reproduce a run without its optimizer, scheduler, data-order, random-state, and configuration state.

17.8 Summary

DPO converts a chosen–rejected preference pair into a direct update of an autoregressive policy. The central objects are sequence-level log probabilities for the policy and a frozen reference. Their difference defines a reference-relative score; comparing that score for the chosen and rejected responses yields the margin optimized by the DPO logistic loss.

The formal bridge to RLHF begins with KL-regularized reward maximization. Its optimal policy can be written as an exponential tilt of the reference, allowing a reward difference to be represented by policy-to-reference log-probability differences. The prompt-only normalization cancels in the Bradley–Terry comparison, which is why an explicit RM need not be fitted for the basic DPO objective. The resulting method avoids rollout collection, PPO clipping, Value Model fitting, and advantage estimation during its offline update.

Those simplifications do not remove the limits of preference data. Annotation noise, response-length and style bias, reference dependence, distribution shift, and insufficient coverage remain policy-level risks. PPO-based RLHF and DPO therefore offer different interfaces to feedback: PPO can optimize new on-policy behavior and flexible rewards, whereas DPO offers a compact, offline likelihood objective for a fixed preference corpus. Choosing between them is a question about available feedback and evaluation evidence, not a generic ranking of algorithms.

References

[91] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model," in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023, pp. 53728–53741. doi: 10.48550/arXiv.2305.18290.

[92] M. Gheshlaghi Azar *et al.*, "A General Theoretical Paradigm to Understand Learning from Human Preferences," in *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*, in Proceedings of Machine Learning Research, vol. 238. PMLR, 2024, pp. 4447–4455. [Online]. Available: https://proceedings.mlr.press/v238/gheshlaghi-azar24a.html

Chapter 18: Group Relative Policy Optimization

Abstract

Group Relative Policy Optimization (GRPO) is an online policy-optimization method that compares several responses sampled for the same prompt. Rather than training a separate Value Model to estimate an advantage, it uses the group's reward distribution to construct a relative, advantage-like signal for each response. This chapter develops group-normalized rewards, the PPO-style clipped objective and reference-model constraint retained by GRPO, and the resulting rollout loop. It distinguishes GRPO from PPO and Direct Preference Optimization, explains why verifiable tasks are a natural setting for group-relative learning, and identifies the sampling, reward, and stability conditions on which the method depends.

18.1 Introduction

Chapter 16 formulated an autoregressive language model as a policy and developed PPO as an actor–critic method for optimizing rollout rewards. Chapter 17 then presented DPO, which uses fixed preference pairs in an offline likelihood objective. Group Relative Policy Optimization (GRPO) is closer to PPO in its dataflow: it samples new responses from a behavior-policy snapshot, scores them, and applies a constrained policy update. Its distinctive change is the baseline used to assign credit.

In the common outcome-reward setting, a completed response receives one scalar score. PPO usually learns a Value Model to estimate whether each token action led to a better or worse return than expected from its prefix. GRPO instead samples several responses for the **same** prompt and asks a simpler comparative question: which sampled responses did better than the other responses in this group? The group mean provides a prompt-local baseline, and the group spread provides a local scale. GRPO was introduced in DeepSeekMath as a PPO variant that forgoes the critic model in favor of group-relative rewards [93].

The method is therefore neither ordinary supervised learning nor offline preference optimization. Its central loop is

$$\text{prompt} \to \text{group Rollouts} \to \text{score} + \text{normalize} \to \text{clipped update} + \text{KL} \to \text{repeat}. \tag{165}$$

This chapter develops that loop without treating it as a full account of Reasoning RL. In particular, it uses verifiable rewards to explain why group comparisons can be useful, but leaves the design of Outcome Reward, Process Reward, and verifier systems to the next chapter.

18.2 Group-Based Rollouts and Relative Rewards

Let x be a prompt drawn from a declared distribution $\mathcal{D}_{\text{prompt}}$. At one rollout step, freeze the current behavior policy as π_{old} and sample a group of G responses,

$$y_i = \left(a_{i,0}, ..., a_{i,T_i-1}\right) \sim \pi_{\text{old}(.\mid x)}, \quad i \in \{1, ..., G\}. \tag{166}$$

The responses in Equation 166 share the prompt and sampling policy revision, but they may have different lengths, stop reasons, and reward outcomes. A reward procedure assigns each response a scalar r_i. It may be a learned Reward Model score, a rule-based verifier, a program-execution result, or a declared combination of such signals. The score is an **absolute reward** in the sense that the reward procedure evaluates each response independently; it need not be a probability or a calibrated utility, as Chapter 15 explained for learned Reward Models.

The group statistics are

$$\mu_r = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad s_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\left(r_j - \mu_r\right)^2}. \tag{167}$$

A robust group-relative normalization is

$$\hat{A}_i = \frac{r_i - \mu_r}{s_r + \delta}, \quad \delta > 0. \tag{168}$$

Here $\hat{A}_i$ is positive when response i scored above its group mean and negative when it scored below it. It is **relative reward information**: adding the same constant to every r_i leaves it unchanged.

Dividing by the within-group spread makes the scale comparable across prompts whose raw reward magnitudes differ. The small δ is an implementation safeguard for zero or near-zero spread. The original GRPO formulation normalizes rewards by their group mean and standard deviation; precise choices for variance convention, clipping, reward transforms, and zero-variance handling vary across implementations [93].

Equation 168 is advantage-like, but it is not the learned value-based advantage $Q^{\pi(s_t,a_t)} - V^{\pi(s_t,a_t)}$ introduced in Chapter 16. It compares whole sampled responses for one prompt rather than predicting a return from every token prefix. In an outcome-reward implementation, the same $\hat{A}_i$ is often broadcast to every valid response-token position in y_i:

$$\hat{A}_{i,t} = \hat{A}_i, \quad 0 \le t < T_i. \tag{169}$$

This gives all generated tokens in a successful response a shared positive signal and all tokens in a relatively poor response a shared negative signal. It is a coarse credit assignment, not evidence that every token causally produced the final outcome. More granular reward assignments exist, but they are outside this chapter's scope.

18.2.1 Why a Group Can Replace a Separate Value Model

The expected return from a prompt can vary substantially with prompt difficulty, answer length, and reward scale. A learned critic in PPO attempts to predict that expected return from each state. GRPO replaces this prediction with evidence available at rollout time: for the same prompt, the sampled responses themselves estimate a local performance distribution. The group average acts as a baseline, so an update emphasizes responses that are better or worse than their siblings rather than merely responses with large raw scores.

This can eliminate the separate Value Model, its optimizer state, and its value-loss tuning. It does not eliminate the need for a baseline in the conceptual sense, nor does it create a precise token-level value estimate. Group-relative learning is most informative when the group contains meaningful variation. If every response is identically wrong, identically correct, or receives the same reward, then $s_r = 0$ and every numerator in Equation 168 is zero. Such a group supplies no ranking signal after the chosen zero-variance rule is applied.

18.3 The GRPO Policy Objective

Although GRPO changes the advantage estimator, it retains PPO's local trust mechanism. For action $a_{i,t}$ sampled under π_{old}, define the token probability ratio

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(a_{i,t} \mid x, a_{i,<t})}{\pi_{\text{old}}(a_{i,t} \mid x, a_{i,<t})}. \tag{170}$$

π_{old} is the rollout behavior policy, whereas π_{ref} below is the behavioral anchor. They have different jobs. The ratio in Equation 170 allows limited reuse of samples collected before the current update. The reference model constrains longer-term drift across many such rollout batches.

Let $\kappa_{i,t}(\theta)$ denote the declared KL penalty between the current and reference next-token distributions at the same prefix,

$$\kappa_{i,t}(\theta) = \mathrm{KL}\Big(\pi_{\theta(\,\cdot \mid x, a_{i,<t})} \,\Big\|\, \pi_{\text{ref}(\,\cdot \mid x, a_{i,<t})}\Big). \tag{171}$$

One representative GRPO objective is

$$\mathcal{J}_{\text{GRPO}(\theta)} = \mathrm{E}\Big[\tfrac{1}{G}\sum_{i=1}^{G} \tfrac{1}{T_i}\sum_{t=0}^{T_i-1} \big(\min\big(\rho_{i,t}(\theta)\hat{A}_i, \mathrm{clip}\big(\rho_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon\big)\hat{A}_i\big) - \beta\kappa_{i,t}(\theta)\big)\Big]. \tag{172}$$

The expectation in Equation 172 is over prompt batches and groups generated by π_{old}. The $\frac{1}{T_i}$ factor is one common response-length normalization; systems must declare whether they use it because changing the denominator changes how long responses are weighted. The objective is representative rather than universal. In particular, practical systems may use a sampled KL estimator, an alternative reduction over tokens, or a different reward-normalization convention.

For $\hat{A}_i > 0$, the clipped term gives the policy an incentive to increase the likelihood of the response's sampled tokens, but only until the ratio exceeds $1 + \varepsilon$. For $\hat{A}_i < 0$, it discourages those tokens only

until the ratio falls below $1 - \varepsilon$. This is the same local update-control idea as PPO's clipped surrogate [94]. The reference penalty is separate: β controls the reward-versus-drift trade-off relative to π_{ref}, while ε restricts how far one update can move from π_{old} on one rollout batch.

18.3.1 Reward Normalization Is Part of the Objective

Group normalization changes the optimization signal rather than merely making logs look comparable. If raw rewards are shifted by a prompt-specific constant, Equation 168 is unchanged. If the reward spread is very small, normalization can magnify tiny reward differences unless the implementation explicitly guards against it. A discrete verifier can also produce a group with only zeros and ones; the resulting advantage depends on how many samples passed, not only on whether one individual response passed.

For this reason, reward transforms, reward clipping, standard-deviation convention, δ, and the treatment of all-equal groups are algorithmic choices. A system should not silently replace an all-equal group's zero advantage with a global batch statistic and still call the result the same group-relative objective. Such a fallback may be useful, but it changes the baseline and should be evaluated as a different design.

18.4 Training Loop, Sampling, and Verifiable Rewards

The GRPO loop consumes fresh online data. Each prompt yields several responses under a fixed behavior-policy revision; scores and advantages are then frozen while the policy performs a limited number of clipped-update epochs. After that, the old group is discarded and a new group is sampled from the updated policy.

Stage	Operation	State that must remain attached
1	Sample prompts and freeze π_{old}	Prompt distribution, Chat Template, tokenizer, policy revision, and decoding settings.
2	Generate G Rollouts per prompt	Token IDs, stop reasons, masks, old-policy log probabilities, and response lengths.
3	Score every response	Reward-function revision, verifier or RM configuration, parsing result, and per-response reward components.
4	Normalize within each group	Group membership, mean, spread convention, δ, zero-variance policy, and advantages.
5	Run limited policy epochs	Reference revision, KL estimator, ε, β, valid-token denominator, optimizer state, and diagnostics.
6	Evaluate and resample	Independent evaluation, reward statistics, KL, pass rate, response length, and checkpoint identity.

Table 25: A GRPO iteration remains an on-policy dataflow. The group identity and the behavior-policy revision are part of the objective, not incidental batch metadata.

Group size G determines a direct statistical and systems trade-off. A larger group gives a more informative local ranking distribution and increases the chance that a difficult prompt contains both a strong and weak response. It also multiplies generation, reward-scoring, and memory or storage cost per prompt. A small group is cheaper but may have unstable means and spreads; $G = 1$ gives no nonzero centered relative advantage at all. The useful group size depends on task difficulty, reward reliability, decoding diversity, and the available rollout budget rather than on a universal constant.

GRPO is particularly natural when a response can be checked with a reproducible external rule. A mathematics answer can be compared with a known answer under a strict parser; a code solution can be run against tests; a symbolic derivation or formal proof can be checked by a declared system. These **verifiable rewards** can be sparse but comparatively reliable for the property they actually test. They do not certify every desirable property of the response: a parser can be gamed, a test suite can be incomplete, and a correct final answer does not necessarily validate every intermediate statement. The verifier, format convention, and failure behavior therefore define the operational reward function.

A learned RM is also compatible with GRPO, but a group-relative advantage does not remove the RM's bias, calibration, or distribution-shift limits described in Chapter 15. The relative construction can make a prompt-local comparison useful, yet a policy may still exploit features that raise the RM score without improving external quality. Independent evaluation remains necessary whether rewards come from a learned judge or an automatic checker.

18.5 GRPO, PPO, and DPO

GRPO and PPO are both online policy-optimization methods. They generate responses from a behavior policy, use an old-policy ratio, and commonly apply clipping and reference-model regularization. The difference is the advantage source. PPO learns or shares a critic that estimates value from token prefixes and can use GAE. GRPO obtains a response-level baseline from other samples for the same prompt and commonly avoids a separate Value Model. This reduces one substantial model and state burden, but it exchanges learned value prediction for extra rollout sampling and coarser outcome-level credit assignment.

DPO differs more fundamentally. It optimizes a fixed dataset of chosen–rejected pairs using policy and reference likelihoods; it does not require fresh Rollouts, a behavior-policy ratio, clipping against π_{old}, or group rewards. Its primary failure boundary is the coverage and quality of the offline preference corpus. GRPO can explore fresh policy behavior and consume a numeric reward from a verifier or RM, but it incurs the cost and instability controls of online generation. Neither method is automatically preferable: DPO offers a compact offline preference-learning interface, PPO offers flexible actor–critic optimization, and GRPO offers a group-relative alternative when multiple scored samples per prompt are feasible.

Property	GRPO	PPO and DPO contrast
Update data	Online groups of G Rollouts per prompt.	PPO uses online Rollouts; DPO uses offline chosen–rejected pairs.
Advantage or margin	Within-group normalized reward.	PPO uses a learned value-based estimate; DPO uses a reference-relative likelihood margin.
Auxiliary models	No separate Value Model in the basic formulation.	PPO normally has a critic; DPO needs neither a critic nor an explicit RM.
Feedback interface	Scalar scores from an RM, verifier, or other declared reward function.	PPO similarly accepts scores; DPO consumes pair orderings.
Main trade-off	Avoids critic state but requires multiple sampled responses.	PPO trades sampling for a critic; DPO trades online adaptation for offline simplicity.

Table 26: GRPO changes the online advantage estimator rather than replacing PPO's trust mechanisms. DPO has a different, offline data interface.

18.6 Stability and Failure Modes

GRPO does not make policy optimization insensitive. A large learning rate, excessive reuse of one rollout group, broad clipping range, weak KL control, or a poorly specified valid-token denominator can still produce abrupt policy changes. The diagnostics of Chapter 16 remain relevant: monitor objective terms, ratio and clip fractions, KL statistics, response length, entropy, finite values, and independent task results. Chapter 8′s numerical checks and Chapter 11′s checkpoint discipline also apply unchanged.

Reward variance has a special role. Groups with no reward variation provide no centered signal; groups whose scores are dominated by noise can create arbitrary positive and negative updates; groups with a single accidental high score can overemphasize an outlier. Increasing G may improve the chance of useful contrast, but it also costs more Rollouts and does not repair a biased reward function. Insufficient sampling diversity can make every response nearly identical, while excessive diversity can produce mostly invalid outputs whose comparisons teach little about the desired regime.

The same reward-hacking and length-bias concerns recur in a new form. A policy can learn to exploit a verifier's parser, a test-suite loophole, or an RM's stylistic heuristic. A reward that favors long answers

can alter both pass probability and group ranking; sequence-length normalization in the objective does not by itself establish a fair reward. Rapid KL growth can signal excessive movement away from the reference, whereas persistently negligible KL can indicate that rewards, advantages, or update settings are not producing useful learning. These are hypotheses to investigate with protected evaluation, not standalone diagnoses.

18.7 Implementation Contracts

The rollout contract must version the prompt distribution, Chat Template, tokenizer, policy and old-policy revisions, decoding parameters, group size G, maximum response length, stop-token policy, and response parser. It must preserve prompt identifiers, group membership, token IDs, response and action masks, old-policy log probabilities, stop reasons, and per-response lengths. A prompt token, padding token, and post-termination position must contribute to neither the policy objective nor its per-token KL term.

The reward contract must name the reward-function revision, whether the score comes from an RM, verifier, test executor, or mixture, and the serialization, parsing, timeout, failure, and reward-composition rules. It must retain the raw r_i, group mean, spread convention, δ, normalized $\hat{A}_i$, and all-equal-group policy. Tests should confirm that permuting group order only permutes the corresponding advantages; shifting every raw reward by a common constant leaves Equation 168 unchanged; and an all-equal group produces zero advantage under the declared fallback.

The optimization contract must distinguish π_{old} from π_{ref}, record the reference revision, exact KL estimator, ε, β, number of policy epochs, response-length reduction, optimizer and scheduler states, precision, effective batch size, gradient accumulation and clipping, and checkpoint policy. Logs should separately report raw reward, normalized advantage, verifier pass rate or RM score, reward spread, zero-variance-group rate, policy loss, ratio and clip fractions, KL, entropy, response length, throughput, and independent evaluation. Recoverable checkpoints require the policy, optimizer, scheduler, random states, prompt-data cursor, objective configuration, and any incomplete rollout-buffer state, following Chapter 11.

18.8 Summary

GRPO samples several responses for one prompt and turns their relative reward performance into an advantage-like training signal. Centering and scaling the group's rewards makes a response positive when it is better than its peers and negative when it is worse. This prompt-local baseline can replace a learned Value Model when response-level comparisons provide enough information for the training task, but it does not create fine-grained credit assignment or remove the need for careful reward design.

The policy update remains PPO-like. A frozen behavior-policy snapshot supplies probability ratios, clipping limits reuse of a rollout batch, and a distinct reference model supplies KL control. The resulting method is online: new policy behavior is generated, scored, normalized within groups, updated cautiously, and sampled again. The old policy and reference policy are different objects and must never be conflated.

GRPO is well suited to tasks with reproducible score functions, including mathematical answers, executable code, and symbolic checks, because multiple samples can expose prompt-local differences in correctness. Its limits are equally practical: group size raises sampling cost, equal or noisy rewards weaken learning, learned scores can still be hacked, and verifiers only reward what they actually check. GRPO, PPO, and DPO therefore represent different interfaces to feedback and compute, not a universal hierarchy of post-training methods.

References

[93] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024, doi: 10.48550/arXiv.2402.03300.

[94] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017, doi: 10.48550/arXiv.1707.06347.

Chapter 19: Reasoning RL, Rollouts, and Verifiable Rewards

Abstract

Reasoning-oriented post-training changes the feedback interface available to a language model. In domains such as mathematics and programming, a sampled solution can often be checked by an executable or rule-based procedure rather than ranked only by a human preference. This chapter treats a Rollout as an autoregressive reasoning trajectory, distinguishes Outcome Reward, Process Reward, learned Reward Models, and deterministic verifiers, and explains why multiple samples can support both online policy optimization and inference-time selection. It develops the idealized success probability of Best-of-N sampling, clarifies the role of Pass@k and self-consistency, and identifies the credit-assignment, diversity, verification, and curriculum conditions that limit reasoning RL.

19.1 Introduction

Chapters 15–18 developed post-training around preferences, learned Reward Models, PPO, DPO, and GRPO. Those methods accept several kinds of feedback, but generic preference data leaves a difficult question unresolved: how can a system obtain a reliable scalar signal for a newly generated, long solution? Reasoning tasks sometimes offer a narrower but powerful answer. A numerical answer may be checked against a known value, a program can be executed against tests, and a formal object can be checked against declared rules. The resulting signal is often called a **verifiable reward**.

The important shift is not that every reasoning task has a perfect automatic judge. It is that correctness can be operationalized for some tasks more reproducibly than a broad human-style preference. This makes it practical to generate new candidate solutions, verify or score them, and use the outcomes for online policy optimization. DeepSeekMath is one example of group-based RL on mathematical tasks with reproducible outcome checks [95]. This chapter develops the reusable ideas behind that setting without treating any one reasoning model as a template for all systems.

The central dataflow is

$$\text{prompt} \to \text{Rollouts} \to \text{evaluate} \to \text{select or update}. \tag{173}$$

The two endings in Equation 173 must remain distinct. At **inference time**, a fixed policy spends extra compute generating and selecting candidates for one prompt. At **training time**, scored Rollouts are used to update parameters, after which a new policy generates a new data distribution. PPO and GRPO provide the update machinery; the focus here is the structure and quality of the Rollouts and rewards they consume.

19.2 Reasoning as Sequential Generation

Let x be a task prompt drawn from a declared distribution $\mathcal{D}_{\text{prompt}}$. A Rollout is one complete sampled continuation of an autoregressive policy,

$$y = (a_0, \ldots, a_{T-1}), \quad y \sim \pi_{\text{old}(\cdot \mid x)}, \quad \pi_{\text{old}(y \mid x)} = \prod_{t=0}^{T-1} \pi_{\text{old}(a_t \mid x, a_{<t})}. \tag{174}$$

The tokens in y can include intermediate textual steps, tool calls, code, delimiters, and a final answer. A parsing procedure $g(x, y)$ extracts the object to be checked: for example, an integer, a program artifact, or a proof term. The visible sequence can make intermediate work available to a reward procedure, but it is not by itself a guarantee that every textual step faithfully describes an internal computation. Operationally, it is a trajectory of token actions that can be sampled, scored, and trained on.

For one prompt, a rollout procedure commonly draws N candidates,

$$\mathcal{Y}(x) = \{y_i\}_{i=1}^{N}, \quad y_i \sim \pi_{\text{old}(\cdot \mid x)}. \tag{175}$$

The candidates in Equation 175 need not be independent in a strict probabilistic sense. They share a model, prompt, decoding policy, and often a common high-probability mode. Temperature, nucleus sampling, prompt perturbations, and search policy affect their diversity. Nevertheless, collecting

several candidates exposes outcomes that one greedy continuation would never reveal. This is useful both when a trainer needs contrast for an update and when a deployed system needs a stronger answer for one fixed prompt.

19.2.1 Feedback Surfaces and Credit Assignment

An **Outcome Reward** assigns a scalar after the completed response. Write it as $R_{\text{out}(x,y)}$. It may be a learned Reward Model score, a human judgment, or a task score based only on the final artifact. A deterministic verifier is a special operational interface: given a declared checker v, it can produce, for example,

$$R_{\text{ver}(x,y)} = v(x, g(x,y)) \in \{0,1\}. \tag{176}$$

The binary form in Equation 176 is illustrative, not required. A verifier can return a score, partial test count, compile result, or structured diagnostic. Its key property is reproducibility under a specified parser, checker version, execution environment, and timeout policy. The fact that an evaluator is automatic does not make it complete: a weak parser, incomplete test suite, or incorrect reference answer defines a weak operational target. Training verifiers to rank multiple sampled math solutions is an early example of using generated candidates and an evaluator to improve final selection [96].

A **Process Reward** instead provides feedback at intermediate positions or declared reasoning steps. If the trajectory has step boundaries, a process model can emit $R_{\text{proc}}, t(x, a_{\leq}t)$ before the final answer. The distinction is shown in Table 27.

Feedback surface	What is evaluated	Central limitation
Outcome Reward	A completed response or final artifact.	Sparse: it does not locate the earlier error that caused a failure.
Verifiable Reward	A parsed output under a deterministic or executable checker.	It rewards only the checker specification and can be exploited through its gaps.
Process Reward	Intermediate reasoning prefixes or steps.	It needs trustworthy step boundaries and substantially denser supervision or verification.
Preference-based Reward	A comparison between candidate responses.	It measures the supplied preference rubric, not necessarily task correctness.
Learned Reward Model	A model-predicted scalar for a response or prefix.	It can be miscalibrated or overoptimized outside its training distribution.

Table 27: Feedback mechanisms differ in where they attach supervision, not only in whether their output is a scalar. A system must retain the origin and operational definition of every reward.

Long-horizon reasoning makes credit assignment difficult. Suppose a trajectory receives $R_{\text{ver}} = 0$ after twenty steps. The outcome does not reveal whether the first incorrect transformation, a later arithmetic slip, a malformed final answer, or the verifier itself caused failure. PPO's critic and GAE in Chapter 16 seek token-level estimates from such returns; GRPO in Chapter 18 uses relative outcome performance within a group. Neither automatically turns a final binary label into causal attribution for every preceding token.

Process supervision is motivated by this gap. A Process Reward Model (PRM) can identify an implausible or incorrect intermediate step before the final result, creating denser feedback for training or search. The distinction between process supervision and outcome supervision, along with its data cost and annotation reliability, is central to work on step-level mathematical verification [97]. A process signal is not intrinsically safer than an outcome signal: a PRM can reward locally plausible prose that is globally inconsistent, and optimizing it can create its own shortcut behaviors.

19.3 Multiple Rollouts, Best-of-N, and Selection

For a fixed prompt, let $C(x,y)$ be the event that a Rollout is correct under an intended task criterion, and let

$$p_x = \mathrm{P}_{y\sim\pi(\,.\mid x)}[C(x,y)=1]. \tag{177}$$

If N Rollouts were independent draws with the same success probability p_x, the probability that at least one succeeds would be

$$\mathrm{P}(\text{at least one success}) = 1-(1-p_x)^N. \tag{178}$$

Equation 178 is an idealized **coverage** calculation. It says nothing about whether the system can recognize the successful candidate. An exact verifier can select a passing response whenever one occurs. A learned scorer may rank an incorrect response above the correct one, and self-consistency may select a common answer that is not correct. In practice, Rollouts are also correlated, so their effective diversity is lower than the independent-draw assumption suggests. Repeated near-identical samples can make increasing N much less useful than Equation 178 predicts.

Best-of-N means generating N candidates and selecting one using a declared score or verifier. The selection rule might be

$$\hat{y} = \arg\max_{y_i \in \mathcal{Y}(x)} S(x, y_i), \tag{179}$$

where S can be a verifier-derived score, an Outcome Reward Model, a Process Reward aggregation, or another declared ranking function. The selector in Equation 179 is as important as the generator. If S is merely a weak proxy for correctness, allocating more candidates can increase the opportunity to exploit the proxy rather than improve true task success.

19.3.1 Pass@k and Self-Consistency

Pass@k is an evaluation convention for the probability that at least one of k attempted solutions succeeds. If a finite set of n samples contains c correct solutions, a common estimator is

$$\widehat{\text{Pass@k}} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \tag{180}$$

The estimator in Equation 180 treats the k attempts as a subset of the observed n samples. It is an evaluation statistic, not a deployed selection policy and not the same object as the idealized model probability in Equation 178. Its interpretation depends on the sampling protocol, correctness checker, and budget. Pass@k became a standard way to report the benefit of repeated program samples in code generation [98].

Self-consistency uses sampled reasoning paths differently. It groups candidates by an extracted final answer and selects the most supported answer, often by a majority or weighted vote. It can help when multiple diverse trajectories reach one correct result and no stronger verifier is available. It is not a proof procedure: shared model biases can make the same wrong answer frequent, and answer parsing can merge or split semantically equivalent outputs. Self-consistency is therefore a selection heuristic whose benefit depends on diversity and answer aggregation, not a substitute for external correctness evidence [99].

19.4 Training-Time and Inference-Time Compute

Additional Rollouts serve two different budgets. During training, prompts are sampled, multiple responses are generated, and rewards become examples for an optimizer. In a PPO-style loop, the sequence is

$$\text{prompts} \to \text{Rollouts} \to \text{rewards} \to \text{advantages} \to \text{policy update}. \tag{181}$$

Chapter 16 derives the policy-gradient and PPO components of Equation 181. Chapter 18 replaces a learned Value Model with group-relative advantages when several Rollouts of one prompt provide useful contrast. In both cases, more training Rollouts can improve the statistical signal only if their rewards are informative and the update remains stable. They also consume generation, evaluation, storage, and optimization compute; they are not free labels.

At inference time, the model parameters stay fixed. The system instead spends a prompt-local budget on multiple candidates, verifier calls, voting, or search,

$$\text{fixed policy} \to \text{candidate or search budget} \to \text{selection} \to \text{one answer}. \tag{182}$$

Training-time Rollout scaling can change future p_x by changing the policy. Inference-time scaling only tries to exploit the current policy's existing success distribution and a selection mechanism for the current prompt. Conflating the two obscures a basic systems choice: a test-time system may improve an answer without learning anything, while an RL update may learn from unsuccessful-looking trajectories yet yield no immediate answer for the sampled prompt.

Search extends Best-of-N when the generator branches conditionally on partial trajectories rather than drawing only independent full completions. A PRM can score prefixes, an outcome verifier can score leaves, and a proposal policy can allocate more candidates to uncertain branches. Such methods make selection more adaptive, but their value depends on the reliability of the intermediate score and on task difficulty. Empirical work on test-time compute finds that no single allocation dominates across difficulty regimes; additional search can also overoptimize an imperfect PRM [100].

19.5 Failure Modes, Exploration, and Curriculum

Reasoning RL fails when the reward interface and the candidate distribution fail together. Sparse outcome rewards are particularly limiting when almost every Rollout fails: there is little evidence about which direction improves the policy. In GRPO, a group with all identical rewards has no centered within-group signal. The converse problem also occurs: if almost every candidate passes, the training task offers little discrimination. A curriculum can control this by shifting task difficulty as the policy changes, but it is a data-distribution choice that must be versioned and evaluated rather than a universal remedy.

Exploration is equally important. Low temperature or narrow decoding can produce correlated samples whose apparent count hides low effective diversity. Extremely broad sampling can instead produce malformed, off-distribution, or trivially failing candidates. The goal is not maximum textual variation; it is enough variation in valid solution strategies for the reward procedure to distinguish useful behavior. Group size, decoding configuration, maximum length, and stop rules jointly define this trade-off.

Verifiers create their own reward-hacking boundary. A solution may exploit answer formatting, parser ambiguity, an incomplete test suite, a timeout behavior, an uninitialized random seed, or a leaked reference pattern. A model can also learn to emit unnecessarily long reasoning traces if length is indirectly rewarded, or to imitate memorized answer forms without solving new instances. Held-out tasks, adversarial checker tests, length-controlled evaluation, and contamination checks are necessary because a rising verifier pass rate alone proves only improvement under that verifier.

Finally, one must distinguish an outcome's correctness from an explanation's quality. A correct final answer can contain invalid intermediate claims; an incorrect final answer can follow mostly sound steps before one slip. Process supervision can address this distinction only when its step labels or process checks are themselves dependable. It should be treated as a richer feedback interface with its own calibration and distribution-shift risks, not as a guaranteed solution to long-horizon credit assignment.

19.6 Implementation Contracts

The rollout contract must version the prompt source and split, tokenizer, Chat Template, policy and reference revisions, decoding parameters, maximum response length, stop rules, tool permissions, and parser g. It must retain prompt identifiers, each candidate's group membership, token IDs, action masks, old-policy log probabilities when training, stop reasons, generated length, and any tool traces. A change to response boundaries or answer extraction changes both the reward and the effective trajectory.

The verification contract must identify the checker implementation, versioned test cases or reference answers, execution image, time, memory, and randomness limits, serialization format, parser failures, score aggregation, and treatment of timeouts or unsupported outputs. It must distinguish a deterministic outcome check from a learned RM and retain their raw component scores separately. Tests should include adversarially formatted candidates, equivalent answers, malformed outputs, and deliberately wrong solutions, so a passing result has a documented meaning.

The optimization contract must state whether the algorithm uses PPO, GRPO, or another update rule; retain its reward-to-advantage transformation; and record the behavior-policy, reference-policy, reward, and verifier revisions. It must also record group size, reward normalization, valid-token denominator, KL estimator, clip range where applicable, optimizer state, precision, gradient controls,

and checkpoint state. Evaluation should report task-defined success alongside Rollout count, Pass@k protocol, selector or verifier revision, response length, diversity statistics, reward distribution, and protected held-out results. Chapter 11′s resumability requirements apply to every stateful rollout buffer and curriculum cursor.

19.7 Summary

Reasoning RL operates on the same autoregressive policy structure as other post-training methods, but it often has access to feedback grounded in a task's executable correctness condition. A Rollout is a sampled token trajectory whose final artifact, and sometimes intermediate steps, can be evaluated by an Outcome Reward, a Process Reward, a learned Reward Model, a preference comparison, or a deterministic verifier. These interfaces differ in where they attach signal and in the failure modes they expose.

Multiple Rollouts can improve coverage and selection. Under an unrealistic but informative independence assumption, their chance of containing a correct solution grows as in Equation 178. Real systems also need diversity and a selector that recognizes success, so Best-of-N, Pass@k, self-consistency, and verifier-guided search are related but noninterchangeable tools. More Rollouts during training supply online optimization data; more Rollouts at inference spend a fixed policy's compute budget on one prompt.

The reward interface remains the limiting object. Sparse or uniform rewards weaken learning, weak verifiers invite shortcuts, correlated samples reduce the benefit of scaling, and process signals require their own trustworthy supervision. PPO and GRPO specify how a policy can be updated from Rollouts, but reliable reasoning systems also require careful candidate generation, reward design, selection, curriculum control, and independent evaluation. Chapter 21 develops the policy-versioning, filtering, and iterative-improvement loop that turns these Rollouts into a moving training distribution.

References

[95] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024, doi: 10.48550/arXiv.2402.03300.

[96] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021, doi: 10.48550/arXiv.2110.14168.

[97] H. Lightman *et al.*, "Let's Verify Step by Step," in *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2305.20050.

[98] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, and others, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021, doi: 10.48550/arXiv.2107.03374.

[99] X. Wang *et al.*, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2203.11171.

[100] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters," *arXiv preprint arXiv:2408.03314*, 2024, doi: 10.48550/arXiv.2408.03314.

Chapter 20: Post-Training Evaluation and Alignment Trade-offs

Abstract

Post-training changes how a language model responds, not merely its next-token loss. Its evaluation must therefore assess a collection of capability, instruction-following, correctness, safety, preference, style, and operational properties under declared generation and scoring protocols. This chapter develops evaluation as a multi-objective system rather than a single benchmark score. It distinguishes human, deterministic, and model-based evaluation; explains pairwise comparison, LLM-as-a-Judge, sampling-based reasoning metrics, and Reward Model validation; and analyzes distribution shift, proxy overoptimization, benchmark contamination, capability regression, catastrophic forgetting, and the alignment tax. It concludes with a regression-suite design and the contracts needed to compare post-training checkpoints reliably.

20.1 Introduction

Post-training is judged by behavior. SFT changes the distribution of instruction-response continuations; preference optimization changes the relative likelihood of candidate responses; PPO and GRPO change a policy through rewards obtained from new Rollouts. A lower training loss, a higher Reward Model score, or a successful update step is evidence about one interface in this process, not a complete measure of the resulting assistant.

The difficulty is structural. A response can be helpful but factually wrong, correct but noncompliant with a requested format, safe but needlessly over-refusing, preferred by a judge but excessively verbose, or strong on an average benchmark while failing a rare high-impact case. These are not merely noisy measurements of one hidden scalar. They are partially distinct objectives that can conflict under a changed prompt distribution or decoding policy. The closing task of post-training is therefore not to identify one champion metric, but to define which evidence supports each claim and which regressions are unacceptable.

This chapter closes the Post-training section. It builds on Chapter 15's distinction between a Reward Model score and human judgment, Chapter 16's warning about reward overoptimization, and Chapter 19's distinction between training-time and inference-time sampling. It does not introduce a new alignment algorithm, inference-serving system, or application framework.

20.2 What Post-Training Evaluation Measures

Let θ denote a checkpoint, $\mathcal{D}_{\text{eval}}$ an evaluation prompt distribution, and c a complete evaluation configuration: Chat Template, system policy, decoding settings, tools if any, answer parser, and scorer. For a dimension j, define a score abstractly as

$$M_{j(\theta;\mathcal{D}_{\text{eval}},c)} = \mathrm{E}_{x\sim\mathcal{D}_{\text{eval}}}\Big[s_{j(x,y;c)}\Big], \quad y \sim \pi_{\theta(\,\cdot\,\mid\,x,c)}. \tag{183}$$

The notation in Equation 183 makes an often-hidden dependency explicit. Changing temperature, maximum length, system message, answer extraction, or evaluator can change the distribution of y and the meaning of s_j. A reported score is therefore a property of a checkpoint **under a protocol**, not an intrinsic and context-free number attached to its parameters.

The relevant object is a vector of evidence,

$$\boldsymbol{M}(\theta) = \big(M_{\text{cap}}, M_{\text{inst}}, M_{\text{correct}}, M_{\text{reason}}, M_{\text{safe}}, M_{\text{style}}, M_{\text{oper}}\big). \tag{184}$$

Here M_{cap} represents general capability, M_{inst} instruction following, M_{correct} factual or task correctness, M_{reason} reasoning and coding behavior, M_{safe} harmlessness and policy adherence, M_{style} response quality and verbosity, and M_{oper} operational behavior such as response length or latency when it is relevant. The entries are not assumed to share a unit or be combined automatically.

20.2.1 Capability and Alignment Are Different Claims

Capability evaluation asks whether a model can solve or perform a task: answer a factual question, reason through a problem, write executable code, or use supplied evidence. It is often strongest when

an answer can be checked deterministically or against a well-specified reference. **Alignment evaluation** asks whether the model behaves according to a desired policy: follows the user's constraints, is helpful, refuses harmful assistance appropriately, preserves useful context, and avoids undesirable style or interaction patterns.

Neither category subsumes the other. An assistant can know a correct answer yet violate a length, language, or safety constraint. Conversely, a polished and compliant response can be unhelpful because it omits a crucial fact. Instruction following is especially useful to separate from general fluency: IFEval evaluates a set of verifiable natural-language constraints instead of relying only on a judge's impression of compliance [101]. Safety and harmlessness similarly require a declared threat model and policy boundary; they are not proven by a generic helpfulness score.

Average capability also differs from worst-case behavior. Let $\mathcal{G}$ be a set of meaningful prompt groups, such as languages, domains, risk tiers, or interaction lengths. An aggregate score may be written as

$$M_{\text{avg}} = \sum_{g \in \mathcal{G}} w_g M_g, \quad M_{\text{tail}} = \min_{g \in \mathcal{G}} M_g. \tag{185}$$

M_{avg} summarizes an explicitly weighted population; M_{tail} exposes the weakest declared group. Neither is universally the right deployment criterion. A high average can conceal an unacceptable safety or accessibility failure, while a strict minimum can be unstable when groups are small. The evaluation owner must decide which groups and thresholds represent an actual requirement.

20.3 Evaluation Mechanisms and Their Limits

Evaluation methods differ in the evidence they produce. A deterministic verifier checks a property specified by an executable rule: unit tests, a formal proof checker, an exact answer parser, or a constrained-format instruction. It is usually the strongest option for the property it actually decides, but it cannot judge qualities absent from its rule. Human evaluation can inspect relevance, nuance, safety, factual support, and trade-offs that resist complete formalization, but it is costly and its rubric, expertise, disagreement, and sampling strategy determine what its labels mean.

Automated learned scorers occupy a third role. A Reward Model or LLM judge can evaluate many open-ended responses under a natural-language rubric. This is valuable for fast iteration and broad coverage, but it adds another model and another distribution shift to the evaluation stack. Table 28 separates the role of these mechanisms rather than treating automation as a single category.

Mechanism	Useful evidence	What it does not establish
Deterministic verifier	Reproducible pass or failure for a declared property.	Broad helpfulness, factual scope beyond the rule, or explanation quality.
Benchmark metric	Comparable performance on a fixed prompt, protocol, and scoring rule.	Generalization beyond the benchmark distribution or contamination-free capability by itself.
Human evaluation	Judgment under a stated rubric, including nuanced trade-offs.	A universal value function or full coverage at feasible cost.
Pairwise comparison	Relative preference between two candidates in context.	A stable absolute score across arbitrary prompts and response sets.
Reward Model or LLM judge	Scalable rubric-conditioned scores or rankings.	Independent ground truth; calibration and bias must be audited.

Table 28: Evaluation mechanisms make different claims. Strong evaluation uses them as complementary evidence rather than silently substituting one for another.

20.3.1 Pointwise, Pairwise, and Human Evaluation

Pointwise evaluation assigns one response a score or pass/fail outcome. It is efficient when a reference answer or verifier exists, but an absolute score can be sensitive to rubric wording and calibration.

Pairwise evaluation asks which of two responses better satisfies a criterion. It directly represents the relative supervision introduced in Chapter 15 and often makes subtle trade-offs easier to judge, though ranking many systems requires more comparisons and a pairing policy.

For a prompt x and two candidate systems A and B, a pairwise evaluation can be summarized by a win probability

$$W_{A,B} = \mathrm{P}(\text{A wins} \mid x, y_A, y_B, c), \quad y_A \sim \pi_{A(.\mid x,c)}, \quad y_B \sim \pi_{B(.\mid x,c)}. \tag{186}$$

The probability in Equation 186 is conditional on candidate sampling and judging protocol. It is not a context-free quality ordering. Response order should be randomized or swapped, ties should be represented when the rubric permits them, and confidence intervals should reflect prompt and judge variation. Human evaluations should additionally retain the rubric version, labeling expertise, disagreement, and any reference material available to evaluators.

20.3.2 LLM-as-a-Judge

LLM-as-a-Judge presents a response, or a response pair, to a judge model together with a natural-language rubric. Its advantages are substantial: it scales more easily than human annotation, has lower marginal cost, can evaluate flexible criteria, and may produce a rationale that supports auditing. It is particularly useful for exploratory evaluation and for scoring open-ended outputs that lack an exact reference.

It is not a deterministic verifier. The judge's capability bounds the errors it can detect; a judge can be misled by an incorrect candidate even when it could solve the task independently. It can also have position bias, verbosity bias, prompt sensitivity, correlated errors with the evaluated model family, and possible self-preference. Controlled studies of LLM judges document position, verbosity, and self-enhancement concerns alongside the advantages of scalable pairwise and pointwise evaluation [102].

The appropriate response is not to discard model-based judging, but to make it auditable. Freeze and version the judge, rubric, prompt, examples, temperature, response order, and aggregation rule. Use order swaps for pairwise judgments; evaluate selected samples against independent human labels or deterministic checks; stratify by response length and task type; and report disagreement rather than forcing an apparent precision the judge does not possess. A judge score is evidence about a judge-conditioned rubric, not evidence that the target model is correct in every relevant sense.

20.4 Evaluation Suites, Sampling, and Regression

No benchmark suite needs to enumerate every possible task, but it must cover the failure modes its deployment claims make important. Table 29 is a practical matrix for a post-training regression suite. The goal is not to maximize every row independently; it is to prevent one strong aggregate from hiding a material loss elsewhere.

Dimension	Question and evidence	Regression guard
General capability	Can the checkpoint retain knowledge, language understanding, and transfer tasks?	Compare protected task slices with the pre-post-training baseline.
Instruction following	Does it satisfy explicit user constraints and output formats?	Use verifiable constraints and adversarially varied instructions.
Correctness and factuality	Are claims or task answers supported under a stated scorer?	Use references or checkers where available; human-audit open-ended cases.
Reasoning and coding	Can sampled solutions solve multi-step or executable tasks?	Report Pass@k, sample budget, verifier, and selected-answer metric separately.
Safety and harmlessness	Does behavior meet the declared policy across benign and adversarial prompts?	Track both unsafe compliance and inappropriate refusal on legitimate prompts.
Style and interaction	Is the response concise, relevant, and usable for the intended audience?	Control for length and evaluate preference slices with audits.
Operational behavior	Are response length, latency, tool cost, or format reliability acceptable?	Use fixed serving and decoding conditions, not an unspecified default.

Table 29: An evaluation suite should connect each claimed property to a protocol and a guard against a plausible regression. Rows may be added or removed only with a deployment-specific rationale.

Sampling changes the meaning of reasoning and coding evaluation. Chapter 19 derived the idealized probability that at least one of several independent Rollouts succeeds and distinguished it from a selector's ability to identify that Rollout. Pass@k measures coverage under a declared sampling budget; a Best-of-N or verifier-selected result also measures selection quality. Neither should be compared with a greedy Pass@1 result as though they used the same inference-time compute. Record the number of samples, decoding parameters, maximum length, answer parser, verifier version, and whether selection was oracle, learned, or rule based.

Regression testing compares the same suite across a candidate checkpoint and one or more baselines. For every dimension j, define the change

$$\Delta_j = M_{j(\theta_{\text{candidate}})} - M_{j(\theta_{\text{baseline}})}. \tag{187}$$

The vector $\left(\Delta_j\right)_j$ is more informative than an unqualified average. Release criteria can require a safety gain, permit a bounded loss on a low-risk dimension, or reject any degradation in a protected capability slice. They should also include threshold-crossing and worst-case tests, because a small global mean change can conceal a large failure in one subgroup. A regression suite is a product requirement encoded as an experiment, not merely a dashboard after training has ended.

20.5 Proxies, Distribution Shift, and Overoptimization

Post-training frequently optimizes a proxy: an RM score, a judge score, a benchmark score, a verifier pass rate, or a short preference rubric. Let $R(x, y)$ be such a proxy and $Q(x, y)$ the external quality property ultimately desired. A policy update that solves

$$\theta_R^* = \arg\,\max_\theta ; \mathrm{E}[R(x, y)] \tag{188}$$

need not solve the analogous objective for Q. Equality would require the proxy to preserve the relevant ordering of policy-generated responses on the distribution reached during optimization. That is a strong condition: an RM can reward polite length, an LLM judge can prefer an elaborate answer, and a benchmark can reward memorized examples. Goodhart's Law is the practical warning that a proxy which becomes an optimization target can cease to measure the intended property reliably.

Reward overoptimization is this effect in the RM setting. Chapter 15 explained that pairwise RM accuracy does not make a scalar score a universal utility, and Chapter 16 showed why PPO constrains policy drift with reference KL. Nevertheless, optimizing strongly enough against a fixed learned RM can produce higher R while independent human preference or task quality stalls or declines.

RewardBench supplies one useful source of held-out ranking evidence for RMs, but it cannot replace evaluation of the policy that eventually searches for high scores [103]. The discrepancy must be measured on protected human, task-grounded, or adversarial evaluations.

Distribution shift compounds the issue. A Reward Model, judge, or benchmark is trained or designed around a particular set of prompts and response styles, whereas the optimized policy can generate longer, more polished, unusual, or adversarial continuations. Judge and policy errors can also be correlated when they share a model family or training data. Evaluation therefore needs a mixture of in-distribution checks, deliberately difficult contrast sets, independent judges or humans, and deterministic verifiers when a property permits them.

Benchmark contamination is a separate validity threat. If benchmark test items, answers, or strong near-duplicates enter Pretraining, SFT, preference data, or evaluator prompting, the reported performance can exaggerate generalization. The issue is not limited to exact string overlap, and for closed training corpora it can be difficult to quantify. Work on contamination argues that training on a benchmark's test split can invalidate the intended evaluation claim and inflate performance estimates [104]. Provenance, release dates, deduplication records, contamination probes, and fresh or private holdouts are all useful controls, but none should be reported as a guarantee beyond the evidence they provide.

20.6 Alignment Trade-offs and Capability Regression

Alignment tax names an observed trade-off in which a post-training change improves a desired alignment objective while reducing performance on another protected capability or distribution. In the notation of Equation 187, a simple empirical pattern is $\Delta_{\text{safe}} > 0$ or $\Delta_{\text{inst}} > 0$ together with $\Delta_{\text{cap}} < 0$ on a relevant suite. This is not a theorem that alignment must reduce capability. It is a reason to report a vector of outcomes and a Pareto trade-off rather than celebrating one gain in isolation. Studies of RLHF have measured alignment-forgetting trade-offs and treat mitigation as a multi-objective problem [105].

Capability regression is the observed loss on a declared post-training comparison. **Catastrophic forgetting** is a stronger mechanism-oriented concern: adaptation on a narrower distribution can impair behavior that the earlier model could express, sometimes through a changed inferred task or prompt distribution rather than a simple deletion of all underlying knowledge. Analyses of language-model fine-tuning show why improvements on the fine-tuning distribution do not establish preservation outside it [106]. The operational response is the same: retain the pre-change checkpoint, evaluate protected capability slices, and diagnose the loss by prompt form, language, task, and sampling protocol before attributing it to one mechanism.

Safety has its own two-sided regression boundary. Lower unsafe compliance can be a genuine improvement; higher refusal on harmless, legitimate requests can be an over-refusal failure. Similarly, an instruction-tuned model can gain polished response style while losing concise directness or specialized-domain behavior. The desired result is not maximal refusal, maximal length, or maximal judge score. It is an explicitly chosen Pareto region whose thresholds reflect the deployment's risks and users.

20.7 Implementation Contracts

The suite contract must version every prompt set, split, task definition, rubric, reference answer, verifier, contamination audit, and prompt-group label. It must identify which data are public, private, newly collected, or potentially exposed during Pretraining and post-training. It must retain the exact evaluation configuration c from Equation 183: system policy, Chat Template, tokenizer, decoding parameters, maximum length, tool environment, answer parser, retry policy, and scorer revision.

The judging contract must distinguish deterministic verifier, human labeler, RM, and LLM judge. For human evaluation, record the rubric, expertise requirements, candidate blinding, order randomization, ties, disagreement, aggregation, and audit sample. For an LLM judge, record the model revision, complete judge prompt, few-shot examples, temperature, response ordering, output parser, repeated-call

policy, and order-swap procedure. For a learned RM, retain the model, scalar-readout, normalization, and calibration conventions specified in Chapter 15. A judge result without this identity is not reproducible evidence.

The regression contract must name the baseline checkpoint, candidate checkpoint, suite version, statistical aggregation, uncertainty procedure, acceptance thresholds, and exception policy for every protected dimension. It should log raw score numerators and denominators where meaningful, response length and latency distributions, Pass@k sampling budget, failure examples, and both average and grouped results. A release decision should retain the report, configuration, raw outputs permitted by the data policy, and any red-team or human-audit findings. Chapter 11′s checkpoint and experiment-tracking discipline applies: an evaluation comparison is not recoverable if its model revision, data identity, or inference configuration is lost.

20.8 Summary

Post-training evaluation is a multi-objective comparison of behaviors under a declared generation and scoring protocol. Capability, instruction following, correctness, safety, preference, style, and operational behavior make different claims and require different evidence. Deterministic verifiers, benchmarks, human evaluation, pairwise comparison, Reward Models, and LLM judges are complementary tools with distinct validity limits.

LLM-as-a-Judge makes broad rubric-conditioned evaluation affordable, but it inherits position, length, capability, prompt, and correlation limits. Reward Models and benchmark scores are also proxies: once a model is optimized against them, their relationship to the intended quality can change. This is why independent evaluation, contamination controls, and regression suites are required rather than optional presentation polish.

The alignment tax and catastrophic forgetting are not arguments against post-training. They are reminders to compare a candidate checkpoint with a baseline across protected dimensions, groups, and failure cases. A reliable closing evaluation system records the protocol, reports the trade-offs, detects regressions before release, and reserves strong claims for the evidence its tools can actually support.

References

[101] J. Zhou *et al.*, "Instruction-Following Evaluation for Large Language Models," *arXiv preprint arXiv:2311.07911*, 2023, doi: 10.48550/arXiv.2311.07911.

[102] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. doi: 10.48550/arXiv.2306.05685.

[103] N. Lambert *et al.*, "RewardBench: Evaluating Reward Models for Language Modeling," *arXiv preprint arXiv:2403.13787*, 2024, doi: 10.48550/arXiv.2403.13787.

[104] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle, and E. Agirre, "NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark," *arXiv preprint arXiv:2310.18018*, 2023, doi: 10.48550/arXiv.2310.18018.

[105] Y. Lin *et al.*, "Mitigating the Alignment Tax of RLHF," *arXiv preprint arXiv:2309.06256*, 2024, doi: 10.48550/arXiv.2309.06256.

[106] S. Kotha, J. M. Springer, and A. Raghunathan, "Understanding Catastrophic Forgetting in Language Models via Implicit Inference," in *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2309.10105.

Chapter 21: On-Policy Alignment and Iterative Policy Improvement

Abstract

On-policy alignment improves a language-model policy using responses generated by a recent version of that same policy. The apparent simplicity of this idea hides an important systems and statistical contract: every selected response must be traceable to the behavior policy, generation configuration, reward or verifier version, and filtering decision that produced it. This chapter develops the rollout lifecycle, policy-version semantics, exploration and selection, rejection-style policy filtering, and Expert Iteration as an iterative improvement pattern. It then treats DAPO as a concrete modern design, explaining its dynamic sampling, asymmetric clipping, token-level objective, and overlong-response shaping. The emphasis is on how online data, relative rewards, and policy updates form a moving training distribution rather than a static preference dataset.

21.1 Introduction

Chapters 15–18 introduced a progression from preference data to Reward Models, PPO, DPO, and GRPO. The distinction between offline preference learning and online policy optimization is now central. An offline method can train on a fixed collection of comparisons. An on-policy method instead asks a recent language-model policy to generate candidate responses, scores those responses, and uses them to update the policy that will generate the next collection. The data distribution therefore changes as training proceeds.

This feedback loop is especially useful when a verifier can score newly sampled solutions, as in the reasoning settings of Chapter 19. It is also delicate. A response may be excellent under one policy version but be treated incorrectly after the learner has moved far away from that behavior policy. A filter that improves average reward may remove the very difficult cases required for robust generalization. A rollout worker can produce many candidates, but no amount of throughput repairs an ambiguous reward contract. This chapter focuses on these algorithmic interfaces. The distributed worker topology used to execute them is deferred to Chapter 22.

21.2 On-Policy Data and Policy Versions

Let π_{θ_v} denote the behavior policy at version v, and let π_θ denote the policy currently being optimized. For a prompt $x \sim \mathcal{D}$, a rollout $y = (y_1, ..., y_T)$ is sampled autoregressively from the behavior policy,

$$y \sim \pi_{\theta_v}(. \mid x) = \prod_{t=1}^{T} \pi_{\theta_v}(y_t \mid x, y_< t). \tag{189}$$

The word **on-policy** means that optimization uses data from the policy being improved, or from a deliberately bounded recent behavior version. By contrast, **off-policy** data comes from a behavior distribution that is not guaranteed to be recent for the update; a fixed preference dataset is a particularly important offline source. A stale rollout is therefore not a separate objective: it is a behavior-policy sample whose version is becoming too distant for the declared on-policy approximation. Practical systems may tolerate limited lag, but they must not silently relabel arbitrary historical data as fresh Rollouts.

This does not mean that a production learner and generator must share one instantaneous set of parameters. In practice, an update may reuse one rollout batch for several optimization epochs, or a generator may finish requests while a learner begins a later update. The essential requirement is that this lag is recorded and controlled.

A useful rollout record contains more than text:

$$r = (x, y, v, \ell_{\text{old}}, m, R, h), \tag{190}$$

where ℓ_{old} is the behavior-policy token log-probability, m is the response-token mask, R is the reward or verifier result, and h identifies the generation, reward, and selection configurations. The record

makes an update auditable. In particular, a PPO- or GRPO-style importance ratio compares a candidate policy to the actual behavior policy,

$$\rho_{t(\theta;v)} = \frac{\pi_{\theta(y_t \mid x, y_{<}t)}}{\pi_{\theta_v}(y_t \mid x, y_{<}t)}. \tag{191}$$

When θ drifts far from θ_v, this ratio becomes more variable and a nominally on-policy update becomes a weak approximation. Policy versions, log-probabilities, response masks, and a maximum allowed lag are therefore training data, not incidental logging.

21.2.1 The Rollout Lifecycle

An online alignment iteration has a compact but consequential lifecycle:

Roll out → score → select → update → refresh version

The prompt distribution may be fixed, sampled from a curriculum, or refreshed from a task pool. The generation configuration—temperature, maximum response length, stop conditions, and any format constraint—changes which trajectories are observed, so it belongs in h in Equation 190. Scoring can be a learned Reward Model, a deterministic checker, a process signal, or a combination, but its version must be equally explicit. Finally, the selected records are transformed into token-level learning signals and used for a bounded number of updates before another rollout round begins.

Exploration is not merely random noise. Sampling temperature, multiple Rollouts per prompt, prompt diversity, and task difficulty determine whether the learner observes useful alternatives. Very low diversity repeatedly trains on one familiar solution pattern; unconstrained diversity can spend most computation on malformed or unscorable trajectories. Chapter 19 explains why multiple candidates can increase both learning signal and inference-time success probability. Here the additional point is that sampling policy and selection policy jointly define the next training distribution.

21.3 Selection, Filtering, and Iterative Improvement

When rewards are sparse or evaluation is expensive, systems often retain only a subset of sampled responses. Let $w(x, y)$ be a nonnegative selection weight; binary policy filtering uses $w \in \{0, 1\}$. The distribution after filtering is, schematically,

$$p_{\text{selected}(y \mid x)} = \frac{\pi_{\theta_v}(y \mid x) w(x, y)}{\sum_{y'} \pi_{\theta_v}(y' \mid x) w(x, y')}. \tag{192}$$

This expression exposes a trade-off. Rejection-style selection can remove invalid outputs, failed verifier checks, duplicate answers, or obviously low-quality candidates. It can also bias the retained data toward short, easy, stylistically preferred, or already-solved cases. It is therefore not the distribution-preserving accept–reject correction used in exact speculative sampling (Chapter 27). It is a deliberate change to the data distribution and must be evaluated as such.

Verifier-guided selection is particularly attractive when correctness is executable. A mathematics checker, unit-test suite, or symbolic validator can identify candidates that satisfy a declared condition without claiming to judge every dimension of response quality. A learned score can rank candidates where no deterministic verifier exists, but it imports the calibration, style bias, and reward-hacking risks discussed in Chapters 15 and 20. A filter should retain its reason code and score, not only its final accept/reject bit.

21.3.1 Expert Iteration and Self-Improvement

Expert Iteration is a general pattern in which a current policy produces or helps search for solutions, a stronger selection procedure identifies better behavior, and the policy is trained to imitate or optimize toward that selected behavior. In the original formulation, planning provides the stronger expert signal while a neural policy generalizes it [107]. In language-model reasoning, the selector may instead be a verifier, a search procedure, or a reward rule:

$$\pi_{\theta_v} \rightarrow \text{sample or search} \rightarrow \text{select verified candidates} \rightarrow \text{update } \pi_{\theta_{v+1}}. \tag{193}$$

The loop is not guaranteed to improve itself. If the selector favors a superficial format, the next policy can become better at that format rather than at the task. If generated data lacks unsolved cases, the

learner receives little new corrective information. ReST is one example of alternating generation and reward-filtered improvement for language models; it illustrates both the practical appeal of selected self-generated data and the need to state the filter and update regime precisely [108].

21.4 DAPO: A Concrete On-Policy Design

DAPO (Decoupled Clip and Dynamic sAmpling Policy Optimization) is an open implementation-oriented design for large-scale LLM reasoning reinforcement learning. Its reported formulation builds on group-relative Rollouts, but changes several details that become important for long reasoning responses: dynamic sampling, a decoupled clipping range, a token-level policy-gradient loss, and overlong-response shaping [109]. These choices are useful to study because they make otherwise implicit decisions about which Rollouts enter an update and how individual tokens are weighted.

For a prompt q with answer a, sample a group $(o_i)_{i=1}^{G}$ from an old policy. In the binary-verification setting of the original formulation, define

$$c(q) = \sum_{i=1}^{G} 1[\text{equivalent}(a, o_i)]. \tag{194}$$

Groups with $c(q) = 0$ or $c(q) = G$ have no within-group correctness variation and give zero standardized relative advantage. Dynamic sampling oversamples prompts and retains groups satisfying $0 < c(q) < G$, then refills the batch as necessary. This can concentrate rollout compute on prompts that currently provide a contrastive learning signal. It must not be interpreted as a universal rule: it can underrepresent consistently solved, consistently unsolved, or non-binary-reward tasks.

21.4.1 Ratios, Asymmetric Clipping, and Group Advantages

For each response token, DAPO uses the behavior-policy ratio

$$r_{i,t}(\theta) = \frac{\pi_{\theta(o_{i,t} \mid q, o_{i,<t})}}{\pi_{\theta_{\text{old}}}\big(o_{i,t} \mid q, o_{i,<t}\big)}. \tag{195}$$

With group rewards R_i, let $\mu_R = \frac{1}{G}\sum_j R_j$ and $s_R = \sqrt{\frac{1}{G}\sum_j \big(R_j - \mu_R\big)^2}$. A representative relative advantage is

$$\hat{A}_i = \frac{R_i - \mu_R}{s_R + \varepsilon}. \tag{196}$$

The exact reward and normalization conventions are implementation contracts. In the published binary-reward setting, the same response-level relative advantage is applied to its valid tokens. The clipped local objective is

$$\ell_{i,t}(\theta) = \min\Big(r_{i,t}(\theta)\hat{A}_i, \text{clip}\big(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\big)\hat{A}_i, \Big). \tag{197}$$

PPO commonly presents a symmetric interval around one. DAPO separates the bounds and, in its Clip-Higher design, uses an upper allowance that can be larger than the lower one. Increasing $\varepsilon_{\text{high}}$ permits a larger probability increase for advantageous tokens; keeping ε_{low} relatively small limits large probability decreases. This is a design for maintaining useful exploration, not a proof that asymmetric clipping is always preferable. The ratio and clipping behavior should be monitored alongside KL drift and entropy even when an implementation does not include the same explicit KL term used by classical RLHF in Chapter 16.

21.4.2 Token-Level Loss and Long Responses

Let $N_{\text{tok}} = \sum_{i=1}^{G} |o_i|$ count valid response tokens in a group. A token-level group objective can be written as

$$J_{\text{DAPO}(\theta)} = \text{E}\left[\frac{1}{N_{\text{tok}}}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\ell_{i,t}(\theta)\right]. \tag{198}$$

This differs from first normalizing each response by its own length and then averaging responses equally. Under Equation 198, a longer response contributes more valid token terms. That can give a correct long reasoning trace more influence, but it also makes response-length behavior part of the

optimization objective. Token masks must exclude prompt, padding, and post-termination positions; otherwise the apparent change in weighting is an indexing bug rather than an algorithmic choice.

The original DAPO system also uses a shaped length penalty near a maximum rollout length. With a maximum $L_{\max}$ and a soft interval L_{cache}, one representative form is

$$R_{\text{length}(y)} = \begin{cases} 0 & |y| \le L_{\max} - L_{\text{cache}} \\ \frac{L_{\max} - L_{\text{cache}} - |y|}{L_{\text{cache}}} & L_{\max} - L_{\text{cache}} < |y| \le L_{\max} \\ -1 & |y| > L_{\max}. \end{cases} \tag{199}$$

The soft region avoids treating every near-limit response identically, while the terminal penalty discourages trajectories that exhaust the length budget. Its purpose is not to make all answers short. A length penalty must be checked against task correctness, because some problems genuinely require a longer derivation.

21.5 Data Quality, Modes, and Monitoring

Online data quality is conditional on the current policy. A batch with high mean reward can still be low value if it contains duplicates, template exploitation, nearly identical easy prompts, or reward artifacts. Conversely, a low reward batch can be informative if it contains calibrated hard cases and enough variation to form a stable relative signal. Curriculum, prompt-source mixture, sampling configuration, verifier coverage, and selection thresholds should therefore be versioned with every update.

Signal	What it can reveal	Required interpretation
Policy lag and ratios	Stale behavior data or overly large updates.	Read with update epochs, clip fraction, and policy-version spread.
Group reward variance	Whether relative advantages are informative.	Separate all-success and all-failure groups from noisy rewards.
Selection rate	Filter strictness and rollout-compute waste.	Stratify by prompt source, length, and difficulty.
Response length and stop rate	Overlong behavior, truncation, or shortcut responses.	Check jointly with correctness and length shaping.
Entropy, KL, and clip fraction	Exploration loss or excessive policy movement.	Compare against behavior and reference-policy contracts.
Held-out task and regression scores	Whether online reward gains transfer.	Use the multi-objective suite of Chapter 20.

Table 30: Online alignment metrics must be interpreted jointly. No single rollout statistic establishes durable policy improvement.

The principal failure modes are coupled. Overly strict filtering can cause mode collapse; weak filtering can train on invalid outputs. A stale rollout pool can invalidate an assumed on-policy update; excessively fresh but narrow data can overfit one policy mode. A verifier can be objective about a narrow property yet still be exploitable through answer formatting or loopholes. The remedy is not a larger reward number, but a reproducible decomposition: inspect prompt distribution, generation configuration, selection decision, reward or verifier trace, advantage statistics, update size, and held-out behavior in that order.

21.6 Implementation Contracts

An implementation should treat on-policy alignment as a versioned data pipeline. Each rollout must record its behavior-policy version, token log-probabilities, response mask, prompt identifier, generation configuration, reward or verifier version, and selection reason. The learner must enforce a bounded policy lag and reject or explicitly reweight records outside that contract. Prompt, completion, padding, and terminal tokens need unambiguous masks; only intended completion tokens receive policy-gradient loss.

Group construction must specify group size, reward normalization, zero-variance handling, and whether all-success or all-failure groups are retained. Filters should preserve enough metadata to audit selection bias by source, difficulty, length, language, and reward mode. Length limits, shaped penalties, stop conditions, and answer parsing must be evaluated with correctness rather than optimized as

isolated proxy metrics. Finally, online metrics need held-out, human, or deterministic regression evidence as specified in Chapter 20.

21.7 Summary

On-policy alignment replaces a static training corpus with a moving distribution produced by a recent policy. That change creates a powerful route to verifier-guided learning and iterative policy improvement, but it makes provenance, policy versioning, exploration, selection, and distribution shift first-class algorithmic concerns. Rejection-style filtering and Expert Iteration can turn sampled candidates into stronger supervision only when their selectors are reliable and their biases are measured. DAPO makes four practical choices explicit: retain informative mixed-outcome groups, separate clipping bounds, weight valid tokens directly, and shape overlong responses. These are levers in a coupled online system, not independent recipes. The next systems chapter will address how Rollout, reward, policy, and critic work is organized and scheduled at distributed scale.

References

[107] T. Anthony, Z. Tian, and D. Barber, "Thinking Fast and Slow with Deep Learning and Tree Search," *arXiv preprint arXiv:1705.08439*, 2017, doi: 10.48550/arXiv.1705.08439.

[108] C. Gulcehre *et al.*, "Reinforced Self-Training (ReST) for Language Modeling," *arXiv preprint arXiv:2308.08998*, 2023, doi: 10.48550/arXiv.2308.08998.

[109] Q. Yu *et al.*, "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," *arXiv preprint arXiv:2503.14476*, 2025, doi: 10.48550/arXiv.2503.14476.

Chapter 22: Distributed RL Training Systems

Abstract

Large-scale language-model reinforcement learning is a distributed dataflow in which a recent policy generates trajectories, reward procedures evaluate them, and a learner turns the resulting token-level records into a new policy version. This chapter explains how Actor, Rollout, Reward, Critic, and Reference roles are placed and coordinated across a cluster; why policy synchronization, bounded rollout staleness, and typed trajectory interfaces are correctness conditions; and how FSDP-backed training, high-throughput inference, queues, recovery, and observability interact. Ray and veRL provide concrete systems examples, but the chapter develops the underlying organization independently of one framework.

22.1 Introduction

Chapters 16, 18, 19, and 21 define the algorithmic loop of online post-training: sample prompts, generate Rollouts from a behavior policy, evaluate them, construct an update signal, optimize for a bounded number of steps, and refresh the policy. On one accelerator, this can appear to be a sequence of ordinary model calls. At useful scale, however, the calls have incompatible execution profiles. Generation is an autoregressive inference workload whose cost grows with response length and benefits from continuous batching; a policy update is a synchronized training workload with activations, gradients, and optimizer state; reward evaluation may be a model forward pass, a CPU-bound verifier, or a sandboxed program execution. The system must coordinate all three without silently changing the on-policy data contract.

This chapter studies that coordination. It does not re-derive PPO, GAE, GRPO, DAPO, verifier design, or preference modeling. Instead, it treats those procedures as consumers and producers of explicitly versioned records. A **worker** here is a scheduled resource-owning process or group of processes. It is distinct from the **actor** in actor–critic terminology: an **Actor Worker** owns the trainable policy computation, whereas a Ray actor is one programming-model abstraction that can host a worker role.

22.2 System Roles and the RL Dataflow

An online RL iteration has two kinds of model state. The **behavior policy** π_{θ_v} at version v generates a trajectory. The **learner policy** π_θ is the state currently receiving updates. For a bounded part of an iteration they may denote the same checkpoint, but a distributed system must not assume that every worker observes a weight update at the same instant. Chapters 16 and 21 establish why old-policy log probabilities and behavior-policy identity remain part of the PPO- or GRPO-style objective.

Let one transmitted trajectory record be

$$\tau = (x, y, v, \ell_{\text{old}}, m, R, Z, h), \tag{200}$$

where x is a prompt, y is the generated token sequence, v is the behavior-policy version, ℓ_{old} stores behavior-policy token log probabilities, m is the action mask, R is a reward or verifier result, Z denotes optional value predictions or other algorithm-specific fields, and h carries the serialization, sampling, and provenance metadata. The learner may add advantages, returns, reference log probabilities, and losses, but it must not lose the fields that identify the distribution from which y was drawn.

Role	Primary computation	State and failure boundary
Actor or Policy Worker	Policy log probabilities, policy loss, backward pass, and optimizer update.	Trainable policy, optimizer and scheduler state, distributed-training topology, and a checkpoint commit boundary.
Rollout Worker	Autoregressive generation under one published behavior-policy version.	Inference weights, KV Cache, sampling configuration, request identity, and generated-token provenance.
Reward or Verifier Worker	Reward Model scoring, deterministic checking, program execution, or reward composition.	Reward or checker revision, parsing and timeout policy, environment image, and raw score trace.
Critic Worker, when used	Value predictions and value loss for actor–critic algorithms.	Value parameters, optimizer state, target convention, and alignment with the behavior-policy rollout.
Reference Worker, when used	Reference-policy log probabilities or KL-related quantities.	Frozen reference revision, tokenizer and Chat Template compatibility, and response-token masking.

Table 31: A distributed RL system separates roles whose state changes at different rates and whose execution profiles differ. GRPO-like methods may omit a Critic Worker; algorithms without a reference constraint may omit a Reference Worker.

Table 31 is a logical decomposition, not a requirement that every role have a dedicated process. A small system can co-locate several roles, while a large system can partition one role across many devices. The useful invariant is ownership: an observation must retain the exact policy, reward, and formatting identities that produced it even if the physical workers are replaced or moved.

22.2.1 End-to-End Lifecycle

One conservative synchronous iteration is

$$\textbf{publish } \pi_{\theta_v} \rightarrow \textbf{generate } \tau \rightarrow \textbf{score and validate} \rightarrow \textbf{construct learning fields} \rightarrow \textbf{update } \pi_\theta \rightarrow \textbf{commit } \pi_{\theta_{v+1}} \rightarrow \textbf{synchronize or serve the new version}$$

The word **publish** means more than exposing a file path. Rollout Workers need an atomic association between the weights they load and the version identifier they place in Equation 200. The learner must similarly know which behavior version supplied a batch before it evaluates an importance ratio. A system can overlap phases, but only by making the overlap explicit: a trajectory produced by v may be processed while the learner prepares $v + 1$, subject to a declared maximum lag.

This lifecycle also separates **training** from **inference** inside one job. The Actor Worker executes a training forward and backward pass, and may be sharded with FSDP. The Rollout Worker executes Prefill and Decode, maintains a KV Cache, and benefits from the scheduling techniques of Chapters 23–29. Treating its memory and batch policy as though it were a training worker often produces poor utilization or an incorrect view of the job's capacity.

22.3 Placement, Colocation, and Scheduling

The first deployment choice is whether roles share accelerators. In a **colocated** design, a training and generation role use the same resource pool at different times. It can avoid copying a large policy between distinct pools and can be economical on a small cluster. Its main cost is phase interference: a training step and a latency-sensitive Decode workload cannot both own the same accelerator memory at full capacity. Switching a model between a sharded training layout and an inference layout can also require reshaping or transferring state.

In a **disaggregated** design, a learner pool, a rollout-serving pool, and optional reward or verifier pools receive separate resources. This permits generation to continue while the learner updates, and it allows each pool to use the parallelism appropriate to its workload. The price is extra weight synchronization, data movement, and coordination. There is no universally better deployment. The decision depends on model size, rollout-to-update ratio, sequence lengths, verifier cost, network bandwidth, and the degree of staleness that the algorithm permits.

Placement choice	What it can improve	What must be measured
Colocate policy training and Rollouts	Avoided duplicate model residency and possibly cheaper local handoff.	Phase switching, allocator peaks, interference, and idle time between phases.
Disaggregate learner and Rollouts	Independent training and serving schedules; role-specific parallelism.	Weight-transfer cost, rollout lag, network use, and trajectory-queue growth.
Separate Reward or Verifier pool	Scales expensive model scoring or heterogeneous checking independently.	Queue delay, sandbox capacity, reward-version consistency, and result ordering.
Co-locate tightly coupled ranks	Locality for high-bandwidth collectives or model-parallel groups.	Node capacity, fault domain, and whether packing prevents other work from scheduling.

Table 32: Placement is a workload-specific resource decision. A lower weight-transfer cost can be outweighed by reduced overlap, while a more elastic layout can be limited by version lag or the network.

Ray is one useful orchestration layer because it expresses both persistent actor-style computation and short-lived tasks through a distributed runtime [110]. Its placement groups reserve resource bundles atomically and let a job pack related resources for locality or spread them across nodes for a different fault domain [111]. These mechanisms are useful for assigning a Tensor-Parallel rollout group, an FSDP learner group, and a CPU-based verifier service, but they do not choose an RL algorithm or make an unsafe placement correct. In particular, every role should declare its CPU, GPU, memory, and any topology-sensitive requirements rather than relying on scheduler defaults.

22.3.1 GPU Allocation and Backpressure

The slowest stage determines steady-state throughput. If Rollout Workers generate completions faster than reward evaluation or training can consume them, the trajectory queue grows until host memory, object-store capacity, or durable storage becomes the bottleneck. If the learner is faster than generation, it waits for fresh on-policy examples or is tempted to reuse stale trajectories beyond its declared update regime. The appropriate response is **backpressure**: bounded queue capacity and admission rules that slow an upstream producer, limit prompts in flight, or defer lower-priority work before an unbounded buffer silently changes the job's failure mode.

For an average rollout arrival rate λ_{roll} and downstream consumption rate μ_{down}, a queue is stable only in the simplified steady-state sense that

$$\lambda_{\text{roll}} < \mu_{\text{down}}. \tag{201}$$

The inequality is not a complete performance model. Arrival times are bursty, response lengths vary, and a single slow verifier can delay a group. Its value is diagnostic: adding Rollout GPUs while μ_{down} is fixed can raise the queue length rather than improve completed RL iterations. Queue occupancy, waiting time, rejection rate, and the policy-version distribution of queued trajectories must therefore be monitored together.

22.4 Synchronization, FSDP, and Inference Engines

Policy synchronization is the boundary between learning and generation. After an update, a colocated system may reuse resident parameters after an explicit phase transition. A disaggregated system must publish a consistent new weight version to its Rollout Workers. The mechanism can range from a checkpoint-like artifact to an in-memory transfer or a backend-specific resharding path. What matters algorithmically is that no rollout record claims version $v+1$ unless the generator actually used parameters associated with $v+1$.

Define the learner's accepted version lag for trajectory τ as

$$\Delta_{v(\tau)} = v_{\text{learner}} - v_{\text{behavior}(\tau)}, \quad 0 \le \Delta_{v(\tau)} \le \Delta_{\max}. \tag{202}$$

The upper bound $\Delta_{\max}$ is a data-contract choice, not a cosmetic metric. PPO and GRPO use behavior-policy log probabilities in their ratios, so a large lag increases distribution mismatch and makes a nominally on-policy update less reliable. Chapter 21 treats this as an algorithmic condition; the system

realizes it with atomic version publication, queue admission or eviction, and an explicit decision about unfinished requests during a refresh.

FSDP is relevant on the learner side because actor and critic models may require parameter, gradient, and optimizer-state sharding. Chapter 12 explains the All-Gather and Reduce-Scatter lifecycle, memory peaks, and checkpoint representations for that role. It should not be assumed that an FSDP live layout is directly consumable by an inference engine. The synchronization interface has to map the logical model state to the rollout engine's accepted partitioning and precision. A fast transfer that changes tied weights, tokenizer assumptions, parameter names, or model revision identity is not a valid optimization.

High-throughput engines such as vLLM are relevant to Rollout Workers because they provide efficient autoregressive generation and KV Cache management; the vLLM system design is described by Kwon et al. [112]. They do not make rollouts an ordinary serving request. An RL system must additionally retain sampled token IDs, old-policy log probabilities, stop reasons, action masks, and the exact sampling configuration. veRL's HybridFlow design is a representative example of separating the RL dataflow from the mapping of training and generation computation to devices, and it explicitly integrates FSDP- and inference-engine backends [113], [114].

22.5 Trajectory Data Plane and DataProto

The trajectory queue is an interface between independently scheduled roles, not an incidental Python list. It must carry fixed-shape or padded tensor fields alongside variable-length and structured metadata. A useful schema partitions the record into three classes:

- tensor fields, such as token IDs, attention and action masks, old log probabilities, values, rewards, returns, and advantages;
- per-sample non-tensor fields, such as prompt identifiers, tool traces, verifier diagnostics, stop reasons, and source records; and
- batch metadata, such as behavior-policy version, tokenizer and Chat Template identity, reward and reference revisions, padding convention, and the producer's serialization policy.

The separation prevents two common mistakes. First, a field that is indexed with each response cannot be treated as one batch-global value; prompt and response metadata must remain aligned when a group is repeated, filtered, packed, or shuffled. Second, a tensor whose dtype, shape, or mask changes must not be smuggled into opaque metadata, where a learner cannot validate it before computing a loss. veRL's `DataProto` is one concrete interface of this kind: it separates batched tensors, non-tensor batch data, and metadata and enforces a shared leading batch dimension for its declared fields [114].

DataProto is not a new RL objective. It is a data-interface discipline that lets a controller or worker group dispatch, slice, repeat, and merge records without guessing which fields are per-token, per-sample, or per-batch. A framework-independent system needs the same distinctions even if it uses a different container. The stable identity of Equation 200, including group membership for GRPO and selection history for DAPO-like filtering, remains more important than the container's class name.

22.5.1 Queues, Storage, and On-Policy Lifetime

On-policy trajectories have a short statistical lifetime. Once their policy version exceeds the accepted lag in Equation 202, they should be rejected, explicitly reweighted by a declared algorithm, or retained only for diagnostics. An online RL queue is therefore not automatically an off-policy replay buffer. Keeping old trajectories because they are expensive to generate can improve hardware utilization while degrading the approximation that justified the update.

The system must specify whether a queue is memory-resident, object-store-backed, or durably persisted. Memory-resident transfer minimizes latency but makes worker failure and capacity pressure important. Durable storage improves recovery and auditability but adds I/O and serialization cost. In either case, each item needs a unique identifier, producer version, completion state, checksum or integrity boundary where appropriate, and idempotent consumption semantics. A retry must not train twice on the same accepted trajectory merely because an acknowledgement was lost.

22.6 veRL as a Concrete Systems Model

veRL is an open-source realization of the HybridFlow design. HybridFlow observes that an RLHF dataflow has large distributed training or generation programs at its nodes and many-to-many state transfers at its edges; it therefore combines centralized orchestration with distributed execution inside worker groups [113]. This is a useful architectural lesson: a single Python-level training loop should not have to micromanage every collective inside FSDP or every model-parallel inference rank.

In veRL terminology, a controller coordinates role-specific workers and their data movement, while model backends can supply the training and rollout implementations. Its public documentation emphasizes flexible mapping of models onto different GPU sets and integrations with FSDP, Megatron-LM, and vLLM [114]. `DataProto` gives those components a common batch interface. These names should be read as one concrete implementation, not a universal taxonomy: another system can use service endpoints, message queues, or a compiled runtime and still need the same policy-version, trajectory-schema, placement, and recovery contracts.

The framework example also exposes a limitation. A centralized controller that receives every full trajectory can become a data-plane bottleneck even if GPU workers are fast. Recent veRL documentation identifies this explicitly for controller-routed `DataProto` transfer and motivates more streamed dispatch paths [114]. The general conclusion is not that every system should be fully asynchronous. It is that control-plane decisions, bulk trajectory transport, and distributed model computation should be profiled separately; otherwise a faster rollout engine can simply move the bottleneck to orchestration.

22.7 Recovery, Observability, and Debugging

Fault recovery begins by deciding which lifecycle boundary is committed. A learner checkpoint should include the policy, critic where applicable, optimizer and scheduler states, random states, prompt-data cursor, and FSDP or other distributed-state metadata described in Chapters 11 and 12. The RL-specific addition is a record of the currently published behavior version, reference and reward identities, queue or rollout-buffer state, accepted-lag rule, sampling configuration, and any curriculum or selection state. A restored learner that cannot identify which trajectories are valid for its next update has not restored the same RL job.

Worker failures need role-specific handling. A failed Rollout Worker can often retry an unfinished request if the published policy version and sampling seed or request identity are preserved. A verifier retry must retain its checker image, timeout, and external environment version. A failed learner during an update requires rollback to a checkpointed commit boundary; partially applied optimizer updates or partially acknowledged trajectory consumption should not be silently combined with a retry. Ray actors can be configured with restart and task-retry policies, but defaults and placement-group behavior must be made explicit rather than assumed [111].

Observed signal	First question	Likely system boundary
Growing rollout queue	Which stage has lower completed-item throughput: generation, reward, or learner consumption?	Backpressure, reward capacity, learner step time, or stale-data admission.
High GPU use but low RL iterations per hour	Are inference and training phases contending, or is queue / serialization time exposed?	Placement, weight synchronization, host transfer, or controller bottleneck.
Rising policy lag	Which weight version generated each queued group, and is the learner accepting it?	Publication cadence, rollout duration, queue backlog, or missing admission control.
Reward result mismatch	Did policy, reward, parser, and Chat Template versions agree for the same token sequence?	Data schema, reward worker, response boundary, or stale service.
Collective stall or learner OOM	At which FSDP lifecycle boundary did the failure first occur?	Wrapping, prefetch, checkpointing, rank divergence, or overlap policy.

Table 33: Distributed RL diagnostics should locate a failure at a dataflow boundary before changing an algorithmic hyperparameter. A policy-loss anomaly can originate in a queue, version, or reward contract rather than in PPO or GRPO itself.

The primary system metrics are not interchangeable with learning metrics. Log requests and output tokens per second, queue occupancy and age, reward latency, learner valid tokens per second, weight-synchronization duration, GPU memory and utilization by role, and FSDP collective wait. Alongside them, retain the algorithmic signals of Chapters 16, 18, and 21: reward distribution, advantage statistics, KL, entropy, ratio and clip fractions, pass rate, response length, selection rate, and policy lag. A rising reward with an increasing backlog does not establish improvement; it may only show that the system has changed which examples reach the learner.

22.8 Implementation Contracts

The role contract must declare which worker groups own policy, rollout, reward or verifier, critic, and reference computations; the model revision and tokenizer / Chat Template used by each; the resource bundle and placement policy; and whether roles are colocated or disaggregated. It must separately state the training and inference parallelism plans, including FSDP ownership, rollout-engine partitioning, precision, and the version-publication mechanism. No worker may claim a policy version without an atomic association to the parameters it used.

The trajectory contract must define the schema in Equation 200, tensor shapes and dtypes, padding and action-mask semantics, per-sample versus batch metadata, serialization format, queue capacity, producer acknowledgement, deduplication key, and maximum accepted version lag. It must preserve group membership, raw reward components, checker traces, and selection decisions where an algorithm needs them. Tests should deliberately repeat, filter, truncate, and reorder a batch to verify that all aligned fields follow the same sample indices.

The recovery and observability contract must define publish, enqueue, consume, update, and checkpoint commit boundaries; retry and idempotency rules; durable-artifact locations; and the conditions under which a queued trajectory is discarded after a restart. It must log role-specific throughput, queue age, policy lag, synchronization time, reward latency, memory peaks, collective waits, and all algorithmic diagnostics required by the chosen objective. An end-to-end test should kill a worker at each boundary, restore from a fresh process, and show that the next learner update consumes a well-defined set of trajectories.

22.9 Summary

Distributed RL training for LLMs is a versioned dataflow, not merely a larger PPO or GRPO batch. Actor Workers optimize a sharded training state; Rollout Workers run an autoregressive inference workload; Reward, Verifier, Critic, and Reference Workers provide different forms of evaluation or control. Correctness depends on preserving the behavior-policy version, sampled log probabilities, token masks, reward provenance, and formatting identity alongside every trajectory.

Placement and scheduling determine whether these roles interfere or overlap. Colocation can avoid weight movement but forces phase sharing; disaggregation enables role-specific scaling but creates synchronization, queue, and staleness costs. FSDP handles learner-state capacity, while inference engines such as vLLM address high-throughput generation. Ray and veRL illustrate how resource bundles, worker groups, controller logic, and typed data containers can realize the resulting system, but no framework removes the need for bounded backpressure, explicit policy publication, durable recovery boundaries, and end-to-end observability.

References

[110] P. Moritz *et al.*, "Ray: A Distributed Framework for Emerging AI Applications," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, USENIX Association, 2018, pp. 561–577. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/moritz

[111] Ray Contributors, "Placement Groups." 2026. [Online]. Available: https://docs.ray.io/en/latest/ray-core/scheduling/placement-group.html

[112] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[113] G. Sheng *et al.*, "HybridFlow: A Flexible and Efficient RLHF Framework," *arXiv preprint arXiv:2409.19256*, 2024, doi: 10.48550/arXiv.2409.19256.

[114] verl Contributors, “verl: Volcano Engine Reinforcement Learning for LLMs.” 2026. [Online]. Available: https://github.com/verl-project/verl

Part V — Inference and Serving

Chapter 23: LLM Inference Fundamentals

Abstract

A trained decoder-only language model generates by repeatedly converting a prefix into next-token logits, selecting one token, and extending the prefix. This chapter separates prompt Prefill from incremental Decode, derives common sampling rules from logits, and explains the role and memory cost of the KV Cache. It then introduces the serving metrics that distinguish an interactive system's first-token latency, per-token generation latency, and aggregate throughput. The goal is a precise execution model for later chapters on cache management and serving optimization.

23.1 Introduction

Pretraining teaches a decoder-only Transformer to assign conditional probabilities to token sequences. At inference time, the same computation is no longer evaluated against a known continuation. A serving system receives a prompt, uses the model to form a distribution for the next token, selects one token according to a decoding rule, and feeds that selected token back as part of the next prefix. The resulting dependence makes generation sequential even when the prompt itself can be processed with substantial parallelism.

Chapter 20 closed the Post-training section by evaluating model behavior under a declared generation protocol. The Inference and Serving section now turns to the execution path that realizes such a protocol. Chapter 5 derived the Causal Language Modeling objective, and Chapter 3 defined causal Self-Attention, key-value heads, and their cached tensors. Here those pieces are treated as an inference program. The discussion stops before cache virtualization, quantization, speculative decoding, continuous batching, and distributed serving. Those techniques optimize the execution model established here; they do not replace it.

Let $\mathcal{V}$ be the vocabulary with size $V = |\mathcal{V}|$. A prompt is a sequence of token IDs $x = (x_1, ..., x_S)$, where S is its current length. The model parameters θ are fixed during ordinary inference. When the model has processed the current prefix, it produces a vocabulary-logit vector $z_t \in \mathbb{R}^V$ for the next generated position t. A decoding rule converts z_t into either a deterministic choice or a random draw.

23.2 From Training to Inference

The training and inference computations share the same conditional distribution, but they use it for different purposes. In teacher-forced training, the corpus supplies every token of a complete sequence. The model can therefore score many next-token targets in parallel, subject to the causal Attention Mask. The loss measures the probability assigned to observed targets; Chapter 5 shows how those token-level terms aggregate into the negative log-likelihood.

At inference time, a future target is unavailable. After the model forms $p_{\theta(\cdot \mid x_1,...,x_S)}$, a decoding policy must select a token $\hat{x}_{S+1}$. That token becomes part of the context for the next forward step. For a generated continuation $y = (y_1, ..., y_N)$, this feedback loop has the form

$$y_t \sim q\Big(\cdot \mid x, y_{\{<t\}}\Big), \quad y_{\{<t\}} = \Big(y_1, ..., y_{\{t-1\}}\Big), \tag{203}$$

where q is the effective decoding distribution. It may equal p_θ, or it may be a transformed and truncated version of the model distribution. The distinction is important: the language model defines logits and conditional probabilities, whereas a serving policy defines how to choose among them. A change in temperature or sampling threshold can change generated text without changing θ.

The loop terminates when it selects an end-of-sequence token, reaches a declared output-token limit, or satisfies an application-level stop condition. A stop string is not normally a token-level property of the probability model. It is a rule applied to decoded output, often after checking a token sequence for one of several configured suffixes. The serving contract must specify whether the stop tokens themselves are returned and whether they remain in the internal context.

23.3 Prompt Processing: Prefill and Decode

Inference has two execution regimes. The first is **Prefill**, also called prompt processing. Given all S prompt tokens, the system runs the Transformer over the complete prompt, constructs the cache needed by future attention layers, and obtains logits for the first output token:

$$(x_1, ..., x_S) \to (K_1, V_1, ..., K_L, V_L, z_0). \tag{204}$$

Here L is the number of Transformer blocks, and (K_l, V_l) denotes the stored key and value tensors for layer l. Causal masking preserves the autoregressive dependency rule, but all prompt positions are present, so the projections, attention computations, and Feed-Forward Networks can operate on many token positions concurrently. For long prompts, this stage can contain much more arithmetic work than one decoding step.

After the first selected token is appended, the system enters **Decode**. Instead of reprocessing the entire prefix, the new token is embedded and passed through one incremental Transformer step. At each layer, its new query attends to cached keys and values from the prior prefix, while its new key and value are appended to the cache:

$$((K_l, V_l), y_t) \to (K_l^+, V_l^+, z_t), \tag{205}$$

where the superscript $+$ denotes the cache after the new token has been incorporated. Sampling from z_t produces y_{t+1}, which triggers the next step. Autoregressive generation is therefore sequential over output tokens, not necessarily over the input prompt. Large Transformer inference studies distinguish the high-parallelism prefill workload from the less parallel incremental-generation workload for precisely this reason [115].

Phase	Input available at once	Persistent state produced or consumed	Primary user-visible consequence
Prefill	All prompt tokens for each request.	Constructs keys and values for every prompt position; produces first-token logits.	Dominates Time to First Token for a long prompt.
Decode	One newly selected token per active request per step.	Reads the prefix KV Cache and appends one layerwise key-value pair.	Determines incremental token delivery rate.

Table 34: Prefill and Decode use the same parameters but expose different shapes and bottlenecks. Prefill processes a known token block; Decode extends an unknown continuation one token at a time.

23.3.1 The KV Cache

Without a KV Cache, decoding token y_t would require recomputing the keys and values of every earlier token at every layer. The cache preserves those earlier projections. The current token still needs a query, key, and value projection, but attention can compare its one new query against stored keys and combine stored values rather than regenerating the prefix's projections. This avoids redundant prefix work and makes incremental decoding practical.

Use the attention notation of Chapter 3: g is the number of key-value heads and d_h is head dimension. For a batch of B active sequences with current length S, one layer stores

$$K_l, V_l \in \mathbb{R}^{B \times g \times S \times d_h}. \tag{206}$$

If every cached scalar uses b_{cache} bytes, the cache footprint for all layers is approximately

$$M_{\text{KV}} = 2BLSgd_h b_{\text{cache}}. \tag{207}$$

The factor two accounts for keys and values. This accounting excludes model weights, activations and temporary workspaces, allocator overhead, and padding or fragmentation. It nevertheless exposes the central scaling law: cache memory grows linearly with sequence length, batch size, layer count, key-value head count, head dimension, and bytes per cached scalar. Grouped-Query Attention and Multi-Query Attention reduce g, which Chapter 3 explains as an architectural trade-off that is especially valuable during decoding.

Model-weight memory and KV Cache memory are distinct. Weight memory is largely fixed for a loaded model and depends on parameter count and weight dtype. KV Cache memory is request-dependent state. It grows while a request generates, varies across prompts and stopping times, and limits how many long requests can coexist. Production serving research identifies this dynamically sized cache as a principal source of memory pressure under batched generation [116].

23.4 From Logits to Tokens

The final hidden state at the current position is mapped by the language-model head to logits $z_t = (z_{t,1}, ..., z_{t,V})$. As in Chapter 5, Softmax converts those unnormalized scores to a categorical distribution. In inference, however, this distribution is an input to a decision rule rather than to cross-entropy with a known target.

23.4.1 Greedy Decoding and Temperature

Greedy decoding chooses the most probable token at every step:

$$y_{t+1} = \text{argmax}_{i \in \{1,...,V\}} \, z_{t,i}. \tag{208}$$

Because Softmax preserves the ordering of logits, taking an argmax over logits is equivalent to taking one over probabilities. Greedy decoding is deterministic if the model, tokenizer, prompt formatting, and numerical execution are deterministic. It is often useful when exact reproducibility or a constrained output format matters, but it follows only one local continuation and does not optimize the probability of a complete sequence globally.

Temperature changes the sharpness of the categorical distribution before sampling. For $T_{\text{temp}} > 0$,

$$p_{t,i}^{T_{\text{temp}}} = \text{softmax}_i\left(\frac{z_{t,i}}{T_{\text{temp}}}\right). \tag{209}$$

When $T_{\text{temp}} = 1$, this is the model's ordinary Softmax distribution. As T_{temp} approaches zero, probability concentrates on the largest logit and sampling approaches greedy selection; implementations handle this limiting case by using greedy decoding rather than dividing by zero. Values above one flatten the distribution and increase the relative probability of lower-ranked tokens. Temperature does not add knowledge or correct a model error. It changes how readily the system explores alternatives already represented in its logits.

23.4.2 Top-k and Top-p Sampling

Unrestricted sampling can select tokens from a long, low-probability tail. **Top-k sampling** first retains only the k highest-probability tokens. Let $K_{k(z_t)}$ be their index set. Its truncated distribution is

$$q_{t,i} = \begin{cases} \frac{p_{t,i}}{\sum_{j \in K_{k(z_t)}} p_{t,j}} i \in K_{k(z_t)} \\ \backslash\, 0 \qquad\qquad \text{otherwise.} \end{cases} \tag{210}$$

and y_{t+1} is sampled from q_t. The cardinality is fixed, but the probability mass covered by the retained set can vary widely across contexts. Top-k sampling has been used in open-ended neural text generation as a practical truncation rule [117].

Top-p sampling, or nucleus sampling, chooses the smallest prefix of tokens sorted by decreasing probability whose cumulative mass reaches a threshold p_{nucleus}. If $P_{p_{\text{nucleus}}}(z_t)$ denotes that set, the sampling distribution is renormalized over $P_{p_{\text{nucleus}}}(z_t)$ exactly as in Equation 210. The retained set has variable size: a peaked distribution may require few tokens, whereas a diffuse distribution may require many. Holtzman et al. motivate this adaptive truncation by observing that the low-probability tail of an open-ended language-model distribution can be unreliable for generation [118].

Temperature, Top-k, and Top-p are not interchangeable. Temperature reshapes every probability before a later filter; Top-k fixes a candidate count; Top-p fixes a retained probability mass. Their composition is an implementation decision. A reproducible serving interface must record the order of operations, thresholds, random seed policy, and whether any token-specific penalties were applied before sampling.

23.4.3 Repetition Controls, Stops, and Beam Search

Practical interfaces often transform selected logits based on the generated prefix. A repetition penalty, frequency penalty, or presence penalty changes the relative scores of tokens that have already occurred. Such controls may reduce a visible repetition failure, but they change the effective distribution q in Equation 203 and can also suppress legitimate repetition. They should be regarded as decoding-policy parameters, not learned properties of the base model.

Stopping is likewise part of the policy layer. A request typically has a maximum output-token budget, one or more end-of-sequence or application stop sequences, and perhaps a context-window limit. A system should check a stop condition after selecting a token and before scheduling another Decode step. If a generated token would exhaust the allowed context, the request must stop or follow a separately declared context-management policy; silently exceeding the model's positional contract is not valid inference.

Beam Search keeps several partial continuations and repeatedly expands the most promising ones under cumulative log-probability. It is useful when a task admits a short, sharply scored output and a sequence-level search budget is appropriate. For open-ended LLM interaction, however, deterministic likelihood maximization can produce generic or repetitive text, and stochastic sampling is often more central. This is a task-dependent observation, not a rule that Beam Search is universally inappropriate; empirical work on neural text degeneration makes the distinction explicit [118]. Beam Search also multiplies active hypotheses and hence cache state, so its systems cost differs from a single sampled continuation.

23.5 Context, Batching, and Serving Metrics

The **context length** of an active request is the number of tokens currently available to the Transformer: prompt tokens plus generated tokens, together with any required special tokens. It is not only a model-side maximum. It affects prefill work, attention reads during Decode, and the cache footprint in Equation 207. A request with a short output can still have a high first-token cost if its prompt is long; a request with a short prompt can become expensive over a long continuation because every new token enlarges the prefix read by later decoding steps.

Batch inference processes multiple independent requests together. During Prefill, batching combines prompt tokens from multiple requests to expose larger matrix operations. During Decode, a scheduler may advance several active requests in one step, usually after padding or otherwise representing their differing current lengths. The batch size B in Equation 207 is therefore an operational quantity: increasing it can improve hardware utilization and total throughput, but it also increases cache memory and may delay an individual request while it waits to be admitted or scheduled.

Metric	Typical unit	Interpretation
Time to First Token (TTFT)	seconds or milliseconds	Elapsed time from a request's arrival or dispatch, under a declared convention, until its first generated token is available. It includes queueing and Prefill under many service definitions.
Time per Output Token (TPOT)	milliseconds per token	Incremental time between delivered generated tokens after the first. It reflects Decode scheduling and execution, not only model arithmetic.
Output rate	output tokens per second	The reciprocal of average TPOT under a compatible measurement convention.
Request rate	requests per second	Completed or admitted requests per unit time; it must be reported with prompt and output-length distributions.
Token throughput	tokens per second	Aggregate input and/or output tokens processed per wall-clock time. The included token types must be declared.

Table 35: Serving metrics answer different questions. A high aggregate token throughput does not by itself imply low interactive latency, and a low TTFT does not guarantee rapid subsequent tokens.

For one request with N generated tokens, end-to-end latency contains queueing time, Prefill time, and the accumulated time of its Decode steps. TTFT is closely related to the first two components and the first Decode selection, depending on the service boundary. TPOT summarizes the later Decode work. A throughput-oriented workload may choose a larger batch and tolerate more queueing; an interactive workload usually imposes a tighter TTFT and TPOT target. The same model can therefore have different preferred schedules for different latency objectives.

Prefill is often compute-heavy because it processes many known token positions and can use large matrix operations efficiently. Decode has much less token-position parallelism for a single request. It repeatedly reads model weights and a growing KV Cache while producing a small amount of new-token work, so its realized performance is frequently limited by memory bandwidth or memory access rather than by peak floating-point throughput. This is a workload-level tendency, not a hardware-independent law: model shape, batch size, context length, dtype, kernels, and device topology all matter [115]. Later chapters will examine the optimization techniques that change this trade-off.

23.6 Implementation Contracts

An inference implementation should make the model-data interface as explicit as a training implementation. The tokenizer, special tokens, and Chat Template used to construct the prompt must match the model's expected convention; Chapter 13 explains why formatting is part of the learned interface for chat models. The prompt must fit the declared context policy before Prefill, and the token selected at each Decode step must be appended exactly once before the next model call.

The cache needs a precise ownership and shape contract. For each request and layer, keys and values must have the dtype, head mapping, sequence positions, and batch association expected by the attention kernel. A cache may not be reused across unrelated requests unless an explicit prefix-sharing rule establishes that the corresponding token prefix, model revision, positional convention, and cache representation are identical. When a request stops or is cancelled, its cache allocation must become unavailable to future reads.

The decoding contract should record the complete policy: greedy or sampled mode; temperature; Top-k and Top-p thresholds; repetition controls; random-number-generator and seed behavior; maximum tokens; stop rules; and whether terminal tokens are emitted. Tests on a small model should compare cached Decode logits with logits from a full recomputation on the same prefix, up to the numerical

tolerance of the execution dtype. They should also verify that an identical deterministic policy yields identical tokens, that a fixed seeded sampling policy is reproducible under a declared runtime configuration, and that TTFT, TPOT, and throughput are measured with stated start and end events.

23.7 Summary

Inference turns the conditional distribution learned in pretraining into a sequential execution loop. Prefill processes a known prompt in parallel where possible, constructs the layerwise KV Cache, and produces first-token logits. Decode then adds one selected token at a time, reusing cached keys and values so that earlier prefix projections are not recomputed. The model supplies logits; temperature, truncation, penalties, and stop rules define the serving policy that turns those logits into an output sequence.

KV Cache memory is dynamic request state, distinct from fixed model-weight memory. Its linear dependence on active batch size, sequence length, layers, key-value heads, head dimension, and dtype explains why inference capacity and latency cannot be understood from parameter count alone. TTFT, TPOT, and throughput capture complementary consequences of Prefill, Decode, queueing, batching, and memory traffic. These concepts establish the baseline for the cache-management and execution optimizations considered next.

References

[115] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[116] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[117] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical Neural Story Generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2018, pp. 889–898. doi: 10.18653/v1/P18-1082.

[118] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=rygGQyrFvH

Chapter 24: KV Cache and Memory Optimization

Abstract

The KV Cache makes incremental decoding possible by retaining attention keys and values for prior tokens, but it turns sequence length and concurrency into persistent memory demand. This chapter derives cache memory accounting from Transformer dimensions, distinguishes MHA, MQA, and GQA cache layouts, and explains allocation, fragmentation, paging, prefix sharing, and cache lifecycle. It then introduces cache-specific quantization as a memory-quality trade-off. The chapter develops the memory-management principles that later serving systems use without treating scheduling or model-weight quantization in depth.

24.1 Introduction

Chapter 23 established the two inference phases: Prefill processes a known prompt and constructs the KV Cache, while Decode reuses that state as it appends one token at a time. The cache eliminates a severe source of redundant computation, but it replaces that work with persistent request state. Unlike model weights, which are mostly fixed once a model is loaded, cache capacity depends on how many sequences are active and how long they become.

This distinction changes the resource problem of serving. A model that fits comfortably in accelerator memory at batch size one can admit far fewer concurrent long-context requests than its weight footprint alone suggests. The relevant question is not merely how many parameters the model has. It is how many layerwise key-value vectors must coexist, in which representation, for how long, and with what degree of sharing or waste.

The chapter uses the attention notation of Chapter 3. Let B be the number of active sequences, S their current cached length, L the number of Transformer blocks, g the number of key-value heads, and d_h the head dimension. Let b denote bytes per cached scalar. The discussion assumes a dense decoder-only Transformer and a cache that stores keys and values separately. Exact tensor strides, padding, metadata, and temporary workspaces vary by runtime, but the basic accounting does not.

24.2 KV Cache Structure and Memory Accounting

For one layer and one active sequence, the key cache and value cache each contain a vector for every cached position and every key-value head. Their shapes are

$$K_l, V_l \in \mathbb{R}^{S \times g \times d_h}, \quad l \in \{1, \ldots, L\}. \tag{211}$$

With a batch axis, each cache tensor has shape $B \times g \times S \times d_h$. One tensor contains $BSgd_h$ scalars. Since attention retains both keys and values, the persistent cache across all layers contains approximately

$$N_{\mathrm{KV}} = 2BSLgd_h \tag{212}$$

scalars and therefore occupies

$$M_{\mathrm{KV}} = 2BSLgd_h b \tag{213}$$

bytes. The factor two is not a constant to discard casually: the attention score needs historical keys, while the weighted attention output needs historical values. The remaining factors describe independent multiplicative pressure. Doubling active batch size, cached sequence length, layer count, key-value heads, head dimension, or byte width doubles the approximate persistent cache memory.

Equation 213 is an accounting model, not an allocator specification. A runtime may reserve tokens in blocks, pad a sequence, retain scaling metadata for quantized data, or keep temporary attention buffers. The exact allocated memory can consequently exceed the ideal payload. Nevertheless, the equation identifies the dominant state whose linear growth makes long-context and high-concurrency serving difficult. Large-inference studies likewise separate fixed parameter memory from decoding-time KV state and its associated memory traffic [119].

Attention form	KV heads	Per-layer K or V shape	Both tensors, all layers
MHA	$g = h$	$B \times h \times S \times d_h$	$2BSLhd_hb$
GQA	$1 < g < h$	$B \times g \times S \times d_h$	$2BSLgd_hb$
MQA	$g = 1$	$B \times S \times d_h$	$2BSLd_hb$

Table 36: Ideal KV Cache payload for three attention organizations. Query-head count remains h; changing the number of key-value heads changes the persistent tensors that incremental attention reads.

24.2.1 MHA, MQA, and GQA

Multi-Head Attention (MHA) has one key and one value head for every query head, so $g = h$. Multi-Query Attention (MQA) keeps the h query heads but shares a single key head and a single value head across them, so $g = 1$. Grouped-Query Attention (GQA) lies between those endpoints: several query heads share each key-value head, with $1 < g < h$. Chapter 3 derives this head mapping and its attention computation.

The important memory consequence follows directly from Table 36. The number of query heads controls how many query rows form attention scores; it is not the same as the number of distinct historical key-value vectors. MQA and GQA reduce the latter without reducing the former in the same proportion. During Decode, this lowers cache capacity and the cache data read by attention. Shazeer introduced MQA as a decoding-oriented key-value sharing design [120], while Ainslie et al. frame GQA as an intermediate architecture that can retain much of MHA quality with much of MQA's inference benefit [121].

Head reduction is an architectural choice made when a model is trained or adapted. It is not a generic serving switch that can be applied to an arbitrary MHA checkpoint without changing its key and value projections. A serving runtime must discover the model's actual number of KV heads from the checkpoint configuration and use the corresponding cache shape. Treating query-head count as KV-head count is a common accounting error that can overestimate capacity by a large factor for GQA models.

24.3 Allocation, Fragmentation, and Paged KV Caches

The ideal payload in Equation 213 assumes that every allocated scalar represents a useful cached token. A simple static allocator violates that assumption when it reserves a contiguous region for each request's maximum permitted length. A short request then occupies capacity for tokens it will never generate; a request that terminates early leaves a partially unused reservation. Reserving for the maximum avoids reallocations, but it can reduce the number of requests admitted long before useful cache payload fills memory.

Dynamic contiguous allocation improves on static reservation by growing a request as it generates. It introduces a different problem. Many requests grow and terminate at different times, so free capacity becomes divided into non-adjacent holes. A later request may need a large contiguous region even when the total number of free bytes is sufficient. This **external fragmentation** is allocator-level waste: the memory exists, but its layout prevents an otherwise valid allocation.

Paged allocation avoids requiring one physical span per logical token sequence. Choose a fixed block size of P token positions. A logical cache sequence of length S then uses

$$n_{\text{pages}} = \text{ceil}\left(\frac{S}{P}\right) \tag{214}$$

logical pages per layer and per key-value tensor. A page table maps those logical pages to any available physical blocks. The final page may be partially occupied, but the unused capacity is bounded by fewer than P token positions per tensor rather than by the request's entire unused maximum length.

Figure 4: A paged KV Cache separates logical token order from physical block placement. Attention follows the logical page table; the runtime need not keep a request's cache in one contiguous physical allocation.

PagedAttention applies this virtual-memory-style organization to attention's key-value storage. A logical sequence is represented by fixed-size KV blocks that can occupy non-contiguous physical memory, while the attention computation consults the block mapping to read them in logical order. This organization limits reservation waste, accommodates variable generation lengths, and creates a natural unit for cache sharing. Kwon et al. introduced PagedAttention and the vLLM serving system to address fragmentation and redundant KV duplication under high-throughput serving [122].

Paging does not make cache memory free. Every live logical token still needs key and value storage, and page tables, block rounding, and allocator metadata have costs. The benefit is better utilization of the available capacity and a representation in which growing, terminating, and shared sequences need not be reshaped into a single contiguous buffer. Scheduling policy and continuous batching decide which requests advance; this chapter concerns the cache representation they manage.

24.4 Reuse, Prefix Sharing, and Cache Lifecycle

Three forms of reuse are often conflated. **KV reuse within one generation** is the ordinary Decode behavior from Chapter 23: a request reuses its own past keys and values after each new token. **Prefix caching** records the Prefill result for an already processed token prefix so that a later request with exactly that prefix can begin after it. **Prefix sharing** refers to a memory representation in which two or more live requests reference the same physical KV blocks for an identical prefix. Prefix caching can enable prefix sharing, but a cache lookup and a shared physical allocation are not identical mechanisms.

Suppose R requests share a tokenized prefix of length S_p and each has a distinct suffix of length S_u. Ignoring block rounding and metadata, storing every prefix separately costs a quantity proportional to

$$R(S_p + S_u), \tag{215}$$

whereas physically sharing the immutable prefix blocks costs a quantity proportional to

$$S_p + RS_u. \tag{216}$$

The difference is $(R-1)S_p$ token positions of cache payload per layer and per key-value tensor. Common system prompts, shared document prefixes, and repeated few-shot exemplars can therefore create meaningful reuse opportunities. The equality test must operate on the exact serialized token prefix and the same model revision, tokenizer, positional convention, cache dtype, and relevant attention configuration. Semantic similarity is insufficient: two similar prompts with different token IDs cannot share the same computed keys and values.

Cache lifetime has four operational phases. A request first allocates or acquires blocks during Prefill. It grows by appending blocks during Decode. Its blocks can become reusable after the request terminates, and a retained prefix may remain available for a future compatible request. Eviction removes retained state to recover capacity. Reference counting or an equivalent ownership rule is essential when blocks are shared: one request ending must not invalidate a prefix still referenced by another request.

24.4.1 Long Context and Eviction

For an unbounded generation, the cache grows with S in Equation 213 until it reaches the model's context limit or available memory. Long context makes the cache expensive in two ways. It consumes persistent capacity, and later Decode steps must read more historical keys and values. High concurrency compounds both effects because all active sequences contribute to the batch factor B.

An eviction policy bounds cache capacity by discarding some historical entries. This is not a transparent allocator optimization: discarded keys and values are no longer available to attention, so it

changes the effective context seen by the model. A sliding window preserves recent tokens but may lose an early instruction or relevant document. More selective policies try to retain entries judged important as well as recent ones. H2O, for example, studies an eviction policy that balances recent tokens with attention heavy hitters [123]. Such policies are model- and task-dependent approximations, not a substitute for preserving the full declared context.

The cache owner should therefore state its eviction semantics explicitly: capacity bound, retained-window policy, selection criterion, whether eviction occurs per layer or globally, and how the resulting effective context is reported. An application that claims a certain context length while silently evicting most of it is describing a different capability from full-context attention.

24.5 KV Cache Quantization

KV Cache quantization reduces the byte width b in Equation 213 rather than changing model architecture. If a reference cache uses b_{ref} bytes per scalar and a quantized cache uses an effective b_{quant} bytes, the ideal payload ratio is

$$\frac{M_{\text{KV}}^{\text{quant}}}{M_{\text{KV}}^{\text{ref}}} \approx \frac{b_{\text{quant}}}{b_{\text{ref}}}. \tag{217}$$

The word **effective** matters. Low-bit blocks also require scales, zero-points, group metadata, alignment, and sometimes a recent full-precision region. Dequantization or mixed-precision attention kernels can add compute and memory traffic. The realized saving is consequently smaller or differently shaped than the scalar byte ratio alone.

Keys and values need not have identical quantization behavior. Keys affect attention scores before Softmax; values affect the weighted accumulation after attention probabilities have been formed. Quantization error can therefore alter both which positions receive attention and what information is aggregated from those positions. KIVI studies cache-specific asymmetric quantization and reports that its key and value cache distributions favor different grouping choices, illustrating why a single generic quantization rule need not be equally suitable for both tensors [124].

The practical trade-off is not simply memory against average perplexity. Cache quantization should be evaluated under the intended context lengths, output lengths, sampling policy, batch concurrency, and task distribution. A configuration that preserves short-answer quality may fail a retrieval-heavy long-context task, and a memory reduction that enables a larger batch may add latency if dequantization becomes a bottleneck. General model-weight quantization changes the storage and arithmetic of learned parameters; it is a related but distinct topic reserved for the next chapter.

24.6 Implementation Contracts

The cache contract begins with model identity. The runtime must know layer count, query-head count, KV-head count, head dimension, cache dtype, positional convention, and the exact tokenizer and model revision used to construct a prefix. Its allocated shapes should agree with Equation 211 and its accounting dashboard should distinguish ideal payload from reserved blocks, metadata, temporary workspaces, and model weights.

For paged storage, tests should verify that a logical page table produces the same attention output as a contiguous reference layout for the same prefix. They should exercise page growth at a block boundary, requests with different termination lengths, block reuse after termination, and shared-prefix reference counts. A cancellation path must release only blocks whose last reader has departed. Metrics should report cache utilization and fragmentation separately from total allocated bytes; otherwise a full allocator can be mistaken for a full useful cache.

For prefix reuse, the lookup key must include the full token prefix and all state that affects cached keys and values. For cache quantization, the contract must define bit width, group shape, scale and zero-point representation, residual full-precision policy, dequantization kernel, and accepted numerical tolerance. Tests should compare quantized and reference cache logits or attention outputs on declared lengths and tasks, not merely verify that allocation succeeds. These conditions make a memory optimization auditable rather than an opaque reduction in reported bytes.

24.7 Summary

The KV Cache stores two persistent attention tensors for every cached position, layer, KV head, and active sequence. Its approximate footprint, $2BSLgd_h b$, explains why cache state becomes a first-order serving constraint under long contexts and high concurrency. MQA and GQA reduce the number of stored key-value heads while preserving the query-head organization that forms attention scores.

Memory optimization is therefore a representation and lifecycle problem as well as a byte-count problem. Paged KV Caches replace maximum-length contiguous reservations with fixed logical pages mapped to available physical blocks. Prefix sharing avoids duplicating immutable common prefixes, while explicit lifecycle and eviction rules determine when blocks may be retained or reclaimed. Cache quantization reduces bytes per element but can perturb attention scores and outputs, so its quality and performance must be tested in the actual serving regime. These principles prepare the later study of weight quantization, scheduling, and full serving systems.

References

[119] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[120] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: https://arxiv.org/abs/1911.02150

[121] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.

[122] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[123] Z. Zhang *et al.*, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfcebb2827f8-Abstract-Conference.html

[124] Z. Liu *et al.*, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," *arXiv preprint arXiv:2402.02750*, 2024, doi: 10.48550/arXiv.2402.02750.

Chapter 25: Quantization for LLM Inference

Abstract

Quantization represents selected model tensors with fewer bits while retaining an approximation to their original values. This chapter develops affine integer quantization, its scale and zero-point parameters, and the distinctions among representation, granularity, and calibration. It then explains weight-only and weight-and-activation regimes, Post-Training Quantization and Quantization-Aware Training, activation outliers, and influential LLM methods including LLM.int8(), SmoothQuant, GPTQ, and AWQ. The emphasis is on the end-to-end trade-off among model memory, bandwidth, kernel execution, and output quality rather than on a particular inference library.

25.1 Introduction

Chapter 24 treated KV Cache quantization as a reduction in the persistent state created by active requests. This chapter concerns a different, mostly fixed object: the learned model weights loaded before requests arrive. A model with N parameters occupies approximately Nb bytes when each parameter is stored with b bytes. For sufficiently large models, that footprint can determine whether a checkpoint fits on an accelerator at all. Even when it fits, Decode often rereads a large fraction of those weights for every emitted token, so moving them through the memory hierarchy can limit throughput.

Quantization changes the representation used to store and execute selected tensors. It is not synonymous with low-precision floating-point arithmetic. FP16 and BF16 are floating-point formats with an exponent and significand; Chapter 8 explains their range and precision trade-offs. Integer quantization instead stores an integer code with an associated mapping back to an approximate real value. The mapping introduces error, but it can reduce weight capacity and memory traffic substantially. Whether it also improves latency depends on the execution kernel and hardware, not on the bit count alone.

The chapter uses a dense linear map as its running object. Let $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ be a weight matrix, $x \in \mathbb{R}^{d_{\text{in}}}$ an input activation, and $y = xW$ its output. The same representation questions apply to the projection matrices in attention and feed-forward layers. Cache-specific quantization remains distinct: it changes request-dependent keys and values, whereas this chapter primarily changes learned parameters and, in some regimes, transient activations.

25.2 Quantization as an Approximation Model

For a real-valued scalar x, affine quantization selects an integer code q through

$$q = \operatorname{clip}_{q_{\min}, q_{\max}} \left(\operatorname{round}\left(\frac{x}{s} \right) + z \right), \tag{218}$$

with approximate reconstruction

$$\hat{x} = s(q - z). \tag{219}$$

Here $s > 0$ is a **scale**, z is an integer **zero point**, and q lies in the representable code interval $[q_{\min}, q_{\max}]$. The scale sets the separation between adjacent reconstructed values; the zero point determines which code represents real zero. Rounding produces an irreducible representation error even when x lies in range. Clipping produces a larger error when x lies outside the interval that the selected scale can cover.

For an unsigned k-bit code, a conventional range is $q_{\min} = 0$ and $q_{\max} = 2^k - 1$. Given chosen real endpoints $x_{\min}$ and $x_{\max}$, a common affine calibration rule is

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad z = \operatorname{round}\left(q_{\min} - \frac{x_{\min}}{s} \right). \tag{220}$$

The reconstructed grid covers the chosen interval only approximately because z must be integral. Its quality therefore depends on both the source distribution and the rule used to estimate the endpoints.

25.2.1 Symmetric and Asymmetric Schemes

Symmetric quantization centers the codebook at zero. For signed codes $q \in \{-Q, ..., Q\}$, it commonly uses $z = 0$ and

$$q = \mathrm{clip}_{-Q,Q}\left(\mathrm{round}\left(\frac{x}{s}\right)\right), \quad s = \frac{\max|x|}{Q}. \tag{221}$$

It is simple to implement and maps positive and negative magnitudes symmetrically. It can, however, waste codes when the observed tensor is strongly shifted or has an asymmetric range. **Asymmetric quantization** uses the nonzero z in Equation 218 to place zero and the representable interval more flexibly. This can use a bounded nonnegative distribution efficiently, but it introduces zero-point metadata and can complicate some integer kernels.

Neither adjective states the whole quantization policy. A **datatype precision** specifies the code format, such as FP16, BF16, INT8, or INT4. A **quantization scheme** specifies the mapping, including signedness, symmetry, clipping, and rounding. A **granularity** specifies which elements share the same parameters s and z. A **calibration strategy** specifies how those parameters are selected. Confusing these axes leads to imprecise statements such as "the model uses INT4," which says neither how its scale is shared nor which tensors still execute at a wider precision.

25.3 Granularity, Regimes, and Memory

A single scale for an entire tensor is inexpensive but must accommodate its widest range. Per-tensor quantization uses one pair (s, z) for all entries of W. Per-channel quantization assigns a pair to each output or input channel, according to the kernel's convention. Group-wise quantization partitions a channel or flattened weight matrix into fixed-size groups and gives each group its own parameters. The groups are smaller than a whole tensor but usually larger than one scalar.

Granularity	Shared parameters	Primary advantage	Primary cost
Per-tensor	One s and z for all entries	Minimal metadata and simple packing	A single outlier can enlarge every step size
Per-channel	One pair per selected matrix channel	Adapts to channel-scale variation	More metadata and channel-aware kernels
Group-wise	One pair per fixed block of weights	A practical low-bit accuracy–metadata balance	Group layout, packing, and dequantization complexity

Table 37: Finer granularity restricts the range that each scale must cover, usually reducing quantization error. It also adds parameter metadata and constraints that an execution kernel must honor.

Finer sharing often improves accuracy because each scale represents a narrower local distribution. It is not free: the scale and zero point consume storage, group boundaries affect packing, and the kernel must fetch or apply the correct parameters. At very low bit widths, group-wise weight quantization is a common compromise between one global scale and costly per-element adaptation.

The ideal memory model makes the benefit visible. If a reference weight tensor has N values stored with b_{ref} bytes each, and a quantized representation stores k bits per code, then

$$M_{\mathrm{ref}} = N b_{\mathrm{ref}}, \quad M_{\mathrm{quant}} \approx N\frac{k}{8} + M_{\mathrm{metadata}}. \tag{222}$$

Relative to FP16 weights, ideal INT8 storage is one-half and ideal INT4 storage is one-quarter before metadata, padding, or unquantized layers. Equation 222 applies only to the represented weights. It does not reduce activation buffers, temporary workspaces, or the KV Cache derived in Chapter 24. A deployment can therefore have a much smaller checkpoint while seeing a smaller reduction in total resident memory.

25.3.1 Weight-Only and Weight-and-Activation Quantization

In **weight-only quantization**, a low-bit representation replaces W, while the input activation x and the matrix-product accumulation remain in a wider type, commonly FP16 or BF16. A kernel can unpack and dequantize weight groups during the matrix multiplication rather than materializing

an entire widened copy. This regime reduces the persistent parameter footprint and, when weights dominate memory reads, can reduce bandwidth demand. Its activations still require wide storage and arithmetic.

In **weight-and-activation quantization**, both W and selected activations have low-bit representations, often summarized as W8A8 for INT8 weights and activations. This can reduce traffic and enable integer-oriented matrix kernels more broadly, but activation quantization is harder. Activations are input-dependent, change by layer and token position, and may have rare large values. A static scale that works for one calibration set can clip another input; a dynamically computed scale adds work and may be difficult to fuse into a high-throughput kernel.

FP16 and BF16 remain useful baselines rather than failed forms of quantization. They preserve a floating exponent per value, so they have much wider dynamic range than a fixed-scale integer group. INT8 is often a moderate compression regime with relatively mature hardware support. INT4 gives larger capacity and bandwidth savings, but its coarse codebook makes granularity, calibration, and layer sensitivity more consequential. Mixed schemes retain higher precision for selected layers, channels, activations, or accumulations when a uniform format would create too much error.

25.4 Obtaining a Quantized Model

Post-Training Quantization (PTQ) starts from a completed floating-point checkpoint and chooses a representation without reoptimizing the original training objective. It may consist of a direct range-based conversion, a calibration pass through representative inputs, or a reconstruction procedure that minimizes the error of selected layer outputs. PTQ is attractive when retraining is unavailable or too expensive, but its result is limited by how well the calibration data and error model capture the deployment distribution.

Quantization-Aware Training (QAT) exposes the model to the approximate quantization path during optimization. A simplified forward model is

$$\widehat{W} = \mathrm{dequant}(\mathrm{quant}(W)), \quad y = x\widehat{W}, \tag{223}$$

while a surrogate gradient, often a Straight-Through Estimator, is used through the nondifferentiable rounding operation. The optimizer can then adapt weights to the quantization error. Foundational integer-only inference work combines quantized arithmetic with a co-designed training procedure [125]. QAT can improve accuracy at demanding precisions, but it reintroduces an optimization workflow, compute cost, and a risk that the learned tolerance is too specific to the simulated format or training distribution.

Calibration is a core PTQ contract, not an incidental preprocessing step. The selected examples determine observed ranges, activation statistics, and sometimes layerwise reconstruction objectives. They should cover the expected prompt domains, formatting, and sequence behavior of deployment. A calibration set consisting only of short plain-text prompts may omit code, structured inputs, multilingual text, or long-context patterns that produce different activation scales. More examples are not automatically better if their mixture is unrepresentative; the objective is coverage of the runtime tensor distributions relevant to the chosen quantization scheme.

25.5 Outliers and LLM Quantization Methods

An **outlier** is an unusually large-magnitude tensor value or channel relative to the bulk of its local distribution. In a group with one scale, a single large value increases s in Equation 218. The many ordinary values then occupy fewer distinguishable code levels and have larger relative rounding error. This is especially troublesome for activation quantization because activation ranges vary with the input. LLM.int8() identifies systematic Transformer feature outliers and combines vector-wise INT8 quantization with a mixed-precision path for the exceptional feature dimensions [126].

SmoothQuant addresses a related imbalance by applying an algebraically equivalent channel transformation before quantization. For a diagonal matrix D with positive entries,

$$y = xW = (xD^{-1})(DW) = x'W'. \tag{224}$$

The exact linear map is unchanged in real arithmetic. The choice of D moves channel scale between activations and weights, allowing activation outliers to be smoothed before an INT8 weight-and-activation path. SmoothQuant selects this migration from offline statistics and specifically targets the fact that activations can be more difficult to quantize than weights [127]. The transformation does not eliminate approximation error; it changes where the fixed quantization budget is spent.

25.5.1 GPTQ and AWQ

GPTQ is a one-shot PTQ method for low-bit weights. Rather than scoring an isolated weight error only by $|W - \widehat{W}|$, it uses calibration activations to approximate how a layer's output changes. For a calibration activation matrix X, a representative reconstruction objective is

$$\min_{\widehat{W}} \|XW - X\widehat{W}\|_F^2. \tag{225}$$

The input correlation $X^T X$ supplies approximate second-order information about directions that the layer output is sensitive to. GPTQ quantizes weights sequentially and compensates remaining unquantized values using this information, aiming to limit output distortion at low bit widths [128]. The crucial point is not that every deployment must calculate a Hessian. It is that equal parameter-space errors are not equal functional errors once the activation distribution is considered.

AWQ is also a low-bit weight-only PTQ method, but it uses observed activation statistics to identify salient channels and chooses a hardware-friendly scaling that protects them. Its central premise is that the importance of a weight channel is better inferred from how it interacts with activations than from weight magnitude alone. The method uses offline statistics and avoids backpropagation or layer reconstruction in its stated workflow [129]. GPTQ and AWQ therefore make different approximations and impose different preprocessing and kernel expectations; neither name is a guarantee of identical quality across model families, group sizes, or workloads.

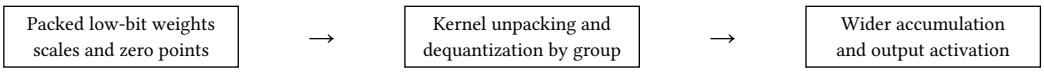

Figure 5: A weight-only kernel commonly combines unpacking, groupwise dequantization, and matrix multiplication. The stored weight representation and the accumulator precision are separate design choices.

25.6 Kernels and End-to-End Trade-offs

The stored bit width is not an end-to-end speed measurement. A low-bit weight may need to be unpacked, multiplied by a scale, and converted into a wider type inside a kernel, as illustrated in Figure 5. Those operations consume instructions and registers. On some devices they can be fused with matrix multiplication and replaced by efficient low-bit tensor operations; on others they can erase a theoretical bandwidth gain. An unsupported group size, layout, or signedness convention may force a fallback path that is slower than the FP16 baseline.

This creates a coupled trade-off. Lowering weight precision reduces M_{quant} in Equation 222 and can allow a larger model or more cache capacity to fit. It can also reduce memory bandwidth per Decode step when the execution is weight-read dominated. But accuracy can fall from rounding, clipping, or accumulated layerwise error; latency can rise from dequantization; and arithmetic throughput depends on actual hardware and kernel support. The reference configuration must therefore state not only "INT4" or "INT8," but also its weight and activation dtypes, quantization axes, group size, scale dtype, packing layout, accumulator type, and kernel backend.

Quantization error rarely distributes uniformly across a Transformer. An attention projection, an embedding or output head, or a feed-forward layer can be more sensitive than its neighbors. Repeated approximate layers can accumulate error, and a model that retains perplexity on a short validation corpus can still fail a long-context, code, or structured-generation workload. Evaluation should compare the quantized model with the exact reference under the intended prompt mixture, context lengths, decoding policy, and output-quality criteria. Averages are useful but do not replace inspection of sensitive tasks or failure tails.

25.6.1 Failure Modes

Table 38 separates common symptoms from their first diagnostic boundary. The remedies are deliberately conditional: replacing every scale or lowering every bit width indiscriminately can obscure the actual cause.

Observed failure	Likely mechanism	First check
Large quality drop in one layer or task	Layer sensitivity, clipping, or a poorly matched group scale	Compare layer outputs and task slices with a wider reference.
Failure on deployment prompts but not calibration prompts	Unrepresentative activation statistics or range selection	Audit calibration mixture, sequence lengths, and prompt formatting.
Erratic results around rare large features	Outlier amplification under shared scales	Inspect channelwise activation ranges and mixed-precision exceptions.
Smaller model footprint but no speedup	Dequantization, unpacking, or unsupported kernel dominates	Profile the selected kernel and verify the intended packed layout is used.
Quality degradation grows with depth	Repeated reconstruction error accumulates across layers	Measure layerwise output error and test a mixed-precision exception policy.

Table 38: Quantization failures arise from representation, calibration, model sensitivity, and execution. A smaller checkpoint alone does not identify which boundary is responsible.

25.7 Implementation Contracts

A quantized checkpoint requires a representation contract. For every quantized tensor or group, it must record code dtype, signedness, symmetry, scale and zero-point dtype, quantization axis, group size, clipping policy, packing order, and the expected accumulator precision. The loader must verify the model architecture and tensor shapes before interpreting packed bytes. A correct INT4 buffer with the wrong group orientation is not a slightly different approximation; it represents different weights. The calibration contract records the source model revision, preprocessing and Chat Template state, sample selection, sequence-length policy, activation collection points, and range or reconstruction objective. Reproducibility requires retaining the calibration seed and statistics, not only the final scale tensors. A deployment contract additionally declares the supported hardware and kernel path, including whether dequantization is fused and which layers intentionally remain at a wider precision. Validation should include a small deterministic set of prompts with reference logits or selected layer outputs, plus an evaluation suite that reflects the intended workload. Check both numerical deviation and task behavior. Measure model memory, allocator-visible memory, and latency separately; report Prefill and Decode behavior separately when they differ. Finally, distinguish a failed quality threshold from a kernel fallback or unsupported format. These are different contract violations and call for different repairs.

25.8 Summary

Quantization maps real tensors to low-bit codes plus metadata and reconstructs an approximation through a scale and, when needed, a zero point. The representation is defined by more than INT8 or INT4: scheme, granularity, calibration, packing, accumulator precision, and execution kernel jointly determine its cost and error. Finer per-channel or group-wise scales can preserve local resolution, while their metadata and layouts increase systems complexity.

Weight-only quantization chiefly reduces persistent model memory and weight bandwidth; weight-and-activation quantization can extend the gain but faces input-dependent activation outliers. PTQ derives a new representation from a completed checkpoint and calibration data, whereas QAT optimizes through a simulated quantization path. LLM.int8(), SmoothQuant, GPTQ, and AWQ illustrate different ways to manage outliers and functional sensitivity rather than one universally optimal

format. A useful inference configuration is therefore one whose memory saving, kernel behavior, quality boundary, and calibration provenance have all been measured under the actual deployment regime.

References

[125] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. doi: 10.1109/CVPR.2018.00286.

[126] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Advances in Neural Information Processing Systems 35*, Curran Associates, Inc., 2022. doi: 10.48550/arXiv.2208.07339.

[127] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 38087–38099. doi: 10.48550/arXiv.2211.10438.

[128] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.17323.

[129] J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proceedings of Machine Learning and Systems*, 2024. doi: 10.48550/arXiv.2306.00978.

Chapter 26: Batching, Scheduling, and LLM Serving Systems

Abstract

An LLM serving system must turn an irregular stream of autoregressive requests into efficient accelerator work while respecting interactive latency and memory limits. This chapter explains why fixed request-level batches waste capacity, develops static, dynamic, and continuous batching, and distinguishes them from microbatching. It then treats Prefill and Decode as competing workloads, derives a KV-Cache-aware admission condition, and examines queueing, fairness, priority, and preemption policies. The result is a systems model for reasoning about throughput, TTFT, TPOT, and resource reclamation without prescribing a specific serving framework.

26.1 Introduction

Chapter 23 defined Prefill, Decode, and the latency quantities used to describe one inference request. Chapters 24 and 25 show that KV Cache state and model representation constrain how many such requests can coexist. A serving system must make those constraints operational. Requests arrive at arbitrary times, contain different prompt lengths, generate an unknown number of output tokens, and occupy different amounts of cache memory as they progress. The accelerator, however, is most efficient when it receives a sufficiently large, regular batch of compatible work.

This mismatch distinguishes LLM serving from a single fixed-shape forward pass. A request cannot usually be dispatched once and forgotten until completion. Each Decode step is another scheduling opportunity, but using that opportunity well requires a scheduler to manage queues, cache capacity, request state, and service objectives together. The chapter studies those decisions. It does not introduce a particular framework, a distributed cluster topology, or Speculative Decoding; those are separate layers built on the scheduling model developed here.

For request i, let a_i denote its arrival time, P_i its prompt length in tokens, and $G_{i(t)}$ the number of tokens generated by time t. Its cached sequence length is

$$S_{i(t)} = P_i + G_{i(t)}. \tag{226}$$

The eventual output length is generally unknown when the request arrives. This uncertainty matters both for queueing and for memory reservation. The notation uses the model dimensions of Chapter 24: L Transformer blocks, g key-value heads, d_h head dimension, and b bytes per cached scalar.

26.2 Why Request-Level Batching Fails

Static batching forms a batch once, starts the requests together, and retains that membership until the batch reaches a completion boundary. This works well for workloads in which every example has similar known work, such as a fixed-shape classifier. It works poorly for autoregressive generation. A request with a short answer finishes while a longer neighbor continues to Decode; its slot may sit idle, and a new request cannot use it until the original batch has been released. A long prompt can also delay the first token of unrelated short prompts if they must enter the same execution unit.

Dynamic batching improves the boundary between batches. The server collects arrivals for a short waiting interval, forms a batch from the queue, executes it, then forms another batch later. It can raise utilization relative to one-request-at-a-time execution and can limit the delay spent waiting for an entire fixed batch to fill. However, once a dynamic batch has begun generation, its membership is still normally fixed. It therefore retains the central length-variability problem: requests finish at different iterations while new arrivals wait outside the active set.

The difference becomes acute for a batch with one request that produces ten tokens and another that produces one thousand. At the tenth Decode iteration, the first request has no useful next-token work left. A fixed batch either carries an inactive slot, compacts only at a coarse boundary, or delays a waiting request that could use the released capacity. ORCA formalized **iteration-level scheduling**:

the server schedules one generation iteration at a time rather than committing to a request-level batch until all its members finish [130].

26.2.1 Continuous Batching and Microbatching

Continuous batching applies that iteration-level view throughout serving. After a Decode step, the scheduler may remove completed requests, admit waiting requests whose Prefill work can begin, and regroup the requests eligible for the next Decode step. A request thus enters and leaves the active set independently of its neighbors. The batch remains a useful accelerator unit, but it is no longer a long-lived ownership boundary.

Microbatching is related but different. A microbatch is a smaller execution unit selected from the work that a scheduler has already chosen to run. It may help bound a Prefill chunk, fit a temporary workspace, or create an execution granularity for Pipeline Parallelism. It does not by itself imply that membership can change after every Decode iteration. Continuous batching is a policy over time; microbatching is a way to subdivide selected work for execution.

Mechanism	When membership changes	Main benefit	Main limitation
Static batching	At a full-batch boundary	Simple regular execution	Completed requests and new arrivals cannot promptly exchange slots
Dynamic batching	When a newly formed batch is dispatched	Better arrival aggregation	Membership stays fixed during a generation
Continuous batching	At Decode-iteration boundaries	Reclaims slots and admits work as requests complete	Requires stateful cache and queue management
Microbatching	Within already selected work	Controls execution granularity and temporary resource use	Does not alone solve request admission or length variability

Table 39: Batching names a grouping decision, but the relevant boundary differs. Continuous batching changes request membership during generation; microbatching changes the size of a dispatched execution unit.

The term **continuous** should not imply zero queueing delay or unbounded admission. An arriving request still waits when the active batch, available cache blocks, or latency policy leave no safe place to run it. The essential change is that a completed request's resources become candidates for reuse at the next iteration, rather than only when an entire historical batch terminates. Modern paged-cache systems make this fine-grained lifecycle practical by allocating and reclaiming cache storage in blocks rather than as fixed maximum-length reservations [131].

26.3 Workload Phases and Queueing

Prefill and Decode have different scheduling shapes. Prefill processes many known prompt tokens and constructs their cache state. Its cost grows with prompt length and benefits from substantial matrix computation. Decode processes one new token per active request at each iteration. It is sequential across output positions and commonly has a stronger dependence on reading model weights and KV Cache state. Chapter 23 develops this execution distinction; here it becomes a source of queue contention.

When the phases share the same accelerators, a scheduler must decide whether a large new Prefill should run ahead of the next Decode step for already streaming requests. Giving Prefill too much priority can improve TTFT for the new request but enlarge TPOT for every active Decode request. Giving Decode strict priority preserves streaming cadence but lets a long queue of prompts accumulate and degrades TTFT. DistServe identifies this prefill-decode interference as a reason to consider phase-

specific resource allocation in some serving architectures [132]. Co-location remains a valid design when its trade-offs match the workload; phase disaggregation is not a universal requirement.

Head-of-line blocking occurs when work at the front of a queue delays other work that could otherwise make useful progress. A very long Prefill submitted under strict FIFO can delay many short interactive prompts. A long generation can hold cache capacity and slow admission even if later requests need little compute. A scheduler reduces such blocking only by adopting a different policy, for example chunking Prefill work, separating queues, or allowing an eligible short request to run first. Each remedy changes fairness or overhead; no policy removes the underlying variability for free.

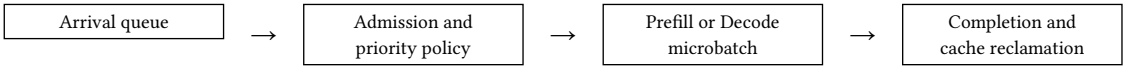

Figure 6: At an iteration boundary, a serving scheduler decides which queued or active requests run next. Completion and cache reclamation feed capacity back into the next decision.

The queue in Figure 6 should represent work state, not merely request IDs. It commonly records arrival time, prompt length, current generated length, stop condition, priority class, predicted or declared output limit, cache-block ownership, and whether the request is awaiting Prefill, active Decode, swapped state, or completion. These fields make policy decisions auditable and permit a cancellation or completion event to release exactly the state it owns.

26.4 Memory-Aware Admission and Reclamation

A candidate request can be computationally cheap enough to run and still be impossible to admit. Its initial Prefill requires cache capacity for P_i tokens, and every later Decode iteration grows its state. Ignoring block rounding, prefix sharing, and metadata, its own cache payload has the Chapter 24 form

$$m_{i(S_i)} = 2Lgd_h bS_i. \tag{227}$$

At scheduling epoch n, let A_n be the proposed active set, M_{weights} the resident model footprint, and $M_{\text{workspace}}$ the temporary memory needed by selected kernels. A conservative one-step feasibility condition is

$$M_{\text{weights}} + \sum_{i \in A_n} m_{i(S_i+1)} + M_{\text{workspace}} \le M_{\text{device}}. \tag{228}$$

Equation 228 does not reserve every request's maximum possible answer. It asks whether the next selected step fits after allowing each active Decode request to append one token. A server may instead reserve a declared output limit, use a statistical length estimate, or admit optimistically and preempt later under pressure. Those alternatives trade utilization against the chance that a live request will later need eviction, swapping, or recomputation.

Admission control is the rule that accepts, defers, sheds, or routes a request before it consumes scarce execution and cache resources. Under high load, accepting every arrival can make all requests slow: queue delays grow, KV Cache capacity becomes fragmented or exhausted, and a late out-of-memory failure wastes earlier work. Deferral keeps the request queued for a future iteration; rejection returns a capacity result promptly; routing selects another compatible replica or service tier. The correct choice depends on the service contract, not only on the local accelerator's instantaneous utilization.

Request completion is the dual operation. A stopping condition, cancellation, error, or timeout releases its private cache blocks and removes its Decode work from the next active set. Shared prefix blocks need reference-aware reclamation, as Chapter 24 explains. Reclamation should occur before the next admission decision so the scheduler can reuse capacity immediately. Delayed or incomplete cleanup produces an apparent memory leak and artificially lowers batch occupancy even when model compute is idle.

26.4.1 Preemption Under Resource Pressure

Preemption temporarily removes a live request from active execution so another request can run. Autoregressive generation permits a natural boundary between tokens: preempting after a Decode iteration preserves a coherent cache prefix. It does not make preemption costless. A server has three broad choices for the interrupted state. It may retain the KV Cache in device memory and simply stop

issuing Decode work; it may swap cache state to another memory tier and later restore it; or it may discard the state and recompute Prefill when the request resumes.

Retaining state has the lowest resume cost but does not free the resource that caused pressure. Swapping can free accelerator memory but introduces transfer delay and bandwidth contention. Recomputing avoids transfer storage but repeats model work proportional to the cached prefix. FastServe explores token-granularity preemption and cache swapping as ways to reduce head-of-line blocking under variable request lengths [133]. The practical policy must account for the request's cache size, expected wait, priority, and the device-to-host transfer path rather than treating all paused requests alike.

26.5 Scheduling Policies and Service Objectives

FIFO serves eligible requests by arrival order. It is easy to explain, makes queue order visible, and can provide a strong fairness baseline. Its weakness is that it does not distinguish a short request from a long one, so a costly front request can create head-of-line blocking. A shortest-job-oriented heuristic tries to favor smaller prompt lengths, output limits, or predicted remaining work. It can improve mean latency when estimates are useful, but generated length is uncertain and an aggressive short-first policy can starve long requests.

Priority scheduling gives selected classes precedence, such as an interactive user over an offline batch job. It is appropriate only when the service contract declares that distinction. Fairness-aware scheduling limits how much service a tenant, request, or priority class can receive over an interval. Aging, virtual service counters, or quotas can prevent a stream of high-priority or short requests from indefinitely delaying another request. These policies are not interchangeable: FIFO emphasizes order, shortest-job heuristics emphasize mean completion time, priority emphasizes declared importance, and fairness emphasizes bounded inequity.

Policy	Primary signal	Potential benefit	Principal risk
FIFO	Arrival time	Transparent baseline and low scheduling overhead	Long requests cause head-of-line blocking
Shortest-job heuristic	Prompt size, output cap, or length prediction	Can reduce mean waiting time	Prediction error and starvation of long work
Priority scheduling	Declared service class or deadline	Protects latency-sensitive work	Low-priority requests can become unbounded
Fairness-aware scheduling	Accumulated service or tenant share	Bounds inequity across request classes	May sacrifice peak local throughput

Table 40: A scheduling policy selects which objective is protected when requests compete. The policy should be stated with its service scope, not inferred from a single throughput number.

Policies also decide how to interleave Prefill and Decode. A system can reserve part of each iteration for new prompts, alternate phase-focused microbatches, cap the number of concurrent Prefills, or use separate queues with an explicit budget. The chosen rule should expose its intent: a maximum queueing delay, a TPOT target, a minimum batch occupancy, or a cost objective. A scheduler that silently optimizes only aggregate tokens per second may deliver excellent offline throughput while producing unacceptable interactive waiting time.

26.6 Metrics and Throughput–Latency Trade-offs

For a request whose first emitted token arrives at t_i^{first} and whose final token arrives at t_i^{last}, the first-token latency is

$$\mathrm{TTFT}_i = t_i^{\mathrm{first}} - a_i. \tag{229}$$

If it emits $G_i^{\mathrm{final}} > 1$ output tokens, a representative average inter-token quantity is

$$\mathrm{TPOT}_i = \frac{t_i^{\mathrm{last}} - t_i^{\mathrm{first}}}{G_i^{\mathrm{final}} - 1}. \tag{230}$$

Queueing delay is the part of TTFT spent awaiting first execution rather than computing Prefill. A system should record it separately, because a high TTFT can arise from overload, a policy choice, or slow prompt computation. Aggregate output throughput can be reported as completed output tokens per second; request throughput as completed requests per second. Neither identifies tail latency, fairness, or how much work was rejected under pressure.

Batch occupancy describes how much of the selected execution capacity is populated by useful request work. GPU utilization measures whether accelerator resources are busy, but a high value can still be spent on poorly prioritized long Prefills or on recomputation after preemption. KV Cache utilization measures used cache payload or allocated blocks relative to cache capacity. It should be reported with fragmentation, reservation, and eviction behavior so that a full allocator is not mistaken for a healthy serving workload.

The central tension is therefore not one-dimensional. Larger batches can improve matrix-kernel efficiency and total throughput, but waiting to form them raises queueing delay. Prefill-heavy batches can improve prompt-processing efficiency, but may disrupt active streaming requests. Strict Decode priority can improve TPOT, but can make TTFT worse. A responsible evaluation reports TTFT and TPOT distributions, queue delay, throughput, batch occupancy, and KV Cache utilization under a declared arrival mixture and service-level objective, rather than choosing one favorable aggregate number.

26.7 Implementation Contracts

The request contract must specify tokenizer and model revision, prompt token sequence, maximum generated length, stopping rule, arrival timestamp, service class, cancellation semantics, and any declared latency objective. The scheduler state must distinguish queued, prefill-active, decode-active, paused, swapped, completed, failed, and cancelled requests. A request must never occupy two incompatible states, and a completion event must be idempotent so that duplicate cancellation signals do not double-free cache blocks.

The memory contract connects scheduler accounting to Chapter 24's cache representation. For every candidate action, the implementation should account separately for model weights, active cache payload, reserved or rounded cache blocks, shared-prefix references, temporary workspaces, and swap buffers. Admission, growth, preemption, and reclamation must be atomic with respect to the allocator: a request cannot be selected for the next Decode step if the blocks needed for that step have already been reassigned. Tests should cover a batch boundary, simultaneous completion and admission, cancellation during queueing, a block-growth boundary, and memory pressure that forces every configured preemption path.

The metric contract records arrival, first-token, each-token, final-token, queueing, preemption, and reclaim timestamps under one clock. It should expose policy name and parameters, batch composition by phase, rejected or deferred requests, cache utilization, and kernel path. Evaluation must compare policies under the same prompt distribution, output-length limits, hardware, model representation, cache policy, and service objective. Otherwise an apparent scheduler improvement may merely be a difference in workload, quantization, or memory budget.

26.8 Summary

LLM serving turns a variable stream of sequential generation requests into accelerator work. Static and ordinary dynamic batching lose capacity because their membership remains fixed while output lengths diverge. Continuous batching instead reschedules at Decode-iteration boundaries, removes completed requests, and admits new eligible work; microbatching is a separate execution-granularity tool.

The scheduler must balance Prefill and Decode, TTFT and TPOT, throughput and fairness, and compute admission against KV Cache capacity. Its decisions are constrained by the persistent memory of each request, not only by its next forward-pass cost. Admission control, resource reclamation, and preemption make those constraints explicit. The next chapter can therefore treat Speculative Decoding

as a change to the work performed per generation step, while this chapter supplies the system that decides which requests may take that step.

References

[130] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "ORCA: A Distributed Serving System for Transformer-Based Generative Models," in *16th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[131] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[132] Y. Zhong *et al.*, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[133] B. Wu *et al.*, "FastServe: Fast Distributed Inference Serving for Large Language Models," *arXiv preprint arXiv:2305.05920*, 2023, doi: 10.48550/arXiv.2305.05920.

Chapter 27: Speculative Decoding and Inference Acceleration

Abstract

Autoregressive Decode exposes only one new target-model token at a time, even when the target model can score a short known continuation in parallel. Speculative Decoding uses a cheaper Draft Model to propose such a continuation, asks the Target Model to verify it in one pass, and commits only the target-authorized prefix. This chapter derives the acceptance correction that makes stochastic speculative sampling distribution preserving, develops a simple expected-speed model, and examines self-speculation, multi-token prediction, cache state, and serving consequences. The aim is to distinguish an exact acceleration technique from an uncontrolled approximation.

27.1 Introduction

Chapter 23 describes ordinary Decode as a feedback loop: the Target Model produces next-token logits, a decoding rule selects one token, and that token becomes the input to the next Target Model step. Even with a KV Cache, a continuation of N tokens normally requires N serial Target Model invocations. Chapter 24 reduces the cache memory that this loop consumes, Chapter 25 reduces the weight footprint, and Chapter 26 decides which requests may take a Decode step. None of those changes removes the token-to-token dependency itself.

Speculative Decoding changes the unit of useful Target Model work. It first lets a cheaper Draft Model propose a short continuation, then evaluates the whole proposed continuation with the Target Model under causal attention. If several proposed tokens agree sufficiently with the Target Model, one Target Model verification pass commits several output tokens. The Target Model remains authoritative: a draft token is never final merely because the Draft Model produced it.

This chapter focuses on draft-and-verify generation. It does not treat general model compression, distributed inference, or every method that predicts multiple candidate tokens. The central question is narrower: how can a system reduce the number of **serial** Target Model Decode iterations without changing the specified Target Model distribution?

27.2 The Sequential Decode Bottleneck

Let $c_t = \left(x, y_{\{<t\}}\right)$ be the prompt x together with the generated prefix before position t. Ordinary stochastic decoding samples

$$y_t \sim p_{\theta(\cdot \mid c_t)}, \tag{231}$$

where p_θ is the Target Model distribution after all declared temperature, truncation, and logit-transform rules have been applied. The selected y_t is required before c_{t+1} is known. Thus, a single request has little token-position parallelism during Decode, even though the Transformer can process the known prompt in parallel during Prefill.

This serial dependency is particularly costly when a Target Model's Decode step is dominated by reading its weights and the current KV Cache rather than by peak arithmetic throughput. A short **known** continuation can often be scored by one causal Target Model pass with less wall-clock cost than performing the same number of independent, sequential Decode iterations. Speculative Decoding exploits that distinction; it does not claim that causality has disappeared [134].

For the remainder of the chapter, let the Draft Model distribution be $q_{\varphi(\cdot \mid c_t)}$, where φ denotes its parameters. The Draft Model may be a separate small model, a reduced execution path through the Target Model, or an auxiliary prediction module. Its implementation varies, but its role is fixed: it proposes likely future tokens cheaply. The Target Model p_θ defines what may be committed.

27.3 Drafting and Parallel Verification

At one speculative iteration, the Draft Model samples or chooses up to γ proposed tokens $\tilde{y}_1, \ldots, \tilde{y}_\gamma$ autoregressively. It produces distributions

$$q_{i(v)} = q_{\varphi}\big(v \mid c_t, \tilde{y}_{\{<i\}}\big) \tag{232}$$

and draws $\tilde{y}_i$ from q_i for $i = 1, \ldots, \gamma$. The notation distinguishes a **proposed token** from a **verified** token: the former is an inexpensive prediction; the latter has passed the target-side acceptance rule.

The Target Model then consumes the known prefix together with the proposed block under its ordinary causal mask. In one verification pass, it computes

$$p_{i(v)} = p_{\theta}\big(v \mid c_t, \tilde{y}_{\{<i\}}\big), \quad i = 1, \ldots, \gamma + 1. \tag{233}$$

The first γ distributions score the proposed positions; the final distribution supplies an additional next-token distribution if every proposal is accepted. The Target Model does not treat the proposals as facts. It evaluates their conditional probabilities exactly as it would evaluate an observed continuation, while causal masking prevents position i from reading later proposals.

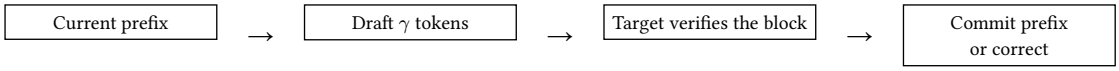

Figure 7: A speculative iteration converts a short draft continuation into Target Model work that can authorize multiple output tokens. The first rejected proposal terminates the candidate prefix.

The phrase **parallel verification** has a precise but limited meaning. The Target Model still respects causal dependencies within the candidate block. It can nevertheless evaluate the logits for all candidate positions in a single block-shaped forward pass, rather than waiting for an external sampling decision between each position. This is the execution opportunity shown in Figure 7.

27.4 Acceptance, Rejection, and Exact Sampling

For stochastic decoding, agreement cannot mean only that the Draft Model's most likely token matches the Target Model's most likely token. The two models generally define different categorical distributions. A naïve rule that commits every draft sample would sample from q_{φ}, not from p_{θ}, and would alter the Target Model's output distribution.

For proposed token $\tilde{y}_i$, exact speculative sampling accepts with probability

$$a_i = \min\left(1, \frac{p_{i(\tilde{y}_i)}}{q_{i(\tilde{y}_i)}}\right). \tag{234}$$

Equivalently, sample $u_i \sim \mathrm{Uniform}(0, 1)$ and accept when $u_i \le a_i$. If $q_{i(\tilde{y}_i)} \le p_{i(\tilde{y}_i)}$, the proposal is always accepted; where the Draft Model places more probability mass on a token than the Target Model does, acceptance corrects the excess. The procedure considers proposals in order and stops at the first rejection.

At a rejection position j, the system samples a replacement from the positive residual distribution

$$r_{j(v)} = \frac{\max\big(0, p_{j(v)} - q_{j(v)}\big)}{\sum_{w \in \mathcal{V}} \max\big(0, p_{j(w)} - q_{j(w)}\big)}. \tag{235}$$

It commits the previously accepted proposals followed by this replacement and discards every later proposal. If all γ proposals are accepted, it instead draws one additional token from $p_{\gamma+1}$. The resulting iteration always commits at least one token and at most $\gamma + 1$ tokens.

The correction in Equation 235 is what makes the method exact with respect to the declared Target Model sampling distribution. Intuitively, acceptance preserves the probability mass where the draft proposal is compatible with the target; residual sampling supplies precisely the target mass that the draft did not supply. This construction, introduced for Transformer generation by Leviathan, Kalman, and Matias and independently developed as speculative sampling for large language models, is distribution preserving up to the numerical behavior of the implementation [134], [135].

27.4.1 Greedy Decoding and Stochastic Sampling

For **greedy** decoding, a simpler deterministic rule is sufficient. The Draft Model proposes its argmax tokens, and the Target Model commits the longest prefix for which each proposal equals the Target Model argmax at that position. At the first mismatch, the Target Model argmax replaces the proposal.

The output therefore matches ordinary greedy Target Model decoding, subject to the same deterministic numerical execution.

For **stochastic** sampling, exactness depends on the acceptance probabilities and residual correction above. In particular, p_i and q_i must use compatible vocabulary identities and the intended sampling transformation. If the service applies temperature, Top-k, Top-p, repetition penalties, or a token mask, it must define whether those operations produce the p_i and q_i used by the acceptance rule. Applying an incompatible transformation only to one side breaks the claimed distributional guarantee.

27.5 Acceptance Rate and Expected Speed

Let A be the number of accepted draft tokens in one speculative iteration, and let $N = A + 1$ be the number of committed tokens, including either a target-side correction or an additional target sample. If each proposal has an approximately independent mean acceptance probability α, then the expected number of committed tokens is

$$\mathrm{E}[N] \approx \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}, \tag{236}$$

with the limiting value $\gamma + 1$ when $\alpha = 1$. This is an explanatory model, not a service-level guarantee. Acceptance probabilities vary by prompt, token position, sampling rule, and request; successive proposals are also correlated. Its value is that it makes the main trade-off visible: a high-quality Draft Model can convert one Target Model verification into several committed tokens, whereas a poor Draft Model commonly produces only the corrective target token.

Let C_{target} be the cost of one ordinary Target Model Decode iteration and let one Draft Model step cost cC_{target}. Under the further idealization that verifying γ positions costs approximately C_{target}, a rough speedup model is

$$S_{\text{ideal}} \approx \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(1 + \gamma c)}. \tag{237}$$

Equation 237 exposes why Draft Model quality alone is insufficient. A larger Draft Model may raise α, but it also raises c. Increasing γ increases the maximum number of tokens a Target Model pass can commit, but also creates more draft work and more candidate suffix that will be discarded after an early rejection. Verification cost itself can grow with candidate length, context length, batching, kernel choice, and cache layout. Real speedup must be measured end to end on the intended workload, not inferred only from acceptance rate.

Quantity	Favorable condition	Why it matters
Acceptance rate α	Draft and Target distributions are close on the workload	More proposed tokens are committed per verification
Draft cost c	Drafting is substantially cheaper than target Decode	Proposal work does not consume the saved latency
Draft length γ	Long enough to exploit accepted prefixes, short enough to avoid waste	Sets both the maximum gain and the rejected-suffix cost
Verification execution	The target can score a short block efficiently	Parallel scoring must be cheaper than equivalent serial Decode steps

Table 41: Speculation pays only when accepted target-authorized work outweighs drafting and verification overhead. The variables interact; optimizing one in isolation can reduce end-to-end speed.

The practical choice of γ is therefore workload dependent. Simple, predictable continuations may support longer drafts; difficult or high-entropy continuations may reject early. Adaptive draft lengths can improve utilization, but they add policy state and need validation under the same latency objectives as Chapter 26. As Table 41 shows, a method with a high mean α can still disappoint if its Draft Model, verification kernel, or batching behavior is expensive.

27.6 Related Drafting Designs

The classical design uses a distinct Draft Model, often smaller than the Target Model but trained with compatible tokenization and formatting. This separation can give a very low c, but it adds model weights, cache state, deployment versioning, and an alignment problem: the Draft Model must remain useful on the Target Model's workload.

Self-Speculative Decoding instead derives a cheap draft path from the Target Model itself. A system may skip selected intermediate layers, exit early, or use a smaller internal predictor while retaining the full path for verification. Draft & Verify is one example: it selectively skips layers to produce draft tokens, then uses the unmodified model to validate the block [136]. Self-speculation can avoid a separately resident Draft Model, but its reduced path must still be cheap enough and accurate enough to justify the additional control flow. It is not exact merely because it shares parameters; exactness comes from target-side verification and correction.

Multi-Token Prediction (MTP) trains or attaches multiple prediction heads so that a shared model representation can propose several future positions. In one formulation, heads predict several offsets rather than only the immediate next token; the candidates may then be verified by a causal Target Model path. MTP can support decoding acceleration, but it is not identical to a small-model speculative scheme: its proposal mechanism is internal and its training objective or heads may be specialized for lookahead. Gloeckle et al. study multi-token prediction as an auxiliary training task, while Medusa uses additional decoding heads and tree-structured verification to create candidate continuations [137], [138]. Any quality-preservation claim still depends on the particular verification rule.

27.7 KV Cache, Batching, and Serving

Speculation introduces provisional state. The Target Model must evaluate the candidate block with the same positional convention and attention semantics as ordinary Decode, which usually creates tentative target-side KV entries for the proposed positions. After verification, the implementation commits only the accepted prefix and the target correction, then discards or rolls back the rejected suffix. A Draft Model has its own cache representation unless it is a self-speculative path with a deliberately shared-compatible representation. Reusing cache tensors across these roles without an explicit ownership contract risks stale keys, invalid positions, or a prefix that no longer matches the committed token sequence.

This state also interacts with the cache-capacity accounting of Chapter 24. A request may temporarily need cache blocks for up to γ speculative positions even though it will ultimately retain fewer. Paged allocation can reduce the cost of such variable-length growth, but it does not remove the need to reserve, reclaim, or roll back blocks correctly. Quantized target weights can change the relative cost of verification and therefore the actual benefit of speculation; Chapter 25's model-weight optimization and speculative acceleration are complementary rather than automatically additive.

Under continuous batching, each active request may commit a different number of tokens after a verification pass. The scheduler in Chapter 26 must account for variable accepted lengths, target verification work, draft work, provisional cache growth, and the next iteration's available capacity. A policy that maximizes accepted tokens for one request can worsen TTFT or TPOT for others if it monopolizes a phase-specific microbatch. The proper evaluation unit is again an arrival mixture with declared latency and fairness objectives, not a single-request speedup.

27.8 Limits and Failure Modes

Speculative Decoding is most useful when ordinary Target Model Decode is expensive, a low-cost proposal mechanism has a reasonable acceptance rate, and the hardware can verify short candidate blocks efficiently. It may be ineffective when the Draft Model is expensive, the continuation is difficult to predict, request batches are already large enough to alter the bottleneck, or another constraint such as long-context cache traffic dominates. The method can increase total arithmetic work even when it reduces serial Target Model iterations; it uses concurrency to exchange some extra candidate computation for lower latency.

Common failures are diagnostic rather than mysterious. A low acceptance rate signals distribution mismatch, an inappropriate sampling transformation, or a draft path too weak for the workload. An excessive γ wastes proposal and verification work after early rejections. A large Draft Model can erase the latency benefit it was intended to create. Poor batching can leave short target verification passes underutilized, while overaggressive batching can damage interactive latency. Additional Draft Model weights, Draft Model KV Cache, provisional target-cache entries, and rollback bookkeeping can turn a nominal algorithmic gain into a memory-capacity regression.

Finally, exactness is conditional. It presumes that the Target Model distribution, tokenizer, context construction, positional indexing, logit transforms, random-number procedure, and residual distribution are implemented consistently. Approximate variants may be useful when a controlled quality trade-off is acceptable, but they should not be described as distribution preserving. Ordinary Decode remains preferable when its simple execution path better meets the workload's latency, memory, reproducibility, or operational constraints.

27.9 Implementation Contracts

The distribution contract must name the Target Model revision, Draft Model or draft path revision, tokenizer, vocabulary mapping, prompt construction, positional convention, temperature, filters, penalties, stop rules, and random-number semantics. Before every acceptance decision, the Target and Draft distributions must refer to the same conditional prefix and the same transformed token support. The implementation must define a safe action when $q_{i(\tilde{y}_i)} = 0$, when a transformed distribution has empty support, or when finite-precision arithmetic makes a probability ratio invalid.

The state contract must distinguish proposed, verified, accepted, rejected, corrected, committed, and discarded tokens. Target-side KV entries for a candidate block must be tagged provisional until the acceptance boundary is known. A rejection must reclaim later provisional entries exactly once, and a cancellation must reclaim both Target and Draft state. Tests should compare short seeded speculative samples against ordinary seeded Target Model samples under the same sampling policy, check that greedy outputs match exactly, and assert that a rejected suffix cannot influence later target logits.

The serving contract must record draft length, accepted-token count, rejection position, target-verification time, drafting time, cache reservation and rollback events, batch composition, and request-level TTFT and TPOT. It should report acceptance-rate distributions rather than only a global mean, because a long or difficult request can have behavior hidden by an aggregate. A deployment decision should compare end-to-end latency, throughput, cache capacity, and quality under the same model representation and scheduler policy as the ordinary Decode baseline.

27.10 Summary

Ordinary autoregressive generation requires one serial Target Model Decode step per output token. Speculative Decoding uses a cheap proposal mechanism to construct a short candidate continuation, then lets the Target Model verify that block in one causal pass. The Target Model remains authoritative: accepted proposals are committed only under its acceptance rule, and the first rejected proposal is replaced by a target-corrected token.

For stochastic generation, acceptance probabilities and residual sampling preserve the Target Model distribution when the implementation uses compatible transformed distributions. The potential gain is governed by accepted tokens per verification, Draft Model cost, draft length, verification efficiency, cache state, and serving policy. Self-speculation and MTP change how candidates are proposed, not the need to state whether target-side verification preserves the intended output behavior. Speculation is therefore a conditional systems optimization, not a replacement for ordinary Decode.

References

[134] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19274–19286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[135] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating Large Language Model Decoding with Speculative Sampling," *arXiv preprint arXiv:2302.01318*, 2023, doi: 10.48550/arXiv.2302.01318.

[136] J. Zhang *et al.*, "Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding," *arXiv preprint arXiv:2309.08168*, 2023, doi: 10.48550/arXiv.2309.08168.

[137] F. Gloeckle, B. Youbi Idrissi, B. Roziere, D. Lopez-Paz, and G. Synnaeve, "Better & Faster Large Language Models via Multi-token Prediction," in *Proceedings of the 41st International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 15706–15734. [Online]. Available: https://proceedings.mlr.press/v235/gloeckle24a.html

[138] T. Cai *et al.*, "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," *arXiv preprint arXiv:2401.10774*, 2024, doi: 10.48550/arXiv.2401.10774.

Chapter 28: Distributed LLM Inference and Parallelism

Abstract

A serving deployment needs more than one accelerator when a model and its runtime state exceed one device's memory, or when one device cannot meet the workload's latency and throughput objectives. This chapter distinguishes replication of complete inference groups from parallel execution of one request, then develops Tensor, Pipeline, Sequence, and Expert Parallelism in the inference critical path. It treats collective communication, Prefill–Decode disaggregation, and multidimensional placement as systems choices whose value depends on memory, topology, workload shape, and service objectives.

28.1 Introduction

The preceding inference chapters considered the execution of a request, the KV Cache that accompanies it, the scheduler that shares a device, and an algorithm that reduces serial Decode iterations. Those optimizations do not remove two fundamental deployment limits. First, model weights, temporary workspaces, and the active KV Cache may not fit in a single accelerator's usable memory. Second, even a model that fits may leave one accelerator unable to satisfy the arrival rate or latency objectives of a production workload. Distributed inference distributes capacity, computation, or both across coordinated accelerators.

The same words used for distributed training appear here, but their operational meaning changes. Chapter 10 derives how ranks synchronize gradients and optimizer updates to preserve one training objective. In serving, parameters are normally fixed; there is no backward pass, optimizer state, or gradient all-reduce. Instead, a request must carry a coherent prefix, positional state, and KV Cache through a sequence of forward computations, often while a scheduler balances many independent requests. The critical question is not whether ranks agree on an update, but whether their placement and communication preserve the logits of the intended model while meeting a latency and capacity target.

Let n_R denote the number of request-serving replicas, n_T the Tensor Parallel degree within one execution group, and n_P its Pipeline Parallel degree. If no other independent dimensions are present, the deployed accelerator count is

$$W = n_R n_T n_P. \tag{238}$$

An **execution group** is the set of ranks that jointly runs one model forward pass under a chosen model-parallel plan. A replica is a complete copy of such a group, not merely one device. An Expert Parallel degree may add another factor in a Mixture-of-Experts (MoE) design, while Sequence Parallelism commonly reuses an existing Tensor Parallel group rather than introducing a wholly independent factor. This accounting is a placement description, not a prescription: the useful degrees are those justified by the model, network, and request distribution.

28.2 Capacity, Throughput, and Inference Semantics

Inference parallelism begins with two distinct questions. **Capacity parallelism** asks how to run one model execution when its weights or request state cannot reside on one accelerator. **Throughput parallelism** asks how many independent request executions can proceed at once. Model-parallel techniques answer the first question by splitting one execution group. Replication answers the second by creating multiple complete execution groups. A practical service commonly needs both: each replica may itself use Tensor or Pipeline Parallelism, while a front-end routes different requests to different replicas.

This distinction prevents a misleading analogy with training. During Data Parallel training, every replica evaluates different examples and later reconciles gradients before advancing the same parameters. A serving replica simply owns a different set of requests. It can decode those requests independently because their prefixes and caches are distinct, and no update must be reconciled after a token is emitted. Inference routing must nevertheless preserve request affinity: after a Prefill, later

Decode steps must reach the ranks that own the corresponding KV state, unless the system explicitly transfers or reconstructs that state.

Inference also exposes phase asymmetry. As Chapter 23 explains, Prefill processes many prompt positions and can exploit comparatively large matrix operations; Decode performs a small incremental step for each generated token. Thus a parallel plan that gives excellent Prefill throughput can still harm interactive Decode latency if it inserts a collective or inter-stage transfer at every token. Pope et al. analyze multidimensional partitioning for Transformer inference under this latency–throughput tension [139]. The relevant unit of evaluation is a realistic mixture of prompt lengths, output lengths, arrival rates, and service-level objectives, not a nominal division of FLOPs by W.

28.3 Tensor Parallelism in the Inference Critical Path

Tensor Parallelism splits the arithmetic of one Transformer layer over n_T cooperating ranks. Its algebra is the same as in Chapter 10, but at inference time the corresponding collective lies directly on the route to the next-token logits. Consider a linear map $Y = XW$, where $X \in \mathbb{R}^{b \times d_{\text{in}}}$ and $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. For a column partition,

$$W = [W^1, ..., W^{n_T}], \quad Y = \text{concat}(XW^1, ..., XW^{n_T}). \tag{239}$$

Each rank owns an output-feature slice and can compute its local product. A following operation may consume the slices directly, or it may require an all-gather to materialize the full feature dimension. For a row partition, split $X = [X^1, ..., X^{n_T}]$ and partition W by its rows. Each rank forms a partial output, and the complete result is

$$Y = \sum_{j=1}^{n_T} X^j W^j. \tag{240}$$

This sum generally requires an all-reduce. A Transformer MLP can arrange its expanding projection as a column-parallel operation and its contracting projection as a row-parallel operation, so that the activation remains local between them. Attention heads can similarly be placed on different ranks before an output projection reconciles the result. This arrangement is the intra-layer model-parallel pattern developed in Megatron-LM [140].

The saving is substantial when one rank could not otherwise hold the layer weights or execute the layer at the needed scale. The cost is recurrent communication. A Decode step visits every Transformer layer, so even a small synchronization repeated at every layer can set the token latency. Tensor Parallel groups are consequently often placed within a high-bandwidth, low-latency locality domain, such as accelerators linked by a fast intra-node interconnect. Crossing a slower network for every layer is possible, but it can turn communication latency into the dominant Decode cost. Group size is therefore a systems decision: a larger n_T lowers local weight memory while increasing collective frequency and potentially making each local matrix multiplication too small to use hardware efficiently.

28.4 Pipeline Parallelism and Request Replication

Pipeline Parallelism partitions model depth. If there are n_P stages, stage j stores a consecutive range of Transformer layers and sends its output activation to stage $j + 1$. During inference, this transfer is principally point-to-point communication: an activation travels forward through the layer stages, then the final stage produces or makes available the next-token logits. A request's Decode loop repeats that stage chain for every emitted token.

Microbatches improve pipeline occupancy when a group is serving many independent request sequences. However, the training bubble formula in Chapter 10 should not be transferred mechanically. Training schedules use both forward and backward waves and can amortize a fixed optimizer-step batch. Interactive inference has no backward wave and often cares about the latency of one request's next token. A single request still traverses every stage serially, so Pipeline Parallelism can solve a capacity problem while adding hop latency. The benefit rises when concurrent Prefill work or many active Decode sequences provide enough independent work to keep stages occupied; the cost rises with imbalanced stages, short batches, and long inter-stage links. GPipe established the layer-stage

abstraction, while large language-model systems combine it with Tensor Parallelism at scale [141], [142].

Replica parallelism is different. A service can make n_R copies of a full execution group, each possibly composed from n_T Tensor Parallel ranks and n_P Pipeline stages. The router assigns a new request to one replica, and that replica retains the request's KV Cache through Decode. Replicas increase aggregate request capacity and can isolate work queues, but they do not make a model fit if each replica is still too small. Conversely, adding more Tensor Parallel ranks to a single group does not create independent request capacity in the same way; the ranks cooperate on the same forward pass and must synchronize.

The following figure records this ownership boundary. It is deliberately logical rather than tied to a serving framework.

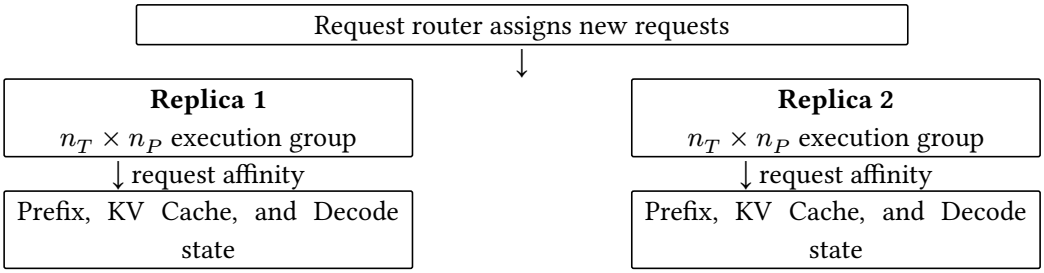

Figure 8: Request replication creates independent complete execution groups. Model parallel ranks within one group cooperate on one request; a request must remain with its cache-owning group unless the service performs an explicit state handoff.

Figure 8 separates **routing** from **execution**. A router can use queue length, expected resource demand, priority, and replica health to choose an entry point, but it must not silently treat a request's KV state as globally available. The scheduling policies in Chapter 26 act within and across these ownership domains.

28.5 Sequence and Expert Parallelism

Sequence Parallelism partitions compatible activation work across ranks, often along the sequence dimension and often in conjunction with a Tensor Parallel group. Its precise implementation varies: some operations retain local token slices, while others all-gather a representation before a layer and reduce-scatter it afterward. For inference, the opportunity depends on the current phase. Prefill has many prompt positions and can expose larger sequence-axis work; a one-token Decode step offers little sequence dimension to divide for a single request. Sequence Parallelism can therefore reduce replicated activation or communication pressure in an appropriate layout, but it is not a universal way to accelerate Decode. The important contract is to specify the shard axis and the gather and scatter boundaries, as Chapter 10 emphasizes.

MoE models introduce **Expert Parallelism**. In an MoE layer, a router maps token representations to a small subset of experts. If experts reside on different ranks, tokens must be dispatched to their selected expert owners, evaluated, and returned to the position that combines their outputs. Let h_i be the representation of token i, let E be the expert set, and let $\mathcal{E}_i \subset E$ be the selected experts. A schematic MoE output is

$$f_{\mathrm{MoE}(h_i)} = \sum_{e \in \mathcal{E}_i} g_{i,e} f_{e(h_i)}, \tag{241}$$

where $g_{i,e}$ is the router's combining weight and f_e is expert e. When different selected experts live on different ranks, the dispatch is commonly an all-to-all exchange: every rank may send a different token subset to every other rank. A second exchange returns the computed outputs. GShard demonstrated conditional expert computation and large-scale automatic sharding, illustrating why routing and placement must be considered together [143].

Expert Parallelism can make large sparse capacity feasible, but it introduces a load-balance problem that ordinary dense Tensor Parallelism does not have. A popular expert may receive too many tokens

while others idle; an uneven request batch can leave the communication fabric busy even when the arithmetic per token is modest. Capacity limits, routing policies, and token distribution therefore affect both quality behavior and serving latency. In an interactive setting, the slowest required expert route can determine when a token is ready.

28.6 Communication on the Serving Critical Path

Distributed inference communicates several different tensor relationships. Naming the operation makes the ownership change explicit and helps expose when a plan can overlap communication with useful work.

Operation	Resulting relationship	Common inference use
All-Reduce	Reduce a tensor across a group and deliver the result to every rank.	Sum row-parallel partial outputs or synchronize a replicated intermediate.
All-Gather	Collect shards so that each rank receives a full logical tensor.	Materialize a feature or parameter view needed by the next operation.
Reduce-Scatter	Reduce a tensor and leave a different shard on each rank.	Return from a replicated intermediate to a sharded activation layout.
Point-to-point	Send one activation or state object from one rank to another.	Transfer activations between Pipeline stages or hand off explicit request state.
All-to-All	Each rank exchanges a distinct payload with every other rank.	Dispatch and return tokens for Expert Parallel MoE computation.

Table 42: Collectives describe which rank owns a result after communication. The appropriate operation follows the tensor partition, not a generic preference for one collective.

For a single communication event with payload q, a minimal cost model is

$$t_{\mathrm{comm}(q)} \approx \alpha + \frac{q}{\mathrm{BW}}, \tag{242}$$

where α summarizes startup and synchronization latency and BW is an effective bandwidth. Collective algorithms, topology, congestion, and message shape change both quantities, so Equation 242 is not a benchmark model. It explains the enduring distinction: frequent small events tend to be latency-sensitive, whereas long KV transfers or large activation exchanges can become bandwidth-sensitive.

For a request, an equally useful decomposition is

$$t_{\mathrm{request}} \approx t_{\mathrm{compute}} + t_{\mathrm{communication}} + t_{\mathrm{synchronization}} + t_{\mathrm{scheduling}}. \tag{243}$$

The terms may overlap, so the expression should not be read as an exact additive accounting. It is a diagnostic model. In particular, Decode repeats a short dependency chain, giving little time to hide a late collective. Prefill can often amortize communication over more tokens and more arithmetic, whereas Decode exposes latency at each output position. The system design should report both phase-specific measurements rather than averaging them into one opaque throughput number.

28.7 Prefill–Decode Disaggregation

Prefill and Decode need not use the same accelerators or the same parallel plan. A **Prefill–Decode disaggregated** service assigns prompt processing to one pool, constructs a request's KV Cache, and then transfers the cache to, or makes it accessible from, a Decode pool. The Decode pool continues the autoregressive loop and owns the active request afterward. The design aims to prevent long prompt computation from interfering with the short, latency-sensitive Decode iterations described in Chapters 23 and 26.

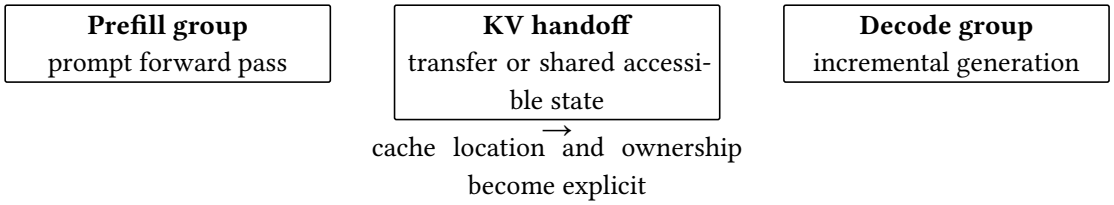

Figure 9: Disaggregation separates the phase that constructs KV state from the phase that repeatedly consumes and extends it. It removes neither cache-placement requirements nor the cost of making the correct state available to Decode.

The benefit is not automatic. Moving a large cache across a constrained link can consume enough bandwidth or delay to erase the queueing benefit. The receiving group must also validate the model revision, tokenizer-derived positions, attention layout, numerical representation, and exact committed prefix before using transferred entries. If cache blocks are shared rather than copied, the service still needs a lifetime, access, and reclamation protocol. DistServe studies the interference caused by colocating the two phases and optimizes their resource allocation and placement separately [144]. Its result motivates phase-aware planning; it does not imply that every workload benefits from a disaggregated design.

Disaggregation can also change parallelism choices. A high-throughput Prefill pool may favor larger batches or a plan that exploits prompt-level parallelism, while a Decode pool may prefer smaller communication groups and a layout tuned for per-token responsiveness. The two pools must agree on a portable cache representation or carry enough metadata to transform it correctly. Incompatible KV head layouts, positional conventions, quantization formats, or partition ownership rules are correctness errors, not merely performance issues.

28.8 Composing Parallelism Dimensions

No dimension is inherently a complete serving strategy. Tensor Parallelism addresses the width and memory of individual layers; Pipeline Parallelism addresses depth placement; Expert Parallelism addresses sparse expert placement; replicas provide request-level capacity. A system may combine them as

$$W = n_R n_T n_P n_E, \tag{244}$$

where n_E is included only when Expert Parallel groups are an independent placement dimension. Sequence Parallelism is omitted from Equation 244 when it reuses the Tensor Parallel group. The formula describes resource ownership, not end-to-end speed: larger products can lower local memory while raising communication, queueing, or underutilization costs.

Observed constraint	Likely first response	Condition to verify
One layer or full model does not fit	Introduce a modest Tensor Parallel group; use Pipeline Parallelism if depth still cannot fit.	Weight and workspace savings exceed added collective and stage-hop latency.
One group fits but request arrival rate is too high	Add complete replicas and route requests with cache affinity.	Each replica has enough queue and cache capacity for its assigned workload.
Sparse experts dominate memory or compute	Place experts across an Expert Parallel group.	Routing balance and all-to-all traffic remain within the latency budget.
Long prompts interfere with interactive Decode	Evaluate phase-specific pools or controlled Prefill–Decode disaggregation.	KV handoff and network cost do not offset reduced phase interference.

Table 43: A parallelism plan should start from a measured resource constraint, then test the new communication and state-management costs under the intended workload.

Table 43 is intentionally conditional. For example, a model that fits on one accelerator may still benefit from replicas before it benefits from model parallelism, because replication avoids per-layer communication. Conversely, a model that cannot fit has no replica-only solution. AlpaServe highlights that model-parallel serving involves a trade-off between its overhead and the opportunity to multiplex bursty workloads across devices [145]. A sound plan maps tightly communicating ranks to favorable topology, keeps request-routing and state ownership explicit, and measures TTFT, TPOT, throughput, queueing delay, and cache utilization together.

28.9 Failure Modes and Scaling Limits

The limiting rank or link on a request's critical path determines the perceived result. Communication bottlenecks arise when collectives are too frequent, payloads are too large, or the chosen group spans an unsuitable network boundary. Pipeline bubbles and unbalanced stages leave devices idle, especially when the active batch is too small to fill the pipeline. MoE routing can create expert hotspots and all-to-all contention. A straggler rank, transient network fault, or collective-order mismatch can delay

every rank in an execution group; the system must distinguish a recoverable slow request from a failed group rather than indefinitely holding cache capacity.

Serving adds failures that are less prominent in training. An admission policy may schedule a request on a computationally available replica whose KV Cache cannot grow to the requested context limit. A handoff can send cache blocks to a Decode group with a different model version or incompatible shard layout. A topology-unaware autoscaler can place Tensor Parallel peers across a slower boundary, changing per-token latency without changing the nominal device count. Insufficient batching can make pipeline or Tensor Parallel communication dominate, while aggressive batching can satisfy throughput measurements but violate interactive latency. These are placement and state-lifecycle failures, not weaknesses in the language-model objective.

28.10 Implementation Contracts

The **placement contract** must declare every execution group, replica, shard axis, Pipeline stage, Expert owner, and communication group. For each tensor that crosses a boundary, it must state the logical shape, layout, collective or point-to-point operation, numerical representation, and post-operation owner. A small deterministic prompt should produce logits matching a single-group reference within the stated numerical tolerance for each supported parallel layout. All ranks in a collective group must execute compatible collectives in the same order; an inference runtime cannot recover correctness by allowing only some ranks to advance.

The **request-state contract** must bind a request identifier to its tokenizer and model revision, committed token prefix, positions, sampling configuration, KV block locations, cache format, and owning execution group. A migration or Prefill–Decode handoff must be explicit, atomic from the request's point of view, and validated before Decode uses the state. Cancellation, completion, preemption, and failure recovery must reclaim every cache allocation exactly once. The contract must state whether state is copied, remotely accessible, or recomputed and how it remains valid while a request is active.

The **observability contract** should report phase-specific TTFT and TPOT, tokens per second, queueing delay, per-group batch occupancy, cache capacity and allocation failures, collective wait time, inter-stage transfer time, expert-load distribution, and retry or restart events. Aggregate throughput cannot diagnose an execution group stalled on one all-reduce or a cache handoff slowed by a network link. Deployment comparisons should preserve model revision, cache representation, request mix, context and output limits, sampling policy, and service objective, so that a claimed parallelism gain refers to the same serving behavior.

28.11 Summary

Distributed inference has two complementary purposes: fitting one request execution across multiple accelerators and serving more independent requests. Tensor Parallelism splits layer computation and introduces frequent collectives; Pipeline Parallelism splits depth and introduces stage-to-stage dependencies; Expert Parallelism routes sparse computation and can require all-to-all exchange. Replicas, by contrast, are complete execution groups that add request capacity while preserving cache affinity. The cost of this placement is communication and state management on the serving critical path. Prefill can often amortize larger operations, while Decode repeatedly exposes synchronization and link latency. Prefill–Decode disaggregation can reduce phase interference only when KV state can be transferred or made available safely and efficiently. A useful deployment therefore chooses its parallel dimensions from measured capacity, topology, cache, and service constraints, then verifies that its placement contracts preserve both the model's logits and the request's state throughout generation.

References

[139] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[140] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019, doi: 10.48550/arXiv.1909.08053.

[141] Y. Huang *et al.*, "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019. doi: 10.48550/arXiv.1811.06965.

[142] D. Narayanan *et al.*, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021. doi: 10.1145/3458817.3476209.

[143] D. Lepikhin *et al.*, "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," *arXiv preprint arXiv:2006.16668*, 2020, doi: 10.48550/arXiv.2006.16668.

[144] Y. Zhong *et al.*, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[145] Z. Li *et al.*, "AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving," in *17th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2023, pp. 663–679. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/li-zhouhan

Chapter 29: Inference System Design and Performance Optimization

Abstract

An LLM serving system is optimized as an end-to-end pipeline, not by accumulating isolated execution tricks. This chapter turns the mechanisms of Chapters 23–28 into a design methodology: characterize the request distribution and service-level objectives, identify the active bottleneck by phase, select one compatible intervention, and validate both performance and model behavior. It relates Roofline-style arithmetic intensity, cache capacity, batching, quantization, speculative decoding, and distributed placement to capacity planning, tail latency, cost per token, and repeatable regression testing.

29.1 Introduction

The previous six chapters established the components of modern LLM serving. A request is Prefilled, assigned persistent KV Cache state, scheduled through incremental Decode, and eventually completed and reclaimed. Its weights can be quantized, its serial Decode loop can be accelerated by speculation, and its computation can span a distributed execution group. None of these mechanisms is an optimization strategy by itself. A modification that improves one measurement can worsen a different measurement that the application actually values.

For example, a larger batch can raise aggregate token throughput while increasing queueing delay and Time to First Token (TTFT). Quantization can reduce weight traffic but add dequantization work, require a different kernel, or exceed a quality tolerance. Speculative Decoding can reduce expensive target-model iterations while paying for drafting, verification, and provisional cache state. More accelerators can increase capacity or make a model fit while introducing communication on every Decode step. The useful question is therefore not "which technique is fastest?" but "which constraint is active for this workload and service objective?"

This closing chapter of the Inference and Serving section develops a measurement-driven answer. It does not rederive attention caching, continuous batching, quantization formats, speculative acceptance, or distributed collectives. Chapters 23–28 remain the sources for those mechanisms. Here they become controlled choices in an end-to-end system design.

29.2 The End-to-End Serving Path

An inference service has an execution path before any Transformer kernel runs. A request arrives with a prompt, generation policy, and often a service class. It can wait in a queue, be admitted only if resources are available, run Prefill, acquire or extend KV Cache blocks, join Decode scheduling iterations, stream tokens, and finally release its state. The following diagram gives the logical path; an implementation may combine or distribute its stages, but each boundary has an observable cost and ownership rule.

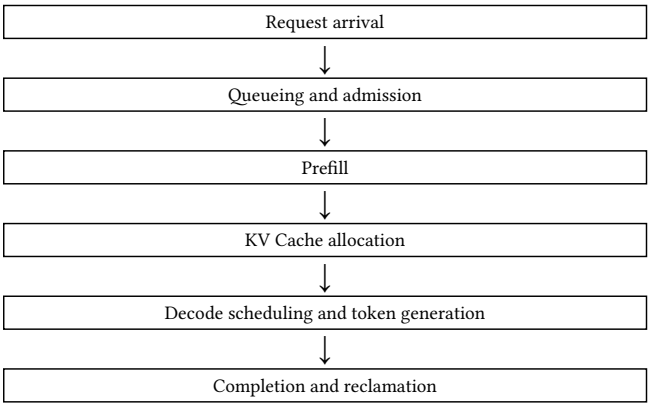

Figure 10: The end-to-end serving path combines request management with model execution. A performance result is interpretable only when it states which of these boundaries are included in its measurements.

Figure 10 separates two kinds of work. Queueing, admission, cache allocation, and reclamation are control-plane or resource-management decisions. Prefill and Decode are model execution phases. Yet the distinction is not a boundary between important and unimportant work: an efficient kernel cannot repair a high TTFT caused by an overloaded queue, and an idle accelerator cannot serve a request whose cache reservation would exceed memory. Chapter 26 develops these scheduler and allocation interactions; the design task is to observe them together.

For request i, let a_i be its arrival time, t_i^{first} the time of its first delivered token, and t_i^{last} the time of its final delivered token. The quantities

$$\text{TTFT}_i = t_i^{\text{first}} - a_i, \quad L_i^{\text{end}} = t_i^{\text{last}} - a_i \tag{245}$$

have different diagnostic value. TTFT contains queueing and prompt processing under the declared measurement boundary. End-to-end latency additionally includes the generated continuation and its repeated Decode steps. TPOT, defined in Chapter 26, describes the intervals after the first token. Reporting only one of these quantities hides where the request spent time.

29.3 Workload and Service Objectives

Optimization begins with the workload rather than an average request. Let S be prompt length, G generated-token count, and A request arrival rate over a measurement window. A workload is characterized by distributions of (S, G, A), not only their means. It also includes concurrency, context limits, sampling controls, model revision, precision, request priorities, cancellation behavior, and the proportion of long-running streams. These variables determine both execution and scheduling: a long prompt changes Prefill work, a long output extends cache lifetime, and a burst of arrivals can make queueing dominate otherwise fast kernels.

The mean can be actively misleading. A small population of requests with very long prompts or continuations may dominate KV Cache occupancy, reserve the scheduler's active slots, and create the p99 latency seen by users. A short offline benchmark can therefore suggest that a configuration is lightly loaded while a production-like length distribution makes it memory-capacity-bound. When requests share fixed system prompts, prefix reuse can change the distribution of allocated cache bytes without changing the distribution of raw prompt tokens; Chapter 24 explains why these are separate facts.

Service-level objectives (SLOs) make such observations actionable. An interactive assistant may constrain p95 TTFT and p95 TPOT. A background workload may tolerate high latency but require high aggregate output throughput. A constrained deployment may prioritize maximum admitted context, cost per output token, or predictable tail behavior. These are not interchangeable objectives. A latency objective specifies a delay target; a throughput objective specifies delivered work per time; a capacity objective specifies the state and concurrency that can be admitted; and a cost objective specifies resources consumed per useful output.

For a random metric Z, its pth percentile is a value z_p such that approximately a fraction p of observations are at most z_p. Thus p50 describes a typical request, while p95 and p99 expose the tail. Percentiles should be paired with the request population, time window, concurrency, and treatment of rejected, cancelled, or timed-out requests. Omitting difficult requests can make a tail metric look excellent while describing a different service.

Objective	Representative metric	Question it answers
Interactive latency	p50, p95, or p99 TTFT and TPOT	How quickly does a user receive the first and subsequent tokens?
Aggregate throughput	Input/output tokens per second; requests per second	How much declared work does the deployment complete under the workload?
Admission capacity	Active requests, context budget, KV Cache utilization	How much state can remain live without allocation failure or unsafe overcommitment?
Cost efficiency	Accelerator time or cost per delivered output token	How efficiently are resources converted into useful output under the SLO?

Table 44: A serving configuration must be evaluated against an explicitly chosen objective. No row is implied by success in another row.

29.4 Bottleneck Classification

A bottleneck is the resource or dependency that limits the declared objective in the measured regime. It is not a permanent property of a model. Prefill and Decode can occupy different regimes for the same request; a small batch and a large batch can occupy different regimes on the same accelerator. The classification below guides investigation rather than replaces profiling.

Compute-bound execution is limited mainly by available arithmetic throughput. Increasing useful matrix-work occupancy, reducing avoidable operations, or changing the arithmetic kernel can help if data movement and scheduling do not become the next limit. Prefill often has relatively high arithmetic intensity because many known prompt positions are processed together, but this tendency depends on prompt lengths, batch construction, model shape, attention implementation, and hardware.

Memory-bandwidth-bound execution is limited by moving weights, activations, or KV state. Decode frequently falls into this regime because it performs modest new-token arithmetic while repeatedly reading large persistent state. Weight quantization can reduce transferred bytes; cache layout or head sharing can change KV traffic; batching can alter reuse and kernel efficiency. None guarantees a speedup if unpacking, dequantization, or another resource becomes limiting.

Memory-capacity-bound execution cannot admit the desired model state, context, or concurrency. Capacity is distinct from bandwidth: an accelerator can have enough bandwidth to process an already-admitted request while lacking enough memory to keep more cache blocks live. Paging, prefix sharing, cache quantization, shorter limits, additional replicas, or a different model layout are possible responses, each with semantic or systems costs. Chapter 24 gives the cache accounting that should precede these choices.

Communication-bound execution spends its critical path waiting for collective operations, inter-stage transfers, or cache handoffs. This regime arises in distributed layouts when Tensor, Pipeline, or Expert Parallel groups cross an unfavorable link or when a small Decode workload cannot hide communication. Adding devices can reduce local memory yet increase token latency. Chapter 28 explains these ownership and topology trade-offs.

Scheduling-bound execution has provisioned arithmetic and memory resources that are poorly converted into useful work because of queueing, fixed batch membership, fragmentation, admission policy, or phase interference. Continuous batching and cache-aware scheduling address this class, but only if the measured delay is actually waiting or underoccupancy rather than a slow model kernel. ORCA's iteration-level scheduling illustrates why autoregressive request lifetimes require a different batching granularity from one-shot inference [146].

29.5 Arithmetic Intensity and Roofline Intuition

A concise way to relate computation and data movement is **arithmetic intensity**. Let F be the number of relevant arithmetic operations and let Q be the number of bytes moved across the limiting memory boundary. Then

$$I = \frac{F}{Q} \tag{246}$$

is the arithmetic intensity in operations per byte. If Π_{peak} is a device's attainable arithmetic ceiling and BW is its attainable bandwidth for the relevant data path, the Roofline-style bound is

$$\Pi \leq \min\left(\Pi_{\text{peak}}, I\ \text{BW}\right). \tag{247}$$

Williams, Waterman, and Patterson introduced this model as a way to expose whether a computation is constrained by arithmetic capability or data movement [147]. Here Equation 247 is an intuition, not a promise of measured performance. Real LLM serving includes multiple memory levels, irregular cache accesses, kernel launch and synchronization costs, communication, queueing, and finite batch effects. Still, it prevents a common category error: a reduction in FLOPs matters most when arithmetic is limiting, whereas a reduction in bytes matters most when bandwidth is limiting.

The contrast between phases follows naturally. Prefill can increase F substantially while also reusing weights and activating efficient matrix operations over many prompt tokens. Decode adds little new-token work per layer but reads weights and an ever-growing history of keys and values. Quantization primarily reduces stored and transferred bytes; batching can change both the effective reuse and the queueing cost; KV Cache management changes persistent state capacity and access patterns. Speculative Decoding changes the number and shape of target-model evaluations rather than simply reducing a fixed count of FLOPs. Distributed execution introduces an additional communication boundary that Equation 247 does not represent. Pope et al. analyze these phase- and partition-dependent inference trade-offs for very large Transformers [148].

29.6 Selecting Compatible Optimization Levers

The mechanisms of Chapters 23–28 should be chosen from an observed bottleneck, not stacked by default. Table 45 summarizes their primary pressure and the condition that can reverse their apparent benefit.

Lever	Primary pressure addressed	Condition that must be measured
KV Cache paging, sharing, or compression	Capacity waste, fragmentation, or long-context cache footprint	Cache traffic, allocation metadata, effective context, and quality remain acceptable.
Weight quantization	Model memory and weight bandwidth	Kernel path, dequantization overhead, and output-quality tolerance justify the lower precision.
Continuous batching	Idle execution capacity from variable request lengths	Extra queueing or phase interference does not violate TTFT or TPOT targets.
Speculative Decoding	Serial target-model Decode iterations	Acceptance, draft cost, verification efficiency, and provisional state produce an end-to-end gain.
Distributed inference	Single-device model capacity or insufficient aggregate service capacity	Communication, topology, load balance, and cache placement do not dominate the critical path.

Table 45: Each serving technique changes a different resource relationship. The primary pressure identifies where to test first, not a guarantee that the technique improves every workload.

KV Cache optimization becomes central when long contexts or concurrent requests exhaust allocator capacity, not merely when the model's weight file is large. Paged allocation and prefix sharing can reduce reservation and duplication waste, while eviction or lossy cache compression changes what information remains available to attention. Weight quantization is most compelling when weights consume scarce memory or repeated weight reads constrain Decode. Chapter 25 explains why a smaller representation must still be evaluated through the actual kernel and quality boundary.

Continuous batching increases useful occupancy across heterogeneous generation lengths, but it cannot change the cost of a particular Decode step. Its central trade-off is service policy: more work

per batch can improve aggregate throughput while delaying newly arrived requests or disrupting active streams. Speculative Decoding is especially relevant when target Decode is a serial, expensive bottleneck and a cheap proposal path is accepted often enough; Chapter 27 emphasizes that acceptance rate alone is insufficient. Distributed inference is justified when a model cannot fit, one group cannot supply the required capacity, or a phase-specific placement is beneficial. Its added devices are not free throughput: collectives, stage transfers, and cache handoffs must appear in the phase-specific latency measurement.

29.7 Profiling, Capacity Planning, and Cost

Profiling locates the active constraint before an optimization is selected. At minimum, a trace should separate queueing delay, admission delay, Prefill execution, Decode execution, kernel time, communication time, batch occupancy, and KV Cache occupancy. Accelerator utilization is useful but incomplete: high utilization can reflect productive work, cache traffic, communication waits, or a batch policy that sacrifices interactive latency. A profile should identify which phase is slow, which resource is saturated, and which requests form the tail.

Capacity planning then combines workload measurements with empirical benchmarks. If requests arrive at rate λ and generate a random G output tokens, the mean offered output-token rate is $\lambda \, \mathrm{E}[G]$. If C_{out} is the measured sustainable output-token rate of one execution group under the same prompt distribution, context limits, sampling policy, and SLO, a necessary average-load condition is

$$\lambda \, \mathrm{E}[G] < n_R C_{\mathrm{out}}, \tag{248}$$

where n_R is the number of independent execution-group replicas. Equation 248 is not a sizing formula. It omits tail lengths, Prefill demand, burstiness, queueing discipline, cache growth, and failover headroom. Its purpose is to show why request rate alone is inadequate: a service whose requests generate twice as many tokens needs roughly twice the output work even at the same arrival rate. A credible plan estimates both prompt and output token distributions, model and workspace memory, cache demand, measured phase throughput, and the desired headroom, then validates the resulting deployment under a representative arrival trace.

Cost efficiency uses the same denominator discipline. If an experiment consumes accelerator-time cost C_{acc} while delivering N_{out} output tokens, then

$$c_{\mathrm{token}} = \frac{C_{\mathrm{acc}}}{N_{\mathrm{out}}} \tag{249}$$

is a conceptual cost per output token. C_{acc} may represent priced accelerator time or an internal resource accounting; it need not assume a particular cloud price. Higher useful utilization can lower Equation 249, but maximizing utilization alone may delay interactive requests past their SLO. Costs should therefore be reported alongside the latency distribution and quality checks that define useful service, not as an isolated throughput contest.

29.8 Regression Testing and the Optimization Workflow

An optimization is only a change relative to a controlled baseline. A regression suite should retain the model checkpoint, tokenizer and prompt construction, precision, hardware topology, cache policy, prompt-length distribution, output-length limits, concurrency, sampling parameters, and scheduling configuration. It should measure p50/p95/p99 TTFT and TPOT, end-to-end latency, throughput, queueing, cache utilization, and relevant resource counters. It should also check output behavior: deterministic requests can compare tokens or logits within a declared numerical tolerance, while stochastic requests need compatible seeds, distributions, or task-level quality evaluation.

The optimization loop is deliberately narrow. First define the workload and SLO; then establish a baseline that meets the same correctness contract. Profile one saturated resource or critical path, choose one intervention whose mechanism matches it, and benchmark again under the same workload. Only then evaluate interactions with a second intervention. This sequence avoids attributing a gain to quantization when a changed batch policy created it, or claiming a speculative gain when the comparison quietly reduced output length. Modern serving systems such as PagedAttention-based

memory management and Prefill–Decode disaggregation exemplify the value of treating capacity, request state, and phase objectives as measured system properties [149], [150].

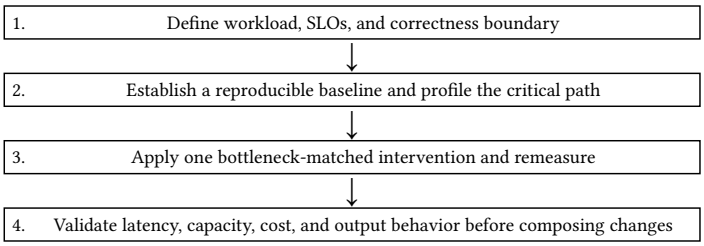

Figure 11: Inference optimization is an experimental loop. The workload and correctness boundary remain fixed while one proposed cause of the measured bottleneck is tested.

Common failures are violations of this loop. Optimizing a synthetic throughput benchmark instead of production-like request lengths can target the wrong phase. Reporting means but not p95 or p99 hides tail queueing and cache effects. Increasing batch size without observing TTFT can exchange one good metric for a failed product. Lower precision does not imply a faster kernel, and more GPUs do not imply lower latency when communication is exposed. Finally, multiple changes can interfere: a quantized target can change speculative verification cost; a cache policy can change the concurrency at which batching becomes useful; a distributed cache handoff can relocate rather than remove a bottleneck.

29.9 Implementation Contracts

The **workload contract** must version the model, tokenizer, prompt format, precision, hardware placement, sampling policy, prompt and output distributions, arrival process, concurrency, context and output limits, service classes, cancellation behavior, and SLO. It must state which events define request arrival, admission, first token, final token, and failure. A benchmark that changes any of these quantities is a new experiment, not a direct comparison.

The **measurement contract** must collect the same timestamps and resource counters under one clock. It should distinguish queueing from Prefill time, Prefill from Decode time, model-weight memory from KV Cache payload and allocator overhead, and compute from communication where the deployment is distributed. Percentiles must include the denominator population and a declared treatment of errors, cancellations, and timeouts. A dashboard that exposes only average tokens per second cannot establish an interactive-latency or cache-capacity claim.

The **correctness contract** must protect behavior while optimizing execution. It records the accepted numerical tolerance, deterministic prompt set or stochastic-evaluation protocol, stop semantics, output-quality threshold, and cache or distribution-preservation claims. It must be possible to associate any performance regression with a fixed checkpoint and runtime configuration, and any quality regression with the representation, kernel, or scheduling change that caused it. These contracts make the final serving configuration diagnosable, reproducible, and safe to evolve.

29.10 Summary

LLM inference is an end-to-end constrained optimization problem. Request distributions, queueing, Prefill, cache allocation, Decode, communication, completion, and reclamation jointly determine the latency, capacity, and cost experienced by a workload. TTFT, TPOT, end-to-end latency, throughput, cache utilization, and p95/p99 tail behavior expose complementary properties; none independently establishes that a system is well designed.

Arithmetic intensity and Roofline-style reasoning help distinguish reductions in arithmetic from reductions in data movement, while memory-capacity, communication, and scheduling analysis supplies the constraints that such a model omits. KV Cache management, quantization, continuous batching, speculative decoding, and distributed inference are therefore conditional levers rather than a fixed stack. The durable methodology is to characterize the workload, define an SLO and correctness boundary, profile the active bottleneck, change one mechanism, remeasure under the same conditions,

and compose only validated gains. This closes the Inference and Serving section with a system-design discipline that remains useful as models, hardware, and serving runtimes change.

References

[146] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "ORCA: A Distributed Serving System for Transformer-Based Generative Models," in *16th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[147] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[148] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[149] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[150] Y. Zhong *et al.*, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2024, pp. 193–210. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

Part VI — Efficient Attention and Long Context

Chapter 30: Efficient Attention and Head-Representation Design

Abstract

Dense attention has two distinct efficiency problems: executing its quadratic interactions without excessive memory traffic, and retaining the Key/Value state that autoregressive decoding repeatedly reads. This chapter separates these concerns. It develops FlashAttention as an exact, IO-aware execution algorithm based on tiling and online Softmax, then compares Multi-Head Attention, Multi-Query Attention, Grouped-Query Attention, and Multi-Head Latent Attention as architectural choices for representing Key/Value information. The resulting distinction clarifies why kernel optimization and KV-state compression are complementary rather than competing interventions.

30.1 Introduction

Chapter 3 derived causal Self-Attention as a tensor program: queries compare with eligible keys, the resulting logits are normalized, and the probabilities aggregate values. That derivation identifies the mathematical function but does not determine how a device should execute it or how a deployed model should represent the state left behind by earlier tokens. Those are separate questions with a common practical consequence: attention can be limited by moving data even when its nominal floating-point operation count appears manageable.

This chapter therefore studies two layers of design. **FlashAttention** changes the execution algorithm for the same dense attention function. It reduces traffic between slow and fast GPU memory by processing the computation in tiles and never materializing the full attention matrix in high-bandwidth memory (HBM). **MHA**, **MQA**, **GQA**, and **MLA**, by contrast, are attention-architecture choices. They determine how many Key/Value (KV) representations exist, or how those representations are compressed, and hence change persistent KV Cache state. A model using GQA can still use FlashAttention; neither choice subsumes the other.

Let B be batch size, T sequence length, L the number of Transformer blocks, H_q the number of Query heads, H_{kv} the number of KV heads, and d_h the head dimension. The symbols follow Chapter 3 except that the more explicit H_q and H_{kv} make head sharing easier to compare. We write b for bytes per stored scalar. Chapter 23 distinguishes Prefill from Decode, Chapter 24 derives KV Cache accounting, and Chapter 29 explains why the active performance bottleneck must be measured rather than inferred from one complexity term alone.

30.2 Attention Cost Is More Than FLOPs

For one head, Scaled Dot-Product Attention first forms scores and then applies those weights to values:

$$S = \frac{QK^T}{\sqrt{d_h}}, \quad P = \operatorname{softmax}(S), \quad O = PV. \tag{250}$$

For a length-T sequence, the two matrix products in Equation 250 require arithmetic proportional to $T^2 d_h$. Across heads, the conventional dense-attention contribution is proportional to $BT^2 H_q d_h$. The all-pairs dependency is mathematical: every legal query-key pair can affect an output. An exact kernel cannot make that dependency linear in T.

Hardware time nevertheless depends on more than arithmetic work. A kernel reads and writes tensors, moves them through a memory hierarchy, launches work, and sometimes synchronizes intermediate results. An implementation can have the same result and nearly the same nominal FLOPs as another while running much faster because it moves fewer bytes over the limiting boundary. This is the IO-aware perspective used by FlashAttention [151].

30.2.1 The Materialization Problem

The following conceptual execution is useful even though framework kernels may fuse some of its steps:

$$Q, K \to S = QK^T \to P = \operatorname{softmax}(S) \to O = PV. \tag{251}$$

If S and P are stored as separate dense tensors, each has roughly BH_qT^2 elements. The device may write scores to HBM, read them to normalize rows, write probabilities, and read probabilities again for the value product. These intermediate transfers can dominate the useful matrix arithmetic at relevant shapes. The issue is not that S is conceptually invalid; it is that a full $T \times T$ representation is a poor execution boundary when it must repeatedly cross a slow memory boundary.

The GPU hierarchy gives the distinction its force. Registers and on-chip SRAM, often exposed partly as shared memory, are small but fast. HBM or global device memory is much larger but slower to access. A high-level model need not depend on particular bandwidth numbers to use this principle: reuse data in fast storage while it is needed, and avoid writing a large intermediate merely because a step-by-step algebraic presentation introduced it.

30.3 FlashAttention: Exact IO-Aware Execution

FlashAttention partitions Q, K, and V into blocks that fit in fast on-chip storage. For one query block, it streams successive Key/Value blocks, computes local scores, updates rowwise normalization statistics and output accumulators, and writes only the completed output block. The computation can be summarized as

$$\text{load a Q tile} \rightarrow \text{stream K/V tiles} \rightarrow \text{update row statistics and output} \rightarrow \text{write O tile} \tag{252}$$

The full probability matrix is not materialized in HBM. The result remains dense attention: every unmasked query-key interaction is still considered. FlashAttention is therefore not an approximate-attention method; up to ordinary floating-point evaluation order, it computes the same mathematical function as Equation 250 while changing the schedule of reads, writes, and reductions [151].

30.3.1 Online Softmax

Tiling introduces a genuine normalization problem. A Softmax row depends on all its logits, but a query tile sees keys only one block at a time. Consider one row whose already processed logits form set A and whose newly processed block forms set C. Define the partial maxima, normalization factors, and unnormalized value accumulators as

$$m_A = \max_{j \in A} s_j, \quad \ell_A = \sum_{j \in A} \exp(s_j - m_A), \quad n_A = \sum_{j \in A} \exp(s_j - m_A) v_j, \tag{253}$$

with analogous quantities m_C, ℓ_C, and n_C for the new block. Let $m = \max(m_A, m_C)$. The combined state is

$$\ell = \exp(m_A - m)\ell_A + \exp(m_C - m)\ell_C, \quad n = \exp(m_A - m)n_A + \exp(m_C - m)n_C, \quad o = \frac{n}{\ell} \tag{254}$$

The merge in Equation 254 is exact in real arithmetic. When a later block has a larger maximum, the earlier partial sum and value accumulator are rescaled to the new stable reference. The final ratio equals the ordinary Softmax-weighted value sum, but neither the global score row nor the global probability row needs to be stored. Max subtraction also controls exponential range, connecting this execution method to Chapter 8′s discussion of numerically stable Softmax.

30.3.2 Memory and Later Improvements

Dense attention's **mathematical interactions** remain quadratic. FlashAttention does not remove the T^2 score relationships or promise that every workload becomes compute-bound. Its key benefit is reducing the additional intermediate-memory footprint and HBM traffic associated with materializing scores and probabilities. Backward computation can recompute local quantities rather than retaining the full attention matrix, trading controlled arithmetic for less activation storage.

FlashAttention-2 retains this IO-aware exactness while improving work partitioning, parallelism, and non-matrix-multiplication overhead [152]. The reusable lesson is not a version-specific kernel recipe. Execution performance depends on how work is divided across available processors and on whether the implementation makes productive use of the memory hierarchy. Shapes, masking form, dtype, dropout, and hardware determine whether a particular kernel path realizes a large gain; a familiar kernel name is not itself a performance result.

30.4 Head-Representation Designs

Chapter 3 introduced attention with a general mapping from Query heads to KV heads. Here the central distinction is restated compactly. MHA assigns independent Key and Value heads to every Query head. MQA keeps many Query heads but shares one Key head and one Value head. GQA shares KV heads within groups of Query heads. These are not three execution algorithms for the same checkpoint: they are architectural configurations of the learned K and V projections.

30.4.1 MHA, MQA, and GQA

For ordinary Multi-Head Attention (MHA),

$$H_{\text{kv}} = H_q. \tag{255}$$

Each Query head has a distinct Key head and Value head. This is the standard multi-head form developed in Chapter 3. Multi-Query Attention (MQA) preserves H_q Query heads but uses one shared Key and one shared Value head:

$$H_{\text{kv}} = 1. \tag{256}$$

The sharing reduces cached state and repeated Decode-time KV reads, which motivated MQA as a faster Transformer decoding design [153]. Its cost is an aggressive restriction: all Query heads must address the same per-token Key/Value representation.

Grouped-Query Attention (GQA) supplies an intermediate choice:

$$1 < H_{\text{kv}} < H_q. \tag{257}$$

Query heads are partitioned into groups, and every group shares one Key head and one Value head. GQA thereby retains more independent KV representations than MQA while reducing them relative to MHA. Ainslie et al. introduce this interpolation and show why it is a useful quality-efficiency design space rather than a binary MHA-versus-MQA decision [154].

30.4.2 KV Cache Consequences

For dense decoder-only attention, the ideal persistent cache payload has the familiar form

$$M_{\text{KV}} \approx 2BLTH_{\text{kv}}d_h b. \tag{258}$$

The factor two accounts for Keys and Values. Decreasing H_{kv} directly decreases cache capacity and cache-bandwidth demand even though H_q can remain fixed. It does not reduce the number of Query rows or eliminate dense query-key interactions. This is why head sharing is particularly consequential during Decode, where each newly generated Query reads a growing history of cached Keys and Values. Chapter 24 gives the broader lifecycle, paging, sharing, and quantization consequences of this equation.

Design	H_{kv}	KV representation	Primary Decode consequence
MHA	H_q	One independent KV head per Query head.	Largest conventional KV Cache and KV-read volume.
MQA	1	One shared KV head for all Query heads.	Minimum head-count cache footprint; strongest sharing constraint.
GQA	$1 < H_{\text{kv}} < H_q$	One KV head per Query-head group.	Intermediate cache and bandwidth trade-off.

Table 46: MHA, MQA, and GQA differ in the learned organization of Key/Value representations. The Query-head count need not change when the KV-head count is reduced.

30.5 Multi-Head Latent Attention

Multi-Head Latent Attention (MLA) targets the same broad Decode bottleneck with a different compression mechanism. Instead of only reducing the number of full KV heads, it stores a lower-dimensional latent representation from which content-related Key and Value information is

reconstructed or used by head-specific projections. MLA was introduced in the DeepSeek-V2 technical report as a KV-compression architecture [155].

For an input hidden state h_t, a schematic latent mapping is

$$c_t = W_c h_t, \quad c_t \in R^{d_c}, \quad d_c \ll 2H_{\text{kv}} d_h. \tag{259}$$

The inequality expresses the design objective, not a universal MLA definition: the latent width d_c is chosen to be much smaller than storing independent full Key and Value vectors. Subsequent projections can derive attention-specific content representations from c_t. A conceptual form is

$$k_t^{\text{content}} = W_K^{\text{up}} c_t, \quad v_t = W_V^{\text{up}} c_t. \tag{260}$$

The constructions in Equation 259 and Equation 260 convey the compression principle without claiming that every MLA implementation has identical factorization, ranks, or kernels. The serving cache can retain the latent state rather than all expanded KV vectors when the attention implementation is designed to use it. That changes both cache representation and the computation that reads or reconstructs it.

30.5.1 Positional Components and RoPE

Content compression interacts delicately with position encoding. Chapter 3 explains that RoPE rotates query and key coordinates so their inner product carries relative displacement. A low-rank latent representation that is convenient for content projections need not preserve the same positional behavior after arbitrary reconstruction. MLA-style architectures can therefore keep a positional Key component separate from compressed content state and combine it with a matching positional Query component during score formation.

Conceptually, one may write

$$q_t = [q_t^{\text{content}}; q_t^{\text{pos}}], \quad k_j = \left[k_j^{\text{content}}; k_j^{\text{pos}}\right], \tag{261}$$

where only the positional components receive the appropriate RoPE transformation. This is an architectural compatibility mechanism, not an instruction to split every Transformer head this way. Its importance is general: a cache-compression design must preserve the positional convention used by the checkpoint, rather than treating position information as an interchangeable post-processing detail.

30.5.2 MLA Compared with Head Sharing

MQA and GQA reduce **how many** independently stored KV heads exist. MLA compresses **what each token stores** into a lower-dimensional latent state. Both can reduce KV Cache capacity and Decode bandwidth, but their quality, projection cost, kernel requirements, and compatibility constraints differ. A comparison should consequently state the representation, cache dtype, sequence length, batch regime, and serving kernels rather than ranking the names in isolation.

Design	Stored per-token state	Main compression mechanism	Important engineering condition
MHA	Full KV vectors for every head.	None beyond ordinary head factorization.	Conventional kernels and largest cache payload.
MQA	One shared full KV pair.	Share KV across all Query heads.	Quality must tolerate maximal sharing.
GQA	One full KV pair per group.	Share KV within Query-head groups.	Group count must match checkpoint configuration.
MLA	Compressed latent KV state, plus required positional state.	Low-dimensional latent representation and reconstruction.	Architecture-specific projections, RoPE handling, and kernel support.

Table 47: Architectural KV-compression strategies. MLA is not merely GQA with fewer heads: it changes the representation retained for each token.

30.6 Kernel Optimization and Architecture Are Complementary

FlashAttention changes neither H_{kv} nor the learned meaning of a Key or Value. It reduces intermediate attention traffic during Prefill and training by executing the same dense operation in a different order. MQA, GQA, and MLA alter the checkpoint's attention representation and reduce persistent KV state or its movement during Decode. The distinctions are summarized separately because a single table that calls one design "more efficient" would confuse different resources.

Execution choice	Mathematical attention function	Primary resource changed	Primary phase
Conventional dense execution	Dense attention	May materialize large score/probability intermediates.	Training and Prefill.
FlashAttention-style execution	Same dense attention	HBM traffic and intermediate-memory footprint.	Training and Prefill; supported Decode shapes.

Table 48: FlashAttention is an execution choice, not a head-sharing architecture. A model with MHA, MQA, GQA, or MLA can require a separate decision about its attention kernel.

During training, FlashAttention can reduce activation-memory pressure and improve attention-kernel utilization, while MQA, GQA, or MLA are fixed architectural choices learned with the model. During inference, FlashAttention may improve Prefill and compatible attention computations; MQA, GQA, and MLA primarily change persistent cache capacity and Decode-time state movement. End-to-end gains can still be limited by projections, model weights, scheduling, communication, or other system costs. Chapter 29's bottleneck-first method is therefore the correct way to evaluate a combined design.

30.7 Failure Modes and Practical Limits

The clean conceptual boundaries above make common errors easier to diagnose. FlashAttention can be unavailable for a dtype, mask layout, head dimension, hardware generation, or runtime path. Very short sequences can leave tile or launch overhead more visible than saved HBM traffic. An incorrect online Softmax merge can cause numerical error or a silent mismatch with a dense reference, especially when masking and reduced precision are involved. It is not enough to report that a kernel was selected; a test must compare its permitted outputs and gradients with an appropriate reference tolerance.

Head-sharing designs have a different risk profile. Treating H_q as H_{kv} misstates cache capacity for MQA and GQA. A checkpoint's head mapping, projection shapes, and positional convention cannot be changed at serving time by merely changing a configuration field. Aggressive KV sharing can reduce model quality, and a nominal cache reduction may not improve latency if weights, scheduler delay, or another operation dominates. MLA adds projection, representation, and positional-encoding complexity; an implementation that expands and stores its full reconstructed KV state would forfeit much of the intended cache benefit.

30.8 Implementation Contracts

An attention-kernel contract must state the score scaling, causal or padding-mask semantics, supported tensor layouts, dtype and accumulation precision, dropout behavior in training, and the numerical tolerance against a reference dense implementation. Tests should include variable sequence lengths, masked rows, boundary tiles, and long rows whose maximum occurs in a later Key block. They should verify both forward outputs and, when training is supported, gradients. Memory reports should separate persistent KV Cache from temporary attention workspace; otherwise FlashAttention's intermediate-memory benefit and a head-design cache benefit will be conflated.

An architectural cache contract must record H_q, H_{kv}, d_h, layer count, cache dtype, the exact model revision, and positional convention. Its expected ideal payload should agree with Equation 258 before allocator rounding and metadata. For MQA and GQA, tests should verify the Query-to-KV head mapping and cache tensor shape. For MLA, they must additionally verify the latent projection, any reconstruction or absorbed-projection algebra, decoupled positional components, and equivalence

to the model's specified attention rule. Benchmarking must state Prefill and Decode lengths, batch concurrency, kernel path, and quality checks; a byte-count reduction alone is not a serving result.

30.9 Summary

Dense attention combines quadratic query-key interactions with an execution problem: a naive schedule can move large score and probability intermediates through HBM. FlashAttention preserves the attention function while using tiles and online Softmax to avoid materializing the full attention matrix in slow memory. It reduces intermediate-memory demand and IO, but it does not make dense attention's all-pairs arithmetic linear.

MHA, MQA, GQA, and MLA address a separate bottleneck: the Key/Value state stored and read across autoregressive steps. MQA and GQA share full KV heads at different granularities, whereas MLA compresses KV information into a latent representation and requires compatible positional handling. FlashAttention can be paired with each of these designs. Chapter 31 then considers the separate architectural choice of computing only a structured subset of attention edges. The useful systems question is consequently not which label is globally best, but whether the measured workload is constrained by attention IO, persistent KV state, computation, or another component of the serving path.

References

[151] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35*, Curran Associates, Inc., 2022, pp. 16344–16359. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract.html

[152] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2307.08691

[153] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: https://arxiv.org/abs/1911.02150

[154] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.

[155] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," *arXiv preprint arXiv:2405.04434*, 2024, doi: 10.48550/arXiv.2405.04434.

Chapter 31: Sparse and Long-Context Attention

Abstract

Long contexts make dense attention expensive because they enlarge both the set of pairwise interactions and the persistent state carried through autoregressive inference. This chapter develops sparse attention as an architectural restriction on which token pairs may interact, distinguishing it from FlashAttention's efficient execution of the same dense connectivity. It examines local, sliding-window, global, block-sparse, strided, dilated, and random patterns; uses Sparse Transformer, Longformer, and BigBird as representative designs; and explains how sparse connectivity, positional extension, KV Cache state, and retrieval jointly determine a model's usable long-context capability.

31.1 Introduction

Dense causal Self-Attention gives each token direct access to every earlier position. This flexible connectivity is one reason decoder-only Transformers can condition generation on a long prefix, but it scales poorly with sequence length. For a sequence of length T, the legal score structure contains on the order of T^2 pairwise interactions. Long contexts consequently pressure attention compute, intermediate memory, KV Cache capacity, memory bandwidth, and inference latency at once.

Chapter 3 develops the dense attention rule and causal mask; Chapter 24 derives the persistent KV Cache cost; and Chapter 30 separates FlashAttention-style execution from KV-representation choices such as MQA, GQA, and MLA. This chapter changes a different object: the **connectivity pattern**. Sparse attention computes only selected query-key pairs. It can substantially reduce the number of interactions, but it may also deny a token direct access to useful distant evidence. The result is an architectural quality-efficiency trade-off, not a free kernel substitution.

Let T be sequence length and let i and j index query and key positions. We use w for a local-window width, g for the number of global positions, and r for a block's token width where needed. The chapter assumes causal decoder-only attention unless a representative encoder-oriented design is explicitly named. Sparse-attention patterns are reusable ideas, but their exact masks, kernels, and training objectives must be stated before their asymptotic claims can be interpreted.

31.2 Dense and Sparse Connectivity

An attention mask can be described by a binary connectivity indicator

$$M_{i,j} \in \{0,1\}, \quad M_{i,j} = 1 \ \text{ if token } \ i \ \text{ may attend to token } \ j. \tag{262}$$

For dense causal attention, $M_{i,j} = 1$ whenever $j \le i$. For sparse attention, only a structured subset of those causal pairs is allowed. The masked score operation still uses the scaling and rowwise Softmax of Chapter 3, but the normalization domain is now the allowed neighborhood:

$$A_{i,j} = \frac{M_{i,j} \exp\big(S_{i,j}\big)}{\sum_{u=1}^{T} M_{i,u} \exp\big(S_{i,u}\big)}. \tag{263}$$

When $M_{i,j} = 0$, the position contributes zero probability. The formula assumes every query has at least one permitted key; causal masks normally retain the diagonal. An implementation may express the same rule through a large negative additive mask or a pattern-aware kernel rather than by explicitly multiplying by M.

This changes the attention graph itself. **FlashAttention** retains dense legal connectivity and improves its execution by tiling and online Softmax. **Sparse attention** omits selected interactions from the model's computation. The two ideas can be combined: a supported kernel can execute a structured sparse pattern efficiently, but an efficient dense kernel does not make a sparse architecture, and a sparse mask does not guarantee a fast implementation.

31.3 Local and Sliding-Window Attention

The simplest sparse pattern gives every token a fixed local history. With causal window width w, a token at position i may attend approximately to

$$\max(1, i - w + 1) \le j \le i. \tag{264}$$

The number of attended keys per query is bounded by w, apart from sequence boundaries. The total number of score interactions is then on the order of Tw rather than T^2, under the assumption that w does not grow with T. Local attention is appropriate when most useful dependencies are nearby, but it creates a direct-access boundary: a token cannot inspect an arbitrary far-away position in one layer.

Sliding-window attention applies such overlapping neighborhoods continuously along a sequence. Neighboring queries share most of their allowed keys, producing a banded attention pattern rather than independent fixed chunks. This avoids artificial disconnections at segment boundaries, but does not restore dense global access. In an autoregressive model, every window must still respect $j \le i$; a symmetric window suitable for a bidirectional encoder is not automatically a valid causal generation mask.

31.3.1 Receptive Field and Information Propagation

The **receptive field** of a representation is the set of input positions that can influence it through the stack. Under a simple causal local window, information can propagate from a position to later positions one local hop per layer. Ignoring nonlinear transformations and boundary effects, after ℓ layers an output can be influenced by positions roughly ℓw steps earlier through repeated relay paths. This is an effective, multi-layer reach, not direct one-layer access.

The distinction matters for long-range reasoning. A deep local stack can carry information across a document, but the signal must pass through intermediate representations and may compete with other information along the route. It is not equivalent to allowing every query to select any prior token directly. Window size, depth, training distribution, and the placement of global connections jointly determine whether this indirect path is useful for a particular dependency.

31.4 Global and Structured Sparse Patterns

Local attention can be augmented with a small set of global positions. A global token may attend broadly, be attended to broadly, or both, according to the declared mask. Special classification positions, task-defined locations, or selected content tokens are possible choices. If most tokens use a width-w local neighborhood and g global positions participate broadly, the interaction count is commonly on the order of

$$Tw + Tg, \tag{265}$$

when w and g are fixed relative to T. The global positions create short paths between distant regions without giving every ordinary token a dense row. Their semantics must be explicit: a global token that reads the whole document but is never visible to other tokens solves a different communication problem from one that is globally bidirectional.

31.4.1 Block, Strided, and Dilated Patterns

Block-sparse attention partitions the conceptual $T \times T$ matrix into $r \times r$ blocks and computes only an allowed subset of blocks. This is often more compatible with accelerator kernels than arbitrary element-wise sparsity because a permitted block contains regular dense matrix work. The realized speedup depends on block size, layout, padding, occupancy, and kernel support; fewer nonzero entries alone do not imply proportional wall-clock improvement.

Other patterns choose long-range edges in a structured way. **Strided attention** permits regularly spaced positions, supplying periodic long-distance paths. **Dilated attention** increases the spacing between reachable positions across heads or layers, expanding reach without making every row dense. **Fixed global positions** devote a known small set of tokens to communication. **Random sparse connections** add irregular graph edges that can shorten paths between distant regions. These choices differ in inductive bias: a regular pattern is easier to reason about and often easier to execute, whereas random or learned-looking connections can improve graph connectivity but complicate reproducibility and kernel design.

31.5 Representative Sparse Architectures

Early Sparse Transformer work factorized the attention matrix into structured patterns to reduce the number of pairwise interactions while retaining paths for long-range information propagation [156]. The important general lesson is not one particular image- or byte-model layout. A sparse pattern must be evaluated as a graph: its edge count controls nominal work, while its path structure controls which information can travel where and how many layers that travel requires.

Longformer is a representative local-global design. Its sliding-window attention gives ordinary tokens local neighborhoods, while task-motivated global attention gives selected positions broad connectivity [157]. This arrangement was designed for long-document processing, where a small number of task-relevant positions can mediate document-wide exchange. Whether a global position should be a special token, a query token, or content selected at runtime depends on the task and changes the model interface.

BigBird combines local, random, and global attention [158]. Local edges preserve nearby structure, global edges supply broad communication, and random edges improve graph connectivity without making the full matrix dense. The original work establishes theoretical properties under its stated pattern and assumptions; those results should not be read as a guarantee that any arbitrary sparse mask preserves dense-attention quality for a particular causal LLM or deployment workload.

Pattern	Allowed connections	Interaction scale	Main limitation
Dense causal	Every previous token.	$O(T^2)$	Largest compute and attention-state pressure.
Local / sliding window	A width-w causal neighborhood.	$O(Tw)$ for fixed w	No direct access to distant tokens.
Local–global	Local neighborhoods plus g broad positions.	$O(Tw + Tg)$ for fixed w, g	Global-token choice becomes part of the task design.
Block-sparse	A declared subset of $r \times r$ blocks.	Pattern-dependent	Kernel efficiency can lag nominal sparsity.
Mixed structured	Local plus strided, dilated, random, or global edges.	Pattern-dependent	Connectivity and implementation are harder to validate.

Table 49: Sparse-attention complexity depends on fixed pattern parameters. The table counts permitted attention interactions, not all runtime costs such as projections, KV state, or kernel overhead.

31.6 Long Context in Decoder-Only Models

Sparse-attention literature includes many bidirectional encoders, but decoder-only generation imposes additional constraints. Every connection must remain causal, Prefill must construct the appropriate cache state, Decode must extend it token by token, and the runtime must support the pattern at the model's actual head dimensions and batch regime. A mask that is useful for long-document classification may need material changes before it can serve an autoregressive language model.

Modern long-context LLMs consequently take several paths. Some retain dense attention while relying on efficient kernels, better positional treatment, GQA or MLA, and KV Cache engineering. Some use sliding-window or other partially structured attention in selected layers. Others combine several mechanisms. Sparse attention is therefore one architectural lever for long context, not a definition of long-context capability. Chapter 30 explains how FlashAttention, MQA, GQA, and MLA address different execution or state bottlenecks that can coexist with a sparse connectivity choice.

31.6.1 KV Cache Does Not Disappear

Reducing attention edges does not automatically remove persistent decoding state. A straightforward decoder implementation may still store per-token Keys and Values for every layer, even if a later query reads only a subset. The ideal dense-cache payload remains proportional to

$$2BLTH_{\mathrm{kv}}d_h b, \tag{266}$$

subject to the actual architecture's cache representation. Sparse connectivity can lower read traffic for a Decode step, but cache capacity may still grow linearly with T. GQA reduces H_{kv}; MLA changes the stored representation; KV quantization reduces effective b; eviction and compression change what historical state remains. Chapter 24 develops those mechanisms. Long-context systems must identify whether their active limitation is score computation, intermediate IO, persistent capacity, cache bandwidth, or a different component rather than assuming that sparse attention resolves all of them.

31.7 Position and Context Extension

Increasing computational context capacity does not guarantee that a checkpoint can use farther positions. Chapter 3 shows how RoPE makes attention scores depend on relative displacement. A model trained chiefly on shorter sequences can nevertheless behave poorly beyond that distribution, even when its mask and memory allocation accept the larger input. It is useful to distinguish three properties: **computational context capacity** is the maximum sequence that the architecture and runtime can process; **positional extrapolation** is behavior at position indices outside the training regime; and **learned long-context capability** is effective use of relevant evidence at those positions. RoPE-based extension methods modify or rescale how position indices enter the rotary transformation. Position interpolation, for example, maps a longer target range into a position range closer to the one seen by the original model and is paired with long-context adaptation in the referenced study [159]. Frequency rescaling and related strategies pursue the same general aim: avoid treating a new positional range as if the original training distribution made it automatically familiar. The specific mechanism, fine-tuning data, and evaluation length are part of the claim; a larger configured maximum alone is not evidence of robust long-context use.

Practical context extension can combine additional long-sequence training, positional scaling, efficient attention kernels, reduced KV-head count, cache management, and structured sparsity. These techniques address different constraints and can interfere. For example, a model may accept a large positional index but run out of cache capacity under concurrency; a sparse pattern may save attention work while long-context training remains insufficient; and a successful kernel benchmark at one length may not predict evidence use at another. No single mechanism defines a usable context window.

31.8 Retrieval and Long Context

Long context and Retrieval-Augmented Generation (RAG) overlap but do not solve the same problem. Long context places more material directly in the model input. RAG selects relevant external units before generation. A large context window can accommodate more source material, but an entire corpus may still be too large, and irrelevant content still consumes Prefill work, context budget, and model attention. Chapter 32 formalizes the retrieval and context-construction boundaries; Chapters 33–39 develop the selection and evaluation machinery.

Conversely, retrieval does not eliminate the value of long context. A selected set of documents may require retaining a long chain of evidence, a large codebase region, or multiple conflicting source passages. The useful system design is often a composition: retrieval controls corpus-scale selection, while the context window and attention architecture determine how much selected evidence can be processed together. Both must be evaluated against the actual evidence distribution and answer task.

31.9 Quality–Efficiency Trade-offs

Dense attention provides flexible direct connectivity but incurs the largest pairwise workload. Sparse patterns reduce permitted interactions by construction, so a relevant distant position can be omitted from a query's neighborhood or be reachable only after several layers. The correct design depends on the task's dependency lengths, the context-length distribution, model depth, training procedure, expected generation regime, and available kernels and hardware. A window that works for predominantly local syntax can be inadequate for a document-level reference, while a broad global pattern can dilute the intended computational saving.

The comparison must also avoid a common systems error: asymptotic interaction count is not latency. Projection layers, cache reads, model weights, block padding, irregular access, kernel launches, batching, and scheduling can dominate a measured path. Sparse attention may be beneficial at lengths where dense attention is untenable yet add overhead at short lengths. It should be compared with a dense baseline under the same model quality target, context length, precision, batch regime, and serving objective.

31.10 Failure Modes

A local window can be too small for a required dependency, and repeated layers may not repair the resulting information bottleneck. Global tokens can be poorly selected, overloaded, or semantically incompatible with a task. Random or structured edges can supply ineffective connectivity for the data distribution, while an apparently sparse pattern can fall back to a slow dense or unsupported kernel. Block padding and shape constraints can erase the expected advantage. These are architecture and systems failures, not merely hyperparameter choices.

Long-context failures cross the same boundaries. Position extrapolation can fail even when the sparse mask admits a long sequence. Long-sequence adaptation data can be unrepresentative. KV Cache capacity can become the active limitation after score computation is reduced. Excessive context can distract the generator, amplify irrelevant material, or give a nominal context limit far beyond the effective usable one. A regression suite should test evidence at varying distances, not only whether the runtime accepts a long input without allocation failure.

31.11 Implementation Contracts

The connectivity contract must define causal direction, every allowed edge family, local-window convention, global-token semantics, block size and indexing, boundary behavior, padding treatment, and the mapping from logical positions to positional indices. Tests should compare a sparse kernel with a dense reference restricted to the same mask, including first and final windows, global-token cases, partially filled blocks, packed sequences, and sequences near the maximum declared length. A dense fallback must preserve the same mask; falling back to unrestricted attention silently changes model behavior.

The long-context contract must record positional-scaling method, position range, tokenizer and serialization rules, KV Cache representation, cache dtype, cache policy, supported kernel paths, and the conditions that trigger fallback. It should separately measure attention compute, temporary workspace, persistent cache, Prefill latency, Decode latency, and quality at several evidence distances. Regression tests must include long-range retrieval of an inserted fact, multi-step dependencies, adversarial irrelevant context, and the expected concurrency regime. These conditions distinguish a claimed configured context length from an auditable capability.

31.12 Summary

Sparse attention changes which token pairs can interact. Local, sliding-window, global, block-sparse, strided, dilated, and random patterns can reduce the number of permitted score interactions below dense $O(T^2)$ attention, but only under declared assumptions about their fixed pattern parameters. They trade direct global access for structured communication paths whose adequacy depends on model depth, training, and the task's dependency graph.

FlashAttention remains a distinct idea: it executes dense attention with lower intermediate IO, while sparse masks alter the attention architecture itself. Neither intervention removes every long-context constraint. KV Cache state can continue to grow with sequence length, positional extrapolation can fail beyond training positions, and a large window can still be overwhelmed by irrelevant information. Effective long-context systems therefore combine only the mechanisms justified by their measured compute, memory, positional, and evidence-selection requirements.

References

[156] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating Long Sequences with Sparse Transformers," *arXiv preprint arXiv:1904.10509*, 2019, doi: 10.48550/arXiv.1904.10509.

[157] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," *arXiv preprint arXiv:2004.05150*, 2020, doi: 10.48550/arXiv.2004.05150.

[158] M. Zaheer *et al.*, "Big Bird: Transformers for Longer Sequences," in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html

[159] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending Context Window of Large Language Models via Positional Interpolation," *arXiv preprint arXiv:2306.15595*, 2023, doi: 10.48550/arXiv.2306.15595.

Part VII — Retrieval-Augmented Generation and Knowledge Augmentation

Chapter 32: Retrieval-Augmented Generation Fundamentals

Abstract

Retrieval-Augmented Generation (RAG) changes the information available to a language model at inference time. An offline pipeline converts an external corpus into retrievable units and an index; an online pipeline ranks units for a query, constructs a bounded context, and asks the language model to generate from that context. This chapter establishes the resulting system vocabulary and separates retrieval success, context-construction quality, and generation quality. It also explains freshness, provenance, context limits, and the boundary between RAG and fine-tuning, providing a foundation for later chapters on retrievers, reranking, and RAG evaluation.

32.1 Introduction

A pretrained language model encodes regularities from its training distribution in parameters θ. This **parametric knowledge** is useful precisely because the model can apply it through ordinary next-token prediction, but it is not an externally addressable database. A model cannot normally expose the source record for an individual learned association, replace one fact without changing parameters, or guarantee that its training snapshot includes a newly published document. Fine-tuning can change the model, but it is an optimization procedure rather than a per-query lookup mechanism.

Retrieval-Augmented Generation (RAG) adds an external knowledge path. Before an answer is generated, a system retrieves text or another structured knowledge unit from a maintained corpus and places selected evidence in the model's input context. The foundational RAG formulation combines a parametric generator with a non-parametric retrievable memory, motivated in part by updateability and provenance on knowledge-intensive tasks [160]. In contemporary systems, the retriever and generator need not be trained jointly; the architectural idea is broader than one particular model family.

This chapter gives the system model that later RAG chapters refine. It distinguishes documents, passages, chunks, retrieval scores, ranking, context construction, and grounded generation. It does not derive BM25, embedding similarity, approximate nearest-neighbor search, reranking, query rewriting, or RAG evaluation protocols. Those are later design choices inside the interfaces defined here. Chapter 1 supplies the tokenizer contract that turns retrieved text into context tokens, Chapter 23 supplies the autoregressive inference model, and Chapter 29 supplies the end-to-end systems perspective needed once retrieval becomes part of a serving path.

32.2 RAG as an External Knowledge System

Let

$$D = \{d_1, \ldots, d_N\} \tag{267}$$

be the retrieval corpus. Each d_i is a retrieval unit, not necessarily one original file: it may be a complete document, a passage, a chunk cut from a larger document, or a structured record rendered as text. A source identifier and metadata connect d_i back to its provenance. Let q be a user query after the system has applied its declared normalization and tokenization conventions.

The RAG architecture has two time scales. **Offline indexing** prepares a corpus before user requests arrive. It parses source material, constructs retrieval units, produces the representations required by a chosen retriever, and stores an index plus metadata. **Online retrieval** receives q, scores or filters units from the prepared corpus, selects candidates, constructs a context, and invokes an LLM. Separating the two paths matters: a change to a source document usually requires reprocessing and index publication, but does not require retraining the base model.

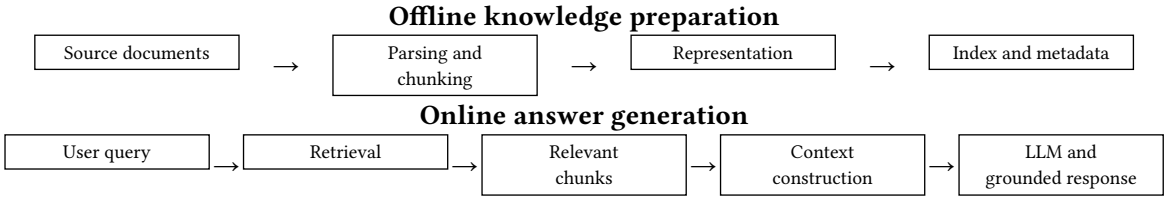

Figure 12: RAG separates corpus preparation from query-time augmentation. The index makes units retrievable; the context builder turns selected units into the bounded model input that the generator actually sees.

The generator remains an autoregressive language model. If $C(q)$ is the context constructed for a query, then a conceptual answer distribution is

$$p_{\theta(y \mid q, C(q))}. \tag{268}$$

Equation 268 does not claim that every generated token is supported by $C(q)$. It makes the intended information path explicit: retrieval changes a conditioning input at inference time, while ordinary generation still depends on the model parameters, the prompt template, and the decoding policy. A system is grounded only to the degree that it retrieves appropriate evidence, presents it intelligibly, and causes the generator to use it.

32.3 Corpus Units and Offline Indexing

The original source corpus and the retrieval corpus should not be treated as identical by default. A source document may contain headings, tables, boilerplate, document-level metadata, and many independently useful claims. Parsing extracts a normalized representation. Chunking then produces retrieval units whose size and boundaries make retrieval and later context use practical. A **passage** often denotes a coherent span; a **chunk** emphasizes a unit selected by an implementation rule. Either can be an element d_i of D in Equation 267.

Each unit should retain at least a stable identifier, its source-document identity, a location or span when meaningful, the text used for representation, and version or freshness metadata. These fields are not secondary bookkeeping. They permit a retrieved chunk to be deduplicated, ordered, filtered, attributed, and invalidated when the underlying source changes. Without them, a system can retrieve text but cannot reliably say which document it came from or whether an answer cites the right revision.

An indexing pipeline produces a searchable representation for every retained unit. The representation may be lexical, sparse, dense, or a combination. Lexical retrieval uses observable terms and their statistics; sparse learned representations retain a high-dimensional feature view; dense retrieval represents queries and units through compact learned vectors; hybrid retrieval combines signals. Information-retrieval literature treats ranking as a distinct problem of matching an information need to corpus items, rather than a property of the generator alone [161]. Dense Passage Retrieval is an example of learning query and passage representations for efficient top-k selection, while sparse systems such as BM25 remain important baselines and components [162], [163].

The choice of representation affects what becomes easy to retrieve, but all variants require a corpus contract. The index must identify the corpus version, parsing rules, retrieval-unit boundaries, representation-model revision when applicable, and the fields that were searchable. An index built before a source deletion, a parser repair, or a policy change may be syntactically valid while operationally stale.

32.4 Online Retrieval and Ranking

At query time, a retriever assigns a relevance score

$$s(q, d_i) \tag{269}$$

to an eligible retrieval unit. The score need not be a calibrated probability, and scores from different retrievers need not share a common numerical scale. Its operational role is ordinal: rank candidates for this query. Let $R_{k(q)}$ be the ordered top-k result set,

$$R_{k(q)} = \mathrm{TopK}_{d_i \in D}\, s(q, d_i) = \left(d_{\pi_1}, ..., d_{\pi_k}\right), \tag{270}$$

where $\pi_1, ..., \pi_k$ order the selected units from higher to lower score under the declared tie rule. Eligibility can include access control, language, recency, document type, tenant, or metadata filters. These filters may prevent an unsuitable unit from reaching the generator even when its unfiltered score is high; they are therefore part of retrieval semantics, not an afterthought.

Retrieval does not imply dense vector search. A lexical system can rank documents by term evidence; a sparse system can score learned sparse features; a dense system can compare learned query and document representations; and a hybrid system can combine or stage those signals. This chapter deliberately treats s as an abstract interface because later chapters must examine how each family represents and searches the corpus. Replacing one scoring method with another can change both the candidate distribution and the kinds of failure a generator encounters.

First-stage retrieval is often asked to preserve plausible evidence among a relatively small candidate set rather than to make a final answer decision. A ranking miss cannot be repaired by a later context builder or generator, because the required evidence was never made available. Conversely, a high-ranked unit can be topically related yet fail to contain the exact fact, scope condition, or source authority needed for a correct response.

32.5 Context Construction and Grounded Generation

Retrieval returns candidate units; it does not yet define the model input. A **context constructor** selects, deduplicates, orders, labels, and serializes candidates for the LLM. With metadata m_i for unit d_i, a conceptual construction is

$$C(q) = \mathrm{compose}\Big(q, \left(d_{\pi_1}, m_{\pi_1}\right), ..., \left(d_{\pi_k}, m_{\pi_k}\right)\Big). \tag{271}$$

The constructor may include document titles, URLs or stable source IDs, dates, section labels, and delimiters that make evidence boundaries visible to the model. Its order can influence attention and generation. Duplicate passages waste scarce input capacity and can make one claim appear more strongly supported than it is. Conflicting passages need their source identity and temporal scope preserved; silently concatenating them does not resolve the conflict.

Context capacity imposes a hard systems boundary. Let τ be the tokenizer from Chapter 1, let $T_{\max}$ be the model's allowed context length, and let N_{out} be a reserved output budget. A request must satisfy a constraint of the form

$$|\tau(C(q))| + N_{\mathrm{out}} \le T_{\max}. \tag{272}$$

The exact accounting also includes any system instructions, role tokens, and output-format markers. This is why retrieving more documents is not automatically better. Additional material can displace stronger evidence, increase Prefill cost, consume context capacity, or distract the generator. A long context window changes the feasible budget but does not remove the need to identify relevant evidence: an entire corpus may still be too large, and irrelevant context still has a cost.

Boundary	Input and output	Necessary success condition
Retrieval	Query $q \longrightarrow$ ranked units $R_{k(q)}$	At least one suitably authoritative and relevant unit survives the candidate boundary.
Context construction	Ranked units $\longrightarrow$ serialized context $C(q)$	Useful evidence fits, remains identifiable, and is not obscured by duplicates, conflicts, or irrelevant material.
Generation	Query and context $\longrightarrow$ answer y	The model follows the evidence, handles uncertainty, and does not invent unsupported claims or citations.

Table 50: RAG quality is compositional. Success at one boundary is necessary but insufficient for success at the next boundary.

Grounded generation is an intended behavioral property, not a consequence automatically supplied by retrieval. A generator can ignore a relevant chunk, combine claims from conflicting sources incorrectly, answer from parametric memory instead of supplied evidence, or manufacture an unsupported citation. Conversely, an accurate answer can be generated after imperfect retrieval if another selected unit contains enough evidence. The provenance shown to a user must therefore be tied to the actual units used in $C(q)$ and evaluated as a claim about support, not merely as a list of nearby links.

32.6 Retrieval Quality, Freshness, and Provenance

Retrieval evaluation begins with a relevance relation rather than with answer fluency. For a query q, let $G(q) \subset D$ be the set of units judged relevant under a declared task definition. A passage-level Recall@k can be written as

$$\text{Recall}\,@k(q) = \frac{|R_{k(q)} \cap G(q)|}{|G(q)|}, \tag{273}$$

when $G(q)$ is nonempty. Precision@k is

$$\text{Precision}\,@k(q) = \frac{|R_{k(q)} \cap G(q)|}{k}. \tag{274}$$

Recall asks how much relevant evidence reaches the candidate set; precision asks what fraction of the returned material is relevant. In tasks with one or a few acceptable evidence passages, a binary Hit@k indicator, $1\left[R_{k(q)} \cap G(q) \neq \varnothing\right]$, is also common. The metrics are useful only with carefully defined relevance judgments: one source can contain an answer string without supporting the required interpretation, and several sources can be relevant but differ in authority or freshness.

Recall is especially important at the first-stage boundary. If $R_{k(q)}$ excludes all useful evidence, downstream ranking or generation cannot recover the omission without another retrieval attempt. Precision also matters because the context budget in Equation 272 is finite. The balance depends on the later pipeline: a broad first stage may favor recall before a later reranker, whereas a direct retrieve-and-generate path may need stronger top-k precision. Ranking quality also depends on position, because a relevant unit at rank one and the same unit at rank k do not have identical chances of surviving a budgeted context constructor.

RAG's freshness advantage follows from separating the corpus from θ. A newly available document can be parsed, chunked, represented, and published in a new index version; it can then become retrievable without a base-model update. This property is valuable, but it is not a guarantee of current or correct answers. The ingestion pipeline can lag, the index can retain an old version, the query can retrieve obsolete evidence, and the generator can still ignore a current source. A responsible system records the source version and retrieval time alongside any freshness claim.

Provenance is similarly an interface, not a decorative citation feature. A response citation should identify a source document and, when possible, the chunk or span that supplied the asserted evidence.

It should not be synthesized from a model's confidence or from a document that was merely retrieved but excluded from $C(q)$. Traceability enables auditing, correction, and source-specific policy enforcement; it does not establish that the cited text entails the answer.

32.7 RAG and Fine-Tuning

RAG and fine-tuning modify different parts of a system. RAG primarily changes the information presented at inference time through $C(q)$ in Equation 268. Fine-tuning primarily changes parameters θ or the behavior learned from a training distribution. A fine-tuned model may follow a domain-specific response format more reliably, use a tool convention, or reason in a desired style even with no retrieved text. A RAG system can expose a newly indexed policy or manual without modifying the base model.

Question	RAG	Fine-tuning
What changes?	Runtime context and access to external units	Model parameters or learned behavior
How is a new source made available?	Update the corpus and publish an index version	Collect data and run an adaptation procedure
What can be inspected at answer time?	Retrieved units, context serialization, and provenance records	Checkpoint and training provenance, not a per-answer source lookup
What does it not guarantee?	Correct use of retrieved evidence or correct citations	Fresh factual coverage or a traceable source for each answer

Table 51: RAG and fine-tuning have different control surfaces. They are often complementary: adaptation can improve how a model consumes evidence, while retrieval supplies current and inspectable task information.

The two techniques are therefore complementary rather than substitutes. An organization can fine-tune a model for instruction following or a specialized output schema while retrieving current documents at runtime. It can also improve retrieval with learned representations without changing the generator. The appropriate boundary depends on update frequency, corpus scale, provenance requirements, latency budget, data governance, and the behavior expected when evidence is absent or conflicting.

32.8 Failure Modes

A **retrieval miss** occurs when the candidate set omits useful evidence. It may arise from incomplete ingestion, an unsuitable retrieval unit, query vocabulary mismatch, an inappropriate filter, or poor ranking. A **context failure** occurs when useful retrieved material is truncated, duplicated, ordered poorly, stripped of its identity, or overwhelmed by conflicting and irrelevant passages. A **generation failure** occurs when the LLM ignores, misreads, overgeneralizes, or contradicts evidence that was available in its context. These boundaries suggest different repairs; asking the generator to be more factual cannot restore a document that the retriever never returned.

Staleness is a cross-cutting failure. An index can contain an obsolete source, a newer source can be absent, or a version change can leave derived representations inconsistent with raw text. Bad chunk boundaries can sever a condition from the statement it qualifies, while duplicate evidence can inflate a weak claim's apparent support. A context overflow can exclude the best unit even after successful ranking. Citation failure is more specific still: an answer can retrieve correct material yet cite an unrelated source, cite a source that does not support the claim, or fabricate a citation entirely.

The practical lesson is not that every RAG system requires every possible safeguard. It is that observability must preserve the boundaries in Table 50. A production trace should make it possible to determine whether a bad answer began with a corpus omission, a retrieval ranking decision, a context-construction decision, or generation conditioned on an already flawed prompt.

32.9 Implementation Contracts

The **corpus contract** must version source identities, access policy, parsing rules, chunk boundaries, unit IDs, metadata schema, deduplication policy, representation inputs, index revision, and publication time. A replacement or deletion must specify whether old units are removed, superseded, or retained with an explicit validity interval. The contract should make a source document's path to one or more d_i units auditable.

The **retrieval contract** must state the query normalization and tokenizer, retriever revision, filters, score semantics, tie rule, candidate count k, and the identifiers and scores returned in $R_{k(q)}$. It should preserve enough trace information to reproduce a result against the same index version. If access control or tenant filters are applied, the system must record them without leaking forbidden corpus content into prompts or logs.

The **context and generation contract** must state the prompt template, source-label format, deduplication rule, ordering policy, context budget, reserved output budget, model revision, decoding policy, and citation-rendering rule. Tests should verify that the serialized context satisfies Equation 272, that each displayed citation maps to a retained source unit, and that a removed or unauthorized unit cannot survive into $C(q)$. Evaluation must separate retrieval coverage, context validity, answer correctness, grounded support, and citation correctness rather than treating a fluent final answer as evidence that every upstream boundary worked.

32.10 Summary

RAG augments a language model with an external information path. An offline pipeline turns source documents into parsed, versioned retrieval units and an index. At query time, a retriever ranks corpus units, a context constructor turns selected evidence into a bounded model input, and an autoregressive generator produces a response conditioned on that input. The score $s(q, d_i)$ and top-k set $R_{k(q)}$ define a retrieval boundary; the constructed context $C(q)$ defines a separate prompt boundary; the answer distribution in Equation 268 defines the generation boundary.

Those boundaries explain both RAG's value and its limits. Updating an external corpus can make information available without retraining the base model, while metadata and source identity can make the answer path inspectable. Neither property guarantees a correct, grounded, or correctly cited answer. High retrieval recall cannot ensure sound context construction, and strong generation cannot use evidence that was never retrieved. Later chapters will develop the retrieval methods, context-selection mechanisms, and evaluation procedures needed to make these interfaces reliable.

References

[160] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020, pp. 9459–9474.

[161] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[162] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.

[163] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

Chapter 33: Embeddings and Semantic Retrieval

Abstract

Dense retrieval maps a query and a retrieval unit into a shared or aligned vector space, then ranks units by a declared similarity function. This chapter develops that abstraction from Dual-Encoder architectures through vector geometry, normalization, and contrastive training. It explains why independently computed document embeddings make semantic search practical, why negative examples determine the learned ranking boundary, and how dimension, domain, chunk granularity, and storage constrain a deployed retriever. The result is a precise interface between the RAG pipeline introduced in Chapter 32 and the vector-search systems developed next.

33.1 Introduction

Chapter 32 treated retrieval as an abstract operation that assigns a score $s(q, d_i)$ to a query q and a retrieval unit d_i. That abstraction deliberately admitted lexical, sparse, dense, and hybrid systems. This chapter makes the **dense** case explicit. Its central motivation is simple: useful evidence need not repeat the words used in a query. A query about an "application crash after a configuration change" may be answered by a passage about "a process terminating after an updated setting," even when their literal term overlap is weak. Conversely, overlapping keywords can identify a passage that discusses the right words but the wrong relationship.

Dense retrieval addresses this gap by learning vector representations whose relative geometry is useful for a retrieval task. It does not make language into a space with objectively correct distances. A similarity score remains a model-dependent ranking signal, and semantic similarity is not identical to factual relevance, source authority, or answerability. The chapter therefore focuses on the interface that a dense retriever provides: what is encoded, how scores are constructed, how the representation is trained, and which assumptions must be preserved when the corpus and index evolve.

The word **embedding** already appeared in Chapter 1, but the objects are different. A token embedding is a trainable row associated with one vocabulary ID inside a language model. A retrieval embedding is usually one learned vector for a larger unit–a query, sentence, passage, chunk, or document–whose role is to support comparison with other units. A retrieval encoder may itself be a Transformer, but its output is an externally stored search representation rather than the decoder's token-by-token residual stream.

33.2 Dense Retrieval as a Ranking Interface

Let the corpus be $D = \{d_1, ..., d_N\}$, using the retrieval-unit convention from Chapter 32. A dense retriever applies an encoder to the query and another encoder to a document or chunk:

$$e_q = f_{q(q)} \in R^r, \quad e_d = f_{d(d)} \in R^r. \tag{275}$$

Here r is the embedding dimension. The encoders f_q and f_d may be identical functions, functions with shared parameters, or separately parameterized networks. They must nevertheless produce vectors in a compatible space. A retrieval score is then

$$s(q, d) = \mathrm{sim}\big(e_q, e_d\big), \tag{276}$$

and the system returns the highest-scoring members of D, subject to the filters and tie rules established in Chapter 32. The score is useful primarily for **ordering**. It is not normally a calibrated probability that a passage is true, sufficient, or safe to place in a prompt.

This differs from lexical matching in what the score can exploit. Lexical systems rely on observable terms and their statistics; dense systems compare learned representations that can align paraphrases, related terminology, or patterns learned from supervision. Neither property is absolute. A dense encoder can miss a rare identifier that lexical matching preserves, while a lexical system can retrieve a precise quotation that a broad semantic representation blurs. This chapter does not derive lexical scoring or hybrid retrieval; the point is only that dense retrieval supplies a different evidence signal, not a replacement for every other signal.

33.3 Dual Encoders and Offline Reuse

A **Dual-Encoder**, also called a **Bi-Encoder**, computes e_q and e_d independently. This independence is the systems property that makes corpus-scale dense retrieval practical. Before requests arrive, the system can encode every retained d_i once, associate its vector with the unit ID and metadata, and build an index. At query time, it computes only e_q and searches among the stored vectors. Dense Passage Retrieval (DPR) made this form particularly influential for open-domain question answering: a question encoder and a passage encoder map their respective inputs into a space scored by an inner product [164].

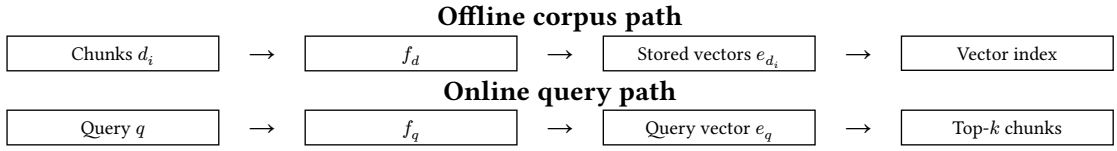

Figure 13: A Dual-Encoder moves corpus work offline. The index stores document-side vectors and identifiers; online retrieval encodes only the new query before ranking compatible stored vectors.

A shared encoder uses one parameter set for both paths, $f_q = f_d$. This is natural for symmetric tasks such as comparing two sentences drawn from similar distributions. Separate encoders allow the model to specialize when queries are short, interrogative, or underspecified while documents are long, declarative, and rich in context. Many search tasks are therefore **asymmetric**: the question "what fixes this?" and a technical explanation of a remediation need not be encoded by exactly the same function. Sharing versus separating parameters is a modeling choice, not a naming convention; the index must record which encoder revision produced every stored vector.

Independent encoding also explains a boundary that later chapters will revisit. A Cross-Encoder may jointly read a query and a candidate passage and can model fine-grained token interactions, but it cannot precompute one candidate representation that is valid for every query. Applying it to every member of a large corpus would discard the offline-reuse property in Figure 13. This chapter does not develop reranking; it establishes why retrieval and later, more expensive comparison stages have different computational roles. Sentence-BERT similarly motivated Siamese encoding so that sentence representations can be compared efficiently rather than requiring a joint model evaluation for each candidate pair [165].

33.4 Embedding Geometry and Similarity

The geometry of the embedding space is meaningful only together with the scoring rule used in training and retrieval. Three common choices illustrate the distinction. The inner product, or Dot Product, is

$$s_{\mathrm{dot}(q,d)} = e_q^T e_d. \tag{277}$$

It depends on both directions and magnitudes. Cosine Similarity divides out the Euclidean lengths:

$$s_{\cos(q,d)} = \cos\big(e_q, e_d\big) = \frac{e_q^T e_d}{\|e_q\|_2 \, \|e_d\|_2}. \tag{278}$$

Euclidean distance instead measures separation,

$$\delta_{\mathrm{L2}(q,d)} = \|e_q - e_d\|_2, \tag{279}$$

and a system ranks smaller distances ahead of larger ones. These forms are related in important special cases but are not interchangeable by default. A model trained to use inner products may intentionally encode useful confidence or frequency information in vector norm. Replacing its score with Cosine Similarity silently changes its ranking function.

L2 normalization transforms a nonzero vector into

$$\hat{e} = \frac{e}{\|e\|_2}, \quad \|\hat{e}\|_2 = 1. \tag{280}$$

For normalized query and document vectors, Dot Product and Cosine Similarity agree:

$$\hat{e}_q^T \hat{e}_d = \cos\big(e_q, e_d\big). \tag{281}$$

Moreover, squared Euclidean distance becomes $\|\hat{e}_q - \hat{e}_d\|_2^2 = 2 - 2\hat{e}_q^T\hat{e}_d$. Thus, on the unit sphere, maximizing Dot Product, maximizing Cosine Similarity, and minimizing squared Euclidean distance give the same ordering. This equivalence is conditional on normalizing both sides and using exact arithmetic. The appropriate similarity function is the one declared by the embedding model's training and serving contract.

Choice	Ranking quantity	What the score retains
Dot Product	$e_q^T e_d$	Direction and vector magnitude.
Cosine Similarity	$e_q^T \frac{e_d}{\|e_q\|_2 \|e_d\|_2}$	Angular alignment after removing magnitude.
Euclidean distance	$\|e_q - e_d\|_2$	Geometric separation; lower is preferred.
Normalized inner product	$\hat{e}_q^T \hat{e}_d$	The same ordering as cosine and squared L2 distance on unit vectors.

Table 52: Similarity functions encode different assumptions. Normalization creates useful equivalences, but it can also remove magnitude information that a particular encoder learned to use.

33.5 Contrastive Training and Negative Examples

A retrieval encoder is not useful merely because it produces a vector. Training must state which query–document relationships should be close and which should be separated. Let (q, d^+) be a positive query–document pair and let $\mathcal{B}(q)$ be a candidate set containing d^+ and negatives d^-. A common contrastive objective is

$$\ell_{\mathrm{ctr}(q,d^+)} = -\log \frac{\exp\left(\frac{s(q,d^+)}{\tau}\right)}{\sum_{d_j \in \mathcal{B}(q)} \exp\left(\frac{s(q,d_j)}{\tau}\right)}, \tag{282}$$

where $\tau > 0$ is a temperature parameter. The loss is small when the positive receives a high score relative to all candidates in the denominator. It is not directly a retrieval metric: it is a differentiable training surrogate whose candidate distribution determines what distinctions the encoder is asked to learn. The scoring function in Equation 282 should match the function used to build the later index, including any normalization convention.

This denominator-based comparison is a retrieval-specific instance of contrastive representation learning: the model must identify the positive association among competing candidates rather than assign an independently meaningful score to one vector alone [166]. The task definition determines which candidates compete and therefore which invariances the resulting space learns.

Negative examples are therefore central rather than incidental. **Random negatives** are often easy: an unrelated passage may already score far below the positive and contribute little gradient. **In-batch negatives** reuse the positive documents paired with other queries in the same batch as negatives for the current query. If a batch contains B aligned pairs, each query can compare against up to $B - 1$ additional document vectors without separately constructing that many examples. This makes larger batches valuable for contrastive retrieval, but it also makes the effective negative set and its sampling policy part of the objective. DPR uses in-batch comparisons and studies both random and BM25-derived hard negatives in its retrieval training procedure [164].

A **hard negative** looks plausible under a lexical or preliminary semantic signal but is not the desired evidence for the query. It can force the model to distinguish near misses that random negatives ignore. Yet hard-negative mining is not infallible. A passage marked negative may be a second valid answer, a useful complementary source, or a different chunk from the same document that supports the query. Such **false negatives** teach the encoder to separate evidence that the application later needs together. Retrieval labels, corpus versioning, and the definition of relevance must consequently be reviewed together rather than treated as independent dataset fields.

33.6 Semantic Retrieval Pipelines and Storage

The dense pipeline specializes the offline and online paths from Chapter 32. Offline processing parses source documents, creates declared retrieval units, applies f_d, stores one vector per retained unit, and

builds a search structure. At query time, the service applies the compatible query tokenizer and f_q, searches the stored vectors by the declared score, and returns the top-k candidate IDs with their scores and metadata. Chapter 34 will examine how a vector-search system can avoid scanning every vector exactly; that systems choice does not change the dense-retrieval interface in Equation 276.

The base storage cost is already instructive. For N stored vectors, dimension r, and b bytes per component, the payload is approximately

$$M_{\text{emb}} \approx Nrb. \tag{283}$$

This excludes IDs, metadata, alignment, replicas, and index overhead. For example, increasing dimension or retaining more chunks increases both memory capacity and the bytes that later similarity kernels must read. More dimensions can offer a model more representational capacity, but they do not guarantee better retrieval; the gain depends on training data, architecture, objective, and the task's required distinctions. Dimension should be selected with quality, storage, bandwidth, and index behavior measured together.

Batch embedding makes offline encoding efficient by applying f_d to many units at once, but it must preserve the unit identity attached to every output row. Long documents make this identity decision consequential. One vector for an entire report can blur a precise claim into a broad average, while smaller passages can expose local evidence to the retriever. The right chunk boundary is a separate design problem, but the representation contract must name which chunk text, title, and metadata fields were encoded. A vector produced from a stale source revision is not repaired merely because its ID still exists.

33.7 Domain Adaptation and Failure Modes

Embedding quality depends on the distribution that shaped f_q and f_d. A model trained largely on general web question answering may not distinguish the terminology, citation conventions, or relevance criteria of scientific literature, source code, legal documents, or multilingual corpora. Domain adaptation can change the encoder or its training pairs so that positive and difficult-negative relationships reflect the intended corpus. It should be evaluated against held-out queries and source revisions, not inferred from an attractive visualization of a few vectors.

Several failures follow directly from the abstraction. A dense retriever can return a passage that is semantically similar but factually wrong, outdated, or unauthorized; similarity does not establish truth or provenance. An ambiguous query can have multiple valid intents, but one vector may collapse them into a single ranking. Rare entities, identifiers, negation, numerical conditions, and cross-language phrasing can be weakly represented. Poor negatives can produce a shallow ranking boundary, while false negatives can actively suppress useful evidence. A model may also exhibit representation collapse, where many inputs become insufficiently distinguishable, or lose fine-grained detail when a long source is compressed into one vector.

Source changes create a final operational failure mode. Updating raw text without recomputing affected embeddings leaves the vector index semantically stale. Changing the encoder is broader still: query vectors and stored document vectors must be generated by compatible revisions. Mixing incompatible spaces can yield numerical scores while destroying their intended ranking semantics. A published index should therefore identify its corpus revision, chunking policy, document encoder, query encoder, dimension, normalization rule, and score convention.

33.8 Implementation Contracts

The **encoder contract** must name the query and document encoder revisions, tokenizer and text-normalization rules, maximum input lengths, pooling or readout method, embedding dimension r, output dtype, and normalization policy. If weights are shared, that fact should be explicit; if they are separate, their compatibility must be tested. Given a fixed input and revision, the implementation should produce an embedding with the declared shape and finite components.

The **training contract** must define the positive-pair provenance, candidate set in Equation 282, negative-sampling and hard-negative-mining policies, temperature τ, score function, batch formation,

optimizer configuration, and evaluation split. It must state how likely false negatives are detected, filtered, or analyzed. Reporting only one contrastive loss cannot establish retrieval quality; recall-oriented retrieval metrics and error slices must be evaluated on a protected query set.

The **index contract** must bind every vector to a retrieval-unit ID, source and chunk revision, document-encoder revision, dimension, dtype, normalization state, score convention, metadata, and index publication revision. A query service must reject or rebuild incompatible vector sets rather than silently comparing different dimensions or encoder spaces. Tests should verify that offline and online preprocessing agree, L2-normalized vectors satisfy the stated tolerance when required, score ordering matches a high-precision reference on a small corpus, and an updated or deleted source cannot return an obsolete vector after its index revision is published.

33.9 Summary

Dense retrieval represents a query and a retrieval unit by compatible vectors, then ranks units through a declared similarity function. Dual Encoders make the corpus side reusable: document vectors can be computed offline, while a new query requires only one online encoding and a vector search. The geometry matters only with the model's score convention. L2 normalization makes Dot Product, Cosine Similarity, and squared Euclidean distance equivalent for ranking on the unit sphere, but it can remove magnitude information that an unnormalized model uses.

Contrastive learning turns relevance pairs and negative examples into a ranking boundary. In-batch and hard negatives can make that boundary more informative, while false negatives, domain mismatch, and stale vectors can undermine it. The next chapter takes the stored-vector interface as given and studies how vector-search systems retrieve high-scoring candidates efficiently at corpus scale.

References

[164] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.

[165] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, Association for Computational Linguistics, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[166] A. van den Oord, Y. Li, and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," *arXiv preprint arXiv:1807.03748*, 2018, [Online]. Available: https://arxiv.org/abs/1807.03748

Chapter 34: Vector Search and Approximate Nearest Neighbors

Abstract

A dense retriever is useful only if its stored vectors can be searched within the latency and memory budget of a RAG system. This chapter starts from exact vector ranking, then develops Approximate Nearest Neighbor (ANN) indexing as a controlled reduction in candidate-search work. It explains Inverted File indexing, Product Quantization, and Hierarchical Navigable Small World graphs; separates ANN recall from retrieval relevance; and traces the consequences of index construction, compression, filters, hardware, and corpus mutation. The central lesson is that an ANN index changes candidate generation, not the meaning of the embedding model or the correctness of the final answer.

34.1 Introduction

Chapter 33 defined a dense retriever by a query embedding e_q and a collection of compatible document embeddings. Once those representations exist, retrieval must answer a systems question: which stored vectors are closest to e_q under the model's declared score, and how quickly can the answer be produced? For a small collection, the direct answer is straightforward. Score every vector, rank the results, and return the best k. At corpus scale, reading and comparing every vector can dominate retrieval latency and memory bandwidth.

Vector search provides data structures and search procedures for this problem. It does not replace the query encoder, document encoder, similarity function, or relevance definition from Chapter 33. Instead, it changes how the candidate set is found. This distinction is essential for diagnosis. A weak answer may originate in an embedding model that placed the right document far away, an approximate index that failed to return a nearby vector, a metadata filter that excluded an eligible item, or a later ranking and generation stage. Those failures require different repairs.

This chapter studies vector-only candidate generation. It does not derive chunking, lexical retrieval, hybrid search, Cross-Encoder reranking, query rewriting, or RAG evaluation. These later choices consume the candidate interface established here.

34.2 Exact Nearest-Neighbor Search

Let $E = \{e_1, ..., e_N\}$ be the stored embedding vectors, each in R^r, and let $s\big(e_q, e_i\big)$ be a score for which larger values are better. The exact top-k set is

$$E_k^{\text{exact}(q)} = \text{TopK}_{e_i \in E}\, s\big(e_q, e_i\big). \tag{284}$$

Brute-force search computes $s\big(e_q, e_i\big)$ for every i, selects the best results, and returns their associated retrieval-unit IDs. For an inner product, the score work is on the order of Nr scalar multiply-add operations per query, followed by top-k selection. The linear scan is exact with respect to the declared vectors, metric, filters, and arithmetic convention. It is therefore the right baseline for measuring index behavior.

Exact does not mean universally expensive. A small corpus can be scanned directly; batches of queries can make dense matrix multiplication highly efficient; and a GPU can evaluate many scores in parallel when the vectors fit the relevant memory hierarchy. Exact search is also useful for offline evaluation and index regression tests. Its limitation is scaling: at fixed dimension, every additional vector adds work and memory traffic to every query. A system that must search a large corpus one request at a time often needs to avoid inspecting most of E.

34.3 Approximate Nearest Neighbors and Recall

Approximate Nearest Neighbor (ANN) search uses an index to examine a selected subset or an approximate representation of the corpus. The goal is not to change the semantic score definition, but to find high-scoring candidates without exhaustively evaluating every stored vector. The trade-off is

explicit: reduced candidate-search work may cause the index to miss an item that exact search would have returned.

For a query with an exact vector-neighbor set $E_k^{\text{exact}(q)}$ and an approximate result set $E_k^{\text{ann}(q)}$, a common index metric is ANN Recall@k:

$$\text{Recall}_{\text{ANN}}@k(q) = \frac{|E_k^{\text{ann}(q)} \cap E_k^{\text{exact}(q)}|}{k}. \tag{285}$$

This metric compares an index with exhaustive vector search under the same embedding space. It is not the same as retrieval-task Recall@k from Chapter 32, which asks whether relevant evidence appears in the result set. An ANN index can have high recall against exact neighbors even when the embedding model retrieves irrelevant passages. Conversely, a lucky approximate miss can occasionally surface a task-relevant item that exact vector ranking would not have chosen. In production, both index recall and task relevance matter; neither substitutes for the other.

ANN methods expose search controls that trade query latency against candidate coverage. More work at query time usually improves ANN recall, but consumes compute, bandwidth, and sometimes queueing capacity. Some indexes also trade greater build time or memory for faster search. There is no configuration that is optimal independently of the corpus, score function, update rate, hardware, workload concurrency, and required quality floor.

34.4 Inverted File Indexes

An **Inverted File** (IVF) index partitions the vector space into coarse regions. Let $c_1, ..., c_M$ be coarse centroids learned from representative vectors. Each stored vector receives an assignment

$$a(i) = \arg\min_{m \in \{1,...,M\}} \delta(e_i, c_m), \quad L_m = \{i : a(i) = m\}, \tag{286}$$

where δ is the coarse distance convention and L_m is the inverted list for centroid c_m. The phrase **inverted file** refers here to lists of vector IDs associated with coarse cells; it is not the term-posting structure used by a lexical search engine.

At query time, the system first ranks the centroids, selects a small set $P_{p(q)}$ of p promising cells, and scores only vectors in the corresponding lists:

$$C_{\text{IVF}(q)} = \cup_{m \in P_{p(q)}} L_m. \tag{287}$$

It then ranks $C_{\text{IVF}(q)}$ by the original or an approximate vector score. The number of probed regions, represented by p, is a key search control. Probing more clusters enlarges the candidate set, tending to improve ANN recall while increasing query work. Probing too few clusters can miss the correct neighborhood when a query lies near a coarse boundary or the centroid partition is poorly aligned with the embedding distribution.

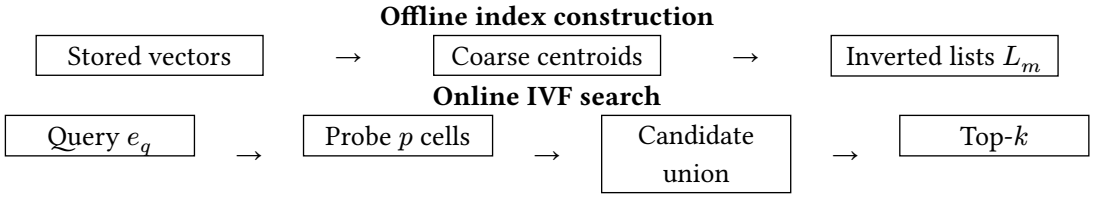

Figure 14: IVF replaces a full corpus scan with centroid selection followed by search inside selected inverted lists. The chosen probe budget determines the candidate-recall versus query-cost trade-off.

34.5 Product Quantization and Vector Compression

Product Quantization (PQ) compresses stored retrieval vectors. It divides a vector into m subvectors,

$$e = [e^1, ..., e^m], \tag{288}$$

and assigns each subvector to a codeword from a small subspace-specific codebook. If $\mathcal{C}_j = \{c_{j,1}, ..., c_{j,K}\}$ is the codebook for block j, the stored code can be written as

$$z_{j(e)} = \arg\min_{\ell \in \{1,...,K\}} \|e^j - c_{j,\ell}\|_2. \tag{289}$$

Instead of retaining every full-precision component, the index stores the m code indices $z_1(e), ..., z_{m(e)}$ and reconstructs or estimates comparisons from the selected codewords. With K codewords per block, the code payload is roughly $m \log_2 K$ bits per vector before IDs and auxiliary structures. PQ therefore trades distance-estimation accuracy for a substantial reduction in vector storage and memory traffic. Product Quantization was introduced for approximate nearest-neighbor search precisely as a compact representation whose distance computations can be organized efficiently [167].

This is different from model quantization in Chapter 25. Model quantization changes how neural-network weights or activations are represented during inference. PQ here compresses the **stored retrieval vectors** and affects the index's candidate-ranking approximation. The two techniques can coexist, but their numerical errors and quality evaluations belong to different parts of a RAG system.

IVF and PQ combine naturally. IVF first limits candidate generation to selected coarse lists; PQ reduces the cost and memory of representing and comparing the vectors inside those lists. The combined index may be much more scalable than a flat full-precision scan, but it introduces two sources of approximation: a relevant vector can be excluded by coarse-cell selection or misordered by compressed distance estimates. Increasing the probe budget, using larger codebooks or more subspaces, or retaining selected full-precision vectors can improve quality at a memory or latency cost.

34.6 Graph-Based Search and HNSW

Graph-based indexes represent stored vectors as nodes joined by selected proximity edges. In a **Hierarchical Navigable Small World** (HNSW) index, upper graph layers contain fewer nodes and support broad navigation, while lower layers contain denser local neighborhoods. Search starts from an entry point high in the hierarchy, greedily moves toward nodes closer to the query, descends through layers, and expands a candidate neighborhood at the base layer. The hierarchy supplies long-range movement before local refinement, rather than explicitly partitioning space into coarse cells.

HNSW's practical controls have clear conceptual roles. Greater graph connectivity stores more edges per node, often improving route quality but increasing index memory. More construction effort can choose better neighbors, at the cost of slower index builds. More query-time exploration visits additional nodes, generally improving ANN recall while increasing latency. The original HNSW work describes this layered proximity-graph construction and its high-recall search behavior [168].

Index family	Candidate mechanism	Key trade-offs
Flat exact	Score every vector.	Exact and simple, but query work and bandwidth grow with the corpus.
IVF	Probe selected coarse lists.	Probe count trades recall for scanned candidates; centroid and list balance can drift as the corpus changes.
HNSW	Navigate a layered proximity graph.	Edges and exploration trade index memory and search work for recall; deletion and compaction need explicit policy.
IVF + PQ	Probe lists and compare compressed codes.	Compression lowers storage and bandwidth but adds distance error; codebooks and vector revisions must remain aligned.

Table 53: IVF searches selected spatial regions, whereas HNSW navigates proximity edges. Neither dominates across corpus sizes, update patterns, recall targets, and hardware.

34.7 Candidate Generation, Filters, and Index Lifecycle

ANN is usually a first-stage candidate generator:

$$e_q \rightarrow \text{ANN} \rightarrow C(q) \rightarrow \text{optional}\backslash \text{ reranking} \rightarrow R_{k(q)}. \tag{290}$$

The candidate set $C(q)$ should be large enough that a later, more discriminating stage has useful alternatives, but not so large that it violates latency or context-construction budgets. This chapter does

not derive reranking; the important boundary is that an ANN miss cannot be repaired downstream if the required item never enters $C(q)$.

Real systems also apply structured constraints. Date, document type, language, tenant, access-control status, or product category may define which vectors are eligible before similarity ranking. **Pre-filtering** restricts the searchable population before or during ANN traversal; it preserves policy boundaries but can leave too few candidates when filters are selective. **Post-filtering** searches a broader set and discards ineligible results afterward; it can waste work and reduce the surviving top-k result count. The correct choice depends on filter selectivity, security semantics, index support, and whether an unauthorized vector is ever permitted to influence logs, caches, or a prompt.

Index build time and query time must be treated separately. Coarse centroids, PQ codebooks, and graph edges may require substantial offline construction, whereas query-time controls determine the work permitted per request. A mutable corpus adds insertion, deletion, and repair semantics. Removing a source might mean deleting its vector, marking it ineligible, rebuilding affected lists or graph neighborhoods, and invalidating cache entries. An index that reports a current document ID while retaining an old embedding, old metadata, or a deleted access policy is operationally stale even if it returns results quickly.

34.8 Memory, Hardware, and Vector Databases

Chapter 33's Nrb payload estimate describes only raw embedding storage. Total vector-search memory may also include compressed codes, graph edges, coarse centroids, codebooks, IDs, metadata, filter structures, shard replicas, allocator overhead, and temporary search state. A graph can spend substantial memory on edges; an IVF design can consume additional memory for lists and centroids; a compressed index can save payload bytes while retaining original vectors for evaluation or reranking. Capacity planning must measure the whole index artifact rather than one vector matrix.

Hardware changes the apparent trade-offs. CPU search benefits from vector instructions, cache locality, and predictable memory access. GPU search can make batched flat scans and selected index operations efficient, but host-device transfer, device-memory capacity, and small request batches can dominate. Many ANN workloads are limited less by arithmetic throughput than by fetching scattered vector, code, or graph data. Johnson, Douze, and Jegou demonstrate how similarity-search design can depend on GPU selection, memory hierarchy, and compressed-domain computation [169].

A **Vector Database** is an operational system that packages vector storage, ANN indexing, metadata, filtering, persistence, mutation, and often replication or distributed execution behind one interface. The name does not establish quality or correctness. It does not choose a compatible embedding model, guarantee ANN recall, solve access-control semantics, or determine whether retrieved text supports a generated claim. These remain explicit retrieval contracts regardless of whether the index is embedded in an application or exposed through a database service.

34.9 Failure Modes and Debugging

Low ANN recall can arise from too little IVF probing, insufficient graph exploration, poor cluster assignment, excessive compression, an unsuitable distance implementation, or a resource cap that truncates search. These are **index-quality** failures: exact search over the same embeddings may still retrieve strong candidates. By contrast, an **embedding-quality** failure persists under exact search because the desired item is not close under the learned score. A **downstream-ranking** failure occurs when the candidate set contains useful evidence but a later selector, context constructor, or generator discards or misuses it. This decomposition prevents an ANN parameter change from being misdiagnosed as an embedding-model improvement or degradation.

Other failures are operational. An index can use the wrong similarity metric or normalization convention, mix document vectors from different encoder revisions, return stale vectors after source updates, suffer memory blow-up from graph edges or replicas, or take too long to rebuild after corpus changes. Filters can silently eliminate the best candidates or, worse, be applied too late for a security boundary. A compression setting may look acceptable on average while collapsing rare

entities, fine numerical distinctions, or close semantic alternatives. These risks require exact-baseline comparison, index-recall monitoring, corpus-version tracking, and task-level evaluation rather than a single latency number.

34.10 Implementation Contracts

The **metric contract** must bind the index to the query and document encoder revisions, embedding dimension, dtype, normalization state, score or distance convention, and tie rule. Tests should compare a small corpus against an exhaustive high-precision implementation and verify that the index's declared exact mode, if any, agrees with Equation 284.

The **index contract** must record the index family, build corpus revision, vector IDs, training sample for centroids or codebooks where applicable, coarse partition policy, compression configuration, graph construction policy, and all search controls. It must define the target ANN Recall@k and the latency evaluation protocol: query batch size, filter distribution, hardware, concurrency, warm-cache state, and percentile statistic. A single average latency without these conditions is not a portable index property.

The **mutation and policy contract** must state insertion, update, deletion, compaction, rebuild, and publication semantics. It must name the metadata and access-control filters applied before and after search, define how vectors excluded by policy are prevented from reaching returned candidates, and preserve the mapping from index result to source and chunk revision. Regression tests should separately measure exact-vector neighbors, ANN recall, task-relevance recall, filter correctness, and downstream ranking quality after every index or encoder revision.

34.11 Summary

Exact vector search supplies the reference candidate set but costs work proportional to the stored collection. ANN indexes reduce that work by narrowing the search or compressing the representation. IVF searches vectors from selected coarse regions; PQ stores compact approximate codes; HNSW navigates a hierarchy of proximity graphs. Their controls expose different combinations of build cost, memory, latency, and ANN recall.

ANN recall measures agreement with exact vector neighbors, not whether a RAG system retrieved relevant or truthful evidence. Reliable systems therefore separate embedding quality, index quality, filter behavior, and downstream ranking. The index must be versioned with the corpus, encoders, metric, policy filters, and mutation rules. Later chapters can build on its candidate set with lexical signals, reranking, query transformation, and evaluation, but none can restore evidence that candidate generation never supplied.

References

[167] H. Jegou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011, doi: 10.1109/TPAMI.2010.57.

[168] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[169] J. Johnson, M. Douze, and H. Jegou, "Billion-Scale Similarity Search with GPUs," *arXiv preprint arXiv:1702.08734*, 2017, doi: 10.48550/arXiv.1702.08734.

Chapter 35: Chunking and Document Segmentation

Abstract

Retrieval-Augmented Generation rarely searches an original source document as one indivisible object. Instead, an ingestion pipeline segments documents into retrievable units, attaches representations and metadata to those units, and indexes them for later selection. This chapter explains why the segmentation policy determines the evidence a retriever can expose, compares length-, boundary-, structure-, and semantics-based policies, and develops the trade-offs in size, overlap, hierarchy, provenance, and maintenance. It closes with operational contracts that make chunk boundaries reproducible, debuggable, and safe to evolve.

35.1 Introduction

Chapter 32 introduced the offline path of a Retrieval-Augmented Generation (RAG) system: source documents are parsed, represented, and indexed before a user query arrives. The apparently small step between parsing and representation is consequential. A retriever usually does not search a whole manual, paper, repository, or book as a single item. It searches units created by a **segmentation** or **chunking** policy:

$$\text{source document} \rightarrow \text{segmentation} \rightarrow \text{retrieval units} \rightarrow \text{representations} \rightarrow \text{index}. \tag{291}$$

The boundaries selected by that policy determine what can later be retrieved together. If an assumption is separated from the theorem it qualifies, a query may retrieve a statement without its scope. If unrelated sections are placed in one large unit, a relevant match can consume context with material that weakens the answer. Chunking is therefore not a cosmetic preprocessing step. It defines the interface between source structure, the retriever described in Chapters 33 and 34, and the context constructor described in Chapter 32.

This chapter treats chunking as a retrieval-system design problem rather than a universal algorithm. It does not derive BM25, dense similarity, approximate nearest-neighbor search, reranking, query rewriting, or a full RAG evaluation protocol. Those components operate on the units defined here. The same segmentation policy can behave differently with another embedding model, query distribution, or context budget, so a useful policy is a measured system choice rather than a fixed percentage copied between corpora.

35.2 Retrieval Units and Segmentation Boundaries

A **source document** is the original object supplied to ingestion: for example, a policy page, technical paper, source file, manual, or database record. A document can contain nested **sections** and **subsections**, and each section can contain one or more **passages**. A **chunk** is a concrete span emitted by a segmentation policy. A **retrieval unit** is the object indexed and returned by a retriever. In a simple pipeline, one chunk is one retrieval unit; in a hierarchical system, a small child chunk may be indexed while a larger parent passage is later supplied to the generator.

Let a parsed document be represented as a token sequence $x = (x_1, \ldots, x_T)$ under the tokenizer contract from Chapter 1. A segmentation policy g produces an ordered collection

$$g(x) = (c_1, \ldots, c_M), \quad c_j = x_{a_j}, \ldots, x_{b_j}, \quad 1 \le a_j \le b_j \le T. \tag{292}$$

The expression in Equation 292 does not require chunks to be disjoint, equal in length, or even determined only from token positions. A structure-aware policy can inspect headings and code fences; a semantic policy can inspect changes in meaning. What every policy must decide is which source span and surrounding information each c_j represents. Dense Passage Retrieval, for example, treats passages as the basic indexed objects and uses a fixed-length passage construction in its open-domain question-answering setting [170]. That choice is one useful baseline, not a definition of a correct RAG chunk.

35.3 Length-Based and Linguistic Segmentation

The simplest family cuts a stream after a fixed number of characters, words, or tokens. Character-based chunking is easy to apply when a parser supplies only plain text, but a character count is weakly related to the model's actual context consumption. Word-based rules are more readable to humans,

yet their relation to downstream token count varies by language and tokenizer. Token-based chunking makes the budget explicit: its length is measured in the same units that compete for the generator's context window.

For a token window of width w and stride s, where $0 < s \leq w$, a sliding-window policy can emit

$$c_j = \left(x_{1+(j-1)s}, \ldots, x_{\min(T,(j-1)s+w)}\right). \tag{293}$$

Adjacent windows overlap by $w - s$ tokens whenever $s < w$. This construction is deterministic, simple to parallelize, and easy to re-run after a source revision. It also makes index cardinality predictable. Its blind spot is meaning: it can split a sentence, detach a heading from its content, or join the final paragraph of one topic to the first paragraph of another.

Sentence- and paragraph-based policies start from natural linguistic boundaries. Respecting these boundaries usually produces units that are easier to read and attribute, and it reduces arbitrary cuts inside an argument. Paragraphs alone, however, can be highly uneven: a one-line list item and a multi-page technical section are both paragraphs in some source formats. A practical policy can therefore accumulate whole sentences or paragraphs toward a token budget, then begin a new chunk at the nearest acceptable boundary. This is still a length-aware policy; the linguistic boundary is a constraint on where it may cut.

Policy family	Useful property	Characteristic risk
Fixed character, word, or token windows	Deterministic size and simple operational behavior	Can cut through syntax, sentences, or a local argument
Sentence or paragraph accumulation	Preserves readable local boundaries	Uneven units and no guarantee that adjacent paragraphs share one topic
Structure-aware segmentation	Retains declared document hierarchy and typed regions	Depends on reliable parsing and source markup
Semantic segmentation	May place boundaries near topic changes	Adds model cost, thresholds, and possible unstable boundaries

Table 54: Segmentation policies expose different assumptions. A robust corpus may use more than one policy for different source types, but each index should record the policy that created its units.

35.4 Structure, Semantics, and Context Preservation

Structure-aware chunking uses boundaries already declared by the source. Markdown and HTML expose headings, lists, tables, and code fences; well-formed documents may expose sections and subsections; source code exposes modules, classes, and functions. A conceptual policy first identifies this hierarchy, recursively segments an oversized structural unit, and attaches the hierarchy to every resulting chunk:

$$\begin{aligned}&\text{document hierarchy} \longrightarrow \text{structural boundaries} \longrightarrow \text{oversized sections}\\ &\longrightarrow \text{recursive segmentation} \longrightarrow \text{chunks plus hierarchy metadata}\end{aligned}$$

The point is not that headings are always semantically correct. A heading can be vague, and PDF extraction can lose the reading order that made a table or caption intelligible. The point is that source structure often supplies evidence about which text should remain together. Structure-aware policies should degrade explicitly when the parser cannot provide trustworthy boundaries, rather than silently claiming the same fidelity for clean HTML and a poorly extracted scan.

Semantic chunking seeks boundaries at changes in content rather than only in length or markup. A system might compare adjacent sentence embeddings, inspect discourse cues, or use a model to propose a topical split. This can avoid grouping two unrelated topics merely because they occupy the same paragraph or window. It can also be expensive at ingestion time, dependent on the chosen model and threshold, and unstable when small source edits alter later decisions. Semantic similarity is not a guarantee that a boundary preserves all logical dependencies: a definition and its exception may look lexically or geometrically different while still needing to be retrieved together.

Context preservation is the common goal. It asks whether a unit carries enough of the surrounding material to support its likely use. An answer may require a definition and its condition, a heading and

the text it scopes, a question and its answer, or a function signature and its implementation. Overlap and structure-aware chunking address this problem differently. Overlap duplicates a neighborhood around every boundary; structure-aware segmentation attempts to place fewer boundaries through meaningful units. Neither removes the need to inspect what the generator actually receives.

35.5 Chunk Size, Overlap, and Boundary Effects

Chunk size is a three-way allocation problem among retrieval specificity, context completeness, and system cost. Smaller chunks often isolate a claim or a definition, reducing irrelevant material in a retrieved result. They also create more vectors or postings, increase index cardinality, and make it easier to separate evidence that must be read together. Larger chunks preserve surrounding context and reduce the number of units, but they can mix several concepts, lower ranking specificity, and consume more of the context budget from Chapter 32.

No numerical size is optimal independently of the corpus. The appropriate range depends on source structure, query granularity, tokenizer behavior, embedding model, retrieval method, context length, and the downstream task. A factual lookup can favor a tightly localized unit; a legal interpretation or a multi-step technical explanation may require a larger parent section. The right question is not whether a chunk is generally "small" or "large," but whether the retrieved unit carries the evidence required for the intended answer without crowding out stronger evidence.

Overlap offers a direct response to boundary loss. Under Equation 293, a fact near the end of one window also appears near the beginning of the next. This improves the chance that a query whose evidence crosses a boundary retrieves at least one adequate unit. The same duplication increases embedding work, storage, index entries, and the likelihood that top-k results contain near-identical text. A context constructor must consequently detect redundant spans rather than treating each retrieved identifier as independent evidence.

The distinction matters operationally. Excessive overlap can make apparent retrieval recall look better because the same source span is represented many times, while adding little new evidence to the generated context. Conversely, zero overlap can create brittle failures at a single arbitrary cut. Measure the effect using the task's candidate recall, context redundancy, answer quality, and latency rather than assuming a universal overlap ratio.

35.6 Metadata, Hierarchy, and Parent–Child Retrieval

Every retrieval unit should preserve a reversible link to its source. At a minimum, a record should carry a stable document identifier, a source revision, a chunk identifier, an ordered source span, its text representation, and any policy-relevant metadata. Useful additional fields include a document title, section path, page or anchor location, timestamp, author, language, content type, and access-control attributes. Chapter 32 explains why these fields are necessary for filtering, provenance, citation, freshness, and debugging; segmentation is the point at which many of them can otherwise be lost.

A stable chunk identifier must be designed carefully. An identifier derived only from ordinal position changes after an insertion near the document's beginning. An identifier derived only from text can collide across repeated boilerplate and changes when harmless normalization changes. In practice, the identity contract should name the source revision, span rule, segmentation-policy version, and canonicalization procedure. It must be possible to distinguish a new representation of the same source span from a genuinely different source span.

Parent–child retrieval separates the unit used for matching from the unit used for generation. Small child chunks are indexed for specificity. When one is selected, the system maps it to a parent section or a bounded local expansion before context construction:

$$\text{small child chunk} \longrightarrow \text{precise retrieval} \longrightarrow \text{parent section or local neighborhood} \longrightarrow \text{context construction} \longrightarrow \text{richer evidence}$$

This pattern can reduce the tension between precise matching and complete context, but it changes the interface: evaluation must distinguish whether the child was retrieved, whether the chosen parent contains the necessary evidence, and whether expansion overflowed the context budget. Hierarchical

approaches such as RAPTOR make the broader idea explicit by organizing text at multiple levels of abstraction for retrieval over long documents [171]. Parent–child retrieval does not require learned summaries or a tree index; even a deterministic section hierarchy is a useful form of parent relation.

35.7 Special Document Forms and Long Contexts

Tables are not ordinary prose with decorative whitespace. A cell value can be uninterpretable without its row label, column header, unit, caption, and nearby qualifier. Naively flattening a table into independent token windows can destroy those relations. A table-aware pipeline should retain headers, row and column associations, caption or source identity, and a declared serialization rule. A system may index a whole small table, coherent row groups with repeated headers, or a text representation paired with the original structured object; the appropriate choice depends on the questions the corpus is expected to answer.

Code has analogous structural dependencies. Arbitrary windows can separate a function from its signature, a method from the class defining its fields, or a call site from imported names. A code-aware policy can prefer module, class, function, method, and block boundaries, while retaining file path, language, symbol name, imports, and line ranges as metadata. It need not make every function a single chunk: very large functions may still require an explicit subdivision rule. The requirement is to preserve the syntax and provenance that give retrieved text its meaning.

Long documents amplify every segmentation decision. Books, standards, technical manuals, papers, and legal texts contain hierarchy, cross-references, and concepts revisited at several granularities. Treating such a document as one flat sequence either produces enormous units or loses the hierarchy that helps a system reconstruct local context. A long-document pipeline should preserve the section path and ordering even if its first-stage index is flat. This keeps later expansion, citation, filtering, and reassembly possible without re-parsing an opaque chunk payload.

35.8 Re-Chunking, Evaluation, and Failure Modes

Changing a chunking policy changes the retrieval corpus. New boundaries can alter chunk identifiers, document-to-chunk mappings, representations, embeddings, sparse features, index entries, and evaluation judgments. A production policy change should therefore be treated as a versioned re-indexing event, not as a harmless configuration edit. Incremental changes are safe only when the system can prove which source revisions and derived units remain compatible with the published index.

Chunking should be evaluated through the downstream system, not aesthetic inspection alone. Relevant signals include retrieval recall and precision, whether an answer-supporting span reaches the context, answer quality, duplicate-context rate, index size, ingestion cost, and query latency. The diagnosis must remain decomposed. Poor retrieval may arise from a weak embedding model or ANN index as Chapters 33 and 34 explain, but it may also arise because the evidence was separated, stripped of metadata, or never emitted as a suitable unit.

Common failures follow directly from this interface. Units that are too small lose conditions and surrounding definitions; units that are too large blur multiple topics and waste context. Arbitrary cuts can break sentences, tables, and code. Excessive overlap produces duplicate results and inflates storage; missing overlap can make one cut decisive. Lost headings or source metadata damage provenance. Semantic policies can create implausible boundaries, and non-deterministic policies make re-indexing difficult to audit. The response is not to select the most sophisticated policy by default, but to make the policy observable, versioned, and evaluated against the target workload.

35.9 Implementation Contracts

The **input contract** must declare accepted source formats, parser versions, canonicalization rules, tokenizer revision where token budgets matter, and the handling of malformed or unparseable content. The **segmentation contract** must name the policy family, target and maximum unit lengths, permissible boundaries, stride or overlap rule, structure-recognition rules, treatment of tables and code, and deterministic tie behavior. It should make clear whether a chunk is a source span, a rendered representation, or both.

The **provenance contract** must preserve document and chunk identifiers, source revision, ordered span offsets, section path, timestamps, policy metadata, and access-control fields. It must define parent links and the expansion rule if parent–child retrieval is used. The **index-update contract** must state when a source update requires re-chunking, re-embedding, deletion, compaction, full rebuild, and publication of a new corpus version. It should prevent a representation generated from one canonical text revision from being presented as evidence for another.

Finally, the **evaluation contract** should measure the declared retrieval and generation task under a fixed corpus revision. It should report unit count, length distribution, duplicate rate, context redundancy, candidate recall, support coverage, latency, and index cost alongside answer quality. Regression examples must include boundary-sensitive cases: a condition following a theorem, a heading that scopes a paragraph, a table whose headers alter interpretation, and code whose signature or import is required to read its body.

35.10 Summary

Chunking determines the retrieval units that an index can expose. Fixed token windows offer simple and reproducible behavior; sentence, paragraph, structure-aware, and semantic policies seek stronger local coherence at different operational costs. Chunk size and overlap balance specificity, context preservation, index cardinality, and redundant retrieval. Tables, code, and long documents require their structural relations to survive segmentation rather than being flattened into arbitrary text.

Reliable RAG systems treat a chunk as a versioned, attributable source span with metadata and, when useful, a parent relation. A segmentation change is an index change: it may require new representations, new identifiers, and new evaluation. Later chapters can compare sparse and hybrid retrieval, reranking, query transformation, and RAG evaluation, but each depends on whether this chapter's policy made the right evidence retrievable in the first place.

References

[170] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. doi: 10.18653/v1/2020.emnlp-main.550.

[171] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv preprint arXiv:2401.18059*, 2024, doi: 10.48550/arXiv.2401.18059.

Chapter 36: Sparse Retrieval and Hybrid Search

Abstract

Sparse retrieval ranks documents through observable lexical evidence, while dense retrieval ranks learned semantic representations. Neither signal is sufficient for every query. This chapter develops Bag-of-Words representations, TF-IDF, BM25, and inverted indexes, then explains how sparse and dense candidate sets can be combined through score fusion and rank fusion. It also introduces learned sparse retrieval, filtering, and operational contracts that keep a hybrid system interpretable, reproducible, and debuggable.

36.1 Introduction

Chapter 33 developed dense retrieval as a learned ranking function over query and document embeddings. Its strength is that a useful passage need not repeat the query's wording. That strength does not make exact lexical evidence obsolete. A model lookup for a package name, a version number, an error code, a scientific identifier, a product SKU, or a rare entity often depends on preserving precisely the terms that occur in the source. Replacing exact matching with a broad semantic signal can turn a query for one identifier into a highly ranked passage about a related but wrong identifier.

Sparse retrieval treats a query and retrieval unit as weighted collections of terms. Most possible terms have weight zero in any one collection, which makes the representation sparse and permits efficient lookup through an inverted index. The family includes classical Bag-of-Words methods such as TF-IDF and BM25 as well as learned sparse models. This chapter uses **lexical** to mean that the score retains an explicit vocabulary-aligned matching interface; it does not mean that every sparse weight is hand-designed.

Sparse and dense retrieval should therefore be viewed as complementary signals. Lexical methods can preserve rare terms and make an obvious match easy to inspect. Dense methods can recover paraphrases and vocabulary mismatch. A **Hybrid Search** system uses both paths, reconciles their candidates or rankings, and passes a broader, more robust candidate set to the later context construction and reranking stages. This chapter does not develop Cross-Encoder or LLM reranking, query rewriting, multi-query retrieval, or RAG evaluation frameworks. Those chapters consume the candidate interface established here.

36.2 Lexical Representations and TF-IDF

Let the collection-specific lexical vocabulary be

$$\mathcal{V}_{\text{lex}} = \left\{t_1, ..., t_{V_{\text{lex}}}\right\}. \tag{294}$$

A Bag-of-Words representation maps a document d to a vector $v(d) \in R^{V_{\text{lex}}}$, with one coordinate per term. The coordinate can record presence, raw count, or a derived weight. Because a document contains only a small subset of the full vocabulary, most coordinates are zero. Bag-of-Words deliberately retains lexical identity while largely ignoring order, syntax, and long-range compositional meaning. It is an abstraction, not a claim that word order never matters for relevance.

Let $f(t, d)$ denote the number of occurrences of term t in document d. **Term Frequency** (TF) makes this observed frequency an evidence feature. A raw count rewards repetition, but not every repeated term is informative. Terms that occur in nearly every corpus item can dominate a count while doing little to distinguish one result from another.

For a collection of N documents, let $\mathrm{df}(t)$ be the number containing t. A conceptual **Inverse Document Frequency** (IDF) is

$$\mathrm{IDF}(t) \approx \log \frac{N}{\mathrm{df}(t)}. \tag{295}$$

When $\mathrm{df}(t)$ is small, the term is comparatively rare and Equation 295 gives it greater weight. The exact smoothing convention differs across systems, especially for terms with extreme document frequencies; the stable idea is that terms common across the collection supply weaker discriminatory evidence than terms found in a small subset.

TF-IDF combines local occurrence and corpus-wide rarity:

$$w_{\text{TF-IDF}(t,d)} = \text{TF}(t,d)\,\text{IDF}(t). \tag{296}$$

Query and document vectors can then be compared through weighted lexical overlap, often an inner product or a normalized variant. Classical information retrieval develops this vector-space view and its assumptions in detail [172]. In a RAG system, the important boundary is practical: a TF-IDF score ranks the terms emitted by the declared analyzer. Tokenization, lowercasing, stemming, punctuation policy, synonym handling, and language analysis can change which apparent strings count as the same term.

36.3 BM25 and Inverted-Index Retrieval

BM25 is a widely used lexical ranking function that refines raw term counting with saturation and document-length normalization. One standard conceptual form is

$$s_{\text{BM25}(q,d)} = \sum_{t \in q} \text{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{\ell_{\text{avg}}}\right)}. \tag{297}$$

Here q is the analyzed query, $|d|$ is document length under the selected analyzer, ℓ_{avg} is average document length in the indexed collection, $k_1 > 0$ controls term-frequency saturation, and b controls the strength of length normalization. A production implementation may use a different IDF smoothing rule, query-frequency treatment, or field-aware extension. The notation in Equation 297 exposes the roles that must remain explicit, not one universally mandated parameterization [173].

The numerator and denominator in the fraction make repeated occurrences valuable but not linearly valuable forever. The first few appearances of a query term can be strong evidence that a document is about that term; the hundredth repetition may be boilerplate, a list, or accidental duplication. Increasing $f(t,d)$ therefore produces diminishing incremental gain. This **term-frequency saturation** prevents repetition from overwhelming the remainder of the query evidence.

Long documents have more opportunities to contain a query term by chance. The $|d\frac{\perp}{\ell_{\text{avg}}}$ factor compensates for this advantage relative to an average-length unit. Larger b gives document length more influence; smaller b moves toward no such correction. The correct setting depends on the retrieval unit distribution. Chapter 35 makes this dependency concrete: a corpus with uniform token windows has a different length profile from one indexed at paragraph or section granularity.

Sparse search is efficient because it uses an **inverted index**. Instead of scanning every document for each query, the index maps a term to a posting list:

$$t \to [(\text{document ID}, f(t,d), \text{positions}, \text{metadata}), \ldots]. \tag{298}$$

Only documents appearing in posting lists for query terms need be considered for a lexical score. A posting can retain a document or chunk identifier, term frequency, positions for phrase or proximity logic, and fields needed for filtering. This is not the **Inverted File Index** (IVF) discussed in Chapter 34. Both names use "inverted," but an inverted index maps lexical terms to documents, whereas IVF maps vectors to coarse vector-space regions. Their stored objects, candidate-generation rules, and failure modes are different.

36.4 Sparse and Dense Signals

Sparse and dense retrievers answer different matching questions. Sparse retrieval asks whether the query's terms, or terms assigned nonzero sparse weights, appear in a unit with informative frequency. Dense retrieval asks whether independently computed embeddings occupy a compatible neighborhood under a learned similarity function. Sparse search is consequently strong for exact names, rare terminology, identifiers, version strings, and evidence that must visibly overlap. Dense search is useful when a relevant source expresses the same intent in different wording.

Signal	Often strong for	Characteristic blind spot
Sparse lexical	Rare terms, identifiers, exact quotations, and inspectable token overlap	Paraphrases, synonymy, and terminology mismatch
Dense semantic	Paraphrases, semantic relatedness, and learned query–passage associations	Fine lexical distinctions, rare strings, and semantically related but factually wrong matches
Hybrid	Robust first-stage candidate generation across both signal types	Score calibration, duplicate management, and added system cost

Table 55: Sparse and dense retrieval have complementary failure modes. A hybrid design aims to preserve candidates that either path finds persuasive, rather than declaring one signal universally sufficient.

The distinction is empirical, not categorical. A sparse query can miss "automobile maintenance" when it asks for "car repair." A dense retriever can retrieve a passage about a semantically related system that has the wrong version, numerical condition, or entity. Work comparing sparse, dense, and more expressive retrieval representations illustrates why sparse precision and dense semantic matching can be productively combined rather than treated as mutually exclusive [174].

36.5 Hybrid Candidate Generation and Fusion

A basic Hybrid Search pipeline runs two candidate generators in parallel:

$$\text{query} \longrightarrow \text{sparse retriever} \longrightarrow \text{sparse candidates}$$
$$\text{query} \longrightarrow \text{dense retriever} \longrightarrow \text{dense candidates}$$
$$\text{candidate union} \longrightarrow \text{fusion} \longrightarrow \text{Top-}k \text{ results} \longrightarrow \text{later reranking}$$

Let $C_{\mathrm{s}(q)}$ and $C_{\mathrm{d}(q)}$ be the candidate sets from sparse and dense paths. Their first-stage union is

$$C_{\mathrm{hybrid}(q)} = C_{\mathrm{s}(q)} \cup C_{\mathrm{d}(q)}. \tag{299}$$

The union can improve recall because a document missed by one retriever remains eligible when found by the other. It also introduces duplicates, a larger candidate set, and the need to decide how much work each path receives. Candidate depth controls both downstream recall and cost. A system that fetches a very large result set from both paths may negate first-stage latency gains or overwhelm a later reranker.

Score fusion combines numeric scores after a declared normalization or calibration. A simple weighted form is

$$s_{\mathrm{hybrid}(q,d)} = \alpha \hat{s}_{\mathrm{dense}(q,d)} + (1-\alpha)\hat{s}_{\mathrm{sparse}(q,d)}, \quad 0 \le \alpha \le 1. \tag{300}$$

The hats in Equation 300 are essential. Raw BM25 scores and dense similarities need not share a scale, range, distribution, or even a comparable notion of distance. Direct addition can make one retriever dominate merely because its numbers are larger. Per-query normalization, score calibration on held-out data, weighted fusion, and learned fusion are alternative ways to construct comparable inputs. None is automatically correct under corpus, query, model, or filter changes.

Rank fusion avoids comparing raw scores directly. Each retrieval path supplies an ordering, and a fusion function rewards documents that rank highly in one or more lists. **Reciprocal Rank Fusion** (RRF) is a compact example:

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k_{\mathrm{RRF}} + \mathrm{rank}_{r(d)}}. \tag{301}$$

Here $\mathcal{R}$ is the set of retrieval systems, $\mathrm{rank}_{r(d)}$ is the one-based rank assigned by system r, and k_{RRF} is a smoothing constant. It is not the top-k retrieval depth. A document at a high rank contributes more than one near the bottom, while a document appearing in both lists receives evidence from both. RRF was introduced as a simple rank-combination method for information retrieval systems [175]. Its

rank-only nature can be attractive when score calibration is unreliable, although it discards differences in score magnitude that may be informative.

36.6 Learned Sparse Retrieval and Filtering

The sparse/dense distinction is not synonymous with classical/neural. A **learned sparse retriever** uses a neural model to assign vocabulary-aligned weights to a query and document while retaining sparsity and inverted-index compatibility. It can learn expansion-like associations that a raw Bag-of-Words representation lacks, yet preserve explicit terms and an efficient posting-list interface. SPLADE is a representative family: it learns sparse lexical and expansion weights under sparsity regularization for first-stage ranking [176].

Learned sparse models add their own contracts. The vocabulary, analyzer, expansion behavior, pruning rule, weight dtype, and index builder determine which nonzero dimensions exist at serving time. A model may improve a paraphrase match by activating a related term, but that activation is still a learned ranking signal rather than an explanation of truth. As with dense retrievers, a source revision requires re-representation before the index can be considered current.

Sparse, dense, and hybrid paths must apply the same eligibility policy. Date, language, tenant, document type, and access-control filters can be applied before candidate generation, after it, or through an index-specific mechanism. Chapter 34 explains the broader pre-filtering and post-filtering trade-off for vector search. The hybrid extension is simple but important: a policy filter applied to only one path changes not only security or governance semantics but also the apparent quality of the fusion procedure.

36.7 Pipeline Design and Failure Diagnosis

An end-to-end first stage can make its boundaries explicit:

$$\text{query} \longrightarrow \text{lexical analysis} \longrightarrow \text{BM25 retrieval}$$

$$\text{query} \longrightarrow \text{compatible query encoder} \longrightarrow \text{dense ANN retrieval}$$

$$\text{candidate union} \longrightarrow \text{score or rank fusion} \longrightarrow \text{candidate set} \longrightarrow \text{later reranking}$$

The lexical analyzer and dense encoder should not be silently assumed to agree. One may lowercase or stem an identifier that the other tokenizes differently; one may accept a language that the other model handles poorly. Candidate identifiers, corpus revisions, and filters provide the interface on which their results can actually be joined. Reranking, addressed next, is a later selective comparison stage rather than a reason to leave these first-stage boundaries unspecified.

Failures should be diagnosed by path. A **sparse-retrieval failure** can arise from analyzer mismatch, vocabulary mismatch, missing aliases, poor field construction, rare-term overemphasis, or unhelpful length normalization. A **dense-retrieval failure** can arise from domain mismatch, semantic confusion, an incompatible embedding/index revision, or an ANN miss. A **fusion failure** arises when the paths contain useful candidates but normalization, weights, ranks, duplication rules, or filtering discard them. Treating all three as "RAG retrieval failure" hides the repairable interface.

Other system failures are coupled to candidate size. Fetching too few items from one path limits complementarity; fetching too many can increase latency and duplicate context. Badly calibrated scores can give one retriever permanent control. Inconsistent filtering can expose an unauthorized candidate on one path or make one path appear weak because it faced a narrower corpus. The appropriate monitoring unit is therefore a query trace that records analyzed terms, sparse postings, embedding revision, each candidate list, filter decisions, fusion inputs, and final candidate order.

36.8 Implementation Contracts

The **sparse contract** must name the analyzer and lexical vocabulary, text fields, normalization, stopword and stemming policy where used, posting format, BM25 or alternative scoring configuration, document-length statistic, and exact corpus revision. It should specify how an analyzed query is represented when no term has a posting list and how phrase, positional, or field behavior affects the score.

The **dense contract** must preserve the encoder, tokenizer, similarity convention, ANN index, and corpus revision from Chapters 33 and 34. The **hybrid contract** must bind the two paths to a common retrieval-unit identifier, eligibility policy, candidate depths, deduplication rule, fusion method, normalization or calibration revision, and final top-k tie rule. A result must remain traceable to the path or paths that supplied it; otherwise, a score regression cannot be assigned to sparse retrieval, dense retrieval, or fusion.

The **evaluation contract** should report sparse-only, dense-only, and hybrid candidate recall separately under the same filters and relevance judgments. It should also report candidate overlap, union size, duplicate rate, fusion ablations, latency by path, tail latency of the joined request, and downstream support coverage. Tests should include exact identifiers, paraphrases, rare entities, conflicting source revisions, filter-sensitive records, and cases where the two paths rank different valid sources. These slices turn complementarity into a measurable claim rather than a slogan.

36.9 Summary

Sparse retrieval preserves lexical evidence through weighted term representations and inverted indexes. TF-IDF balances local term occurrence with corpus-wide rarity; BM25 adds term-frequency saturation and document-length normalization. These methods remain valuable wherever a precise string, rare term, or interpretable overlap is part of the evidence needed for retrieval.

Dense and sparse methods fail differently. Hybrid Search forms a candidate union and combines scores or ranks through an explicitly versioned fusion rule. Score fusion requires normalization or calibration because BM25 and embedding scores are not inherently comparable; rank fusion such as RRF avoids raw-score comparison while giving up magnitude information. A reliable system evaluates sparse, dense, and fusion quality separately, applies filters consistently, and passes a well-specified candidate set to the later reranking stage.

References

[172] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[173] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[174] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, "Sparse, Dense, and Attentional Representations for Text Retrieval," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021, doi: 10.1162/tacl_a_00369.

[175] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.

[176] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 2021, pp. 2288–2292. doi: 10.1145/3404835.3463098.

Chapter 37: Reranking and Retrieval Refinement

Abstract

First-stage retrieval must search a large corpus quickly, so it is usually designed to preserve useful evidence at a generous candidate depth. A reranker makes a more selective comparison over that bounded set before context construction. This chapter develops the recall–precision division of labor, contrasts Bi-Encoder retrieval with joint Cross-Encoder scoring, and explains pointwise, pairwise, listwise, and LLM-based reranking. It also treats diversity, metadata, cascades, latency, and operational contracts as parts of the ranking objective rather than afterthoughts.

37.1 Introduction

The retrieval pipeline developed in Chapters 33–36 deliberately separates broad search from fine discrimination. A dense, sparse, or hybrid retriever must compare a query with a corpus that may contain millions of retrieval units. Its score is therefore designed for a representation that can be indexed or precomputed. The resulting first-stage list is useful only as a **candidate set**: it is an affordable opportunity for a later component to make a more informed relevance judgment.

Reranking is that second-stage judgment. The full pipeline has a clear division of labor:

query → sparse / dense / hybrid retrieval → candidate set
→ reranker → final ranked passages → context construction → generator

The first stage should retain answer-supporting evidence even when its ranking is imperfect. The reranker is then allowed to spend more computation on a modest number of candidates to put the most useful evidence near the top. Final **context selection** is still a separate decision: it turns ranked passages into a bounded, nonredundant prompt with provenance and formatting compatible with the generator. Conflating candidate generation, reranking, and context selection makes it difficult to identify whether an unsupported answer began with a retrieval miss, a ranking error, or a poor prompt assembly decision.

This chapter focuses on retrieval refinement. It does not develop query rewriting, multi-query retrieval, HyDE, iterative retrieval, graph-based retrieval, agentic RAG, or a complete RAG evaluation framework. Those methods can alter the candidate interface considered here, but they do not remove the need to state what a reranker scores and what evidence reaches it.

37.2 Candidate Generation, Recall, and Candidate Depth

For a query q, let the first-stage retriever return an ordered candidate set

$$C_{K(q)} = (d_1, ..., d_K), \tag{302}$$

where K is the **candidate depth**. Let $G(q)$ denote the set of judged relevant units for the query. A simple candidate-recall measure is

$$\text{Recall}@K(q) = \frac{|C_{K(q)} \cap G(q)|}{|G(q)|}. \tag{303}$$

The exact relevance judgments and averaging convention must be stated for a real evaluation, but Equation 303 exposes the structural constraint: no reranker can recover a relevant passage that is absent from $C_{K(q)}$. First-stage Recall@K is therefore an upper bound on the evidence available to a second stage. A reranker can repair an order; it cannot repair a candidate-generation miss.

This explains why first-stage retrieval is commonly recall-oriented. Chapters 33 and 34 showed that a Bi-Encoder and an approximate-nearest-neighbor index exchange exact comparison for scalable search. Chapter 36 showed that sparse and dense paths can expand the set through hybrid candidate generation. These designs may leave related, partially redundant, or merely topical passages in $C_{K(q)}$. That is acceptable when the next stage can distinguish them. A first stage tuned only for its own Precision@K can instead discard the one source that contains the condition, date, or counterexample the generator needs.

Candidate depth is a system parameter, not a harmless constant. If K is too small, first-stage recall constrains every later component. If K is too large, joint scoring becomes expensive, creates more duplicate candidates, and can delay an interactive response. The appropriate value depends on corpus size, query ambiguity, retrieval-unit granularity from Chapter 35, reranker cost, context budget, and the desired tail latency. It should be selected with end-to-end evidence rather than copied from a benchmark configuration.

37.3 Bi-Encoders and Cross-Encoders

Chapter 33 introduced a Bi-Encoder, or Dual-Encoder, in which a query encoder and a document encoder produce independent vectors. The system can precompute document vectors, index them, and score a query with a similarity function. This independent encoding is exactly why a Bi-Encoder scales to a large corpus. It is also a restriction: the document representation cannot condition on the particular query at scoring time.

A **Cross-Encoder** jointly processes the query and one candidate document. Conceptually,

$$(q, d) \rightarrow \text{joint Transformer} \rightarrow r_{\theta(q,d)}, \tag{304}$$

where $r_{\theta(q,d)}$ is a learned relevance score. Because the Transformer's attention can compare query tokens with document tokens directly, the model can recognize a qualifying phrase, a negation, an identifier, or a relation whose importance depends on both inputs. BERT-style passage reranking was an early influential demonstration of applying a jointly encoded pretrained Transformer to query–passage ranking [177].

The same interaction prevents precomputing one reusable score or embedding for every possible query. A Cross-Encoder must run its forward pass for each (q, d) pair, so applying it to the whole corpus would be far more expensive than first-stage vector or lexical retrieval. The two architectures are therefore complements rather than substitutes:

Component	Representation and comparison	Operational role
Bi-Encoder retriever	Encode q and d independently; compare reusable vectors or sparse features	Broad corpus search and high-recall candidate generation
Cross-Encoder reranker	Jointly encode a query–candidate pair; score token-level interactions	Precision-oriented ordering over a bounded candidate set
Context selector	Select, deduplicate, and format highly ranked evidence	Fit useful, attributable evidence into the generator's context budget

Table 56: Candidate generation, reranking, and context selection use different representations and optimize different interfaces. A high Cross-Encoder score does not itself decide how many passages should enter a prompt.

Cross-Encoder scores are ranking signals, not automatically calibrated probabilities of truth, usefulness, or user satisfaction. Their numerical scale depends on the training labels, objective, candidate distribution, and model revision. A deployment should preserve the score for observability, but should not treat a larger raw score as comparable across unrelated model versions or query populations without validation.

37.4 Ranking Objectives

Learning to rank can be organized by the object on which supervision is expressed: an individual item, a pair of items, or a whole ordered list [178]. The categories clarify what a reranker is trained to preserve; they do not prescribe one universally superior architecture.

In a **pointwise** formulation, the model predicts a relevance value for a single candidate, such as $r_{\theta(q,d)}$. Labels may be binary, graded, or derived from interaction data. Pointwise scoring is straightforward

to serve because candidates can be scored independently, but the objective need not directly express whether one candidate should be above another for the same query.

In a **pairwise** formulation, supervision states a relative preference. For candidates d_i and d_j, a standard conceptual model is

$$P\big(d_i \succ d_j \texttt{ mid } q\big) = \sigma\Big(r_{\theta(q,d_i)} - r_{\theta(q,d_j)}\Big). \tag{305}$$

The corresponding negative log-likelihood increases the score gap when d_i should rank above d_j. Pairwise supervision matches the comparative nature of ranking, but the number of possible pairs grows rapidly with candidate-set size. It also requires a policy for ties, incomplete judgments, and pairs that are equally relevant but differ in source authority or freshness.

Listwise approaches optimize properties of an ordered candidate list or a distribution over permutations. They can better align training with a ranking metric that values the placement of several items, but they commonly require more structured labels and more complex training or serving behavior. The important design question is not which label is fashionable; it is whether the supervision and loss represent the downstream intent.

Formulation	Supervision unit	Principal trade-off
Pointwise	A query–candidate relevance label	Simple independent scoring; relative ordering is only indirect
Pairwise	A preferred candidate relative to another candidate	Direct comparative signal; pair construction and cost matter
Listwise	A candidate list or its desired ordering	Closer to list-level ranking goals; labels and optimization are more involved

Table 57: Pointwise, pairwise, and listwise objectives differ in the unit of supervision. Their suitability depends on the available labels and the final ranking criterion, not only on model capacity.

These distinctions apply to both traditional learned rankers and neural rerankers. A Cross-Encoder is an architecture for producing a query-conditioned score; it can be trained under different ranking objectives. Conversely, a pointwise score can be used in a broader list-level selection policy. Keep the modeling objective, score meaning, and final selection rule separate in the system specification.

37.5 Precision, Usefulness, and Context Selection

Reranking is often described as improving **precision**, but relevance itself is multidimensional. A passage can be topically related yet fail to answer the question. It can contain an answer but be too incomplete to support a grounded response. It can be useful evidence but obsolete, unauthorized, or less authoritative than an available primary source. A RAG pipeline should consequently distinguish topical relevance, answer usefulness, evidence quality, source authority, freshness, and eligibility.

The distinction becomes concrete at context construction. Suppose the reranker places five near-duplicate passages from a single source above a complementary source containing a missing assumption. A context constructor that blindly takes the top five can reduce answer quality despite a locally accurate ranker. It may need to suppress duplicates, preserve source identity, prefer a more current document, or select one detailed passage and one independent corroborating passage. These are selection decisions over the reranked list, not reasons to hide metadata from the ranker.

Maximal Marginal Relevance (MMR) is a representative diversity-aware rule. Given already selected passages S, it chooses a candidate that balances query relevance and similarity to what is already selected:

$$\mathrm{MMR}(d \mid S) = \lambda r(q,d) - (1-\lambda)\max_{u \in S} \mathrm{sim}(d,u), \quad 0 \le \lambda \le 1. \tag{306}$$

The first term prefers evidence relevant to q; the second penalizes redundancy relative to S. When λ is large, selection approaches relevance-only ranking. When it is small, diversity has more influence. MMR is a useful conceptual mechanism, not a guarantee that dissimilar passages are useful. An

unrelated result is diverse but does not improve evidence coverage. The original MMR formulation made this relevance–novelty trade-off explicit for document reordering and summarization [179].

Metadata-aware refinement supplies additional constraints or features. Date, document version, source type, language, tenant, permission, and authority can be eligibility filters, ranking features, or tie-breakers. The policy should state which role each field plays. A hard permission restriction must never become a soft relevance preference; a freshness preference should not silently override a source's technical authority. Chapter 34's filtering discussion applies here: a ranking result is meaningful only over the eligible corpus actually considered.

37.6 LLM-Based Reranking and Retrieval Cascades

An LLM can also make query-conditioned relevance judgments. In a **pointwise** prompt, it assigns a label or score to one query–passage pair. In a **pairwise** prompt, it chooses which of two candidates better answers the query. In a **listwise** prompt, it orders several candidates together. Such prompting can express nuanced criteria, including whether a passage contains usable evidence rather than merely a shared topic. Research on LLMs as reranking agents demonstrates that appropriately prompted generative models can be evaluated directly on relevance ranking, while also exposing concerns around novel-data evaluation and ranking cost [180].

The flexibility comes with constraints. Each LLM judgment consumes tokens and introduces latency that may exceed a specialized Cross-Encoder. Its ranking can vary with prompt wording, candidate order, truncation, decoding configuration, and the judge model's own capabilities. Position bias can reward early candidates; verbosity bias can mistake a longer passage for a more useful one; context limits can force the system to omit information needed for a fair comparison. An LLM reranker should therefore be evaluated as a model with a declared prompt, decoding configuration, candidate order policy, and cost budget, not as an oracle.

These trade-offs motivate a **retrieval cascade**:

$$\begin{gathered}\text{large corpus} \longrightarrow \text{cheap retriever} \longrightarrow \text{broad candidate set} \\ \longrightarrow \text{stronger reranker} \longrightarrow \text{smaller set} \longrightarrow \text{optional expensive selector} \\ \longrightarrow \text{final context}\end{gathered}$$

An LLM need not appear in every cascade. A sparse, dense, or hybrid first stage followed by a Cross-Encoder is often the appropriate balance. An optional LLM stage may be reserved for ambiguous, high-value, or difficult requests, provided its added cost and nondeterminism are measured. The end-to-end latency is approximately

$$T_{\text{answer}} = T_{\text{retrieve}} + T_{\text{rerank}} + T_{\text{generate}}. \tag{307}$$

The terms in Equation 307 can overlap in a particular serving implementation, but the decomposition remains useful for diagnosis. Reducing retrieval latency cannot compensate for a reranker whose candidate depth or sequence length dominates time to first token. Conversely, skipping reranking may save milliseconds while causing the generator to process more irrelevant context and produce a less supported answer.

37.7 Metrics and Failure Diagnosis

Reranking should be evaluated at both its own interface and the final system interface. Recall@K asks whether first-stage retrieval exposed relevant evidence. Precision@k asks what fraction of a small displayed or selected prefix is relevant. **Mean Reciprocal Rank** (MRR) rewards placing the first relevant item early. **Normalized Discounted Cumulative Gain** (nDCG) can use graded relevance and discounts results that occur later in the list. These metrics answer different questions; reporting one number without candidate depth, judgment policy, filters, or latency hides the actual ranking trade-off.

A practical diagnostic separates three failure classes. A **candidate-generation failure** means answer-supporting evidence never reached $C_{K(q)}$. A **reranker failure** means the evidence was present but placed too low, confused with a more topical passage, or assigned a stale or poorly calibrated score. A **context-selection failure** means an adequately reranked passage was removed, crowded out by

redundant items, truncated, or formatted so that the generator could not use it. This decomposition avoids blaming the embedding model for an MMR policy or blaming a Cross-Encoder for a sparse-retrieval miss.

Common failures can cross these boundaries. A domain-mismatched reranker can learn lexical shortcuts rather than evidence quality. An overly shallow K prevents recovery; an overly deep K raises latency and may exceed a cross-encoder's input budget. Duplicate passages can absorb ranking positions. A model can prefer verbose text, stale sources, or an early candidate position. Score scales can shift after a model update, making an old threshold invalid. The remedy is an observable trace: query revision, candidate IDs and ranks from each first-stage path, reranker inputs and scores, metadata decisions, selected context, and the model versions responsible for each step.

37.8 Implementation Contracts

The **candidate contract** must identify the corpus revision, retrieval-unit schema, eligibility filters, sparse and dense paths where used, candidate depths, deduplication rule, and the exact order passed to the reranker. It must distinguish a candidate identifier from a source-document identifier, so that repeated chunks and parent expansions remain observable. It should also define fallback behavior when no candidate is eligible or a first-stage service is unavailable.

The **reranking contract** must name the model, tokenizer, query–document serialization, truncation policy, maximum pair length, objective or prompt, score interpretation, batching policy, tie rule, and model revision. A Cross-Encoder must receive the same declared field layout at evaluation and serving time. An LLM-based reranker additionally needs a prompt template, candidate-order policy, decoding configuration, parser for its output, retry policy, and a budget that bounds tokens and latency.

The **selection contract** must specify how the ranked list becomes generator context: final depth, diversity rule, duplicate policy, metadata or authority constraints, source-attribution format, context budget, and treatment of conflicting evidence. The **evaluation contract** should report first-stage recall, reranker ranking metrics, selected-context support coverage, downstream answer quality, tail latency, cost, and slice results for exact identifiers, paraphrases, fresh sources, permissions, duplicate passages, and ambiguous questions. Version all judgments and queries so that a change in corpus, retriever, or reranker can be localized rather than attributed to a generic RAG regression.‘’

37.9 Summary

Reranking refines a bounded first-stage candidate set before evidence enters the generator. The first stage is optimized to make relevant evidence available; a Cross-Encoder or LLM-based reranker can then make a stronger query–passage comparison, at a cost that grows with candidate depth and input length. Pointwise, pairwise, and listwise objectives describe different forms of ranking supervision, while their downstream value depends on the relevance criterion they encode.

The final goal is not an abstract relevance score. It is a compact context containing useful, attributable, eligible, and nonredundant evidence. Diversity-aware selection, metadata policy, and cascaded latency budgets therefore belong to the retrieval design. A debuggable system separately measures candidate generation, reranking, and context selection, preserving the interfaces needed to improve one stage without obscuring a failure in another.

References

[177] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019, doi: 10.48550/arXiv.1901.04085.

[178] T.-Y. Liu, "Learning to Rank for Information Retrieval," *Foundations and Trends in Information Retrieval*, vol. 3, no. 3, pp. 225–331, 2009, doi: 10.1561/1500000016.

[179] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, Association for Computing Machinery, 1998, pp. 335–336. doi: 10.1145/290941.291025.

[180] W. Sun *et al.*, "Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 14918–14937. doi: 10.18653/v1/2023.emnlp-main.923.

Chapter 38: Advanced Retrieval and RAG Architectures

Abstract

A one-pass Retrieval-Augmented Generation pipeline asks one query of one index and gives the resulting context to a generator. That interface is insufficient when a query needs reformulation, several evidence paths, or a representation of document structure. This chapter develops query transformation, multi-query and decomposed retrieval, HyDE, feedback-driven and multi-hop retrieval, hierarchy, graph-based retrieval, and adaptive routing. Its central concern is not adding stages indiscriminately, but selecting a richer retrieval policy only when a measurable limitation of the simple pipeline justifies its cost and failure surface.

38.1 Introduction

Chapter 32 established the elementary Retrieval-Augmented Generation (RAG) path:

$$\text{query} \longrightarrow \text{retrieve} \longrightarrow \text{context} \longrightarrow \text{generate}$$

Chapters 33–37 refined its components with semantic and sparse retrieval, vector indexing, segmentation, hybrid candidate generation, and reranking. The elementary path remains a strong baseline. It is low latency, has a small number of interfaces, and can be inspected as one query, one candidate set, and one context. It fails, however, when the user formulation is a poor retrieval key, when evidence is dispersed across several sources, or when the appropriate next search depends on an earlier result.

An **advanced retrieval architecture** changes one or more of these interfaces. It may transform the query before search, issue several complementary queries, separate a compound question into subproblems, retrieve again after inspecting evidence, move between levels of a document hierarchy, or traverse links in a graph. The common aim is to make the needed evidence reachable without asking the generator to invent a connection that retrieval never exposed.

More stages are not automatically better. Each transformation can drift from the user's information need; each additional search can add latency, duplicate evidence, and another opportunity for a model error to become a retrieval error. This chapter therefore treats advanced retrieval as a controlled policy over a retrieval budget. It does not develop Agent architectures, Tool Calling, general planning, LangChain, LangGraph, or a full RAG evaluation methodology. An LLM may participate in rewriting or routing, but the object of study remains the retrieval architecture.

38.2 Query Transformation and Parallel Retrieval

The question a user asks is not necessarily the string that should be passed to a retriever. Let q_{user} denote the user-facing question and let q_{retr} denote a retrieval-oriented query. **Query Rewriting** constructs

$$q_{\text{retr}} = \rho(q_{\text{user}}), \tag{308}$$

where ρ may resolve a reference from dialogue history, remove conversational framing, make an entity explicit, or use terminology that is more likely to occur in the corpus. The rewrite is an interface transformation, not a new user request. Its contract must preserve the original information need: rewriting What did the report say about it?'' after a discussion of a named report may be necessary; silently replacing a question about a specific model version with a broader question about the product is not.

Query Expansion augments a retrieval query with alternative lexical or semantic cues. Synonyms, abbreviations, entity aliases, and domain terms can improve recall when the corpus and the user use different vocabulary. Classical relevance feedback and query expansion distinguish global reformulations from local changes informed by initially retrieved documents [181]. Expansion has a symmetric risk: every added term or concept can make a result set broader in the wrong direction. This **query drift** is especially damaging in a hybrid system because an expanded lexical path can introduce a large candidate pool that appears plausible but no longer satisfies the original constraint.

Multi-Query Retrieval issues several formulations for one information need. Given

$$q_{\text{user}} \to (q_1, \ldots, q_m), \tag{309}$$

each q_i retrieves a candidate set $C_{K(q_i)}$. The initial pooled set is

$$C_{\text{multi}(q_{\text{user}})} = \cup_{i=1}^{m} C_{K(q_i)}. \tag{310}$$

The union in Equation 310 must be deduplicated by retrieval-unit and source identity before it is passed to the reranking and context-selection stages of Chapter 37. The rank fusion methods from Chapter 36, including Reciprocal Rank Fusion, can combine the per-query result lists without assuming that their raw scores are comparable. Multiple formulations are useful when the corpus may use several names for the same concept. They cost m retrieval operations, enlarge the candidate pool, and can yield several copies of the same evidence.

Query Decomposition solves a different problem. A complex information need is divided into distinct subproblems:

$$\text{complex question} \longrightarrow \text{subquery 1, subquery 2, ..., subquery } n$$
$$\longrightarrow \text{retrieve evidence for each subproblem} \longrightarrow \text{aggregate evidence}$$

Multi-query retrieval expresses one intended search in several ways; decomposition identifies several searches whose answers must be combined. A question asking for the regulator of an organization and the date of that regulator's latest ruling may require two different evidence paths. Decomposition can make coverage explicit, but an incorrect split can omit a condition that links the subproblems or create unsupported intermediate assumptions. The answer generator must not be asked to treat independent subquery answers as mutually consistent without retaining their sources and scope.

Strategy	What it changes	Principal cost or risk
Query rewriting or expansion	The query representation before a first retrieval pass	Intent loss or query drift
Multi-query retrieval	Several formulations of one information need	Extra searches, duplicate candidates, and fusion cost
Query decomposition	One compound need into distinct evidence tasks	Incorrect subproblems or lost dependencies
Iterative retrieval	Later queries conditioned on earlier evidence	Error propagation, loops, and growing latency
Hierarchical or graph retrieval	The search space and relations available to retrieval	Index-construction cost and structural errors

Table 58: Advanced retrieval methods alter different interfaces in the basic RAG path. Their costs are not interchangeable: a query rewrite changes intent risk, whereas a graph index changes ingestion and traversal risk.

38.3 Hypothetical Documents and Retrieval Feedback

Hypothetical Document Embeddings (HyDE) change the representation used for dense retrieval rather than simply adding terms to the original query. A language model first writes a hypothetical passage $\tilde{d}$ that might answer q_{user}. A document encoder then embeds the hypothetical text and retrieves real corpus passages near that vector:

$$q_{\text{user}} \to \text{generate } \tilde{d} \to f(\tilde{d}) = e_{\text{HyDE}} \to \text{TopK}_{d \in \mathcal{D}} \, \text{sim}\big(e_{\text{HyDE}}, e_d\big). \tag{311}$$

The intuition is geometric. A short question and an answer-bearing passage can have different linguistic forms, while a generated passage can resemble the style and content pattern of corpus passages more closely. HyDE uses the generated text as a retrieval representation, not as evidence. It need not be factually correct. The dense retrieval step is meant to ground the query in actual corpus items, and the hypothetical passage must never be presented as a source or merged into an answer as though it had been retrieved [182].

This separation also makes HyDE's limitations clear. Hallucinated entities or relations can steer e_{HyDE} toward the wrong neighborhood. The usefulness of the technique depends on the embedding model's geometry, the prompting policy, and the target corpus. It adds a generation step before retrieval and

should be compared with a query-only baseline under the same candidate depth and latency budget. A plausible hypothetical document is not proof of a useful retrieval representation.

Retrieval with feedback closes a related loop. An initial result set can reveal an alias, a missing constraint, or an entity needed for a more precise follow-up query. In classical relevance feedback, judged or presumed-relevant results inform a revised query; modern systems may use a model to inspect retrieved metadata or passages and propose the revision. The important distinction is between evidence and control. A passage used to form the next query is a tentative signal, not automatically verified support for the eventual answer. The system should record which result informed the revision and should preserve the original query alongside every later one.

38.4 Iterative and Multi-Hop Retrieval

Some questions cannot be answered from a first result list because the next search key is itself contained in the retrieved evidence. Let $q_0 = q_{\text{user}}$, let E_t be the evidence retained after round t, and let u be a query-update procedure. An iterative retrieval loop can be written as

$$q_{t+1} = u(q_0, E_1, \ldots, E_t), \quad E_{t+1} = \text{Retrieve}(q_{t+1}). \tag{312}$$

Unlike Multi-Query Retrieval, the next query in Equation 312 is not available independently at the start. It can depend on an entity, relation, or contradiction exposed by a prior round. Work on interleaving retrieval and intermediate reasoning makes this dependency explicit for knowledge-intensive multi-step questions: what is retrieved next can depend on what earlier evidence has established [183].

Multi-Hop Retrieval is a particular iterative setting in which answering requires connected evidence. A first hop may retrieve an entity description; a relation mentioned there determines the next hop; the answer is supported only after the resulting pieces are combined. Multi-hop question-answering benchmarks such as HotpotQA make the need for multiple supporting documents and attribution visible [184]. Multi-hop dense retrieval formalizes a recursive retriever that conditions later retrieval on the question and previously retrieved passages [185].

The additional expressiveness creates an error-propagation path. If a first result names the wrong entity, a perfectly executed second search can be irrelevant to the original question. Branching over several first-hop candidates can increase recall but causes the candidate space to grow quickly. An iterative system therefore needs to retain provenance by round, limit branching, rerank candidate paths as well as individual passages where appropriate, and distinguish an uncertain intermediate hypothesis from an established source fact.

The retrieval loop should also have an explicit stopping rule. Let Δ_t denote an estimated marginal evidence gain of round t, $R_{\max}$ a maximum number of rounds, and b_t the remaining retrieval budget. A conceptual policy is

$$\text{stop}_t = 1 \quad \text{if } t = R_{\max} \;\text{ or }\; \Delta_t < \delta \;\text{ or }\; b_t \le 0. \tag{313}$$

The gain estimate in Equation 313 is only a controller signal; it is not a reliable statement that the answer is true. It might use new-source coverage, lack of score improvement, result redundancy, a learned confidence estimate, or a fixed budget. Stopping too early leaves an evidence chain incomplete; stopping too late produces an expensive loop of near-duplicate searches. The policy must make this latency–quality trade-off observable.

38.5 Hierarchical and Recursive Retrieval

Chapter 35 introduced Parent–Child Retrieval: index small child chunks for precise matching, then map a selected child to a larger parent section or local neighborhood before generation. This is already an advanced retrieval decision because the object that maximizes first-stage relevance is not necessarily the object that provides complete evidence. The parent expansion must preserve source identity, ordered spans, and the context budget; otherwise, a precise hit can become an opaque block of unrelated text.

Hierarchical Retrieval generalizes the idea to several levels:

document → sections → subsections → chunks

retrieve or route at one or more levels → expand selected evidence

For a book, paper, manual, or long report, a section-level representation can help locate the broad topic, while a child chunk can supply the exact statement. Hierarchy can also guide context expansion: retrieve a detailed passage, then attach its heading, parent scope, or a bounded neighboring span. This does not require a learned tree. A deterministic document hierarchy is valuable when it is extracted reliably and versioned with the source.

RAPTOR is a representative recursive design. It clusters and summarizes lower-level text to construct higher-level representations, permitting retrieval at several abstraction levels rather than only from contiguous leaf chunks [186]. The architecture can help a broad query reach a document-level theme and a narrow query reach a local passage. Its summaries, however, are derived artifacts: they can omit qualifiers, flatten disagreement, or become stale after an underlying source changes. A system should preserve links from every summary to the source spans it abstracts and should not treat a summary as an independent authoritative source.

38.6 Graph-Based Retrieval and Graph RAG

Flat vector retrieval treats each indexed chunk as an independent candidate except for similarity and any metadata filters. **Graph-Based Retrieval** adds an explicit relation structure. Let

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \tag{314}$$

be a retrieval graph, where nodes in $\mathcal{V}$ can represent entities, passages, documents, sections, or concepts, and edges in $\mathcal{E}$ can represent citations, hyperlinks, document containment, entity relations, or declared semantic links. A query can first retrieve seed nodes, then traverse selected edges to gather related evidence. The graph's usefulness depends on whether its edges represent a relation that is relevant to the task; connecting all semantically similar chunks indiscriminately merely turns a neighborhood into another source of redundancy.

A **Knowledge Graph** is a more structured instance in which nodes and edges have declared entity and relation semantics. A conceptual path is

$$\begin{aligned}&\text{query} \longrightarrow \text{entity and relation identification} \longrightarrow \text{graph retrieval}\\ &\longrightarrow \text{relevant nodes and edges} \longrightarrow \text{attributable textual context} \longrightarrow \text{generation}\end{aligned}$$

Knowledge Graph retrieval can be useful for entity-centric and relational questions, where traversal makes a relation explicit that a dense similarity score may not expose. It is not a substitute for text retrieval. Entity resolution errors, incomplete extraction, relation-schema choices, and source updates can make a graph less complete or less current than the documents from which it was built. Graph evidence must retain its supporting source text and timestamps rather than reducing a contested relation to an untraceable edge.

Graph RAG names a family of RAG systems that combine generation with graph-structured text indexes or knowledge graphs; it is not one canonical algorithm. Some designs combine chunk retrieval with entity graphs and local traversal. Others construct community-level summaries for global questions over a corpus. The Graph RAG approach of Edge et al. builds an entity graph and community summaries, then uses those summaries to support query-focused synthesis over a large text collection [187]. That example illustrates a broader point: graph structure can change the unit retrieved and the kind of question a system can support, but it also moves substantial complexity to ingestion, graph maintenance, and provenance management.

38.7 Adaptive Routing and Evidence Aggregation

Not every query deserves the same retrieval policy. An **adaptive retriever** or **router** can choose among no retrieval, one-pass retrieval, Multi-Query Retrieval, iterative retrieval, graph traversal, or a deeper candidate search. The router may use query length, explicit entities, ambiguous references, predicted complexity, source availability, initial candidate confidence, or a latency service-level objective. These are imperfect proxies. A short question can require several sources; a long question can be answerable from one precise passage. Routing should be evaluated against the selected policy's end-to-end evidence quality and cost, not merely against a complexity label.

Retrieval depth has several meanings and should be named precisely. It can mean the first-stage candidate depth K from Chapter 37, the number of expansion levels in a hierarchy, the hop count in a graph, or the number of iterative rounds. Increasing any of these can expose more evidence, but it also raises retrieval, reranking, and context-construction costs. A system that silently changes depth in response to a model confidence score is difficult to reproduce; the route, thresholds, budgets, and observed stopping reason should be recorded with the answer trace.

Advanced retrieval commonly returns evidence from several paths. Before generation, the system must deduplicate spans, group passages by source and revision, retain a useful order, and assess whether the assembled context covers each required subproblem. More passages do not imply more support. Ten near-duplicate chunks provide less independent evidence than two complementary primary sources. Chapter 37's distinction between reranking and final context selection remains essential: a retrieved or highly ranked item still may not fit the final context budget.

Conflicting evidence requires an explicit policy rather than a silent average. The context should preserve source identity, timestamp, version, authority signal, and the exact claim each passage supports. A system may prefer a current official source for a current factual question, surface the disagreement, request clarification, or decline to synthesize an unsupported conclusion. It should not concatenate incompatible claims merely because they are both relevant to a broad query. Conflict resolution is an evidence and provenance problem before it is a generation problem.

38.8 Failure Modes

The first class of failures comes from changing the query. A rewrite can remove a crucial qualifier; expansion can drift toward an associated but irrelevant topic; Multi-Query Retrieval can return redundant variants while hiding the original result in a large pool. Decomposition can assign a false premise to a subproblem. HyDE can encode hallucinated concepts that pull retrieval toward a wrong semantic neighborhood. These are not generic generator hallucinations: they are retrieval-control errors that must be inspected before the answer stage.

The second class comes from repeated or structured search. Iterative retrieval can amplify a wrong early entity, branch into an unmanageable candidate tree, or loop without gathering new evidence. Parent expansion can dilute a precise child hit with irrelevant parent text. Hierarchical summaries can lose a condition present in their leaves. Graph traversal can move from a correct seed through an irrelevant edge, while bad entity linking can attach the query to the wrong graph region. Stale graphs and summaries can conflict with a current source corpus.

Finally, complexity itself is a failure mode. A strong one-pass hybrid retriever with a suitable reranker can outperform a weakly controlled multi-stage pipeline. Every added component requires data, versioning, latency budget, evaluation slices, and a recovery path. Advanced retrieval should be introduced only when a measured failure of the simpler pipeline identifies the missing capability: vocabulary mismatch, insufficient candidate recall, missing multi-hop evidence, long-document scope, relational structure, or an unsuitable fixed-depth policy.

38.9 Implementation Contracts

The **query-control contract** must retain q_{user}, every rewritten, expanded, or decomposed query, the controller or prompt revision, declared intent-preservation policy, and the reason that each query was issued. It must bind multi-query fusion to the candidate identifiers and deduplication rules from Chapters 36 and 37. A HyDE implementation additionally needs the hypothetical-document prompt, generator revision, embedding model, and a guarantee that hypothetical text is never presented as retrieved evidence.

The **iterative contract** must define state retained across rounds, allowed query-update inputs, branch factor, maximum rounds, candidate and reranker depths per round, stopping policy, timeout, and fallback behavior. It should record evidence by round and distinguish retrieved documents from inferred intermediate claims. Tests should include an intentionally wrong first-hop entity, redundant

feedback, a query requiring two independent sources, and a request that must stop because its budget is exhausted.

The **structure contract** must version parent–child links, hierarchy extraction, summary derivations, graph schema, entity-resolution model, edge types, traversal policy, and source-to-node provenance. It must specify how source revisions delete or rebuild nodes, edges, summaries, and indexes. Permission restrictions propagate through every expansion and traversal: an eligible child must not permit the system to expose an ineligible parent or neighboring node.

The **evaluation contract** should compare every advanced route with a fixed one-pass baseline under the same corpus revision. It should report candidate recall, evidence coverage by subproblem, duplicate rate, source diversity, conflict handling, answer support, retrieval and reranking latency, generation-context size, and total cost. Slice tests should include ambiguous language, aliases, compound questions, multi-hop chains, long documents, incorrect entity links, stale graph relations, conflicting source versions, and cases where additional retrieval produces no new useful evidence.

38.10 Summary

Advanced retrieval improves on a single query and single search pass by changing the query, repeating retrieval in response to evidence, or exposing document and graph structure. Query rewriting, expansion, and Multi-Query Retrieval address vocabulary and formulation mismatch; decomposition, iterative retrieval, and multi-hop retrieval address evidence dependencies; Parent–Child, hierarchical, recursive, and graph-based methods change how the corpus is represented and traversed.

These strategies are controlled retrieval policies, not interchangeable features. A useful system preserves the original query, source provenance, and evidence history; routes only when a simple baseline cannot meet the need; and stops when additional retrieval no longer offers enough expected evidence to justify its cost. Chapter 39 evaluates such architectures, but meaningful evaluation begins with the observable interfaces and failure boundaries established here.

References

[181] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[182] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 1762–1777. doi: 10.18653/v1/2023.acl-long.99.

[183] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 10014–10037. doi: 10.18653/v1/2023.acl-long.557.

[184] Z. Yang *et al.*, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2018, pp. 2369–2380. doi: 10.18653/v1/D18-1259.

[185] W. Xiong *et al.*, "Answering Complex Open-Domain Questions with Multi-Hop Dense Retrieval," in *International Conference on Learning Representations*, OpenReview.net, 2021. [Online]. Available: https://openreview.net/forum?id=EMHoBG0avc1

[186] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv preprint arXiv:2401.18059*, 2024, doi: 10.48550/arXiv.2401.18059.

[187] D. Edge *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024, doi: 10.48550/arXiv.2404.16130.

Chapter 39: RAG Evaluation and Diagnostics

Abstract

Retrieval-Augmented Generation is an evidence pipeline rather than a single model call. Its evaluation must therefore separate corpus coverage, candidate retrieval, ranking, context construction, answer quality, grounding, citations, and operational cost. This chapter develops the standard ranking metrics used at the retrieval boundary; distinguishes context, answer, and evidence evaluation; and presents controlled ablations, diagnostic traces, regression suites, and online signals. The central principle is that an incorrect final answer is an observation to explain, not a diagnosis of one component.

39.1 Introduction

Chapter 32 defined the basic Retrieval-Augmented Generation (RAG) path:

query → retrieval → ranking → context construction → generation

Every arrow is a possible source of error. A required fact may be absent from the corpus, present but omitted by retrieval, retrieved but ranked below a cutoff, selected but truncated from the final context, available in the context but ignored by the generator, or stated without support or with an incorrect citation. An incorrect answer alone does not identify which of these events occurred. Conversely, a correct answer can arise from parametric knowledge even when the retrieved context is irrelevant. RAG evaluation must therefore make the evidence path observable rather than report only a final answer score.

This chapter closes the RAG and Knowledge Augmentation section. It builds on the corpus, context, and provenance interfaces of Chapter 32; the first-stage and reranking boundaries of Chapters 33–37; and the trace and policy boundaries of Chapter 38. It applies Chapter 20′s cautions about human and model-based judging to an evidence-grounded setting. It does not introduce an evaluation framework tutorial, Agent architecture, or Tool Use.

39.2 Evaluation Objects and Retrieval Metrics

Let $\mathcal{Q}$ be an evaluation query set. For a query q, let

$$R_{k(q)} = (d_1, \ldots, d_k) \tag{315}$$

be the ordered top-k retrieval list, and let $G(q)$ be the set of retrieval units judged relevant under an explicit task definition. Relevance is not an intrinsic property of a passage. A source can mention a query term without supporting the needed claim, while two passages can each be relevant because they supply complementary parts of a multi-hop answer. The corpus revision, retrieval-unit boundaries, filters, and judging guideline are consequently part of every metric definition.

For binary relevance, Recall@k and Precision@k are

$$\text{Recall}\,@k(q) = \frac{|R_{k(q)} \cap G(q)|}{|G(q)|}, \quad \text{Precision}\,@k(q) = \frac{|R_{k(q)} \cap G(q)|}{k}. \tag{316}$$

Recall@k asks what fraction of known relevant units reaches the top-k boundary; Precision@k asks what fraction of that returned prefix is relevant. A binary Hit Rate records whether at least one relevant unit occurs:

$$\text{Hit}\,@k(q) = 1\left[R_{k(q)} \cap G(q) \neq \varnothing\right]. \tag{317}$$

Hit Rate is useful when one sufficient passage can answer a question, but it does not distinguish a result at rank one from one at rank k, nor does it reward collecting all complementary evidence. Standard information-retrieval evaluation treats these metrics as properties of ranked lists under relevance judgments, not as universal measurements of answer quality [188].

Ranking-sensitive measures make position explicit. Let $r_{\text{first}(q)}$ be the one-based rank of the first relevant item, with a reciprocal-rank contribution of zero when no relevant item is returned. Then

$$\text{MRR} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{r_{\text{first}(q)}}. \tag{318}$$

Mean Reciprocal Rank (MRR) rewards placing one useful unit early. It is appropriate when the first supporting result matters most, but it says little about the remaining evidence needed for a complex answer. For graded relevance labels g_i, where larger values indicate a more useful item at rank i, a common discounted cumulative gain is

$$\mathrm{DCG}\,@k(q) = \sum_{i=1}^{k} \frac{2^{g_i} - 1}{\log_2(i+1)}, \quad \mathrm{nDCG}\,@k(q) = \frac{\mathrm{DCG}\,@k(q)}{\mathrm{IDCG}\,@k(q)}. \tag{319}$$

The ideal DCG, IDCG@k, is the value obtained by ordering the same graded judgments optimally. Normalized Discounted Cumulative Gain (nDCG) therefore rewards highly relevant results near the front while making scores comparable across queries with different relevance-label totals. It is a ranking metric, not proof that an LLM will use the resulting items correctly.

Recall@k is often a first priority for RAG candidate generation. A later reranker cannot recover evidence that the first stage did not expose, as Chapter 37 emphasized. Yet maximizing recall by increasing k indefinitely is not a solution: more candidates create reranking work, duplicate evidence, and a larger set from which a context constructor can admit noise. The useful operating point depends on the downstream context budget, candidate depth, reranker, and latency target.

Metric	Question it answers	Important limitation
Recall@k	How much judged relevant evidence reaches a candidate boundary?	Does not reward early placement or assess generator use.
Precision@k	How much of a retrieved prefix is relevant?	Can penalize a deliberately broad recall-oriented first stage.
Hit Rate@k	Did at least one relevant unit appear?	Ignores rank and complementary evidence.
MRR	How early is the first relevant item?	Ignores later relevant items after the first hit.
nDCG@k	Does the ranked prefix place graded relevance near the front?	Requires defensible graded labels and remains a ranking metric.

Table 59: Retrieval metrics measure different properties of a ranked list. Their meaning depends on relevance judgments, cutoff depth, and the role of the list in the later RAG pipeline.

39.3 Context and Generation Evaluation

The top-k list is not the context that the language model receives. Let $P(q)$ be the set of retrieval units that survive reranking, deduplication, truncation, ordering, and serialization into the final prompt. Let $S(q) \subset G(q)$ be the subset of units sufficient to support the intended answer under the stated task. A useful context-level view is

$$\mathrm{ContextRecall}(q) = \frac{|P(q) \cap S(q)|}{|S(q)|}, \quad \mathrm{ContextPrecision}(q) = \frac{|P(q) \cap G(q)|}{|P(q)|}. \tag{320}$$

These expressions are abstractions, not a claim that all tasks have one complete gold set. Some questions have several valid support paths, and a context can contain a relevant unit that is redundant for a particular answer. Their practical value is to make the difference visible: a retriever may achieve high candidate Recall@k while the prompt loses the key item through a final token budget, poor ordering, repeated chunks, or a parent–child expansion that dilutes a precise result. Context evaluation must also inspect redundancy, contradictory evidence, source diversity, metadata, and ordering, because each changes what the generator can attend to.

Generation evaluation asks a different set of questions. **Answer correctness** asks whether the answer is factually or task-wise right, preferably through a reference, deterministic verifier, or qualified human judgment. **Answer relevance** asks whether the response actually addresses the user's information need. **Completeness** asks whether it covers the necessary parts of the answer. These

properties are related but nonidentical. A relevant response may be wrong; a correct response may omit a required condition; a complete answer may be unusably verbose for an interactive request.

Reference-based evaluation compares an output with gold answers, labeled relevant passages, or annotated support. It can make a narrow property auditable, but creating high-quality references is expensive and may underrepresent valid paraphrases or alternative evidence paths. Reference-free evaluation instead uses the query, retrieved context, model judges, consistency checks, or structural constraints. It is useful for broad coverage and evolving corpora, but it replaces one missing label with assumptions about the evaluator. RAGAS and ARES exemplify component-aware automated evaluation through categories such as context relevance, answer faithfulness, and answer relevance; their existence does not make any generated score independent ground truth [189], [190].

39.4 Faithfulness, Grounding, and Citations

Faithfulness or **groundedness** asks whether the answer's claims are supported by the evidence actually provided to the generator. Let $A(q) = (a_1, \ldots, a_M)$ be a decomposition of an answer into evaluable claims, and let $z_j = 1$ when the final context supports claim a_j under a declared support rule. A conceptual claim-support score is

$$\text{Groundedness}(q) = \frac{1}{M} \sum_{j=1}^{M} z_j. \tag{321}$$

The difficulty is in the support rule. A claim can be directly stated, entailed by several passages together, partially supported, contradicted, or outside the scope of the context. Claim segmentation and entailment judgments can themselves be fallible. An LLM judge can scale the comparison between claims and passages, but it cannot make an unsupported inference reliable merely by expressing a fluent rationale. Chapter 20′s judge-versioning, order-control, and human-audit principles apply directly here.

Groundedness is not answer correctness. A response can be grounded but incomplete because the context lacks a needed fact. It can be correct but unsupported because the model used parametric knowledge after a retrieval miss. It can be relevant but false, or supported by a context that is itself stale or low-authority. These distinctions are diagnostic assets: they identify whether a repair belongs in ingestion, retrieval, context construction, generation, or source policy rather than in generic prompting.

Explicit citations add another interface. **Citation correctness** asks whether a cited source supports the adjacent or associated claim. **Citation completeness** asks whether claims that require support receive suitable citations. **Citation quality** asks whether the cited source is authoritative, current, and appropriate for the claim. A valid-looking source identifier alone establishes none of these. ALCE formalizes end-to-end citation evaluation as a separate dimension alongside answer quality, illustrating why citation coverage and citation entailment should not be collapsed into an answer score [191].

39.5 Human and Model-Based Evaluation

Human evaluation remains essential when answer usefulness, nuanced correctness, explanatory completeness, or source appropriateness cannot be reduced to a deterministic rule. A useful rubric names the evidence available to annotators, whether they may use external knowledge, what counts as a supported claim, how to score uncertainty, and how to treat a correct but uncited answer. It should record annotator expertise, candidate blinding, response order, inter-annotator disagreement, and the aggregation procedure. Human labels are observations under a rubric, not a universal utility function.

LLM-as-a-Judge can evaluate large answer sets cheaply and express natural-language criteria such as relevance or support. It is especially valuable for triage, rapid regression signals, and domains where references are incomplete. Its limitations are the same ones introduced in Chapter 20: position bias, verbosity bias, prompt sensitivity, judge capability limits, self-preference, and correlated errors when

the target and judge share a model family. Controlled work on LLM judging documents these risks alongside its practical scalability [192].

A robust design treats judges as calibrated instruments, not oracles. Freeze the model revision, prompt, rubric, examples, temperature, answer order, parser, and aggregation rule. Swap candidate order in pairwise tests; compare a stratified sample with human labels or deterministic checks; and report uncertainty and disagreement. Use different mechanisms for different claims: a verifier for an executable answer, human review for difficult source quality, and a judge for scalable open-ended slices. No single evaluator validates the full RAG path.

39.6 Evaluation Data and Controlled Ablations

An evaluation set should represent the decisions a real system must make. It needs ordinary user queries and known supporting documents, but also unanswerable requests, ambiguous formulations, multi-hop questions, conflicting evidence, temporally sensitive facts, and queries whose answer depends on a rare identifier or a particular document version. Public benchmark prompts are useful, but the corpus revision, query distribution, and source governance must resemble the operating setting. A synthetic question that perfectly echoes one chunk can make retrieval look easy while failing to test the vocabulary mismatch or ambiguity present in production.

Hard negatives are passages that are lexically or semantically plausible yet do not answer the query. They test whether a retriever or reranker can distinguish an entity's current policy from an outdated revision, a related product from the named version, or a broad topical summary from an answer-bearing passage. They are particularly valuable because random negatives often make a ranking task artificially easy. A hard negative must still be labeled carefully: an apparently wrong passage may be a legitimate alternative support path under a wider task definition.

Distribution shift is continuous rather than exceptional. New documents change vocabulary and source authority; new user populations change query style; a revised chunking policy changes the retrieval-unit distribution; and a model update can alter query rewriting or answer length. Evaluation is therefore a repeated measurement under versioned corpus and protocol conditions, not a one-time benchmark certificate.

Controlled ablations expose a component's contribution. Replacing normal retrieval with an **oracle context** that contains known supporting evidence tests the generator and context format without a retrieval miss. Removing a reranker, varying candidate depth or final top-k, changing chunk size, comparing sparse and dense paths, disabling query rewriting, or changing evidence order each tests one declared interface. The controls must keep the corpus, filters, prompts, generation settings, and scoring protocol fixed. Otherwise a reported improvement cannot be assigned to the purported component.

39.7 Error Attribution and Regression

Diagnostic evaluation begins with the answer trace rather than a single scalar score:

Was the answer wrong? → Was sufficient evidence in the corpus?
→ Was it retrieved? → Was it ranked and included in context?
→ Was the evidence interpreted correctly? → Was the final claim grounded and cited?

This flow yields a concrete taxonomy. A **corpus failure** means sufficient authoritative evidence was absent or unavailable. An **indexing failure** means an eligible source was not represented correctly. A **query-representation failure** originates in analysis, embedding, rewriting, or routing. A **retrieval failure** omits relevant candidates; a **ranking failure** buries them; a **context-construction failure** removes, truncates, duplicates, or obscures them. A **generation failure** misuses available evidence; a **grounding failure** adds unsupported claims; and a **citation failure** links a claim to an unsupported, incomplete, or unsuitable source. Fine-grained RAG evaluation frameworks similarly motivate separate retrieval and generation diagnostics rather than an undifferentiated final score [193].

Layer	Primary evidence	Representative regression guard
Retrieval	Recall@k, Hit Rate, MRR, nDCG, and source-version coverage	Keep corpus, relevance labels, filters, and candidate depth fixed.
Context	Support coverage, precision, redundancy, ordering, and token budget	Compare selected evidence against retrieved candidates and oracle context.
Generation	Correctness, relevance, completeness, and abstention behavior	Fix prompt, model, decoding settings, and answer parser.
Grounding and citations	Claim support, citation correctness, completeness, and source quality	Audit answer-to-context and claim-to-citation links separately.
Systems	TTFT, latency, throughput, context tokens, and cost	Measure quality and service conditions together under the same workload.

Table 60: A RAG regression suite should preserve evidence at each pipeline boundary. An end-to-end answer metric remains useful, but it cannot replace the component evidence needed to locate a regression.

For a metric M_j and a candidate configuration c relative to a baseline b, record

$$\Delta_j = M_{j(c)} - M_{j(b)}. \tag{322}$$

The vector $\left(\Delta_j\right)_j$ is more informative than an aggregate that hides a retrieval, grounding, or latency regression. A release criterion can demand that a new reranker improve nDCG without reducing context support, or that a larger candidate depth improve recall without exceeding a tail-latency budget. This is an application of Chapter 20's multi-objective regression logic to an observable evidence pipeline.

39.8 Online and System Evaluation

Offline labels cannot cover every new document, query, or interaction pattern. Online signals may include source-opening or click behavior where it is meaningful, user corrections, query reformulations, abandonment, explicit feedback, task completion, and escalation to a human. These signals are noisy behavioral observations. A user may open a source because the answer was unclear, reformulate after a correct answer for a new purpose, or abandon because the interface is slow rather than because the retrieval was wrong. They should be joined to sampled human review and trace-level diagnostics, not treated as automatic quality labels.

RAG also has system costs. Measure retrieval and reranking latency, generation latency, Time to First Token (TTFT), total response time, context-token count, requests per second, and monetary or computational cost under an explicit workload. Chapters 23, 26, and 29 explain why those quantities have different bottlenecks. Increasing top-k may raise candidate recall, then add reranking work, increase context length, and increase Prefill cost without improving answer correctness. The relevant objective is a quality–latency–cost frontier, not throughput or a retrieval metric in isolation.

39.9 Failure Modes

The most common evaluation failure is measuring only the final answer or only retrieval. Answer-only evaluation cannot distinguish a missing source from a generator that ignored it; retrieval-only

evaluation cannot detect a context builder that truncates the best passage or a model that fabricates a citation. Another failure is relying on one LLM judge without auditing its prompt, order, and distributional biases. A single evaluator can become the unexamined bottleneck of a supposedly multi-component evaluation system.

Unrealistic synthetic queries, weak relevance labels, omitted unanswerable cases, and hard negatives that are not truly negative can make both ranking and answer metrics misleading. Comparing configurations with different corpora, document versions, filters, prompts, decoding settings, or answer parsers confounds a component change with a protocol change. Evaluation leakage is similarly damaging: if benchmark questions, answers, or known support passages are incorporated into the corpus or model-development loop without disclosure, an end-to-end gain may overstate generalization.

Finally, optimizing directly against one proxy creates a Goodhart risk. A system can learn to maximize an LLM judge, retrieve more passages to raise recall, or attach many citations to raise coverage while reducing usefulness, evidence quality, or latency. Chapter 20's warning applies unchanged: a proxy that becomes an optimization target can stop measuring the property it was chosen to represent. Keep independent audits, protected slices, and full-pipeline traces outside the immediate optimization loop.

39.10 Implementation Contracts

The **dataset contract** must version the corpus, document and chunk identifiers, relevance and support judgments, query set, query distribution, unanswerable and hard-negative policies, source timestamps, and labeling rubric. It must state whether relevance is binary or graded and distinguish a relevant unit from a minimal answer-supporting unit. The **pipeline contract** must retain every query transformation, retriever and index revision, filter decision, candidate list with ranks and scores, reranker inputs, final context order, prompt template, generator revision, decoding configuration, answer parser, and output citations.

The **scoring contract** must name every metric cutoff, averaging convention, treatment of missing or empty relevance sets, judge or verifier revision, claim-segmentation policy, citation-to-claim alignment rule, and uncertainty calculation. Human evaluation requires annotator qualifications, blinding, order randomization, disagreement handling, and audit sampling. An LLM judge requires the frozen prompt, examples, temperature, model version, response-order policy, parser, repeat-call policy, and calibration checks from Chapter 20.

The **regression contract** should compare a candidate with a baseline under the same corpus revision and workload. It should report component metrics, answer quality, grounding, citation quality, latency distributions, context-token use, throughput, cost, and failure slices for vocabulary mismatch, ambiguity, multi-hop evidence, stale sources, conflicts, unanswerable queries, permissions, and long contexts. Retain representative traces for every material regression. A RAG system is diagnosable only when the evidence that crossed each boundary can be reproduced.

39.11 Summary

RAG evaluation is necessarily multi-level. Retrieval metrics establish whether candidate ranking exposes relevant evidence; context metrics establish whether answer-supporting material reaches the prompt; generation metrics establish correctness, relevance, and completeness; grounding and citation metrics establish whether the response is supported by the evidence it presents. These measures complement, rather than substitute for, end-to-end evaluation.

The practical discipline is to preserve the entire evidence path. Build versioned evaluation data, use controlled ablations and oracle contexts to isolate components, inspect trace-level failures, and protect quality, groundedness, latency, and cost together in regression tests. This closes the RAG section with its central engineering principle: a reliable system does not merely produce an answer score; it can explain what evidence was available, what evidence was used, and where a failure entered the pipeline.

References

[188] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[189] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, Association for Computational Linguistics, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.

[190] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 338–354. doi: 10.18653/v1/2024.naacl-long.20.

[191] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 6465–6488. doi: 10.18653/v1/2023.emnlp-main.398.

[192] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. doi: 10.48550/arXiv.2306.05685.

[193] D. Ru *et al.*, "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," in *Advances in Neural Information Processing Systems 37*, Curran Associates, Inc., 2024. doi: 10.52202/079017-0692.